TURING

这就是MCP

艾逗笔（@idoubi） 著



中国工信出版集团
人民邮电出版社
POSTS & TELECOM PRESS

内容提要

这是一本全面、系统、详尽的MCP指南，由业内专家执笔，旨在为读者提供一线开发与使用参考。

本书涵盖MCP的方方面面，从基础概念到实战案例，再到生态系统构建。第1章讲述了MCP的工作原理、诞生背景和应用场景；第2章深入解析了MCP架构；第3章和第4章分别通过实际案例讲解了MCP服务器和MCP客户端的开发；第5章则展示了如何在典型应用场景下使用MCP服务器；第6章介绍了MCP的生态系统。

本书适合所有对MCP感兴趣的读者，包括软件工程师、产品经理、设计师和创业者等。无论你是想了解MCP的基础知识，还是想深入学习MCP的开发和使用，本书都能满足你的需求。

专家推荐

艾逗笔，实战派全栈工程师，超级个体，以“一人抵一团队”的强悍战斗力活跃在AI前沿。作为MCP的早期布道者、深度开发者和一线创业者，他对这一改变AI生态的通信标准有着深刻的洞见。这本书系统解析了MCP的原理与开发实战，案例丰富、干货十足，是开发者深入掌握这一关键基础设施、把握平台型机遇的重要指南。强烈推荐！

——袁进辉，硅基流动创始人 & CEO

作为业内专家，艾逗笔结合丰富的实战经验，剖析了MCP如何解决AI应用集成的痛点，揭示了MCP作为AI时代通信标准的巨大潜力。这本书全面、系统、详尽，是构建AI生态不可或缺的参考读物。

——肖弘，Manus创始人、蝴蝶效应CEO

基于全面的技术与商业研究，艾逗笔讲透了MCP的实用性以及潜力。能在MCP发展早期就学习到这样的洞见，就像“开了外挂”一样！建议每一位大模型应用开发者都来读读这本书。

——John Yang，Same联合创始人

MCP正在重塑AI应用生态，作者艾逗笔结合开发者与创业者的双重经验，系统讲解了MCP的原理与实战。无论是技术探索还是商业布局，本书都极具参考价值。

——低空飞行，ChatWise作者

MCP正在重塑AI应用开发生态。作为该领域具有全球影响力的实践者，艾逗笔充分融合前沿洞察与一线经验，创作出这本兼具理论深度与实践价值的指南，是从业者快速、全面了解MCP的理想读物。

——王新铭，Cherry Studio CEO

艾逗笔是国内少数的实战派专家。这本书梳理了从协议原理到应用开发的完整体系，且涵盖服务器开发、客户端开发、经典使用案例等内容，可谓理论与实践并重，是帮助开发者快速掌握MCP技术、构建AI应用的实用指南。

——谢孟军，DeepChat创始人、积梦智能CEO

2024年11月MCP发布后，艾逗笔敏锐地意识到其颠覆性价值，第一时间扎进该领域。他的导航网站MCP.so整合海量优质MCP服务器，推动了生态发展。作为见证艾逗笔深耕AI前沿的老朋友，我向你推荐《这就是MCP》！它不仅是一本技术读物，更凝

结了一个行业观察者的洞见与热忱。从底层逻辑到应用场景，这本书都讲得务实、通透，既为开发者提供实操抓手，也帮助普通读者看清MCP如何重塑智能交互。

——AJ，WaytoAGI社区发起人

将MCP这种前沿协议在应用层落地，是当下AI的核心挑战。本书结合艾逗笔在MCP创业中的一线经验，通过上百幅图示、翔实的案例，完美架设了从理论到实践的桥梁。对于渴望在MCP领域大展拳脚的实干家，本书不容错过。

——黄益贺，科技博主、newtype社群主理人

MCP是AI时代的“通用语言”和“万能插座”。作者艾逗笔兼具MCP创业者、开发者与布道者的身份，积累了丰富的经验。这本书深入浅出地讲解了MCP的核心原理、开发实践与生态构建，兼具专业性与实用性，强烈推荐给开发者和产品人。

——刘若愚，字节跳动研发工程师、前腾讯高级工程师

第1章

什么是MCP

在软件开发领域存在一个难题，我们可以称其为“*M*×*N*问题”：当多个应用（*M*）需要对接多个外部扩展（*N*）时，如果缺乏统一的标准，每一对组合都得单独适配。这意味着每新增一个应用或一个扩展，整体的集成工作量就会按照*M*×*N*的复杂度增长。例如，从3个应用和4个扩展增加到4个应用和5个扩展时，工作量会从适配12个连接关系增加到适配20个连接关系。

在AI应用开发领域，这个问题尤其突出。AI应用需要调用各类外部工具来扩展自身的能力，比如获取实时信息、访问数据库、处理本地文件等。但AI应用和工具往往来自不同的开发团队，缺少统一的对接标准，这就导致开发者需要为每个AI应用-工具组合单独开发适配逻辑——重复劳动多，沟通成本高，维护难度也越来越大。

通过一个通用的协议，解决AI应用与外部工具的集成难题，是MCP设计的出发点。图1-1中所示是使用MCP降低*M*×*N*（*M*、*N*均为4）问题复杂度的一个示例。

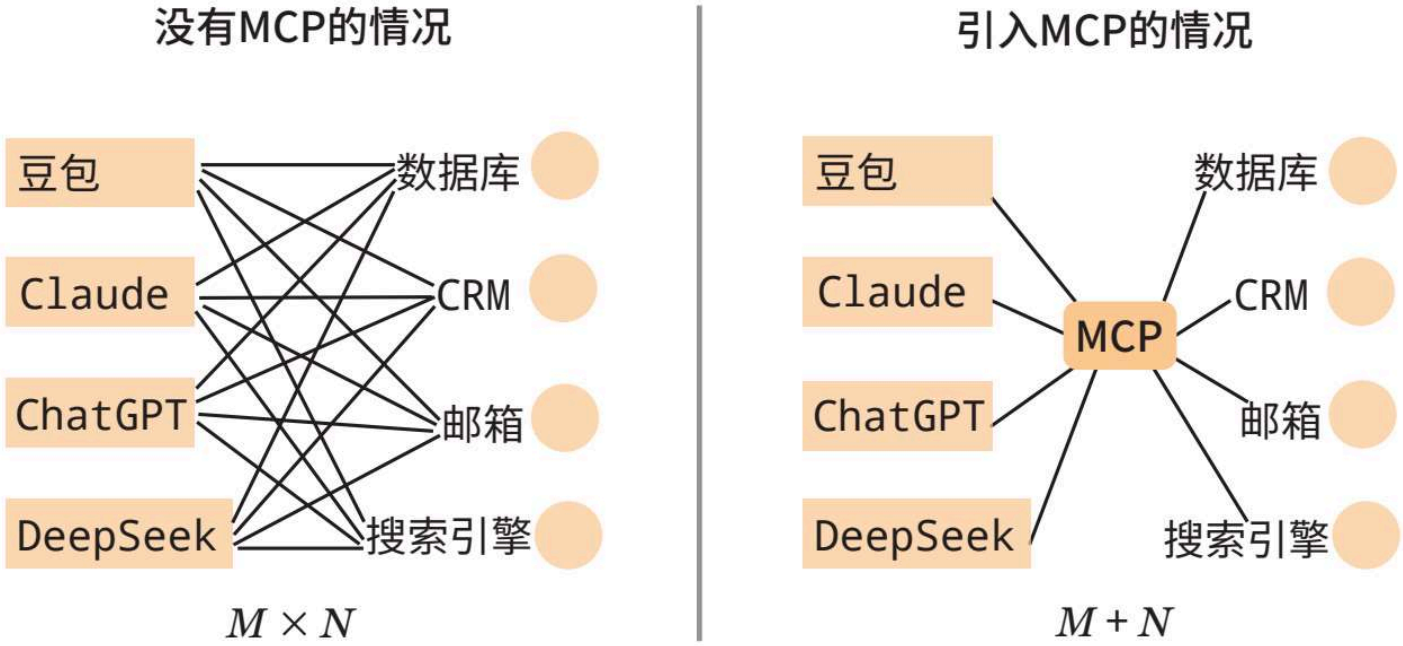


图1-1　通过MCP降低*M*×*N*问题的复杂度

2024年11月25日，Anthropic公司以MIT许可证开源发布MCP，成为AI发展历程中的一个重要里程碑。

MCP是Model Context Protocol的缩写，中文译为“模型上下文协议”。MCP约定了AI应用如何规范地集成外部工具，实现为大模型（Large Language Model）补充上下文（Context）的目的，其本质是应用层协议（Protocol）。

协议即标准。标准的制定，是为了推动行业形成一种共识。共识一旦形成，不仅能降低重复开发的成本，更能推动整个生态系统有序、快速地发展。

我们可以用“秦始皇统一六国”的例子来简单类比MCP诞生的意义。战国时期，七国各自为政，使用不同的文字、货币和度量衡，不同小国之间交流困难，经济、文化、科技发展缓慢。秦始皇统一六国后，为了方便管理，统一了文字、货币和度量衡。从此，各国之间的交流变得方便，经济、文化、科技发展迅速。

“一法度衡石丈尺。车同轨，书同文字。”

——《史记·秦始皇本纪》

1.1 MCP是如何工作的

MCP并不复杂，这个协议的核心是定义了三个角色并明确了它们之间的协作模式。

1.1.1 主机-客户端-服务器

1. MCP主机

本书根据具体情况在主机、客户端、服务器之前添加或省略MCP一词。

MCP主机（有时简称主机）是MCP的中心枢纽，它既是用户与大模型直接交互的终端，又是协调外部资源的调度中心。

主机具有双重客户端的属性。

- 作为与大模型交互的客户端：接收用户输入并请求大模型生成回答。所有支持MCP的AI应用（主要包括AI对话助手、智能体、AI编辑器），都可以看作主机。
- 作为服务协调的MCP客户端：主机创建并管理多个客户端实例（与服务器一一对应）。在请求大模型生成回答的过程中，主机可以通过客户端实例请求外挂的服务器获取额外的数据或者执行特定的任务。主机把从外挂的服务器获取的数据或者执行任务的结果，作为上下文补充给大模型，让大模型生成更加符合用户需求的回答。

2. MCP服务器

MCP服务器（有时简称服务器）是MCP的核心组件，主要负责对接外部数据或服务，并通过标准的数据格式将响应内容发送给MCP客户端。

我们可以将MCP服务器理解为外部数据或服务的一个代理网关，所有的数据交互都通过该代理网关进行。

比如，高德地图（作为数据或服务提供方）开放了其路程规划的API：在MCP出来之前，AI应用需要通过编码的方式，自行对接高德地图API；有了MCP之后，高德地图就可以实现一个MCP服务器，并通过它暴露自己的能力，而支持MCP的AI应用可以直接集成此MCP服务器，不需要额外的对接工作。

3. MCP客户端

MCP客户端（有时简称客户端）是MCP主机与MCP服务器之间的桥梁。客户端“寄生”于主机中，由主机创建并进行调度，客户端与服务器进行连接，从而实现与外部数据或服务的交互。



MCP是基于经典的C/S（客户端-服务器）架构设计的。MCP客户端是相对于MCP服务器的概念，请注意，本书先介绍的是MCP服务器，然后是MCP客户端，目的就是想体现“有了连接服务器的需求，主机才创建了客户端实例”。不过出于表述习惯，我们依然称其为“主机-客户端-服务器”架构。MCP主机要想调用一个外部的MCP服务器，就需要创建一个MCP客户端，才能进行彼此的连接，MCP客户端与MCP服务器是一一对应的。

在本书中，大模型客户端即AI应用（有时简称应用），我们视具体情况不同使用两种表述。

而MCP主机本身是把大模型作为交互对象的客户端，我们称之为“大模型客户端”。尽管在某些场景中，我们容易混淆“大模型客户端”和“MCP客户端”，但在系统架构层面，两者应该明确区分——“大模型客户端”是面向用户（用户可见）的AI应用，“MCP客户端”是由MCP主机创建的进程。

官方给出的MCP架构如图1-2所示（关于MCP架构的相关内容，我们还会在第2章深入讲解）。

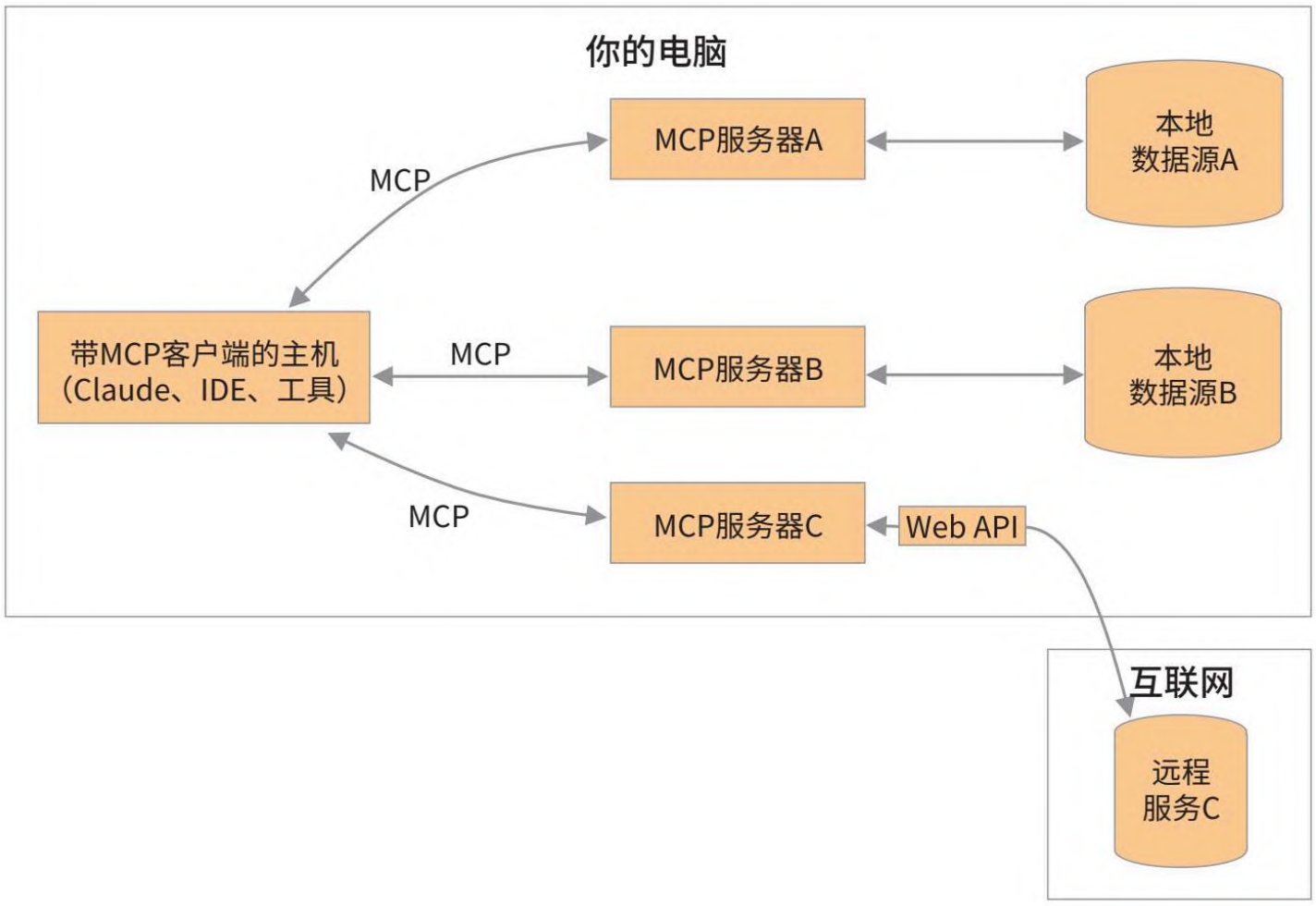


图1-2 MCP架构图

1.1.2 举个例子来理解

本书中的Claude均指Claude桌面版。

用一个具体的例子来理解MCP的运行机制。假设用户要在Claude中使用高德地图MCP服务器来进行路程规划，路程规划的流程如下所示。

1. 高德地图把自己的路程规划能力以API的形式开放；同时提供一个MCP服务器，对接自身的API，提供给所有客户端使用。

2. 用户打开Claude，配置了高德地图MCP服务器的调用方式和密钥。

3. 用户向Claude发送问题：“我想下周从北京开车去上海，帮我规划一条最省时的路线。”

4. Claude请求大模型，告诉大模型有一个叫作高德地图的MCP服务器可用，其中包含一个路程规划工具。

5. 大模型回复Claude，需要请求高德地图MCP服务器调用路程规划工具，查询参数是：“出发地=北京，目的地=上海”。

6. Claude创建一个内部的MCP客户端程序。

7. Claude内部的MCP客户端程序请求高德地图MCP服务器。

8. 高德地图MCP服务器将驾驶路线信息返回给MCP客户端程序。

9. Claude把查到的驾驶路线和用户最初的问题一起发送给大模型，请求大模型回答用户的问题。

10. 有了驾驶路线作为上下文信息，大模型回复用户的问题，返回更为精准的回答。

11. Claude将回答呈现给用户。

本书中包含大量用于辅助理解的流程图，这些图示旨在以直观、结构化的方式呈现系统不同模块之间的交互。为了增强可读性与可视化效果，流程图中的步骤或者说明均作了一定程度的简化，与正文不完全对应。

用一幅图来描述这个过程，如图1-3所示

。

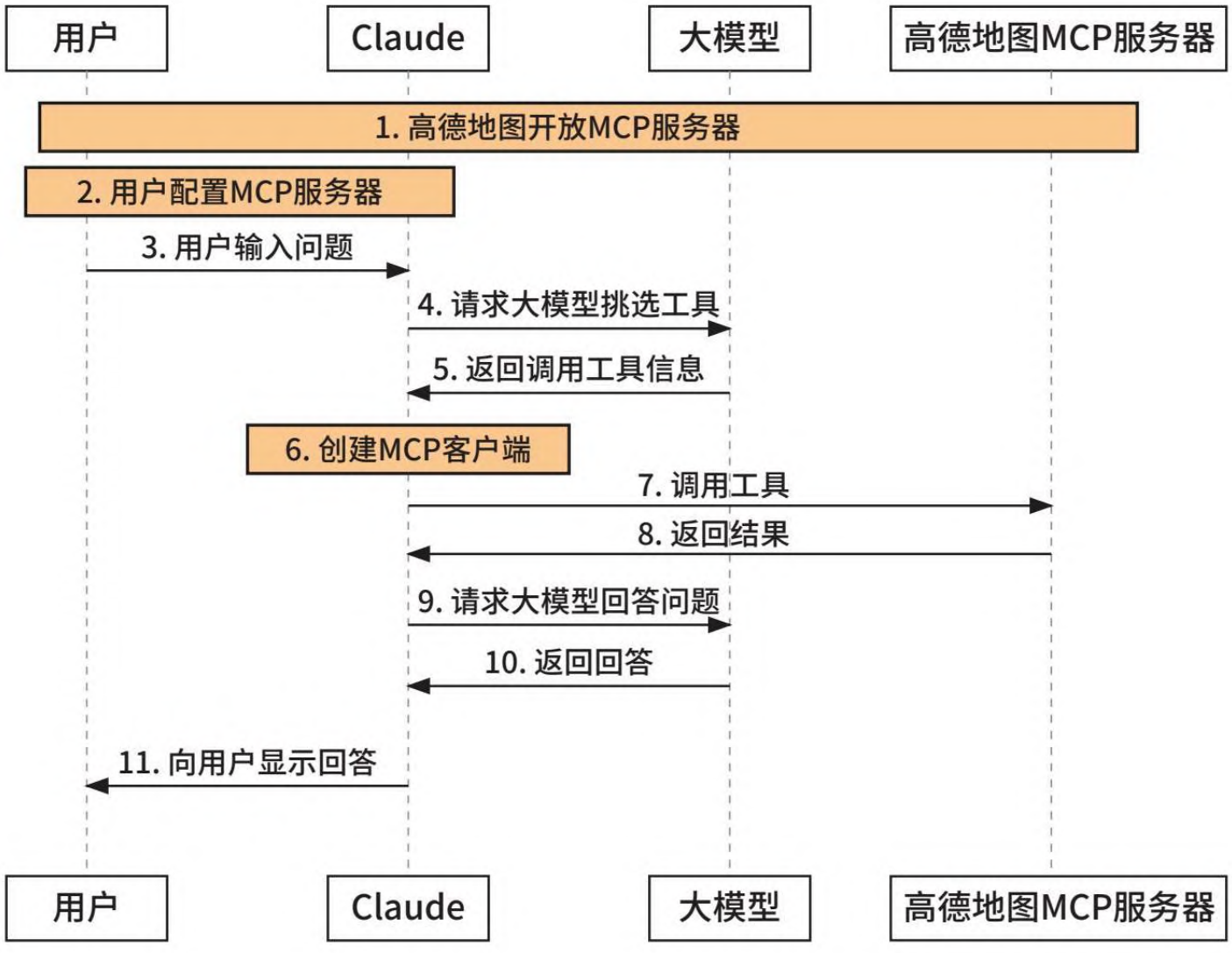


图1-3 在Claude中使用高德地图MCP服务器的交互流程

在这个例子中，Claude是主机，通过外挂高德地图MCP服务器，让大模型有了路程规划能力。

同样的原理，任意提供数据或服务的第三方，都可以通过MCP服务器，暴露自身的数据或服务能力，包括：互联网上的各类服务，本地电脑上的各类文件、数据库等。

任意AI应用，都可以实现MCP，与各类MCP服务器进行通信，将获得的数据作为上下文补充给大模型，旨在让大模型的回答更加精确。

有人把MCP比喻成AI应用的扩展坞，如图1-4所示，我觉得非常贴切。

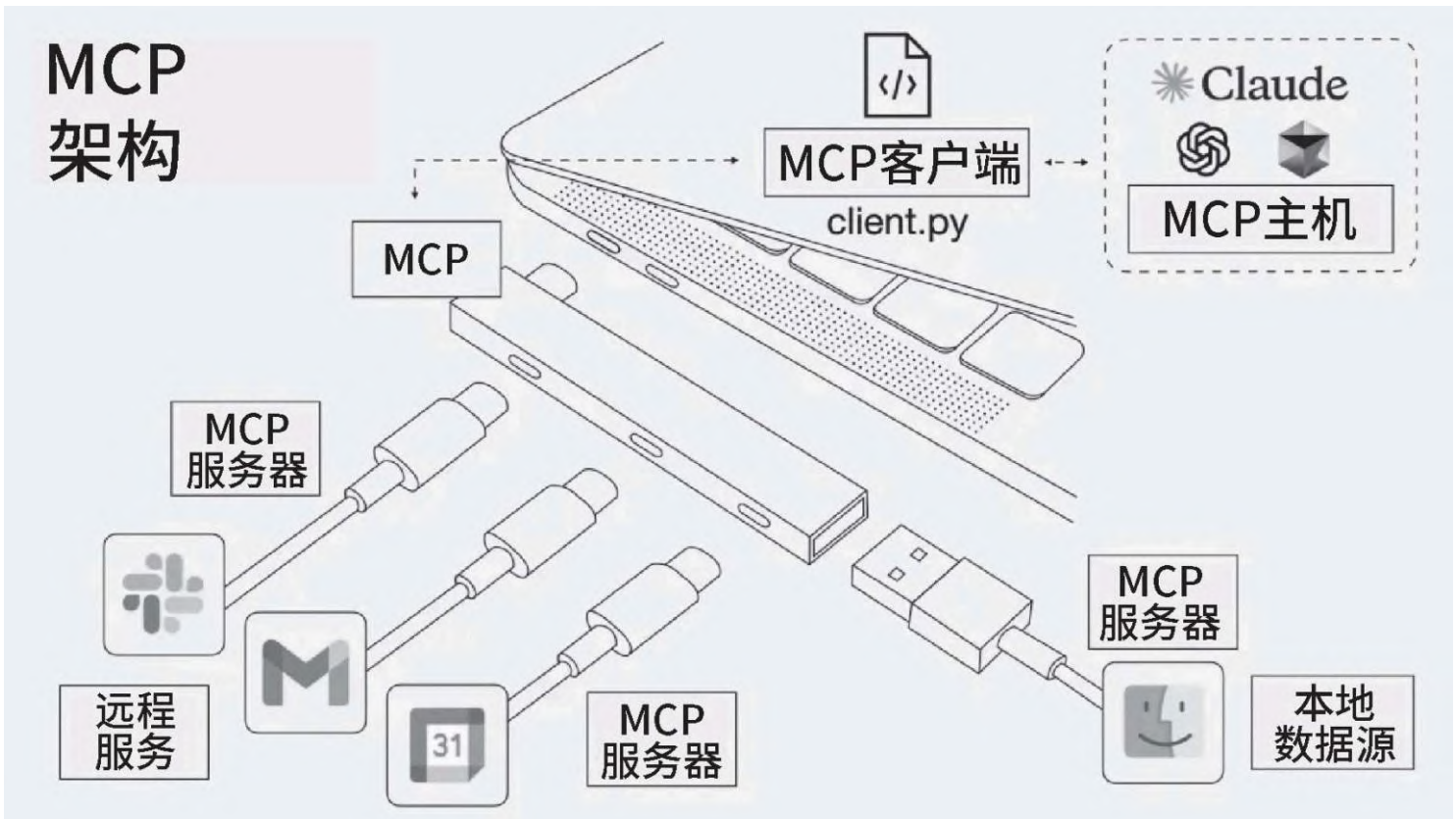


图1-4 MCP与扩展坞

AI应用就像一台只有有限接口的笔记本电脑，无法直接访问外部数据或服务。MCP则扮演了“扩展坞”的角色，通过标准化的协议接口，为AI应用提供连接各种外部系统的能力。

通过MCP，AI应用不再受限于大模型的能力边界，而是能够像连接了扩展坞的笔记本电脑一样，灵活地接入各种外部系统，实现更强大的功能。

1.2 MCP是如何诞生的

1.2.1 大模型视角

2022年11月30日，OpenAI发布了ChatGPT——一款基于大模型的对话产品。ChatGPT以惊人的交互体验迅速出圈（用户通过自然语言对话即可完成编程、写作、推理等复杂任务），让“大模型”这一原本只在技术社区流传的概念首次进入了公众视野。业界迅速达成共识：大模型将成为下一代人机交互的核心载体。

在之后的时间里，大模型技术有了飞速发展，通过对话助手与大模型对话，逐渐成了人们上网获取信息的主要途径。

然而，大模型是通过历史数据进行训练的，这些数据在训练时就已经固定。模型训练完成后，其知识就停留在训练时的状态，无法自动获取训练之后的新信息。也就是说，大模型的“静态知识”与动态世界的“实时需求”之间存在根本性矛盾。

为了突破这一矛盾，解决大模型对训练后信息的获取问题，行业探索出两类主要的解决方案。

第一类：模型微调

模型微调的基本原理是在预训练好的大模型的基础上，使用新的数据集进行额外的训练。通常情况下，模型的基本架构保持不变，只调整全部或者部分参数，从而使模型学习新的知识或适应特定任务。

相比从头训练，模型微调需要的计算资源更少，能够更高效地实现模型的定制化。它的主要局限性在于，模型微调仍然是一个“先训练后部署”的方案，虽然在解决某些专业领域问题时补充了新知识，但不适用于对时效性要求高的场景，比如获取实时新闻、天气信息等。

第二类：上下文补充

上下文补充指的是为大模型提供外部知识库或信息源的技术方案，旨在让大模型突破知识的时效性限制，能够访问专业领域信息，从而提高回答的准确性、时效性和可靠性。

与模型微调不同，上下文补充不需要改变模型参数重新训练，而是在模型生成回答时实时提供相关信息作为输入上下文，让模型能够基于这些最新的信息进行推

理。基于补充的上下文信息，大模型能够回答与训练之后发生的事件相关的问题，弥补静态知识与动态世界之间的鸿沟。

MCP的核心在于C，也就是上下文（Context），从这个角度来看，MCP诞生的初衷就是给大模型补充上下文。

在MCP之前，行业存在如下几种为大模型补充上下文的主流方案。

1. 记忆存储

记忆存储通常由大模型客户端实现。客户端会将用户与大模型的每轮对话内容记录下来，并设定一定的记忆容量。当用户提出新问题时，系统会从记忆库中提取与当前对话相关的信息，作为补充上下文提供给大模型，从而使模型的回答更加连贯，就像赋予了模型“记忆”能力。

不过，这种记忆机制也存在一定的局限性，模型的“记忆”能力主要受限于大模型支持的上下文长度。如果每次都将同一个会话的全部历史内容传入模型，容易因上下文窗口限制而导致信息截断；而若仅选取与当前问题相关的历史内容，则对客户端的实现能力提出了更高的要求，需要依赖相似度匹配等技术来筛选最相关的信息。

2. RAG

RAG（retrieval-augmented generation，检索增强生成）是一种让大模型获取实时信息的重要技术，其工作原理是：当用户发出提问时，AI应用通过向量检索、关键词匹配等方式，从外部知识库或数据源检索相关信息，再把检索到的信息作为上下文提供给大模型，让大模型基于补充的信息进行回答。

RAG的交互流程如图1-5所示。

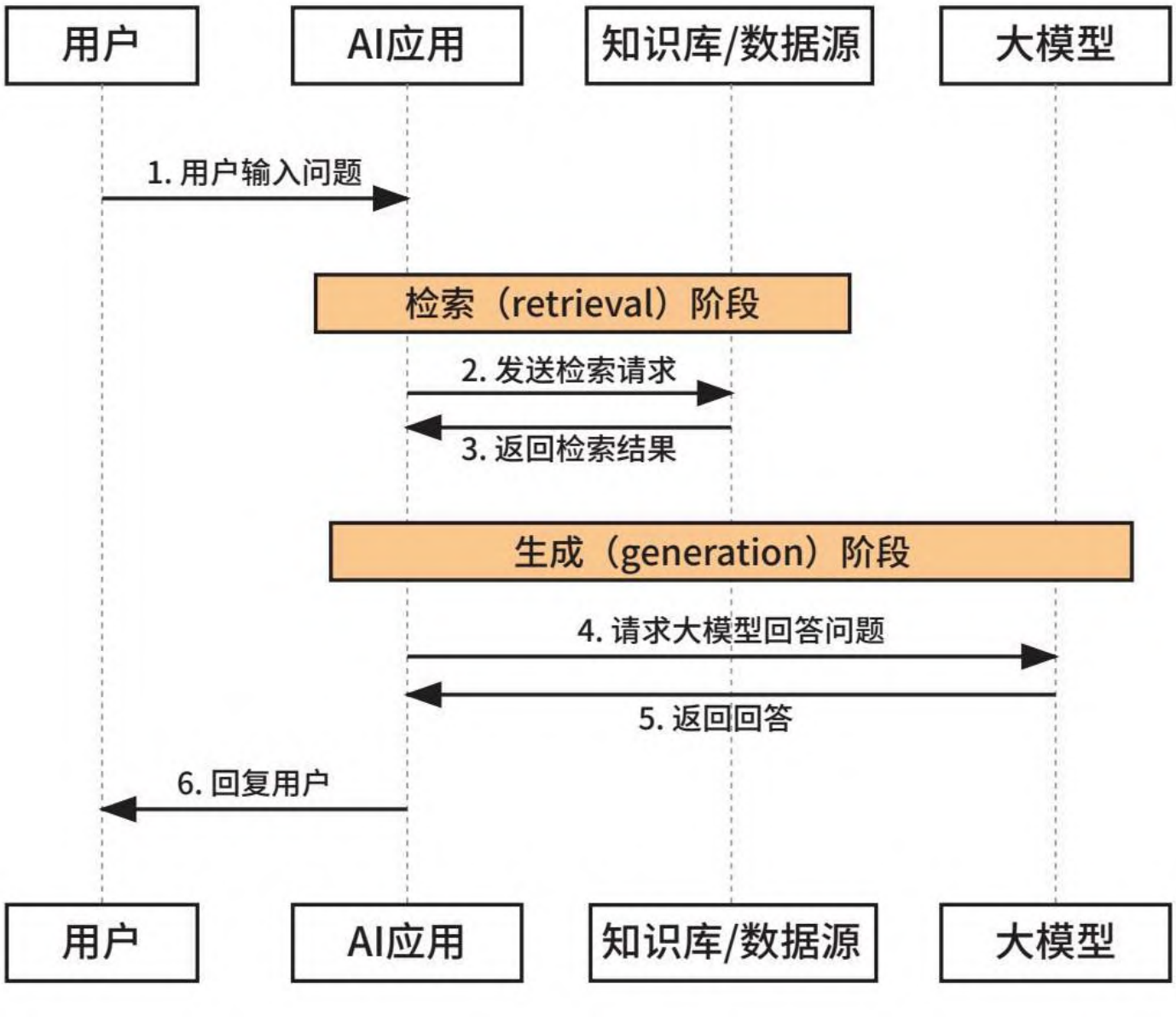


图1-5　RAG的交互流程

RAG技术的主要优势是：不需要重新训练模型（成本低）；可以灵活更新知识库（保持信息的新鲜度）。因此，RAG可以用于联网查询实时信息、跟本地文档（知识库）对话等场景。

RAG技术的局限性在于：大模型的最终回答效果依赖于检索到的信息质量。因此，检索步骤非常关键，需要基于高效、准确的检索算法和优质的数据源，而在实际应用中，这两点往往很难同时满足。

3.　函数调用

函数调用（function　calling）是一种让大模型执行特定任务的机制，允许大模型将自然语言请求转换为具体的函数调用，供AI应用调用外部工具，并将结果反馈给用户。

函数调用的工作原理是：当用户发出提问时，AI应用会将集成的函数列表作为上下文发送给大模型。大模型根据用户输入判断具体调用的函数，并生成相应的调

用参数。随后，AI应用执行该函数并将结果发送给大模型，作为补充信息供其生成最终的总结或回答。

函数调用的交互流程如图1-6所示。

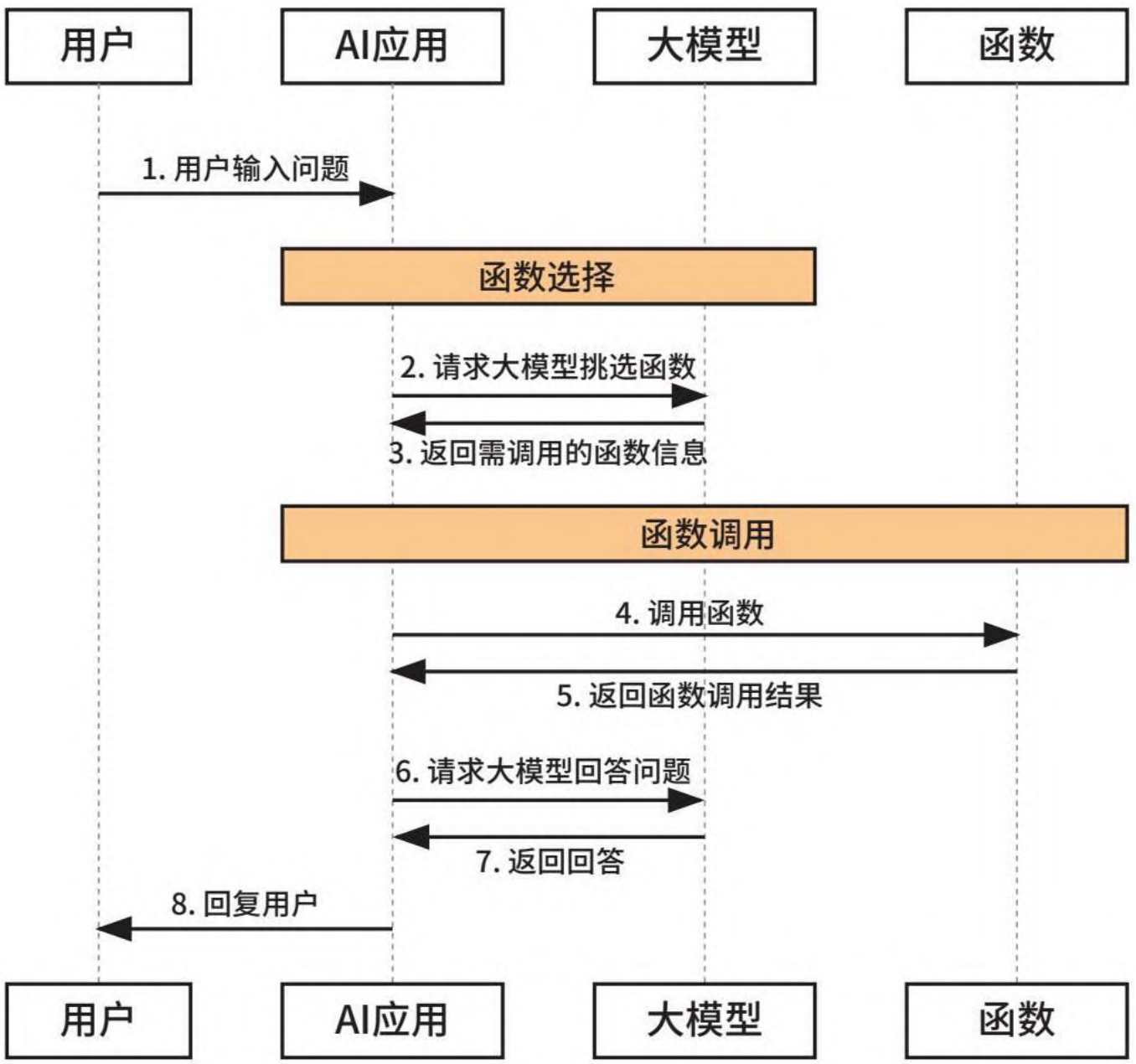


图1-6　函数调用的交互流程

函数调用扩大了大模型的能力边界，通过函数调用的方式，大模型既能外挂各种类型的数据，也能执行各种类型的操作。



函数调用vs　RAG

函数调用跟RAG在外挂数据方面的主要区别在于：RAG是由AI应用根据用户输入直接前往固定的信息源查询相关内容，然后将其作为补充上下文提供给大模型来回答问题；而函数调用是AI应用通过工具函数提前集成多个数据源，由大模型进行调

度，AI应用再动态读取这些数据源，最后将其作为补充上下文提供给大模型来回答问题。

自OpenAI于2023年6月首次在其GPT系列模型中支持函数调用机制以来，各大模型厂商纷纷跟进，函数调用已经成为大模型外挂数据的标配。

然而，17个月之后，MCP出现了。

4. MCP

我们可以认为，MCP是在函数调用的基础上做了进一步的升级和抽象，目的是让AI应用更加简单、高效、安全地对接外部资源，更好地为大模型补充上下文信息。因此，接下来我们用对比讲解函数调用和MCP的方式来了解MCP。

函数调用是一种交互范式，其本质是一种设计模式，定义了AI应用与外部函数的调用规范。

从用户视角看，函数调用的交互流程涉及三个角色：AI应用、函数、大模型；核心交互流程涉及两大核心步骤：函数选择和函数调用。

❑

函数选择：AI应用把函数列表和用户提问发送给大模型，大模型识别用户意图，挑选最适合的函数来满足用户需求。

❑

函数调用：应用通过大模型返回的函数名称和参数，调用函数获取外部数据。

图1-6已经介绍了函数调用的交互流程，此处不再赘述。

MCP是一套通信协议，其本质是为AI应用与外部工具交互而制定的标准化接口规范、数据格式和通信契约。

MCP定义了三个角色：MCP主机、MCP客户端、MCP服务器。跟函数调用相比，MCP相当于是把“MCP客户端-MCP服务器”作为一个黑盒使用。

从用户视角看，MCP的交互流程也涉及三个角色：AI应用、黑盒（MCP客户端MCP服务器）、大模型。MCP的交互流程（以调用工具为例）如下所示（注意，只提及了核心步骤）。

❑

AI应用把用户提问发送给黑盒中的MCP客户端。

❑

黑盒中的客户端请求黑盒中的MCP服务器，获取MCP服务器定义的工具列表。

❑

AI应用把工具列表和用户提问发送给大模型，由大模型进行工具选择。

AI应用根据大模型返回的工具调用参数，通过黑盒中的MCP客户端，向黑盒中的MCP服务器发起工具调用请求。

黑盒中的MCP服务器请求外部数据或服务，将结果返回给MCP客户端。

AI应用得到黑盒返回的外部数据，作为上下文信息发送给大模型生成回答。

可以用一幅图概括MCP的交互流程，如图1-7所示。

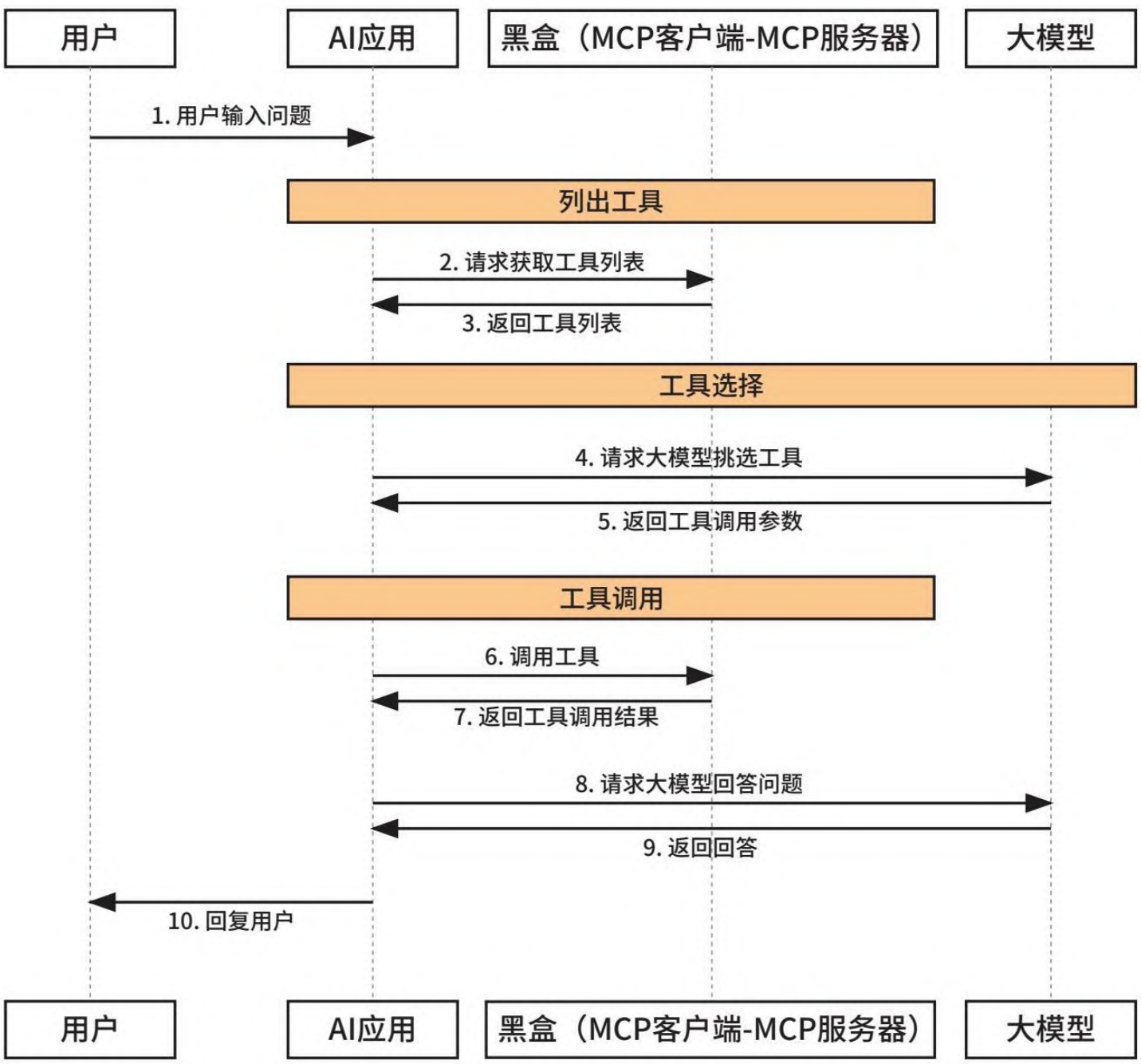


图1-7　MCP的交互流程

函数调用vs MCP

在函数调用机制中，函数是在AI应用内部直接定义的，与外部数据或服务的对接是由AI应用直接完成的。

而在MCP中，工具是在外部MCP服务器中定义的，AI应用添加了MCP服务器，就能获得MCP服务器定义的工具列表。AI应用不直接跟外部数据或服务打交道，而是通过MCP服务器来完成与外部资源的交互。我们可以认为，MCP通过“业务外包”的方式，减轻了AI应用侧的实现负担。

函数调用机制的关键在于函数选择，依赖大模型的推理能力。我们平时说某大模型支持函数调用能力，对其正确的理解应该是此模型在工具选择的任务场景上做了专门的训练优化（函数描述的嵌入表示、特殊格式训练与指令微调）。支持函数调用的模型，在接收到AI应用传递的工具列表时，能够根据用户的请求从中精准选出最合适的工具，并生成结构化的调用参数。得益于模型在函数描述理解和参数填充方面的专门训练优化，其匹配准确度通常优于未经过相关训练的模型。而不支持函数调用的模型，比如gpt-3.5-turbo，也可以通过设置系统提示词的方式实现工具选择的操作，只是匹配准确度没有支持函数调用的模型高而已。

MCP是基于函数调用机制的应用层协议，本质是为AI应用给大模型提供上下文服务的，MCP跟大模型本身没有直接的关系。

因此，可以说某大模型支持或不支持函数调用，但不应该说某大模型支持或不支持MCP。

1.2.2 AI应用视角

1. 智能体的崛起

2023年11月7日，OpenAI在其首届开发者大会上发布了GPTs，允许用户通过简单的自然语言来创建自定义的AI助手。我们不妨将GPTs看成初级形态的智能体。

GPTs的发布，是智能体发展的一个重要里程碑，推动了智能体的大众化。在此之后，各大模型厂商纷纷跟进，推出了各自的智能体平台。

各大模型厂商的智能体与GPTs的实现原理基本一致：由用户在模型厂商的对话产品（比如智谱清言、豆包等）中自定义创建，用户可以设置系统提示词、上传知识库文件、创建工具函数以集成外部API等。

在ChatGPT平台创建GPTs应用的界面如图1-8所示。

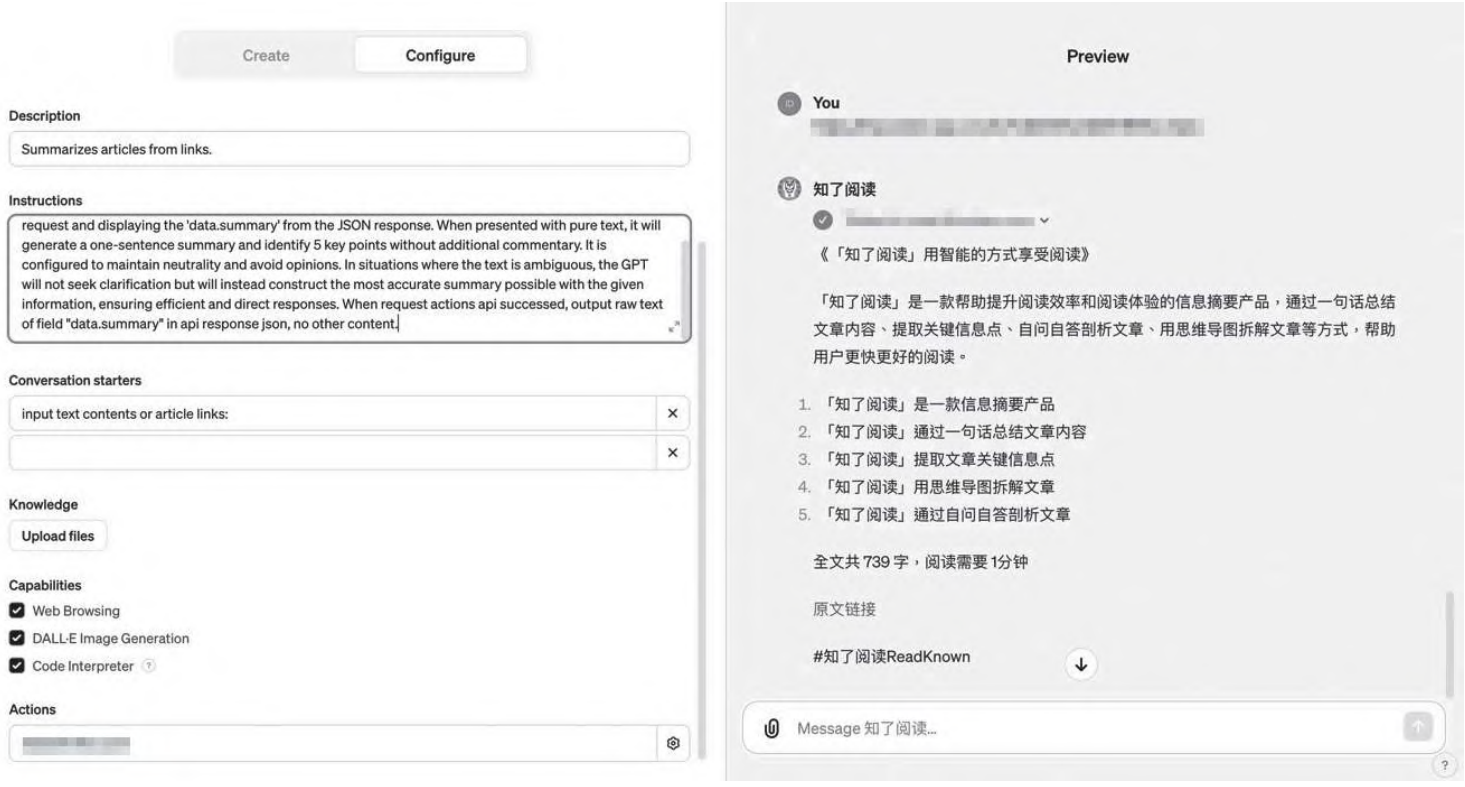


图1-8　在ChatGPT平台创建GPTs应用

不难看出，此类产品实现的核心是对函数调用机制的进一步封装，在大模型智能生成能力的基础上，增加了对业务数据的获取与处理能力。然而，随着智能体数量的激增，这一繁荣的背后暴露了一些亟待解决的问题。

2.　AI应用生态面临的问题以及MCP的解决方案

智能体实在太火了！独立开发者小豆跃跃欲试。他想打造一个“网页文章摘要助手”——用户只需在浏览器中点击按钮，即可调用大模型对文章进行摘要提炼。这类应用看似简单，但小豆在动手实践的过程中遇到了一连串的难题。

他先在ChatGPT平台上创建了一个GPTs，调教得还算满意。然而，当他打算将摘要能力集成进自己的浏览器插件或者部署到其他平台（如智谱清言）时，却发现无

法复用GPTs，只能重新搭建。而如果通过Coze、Dify等第三方平台导出API并将其接入自己的应用，又发现接口格式不统一，适配工作量巨大。

与此同时，有的用户希望摘要助手还能分析本地客户沟通文件，以识别潜在意向客户。但由于GPTs默认只能在云端处理数据，一旦涉及上传操作，用户普遍担心隐私泄露，加之每次文件更新都得重新上传，体验十分割裂。

更棘手的是，当小豆想进一步提升摘要效果、接入Perplexity的联网检索能力时，又得写代码去对接API，难以通过GPTs打通服务链路。服务提供方希望能力被广泛复用，客户端平台则面对众多API选择无所适从，而开发者夹在中间，感受到极高的集成与协作成本。

一个简单的智能体开发需求，却暴露出AI应用生态的三大系统性问题：

- 接口碎片化，导致跨平台兼容性差；
- 数据处理与隐私安全难以兼顾，限制智能能力释放；
- 服务集成与扩展效率低，生态构建成本高。

我们来看看MCP是如何解决上述三大问题的。

第一，解决接口碎片化问题：统一协议标准，降低开发成本。

MCP服务器可复用

MCP服务器只需开发一次，即可在任意支持MCP的AI应用中使用。用户想要使用某个MCP服务器时，只需要为AI应用添加一个配置即可。比起原来的GPTs或智能体，MCP服务器的复用成本更低。

统一对接方式，简化了MCP客户端的开发

在开发AI应用时，若需要接入外部能力，开发者往往需要逐一对接各类外部API。不同的API具有不同的请求地址、参数格式和鉴权方式，多样化的对接方式导致开发工作量大。

而基于MCP为AI应用集成功能时，无论接入多少个MCP服务器，请求和响应的数据格式都是统一的。AI应用只需创建一个MCP客户端，并按照MCP约定的方式传递参数，MCP服务器即可按统一格式返回结果。通过标准化的数据交换格式，MCP显著降低了双方的开发和集成成本。

第二，兼顾数据处理与隐私安全：安全使用，最小暴露。

私有数据可安全使用

MCP服务器可以通过本地进程运行，对接用户电脑中的私有数据，并将其作为上下文补充给大模型。由于数据处理过程全程在本地完成，不需要将原始文件上传至云端，有效避免了敏感信息外泄的风险。

按需提取，最小暴露

与传统外挂知识库一次性上传整份数据不同，MCP服务器采用查询式处理机制。大模型只会收到与当前问题相关的“最小必要信息”，降低了敏感数据暴露的概率。

第三，解决服务接入效率低的问题：功能即服务，快速构建生态。

MCP服务器可方便集成

MCP服务器不仅可以作为外挂插件，被用户添加到AI应用使用，也可以作为独立的服务，被AI应用集成。

跟对接功能API相比，AI应用对接MCP服务器更加简单、灵活。可以把MCP服务器当作积木，每个MCP服务器集成一项或多项原子能力，开发AI应用就跟搭积木一样，快速拼装，大大提升了开发效率。

数据或服务提供方掌握接入主动权

MCP服务器具备可复用的特点，对于数据或服务提供方来说，是一个很大的利好。

MCP通过“配置即接入”的机制，系统性缓解了服务方“主动权分散、接入不可控”的问题。数据或服务提供方只需将能力封装为MCP服务器，并公开其配置地址，即可实现“一次开发，多端复用”。

AI应用无须重复对接

MCP通过统一的接口标准和用户配置机制，帮助AI应用解决了**筛选难、整合难、对接难**的问题。

AI应用只需实现一次MCP接入，就拥有了与所有MCP服务器通信的能力，客户端的处理方式可以完全复用，避免了为每个服务单独写接入逻辑的高昂开发成本。

更重要的是，**服务的整合从“开发接入”转变为“用户配置”**。新增服务不再需要MCP客户端更新代码或发布新版本，只需用户粘贴一个配置地址，就能立即调用该MCP服务器所提供的能力。这一转变极大地降低了服务集成的边际成本，让AI应用可以更轻松地构建“插件市场”或“服务集市”。

1.2.3　技术设计视角

1.　从LSP到MCP

从MCP设计者David　Soria　Parra和Justin　Spahr-Summers的访谈播客可知，MCP的设计受到了LSP的启发。

LSP（Language　Server　Protocol，语言服务器协议）是微软于2016年提出的通信协议，旨在标准化编辑器/　IDE与语言服务器之间的交互。它将代码补全、错误检查、智能跳转等功能从编辑器中解耦，由独立的语言服务器提供。

LSP基于JSON-RPC，通过统一的请求和响应格式，实现了编辑器/　IDE与语言服务器之间的高效通信。LSP的交互流程如图1-9所示。

LSP的核心价值包括以下几点：

- **编辑器中立性**，开发者可在任意支持LSP的编辑器中享受统一的语言智能服务；
- **避免重复开发**，每种语言只需实现一次语言服务器，即可供所有编辑器调用；
- **良好的扩展性**，新增语言支持无须修改编辑器，只需增加语言服务器。

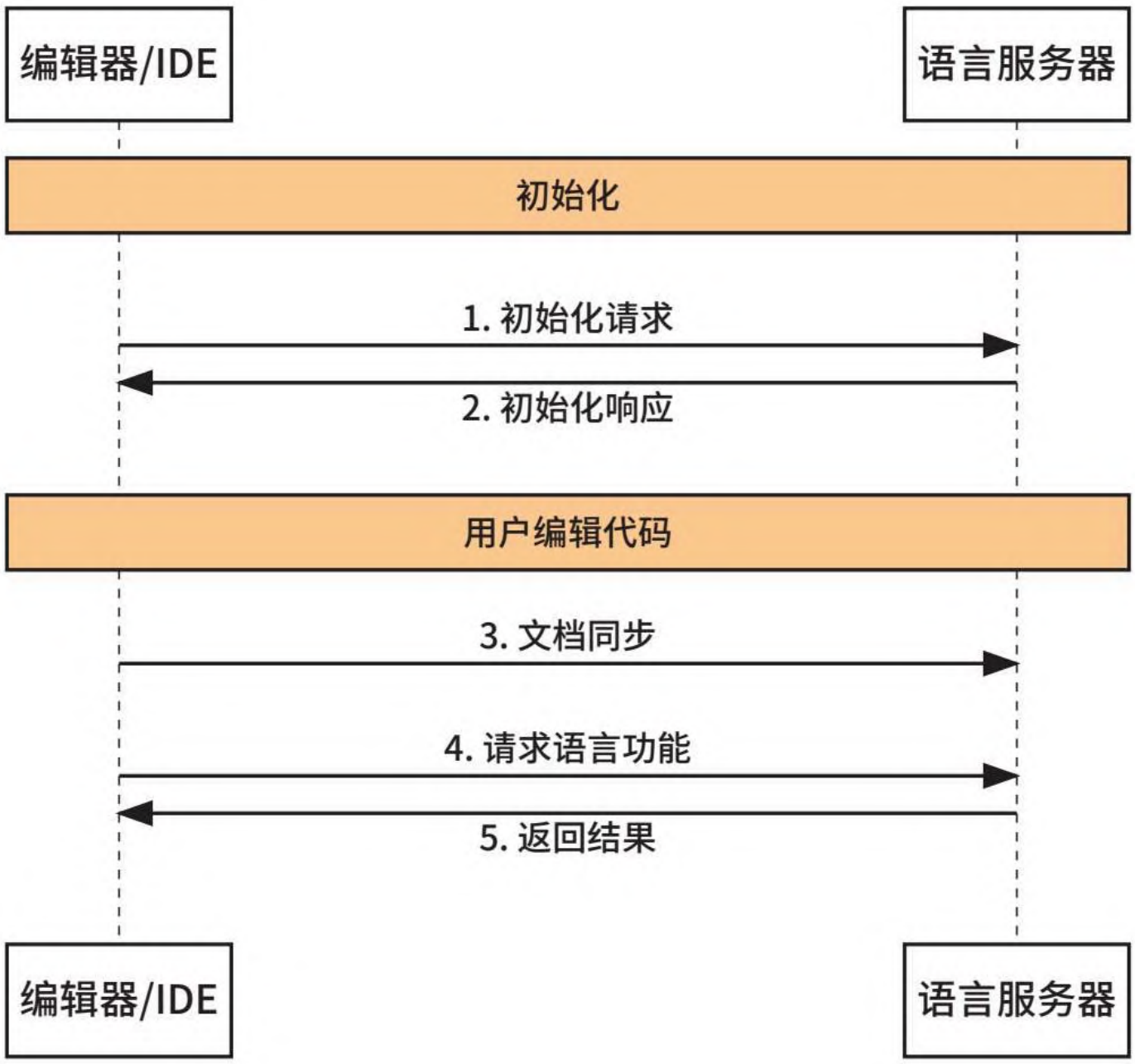


图1-9　LSP的交互流程

MCP借鉴了LSP的架构设计，在应用层定义了三类角色：主机、客户端、服务器。主机是AI应用（如Cursor），可在其内部创建MCP客户端，MCP中的主机+客户端的组合类似于LSP中的编辑器/IDE；MCP服务器则提供具体的数据或服务，类似于LSP中的语言服务器。此外，MCP还借鉴了LSP中的其他几个优秀的设计理念：

- **资源与应用解耦**，通过外部服务器连接数据或服务；
- **支持本地运行**，服务器可作为本地进程运行，通过stdio与客户端通信；
- **统一通信协议**，采用JSON-RPC实现标准化的数据交换；
- **一次实现，多端复用**，客户端和服务器只需开发一次，即可在支持MCP的环境中通用，避免了重复劳动。

LSP遵循了软件工程中“关注点分离”的原则，并实现了“一次实现，处处可用”的工程愿景，成功推动了编辑器生态的标准化和可扩展性。MCP则延续并拓展了“协议驱动”的理念，跟LSP有限的语言服务器数量相比，MCP服务器的规模要大得多，将为AI应用构建出更加繁荣和开放的服务生态。

2. 从HTTP到MCP

MCP发布之后，很多人觉得我们迎来了“AI时代的HTTP”，价值不可限量。

HTTP（HyperText Transfer Protocol，超文本传输协议）是万维网（WWW）的基础协议，定义了客户端与服务器之间如何交换数据。自Tim Berners-Lee博士及其团队于1989年提出万维网的构想，并在1990年到1991年间设计并实现了最初版本的HTTP和Web系统以来，HTTP已经从早期仅支持纯文本传输演化为支持网页、图像、音视频、3D等内容的现代互联网基础协议。它的每一次升级（如HTTP/1.1、HTTP/2、HTTP/3）都与Web技术的进步相互推动，共同构建了当今庞大的Web生态。

MCP的设计在架构上与HTTP类似，也采用了C/S架构，但引入了一个关键的新角色——主机，形成了其核心的主机-客户端-服务器架构。MCP中的主机+客户端的组合，可类比于HTTP中的浏览器，而MCP中的服务器则可类比为HTTP中的Web服务器。

为了更清晰地理解MCP的定位，表1-1对MCP与HTTP进行了对比。

表1-1 MCP vs HTTP

维度	HTTP	MCP
协议层级	应用层，基于 TCP/IP	应用层，可运行于 HTTP 之上
交互模式	用户通过浏览器输入网址，浏览器与 Web 服务器交互	用户通过 AI 应用输入问题，AI 应用创建 MCP 客户端与 MCP 服务器交互
架构模型	客户端-服务器	主机-客户端-服务器
生态目标	构建 Web 应用生态	构建 AI 应用生态

正如HTTP在30多年前开启了Web时代，让互联网日后真正走进了千家万户，改变了人类获取信息和交流的方式，我们可以预见，MCP的出现标志着一个全新的AI原生应用生态正在形成，由此引发的信息变革将比Web时代带来的变革更加深远。

1.3 MCP是怎么火起来的

1.3.1 横空出世

MCP于2024年11月25日发布，可以说是“横空出世”。MCP发布的当天，国内外媒体争相报道，用“震惊、炸裂”来形容MCP的诞生，声称AI行业马上要“变天”了。

我也在第一时间阅读了MCP的文档，我对MCP的价值和未来潜力非常看好。当时，我给出了以下三个判断。

❑

MCP是对函数调用机制的升级，为大模型集成外部资源的方式定义了标准，能够有效扩充大模型的能力边界。

❑

不管是函数调用还是MCP，最大的难点都在于大模型对用户意图的识别。如何从客户端传递的工具列表中选择最适合调用的工具，并且返回准确的调用参数，是整套机制的关键。

❑

MCP能不能成为AI行业的普适性标准，取决于生态的建立和共识的形成。这里面存在一个经典的“鸡蛋悖论”，也就是先有鸡还是先有蛋的问题。如果有足够多的数据或服务提供方基于MCP开放了能力，其他AI应用会选择跟进；如果主流的AI应用支持通过MCP添加扩展，第三方数据或服务提供方也会愿意开放其能力。

1.3.2 靠明星效应破局

MCP发布之初，官方实现了十几个经典的MCP服务器，包括网页内容抓取、时间处理、数据查询等类别，并在Claude中实现了对MCP服务器的接入。

MCP在发布后的两个月内，处于相对沉寂的状态，偶尔有一些新的MCP服务器发布和AI应用宣布支持MCP，但整体讨论热度不高。

从2025年1月底开始，MCP才出现热度上升的势头，这跟几个重要的AI应用的功能迭代有关，比如Cursor、Windsurf。

Cursor支持MCP

2025年1月23日，知名AI编辑器Cursor发布了0.45.*x*版本，开始支持MCP，用户可以添加MCP服务器，读取本地数据或文件，进一步增强Cursor的编程辅助能力。

Windsurf支持MCP

2025年2月13日，另一款知名的AI编辑器Windsurf发布了Wave 3版本，也宣布支持MCP。用户可以在Windsurf中添加各类MCP服务器，实现各类能力的接入。



除了知名的客户端产品支持MCP之外，一些知名的数据或服务提供方，包括Perplexity、Figma等，也把自己的部分能力以MCP服务器的方式开放。这进一步提升了MCP的讨论热度。

2025年3月，伴随着AI领域几个重量级产品的发布和几个重大事件的发生，MCP才迎来了真正的爆发。

以下为典型例子。

Manus发布

Monica AI团队于2025年3月6日发布了Manus——一款通用智能体产品。Manus可以模拟用户操作浏览器的行为，通过AI执行各类自动化任务。

Manus一经发布迅速火遍全网，成为继DeepSeek之后的又一个现象级AI产品。

Manus火了之后，开始有人分析Manus的实现原理，推测其实现过程中用到了大量的MCP服务器，比如Manus的联网检索、浏览器操作、文件创建、命令行调用等工具，毕竟，之前都有MCP服务器实现了类似的功能。

对于一个超级智能体，通过MCP服务器实现各类功能，为大模型补充上下文知识，从而更好地满足用户需求，是一件非常合理的事情，也是MCP发布之初的愿景之一。

据我所知，Manus最初的功能实现并没有用到MCP服务器，但Manus的出圈让MCP迅速火了起来，推动了MCP生态的发展。

a16z发布市场分析报告

2025年3月20日，全球知名的风险投资机构安德森-霍洛维茨（Andreessen Horowitz，简称a16z）基金发布了题为“A Deep Dive into MCP and the Future of AI Tooling”的市场分析报告，并同步公布了MCP市场地图，如图1-10所示。

该报告从资本市场视角对MCP进行了积极的评估与战略展望，进一步提升了全球投资者与科技界对MCP的关注度与认可度。

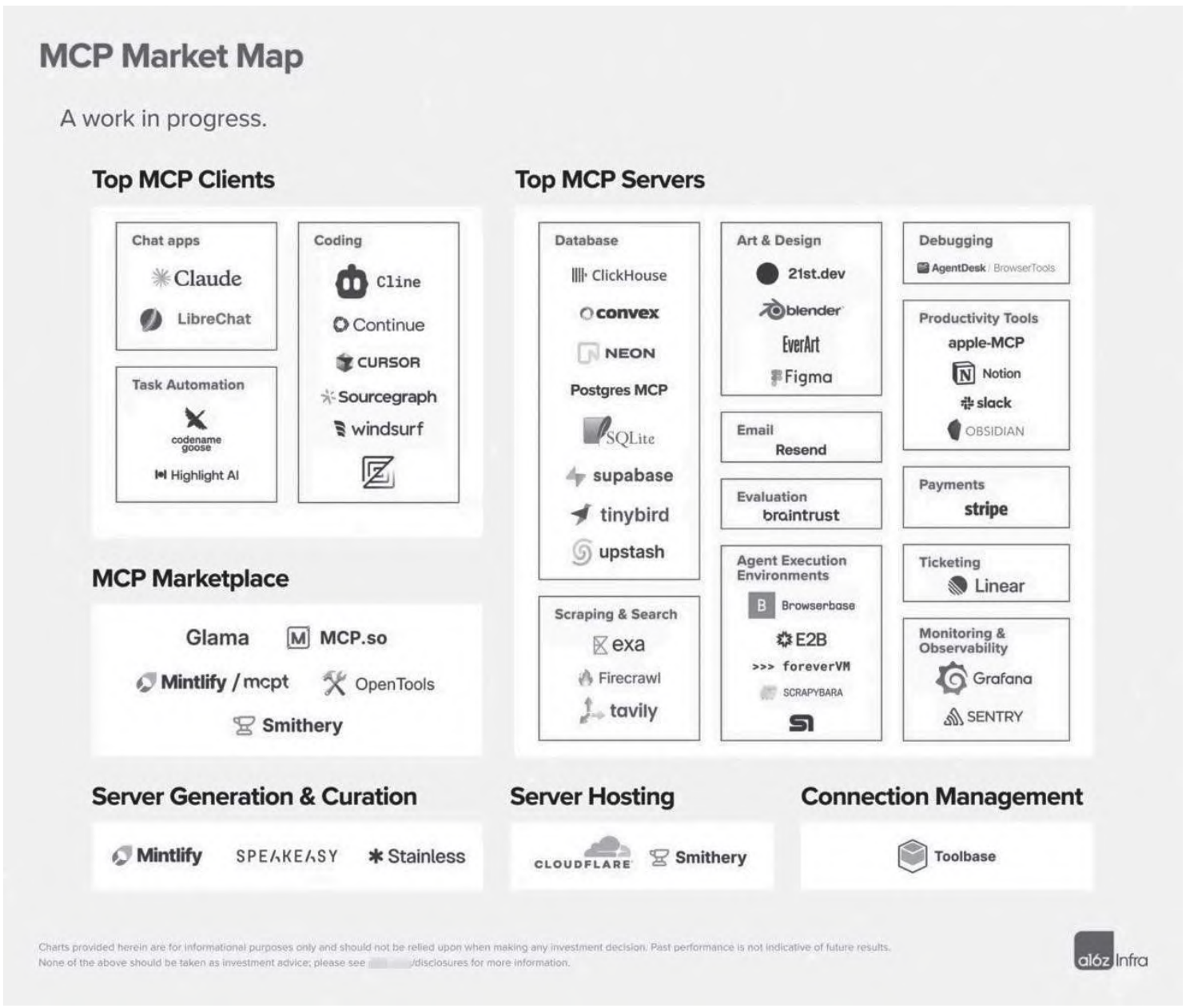


图1-10 MCP市场地图

OpenAI宣布支持MCP

2025年3月27日，OpenAI首席执行官Sam Altman在X发帖，称因为大家都很喜爱MCP，所以OpenAI在其自身的产品中添加了对MCP的支持——Agents SDK率先接入MCP，后续团队也会在ChatGPT桌面版和API中逐步支持MCP。

这一消息再次拉升了MCP的热度。之前行业内不少人认为，OpenAI作为Anthropic最主要的竞争对手，有可能会选择发布一套新的协议，而不是接入竞争对手发布的MCP。

Sam Altman的表态给很多此前对MCP持观望态度的人“吃了一颗定心丸”。随后，各类数据或服务提供方开始积极接入MCP，各大模型厂商、AI应用也纷纷跟进。

1.3.3 靠双边网络效应爆发

除了Manus的发布、a16z的市场分析报告和OpenAI的表态，2025年3月和4月，在国内外AI圈发生的其他几个重要的事件也加速了MCP生态的发展。

❑

3月21日、23日和31日，百度地图、高德地图、腾讯地图相继发布MCP服务器。

❑

3月24日，Zapier推出MCP全流程方案，开发者可以通过MCP服务器接入Zapier支持的上千个API服务。

❑

3月25日，Cloudflare推出MCP云托管方案，开发者可基于Cloudflare部署远程MCP服务器。

❑

4月9日，阿里云百炼平台上线MCP应用市场。

❑

4月10日，谷歌宣布将在Gemini和SDK中添加对MCP的支持。

❑

4月22日，Docker推出MCP目录和工具包。

❑

4月23日，360纳米AI推出MCP工具箱。

❑

4月25日，百度在2025 AI开发者大会上宣布全面支持MCP。

MCP的热度还在持续上升，各类服务平台、AI应用都在积极接入MCP，MCP生态正在不断壮大。

MCP从发布到流行的过程完美诠释了双边市场的发展规律：

初期面临典型的“鸡蛋悖论”——缺乏应用支持导致服务器开发意愿不足，而服务器稀缺又让应用缺乏接入动机。然而，头部产品团队的示范性介入触发了“明星效应”，打破了这一僵局并启动了“双边效应”，使得应用端和服务器端开始相互拉动。随着生态参与者不断增多，“网络效应”逐渐显现。每个新加入者都会为整个网络增值，最终形成正向循环，推动了MCP生态实现爆发式增长。

1.4 MCP能做什么

MCP横空出世之后，许多人看到了Web时代HTTP的影子，对MCP的价值与未来抱有很高的期待。而对于大部分普通开发者而言，可能还需要进一步理解MCP的价值具体体现在什么地方。

在本节中，我们主要探讨MCP的实际应用价值，换句话说，MCP能做什么。

1.4.1 在企业办公场景的应用

想象一下，你是一个企业的老板，你的公司内部有很多的OA（office automation，办公自动化）系统。

员工需要在考勤系统请假，采购人员需要在ERP（enterprise resource planning，企业资源计划）系统报备，销售人员需要在CRM（customer relationship management，客户关系管理）系统录入潜在的客户信息，人力资源需要在招聘系统查看候选人的简历，诸如此类。

如果一个员工负责的事项较多，难免要在各个系统之间来回切换，跟各种类型的企业系统或内部资料打交道。而作为老板，你也需要每天登录各个内部系统，查看各种信息，审批各种请求。

AI的迅速发展，让你看到了通过AI在企业内部提效的可能性。你希望有一个像ChatGPT一样的对话型AI产品，让你的员工可以通过对话快速处理各类信息。

❑

员工A：“我想在清明节前请三天事假，理由是回老家扫墓。”AI自动生成一条请假记录，录入考勤系统。

❑

采购B：“今天买了20台Mac mini M4，发放给新员工使用，报备一下。”AI自动生成一条采购记录，录入ERP系统。

❑

销售C：“今天跟X科技的王总聊了我司的Y产品，王总很感兴趣，约了下周深入沟通。”AI自动生成一条客户沟通记录，录入CRM系统。

❑

人力资源D：“帮我看五份前端工程师的简历，总结五位候选人的优势。”AI自动从简历库读取简历，并将总结输出给HR。

❑

……

你把这个系统的开发需求告诉了研发部，并对这套系统提出了如下要求：

- 能够对接企业内部各套系统，通过对话进行操作；
- 系统部署在内网，内部数据不能上传到公网；
- 研发周期要短，快速上线。

研发部由多个开发小组组成，每个小组负责一个内部系统的开发与维护，而这个对话产品由一个新的小组负责。

在MCP出现之前，这个新产品的开发，需要推动原有的内部系统开发小组将内部系统的核心功能以API的形式暴露，再在新产品里一个个对接内部系统的API。这个过程伴随非常高的沟通和协作成本。

而借助MCP，这个系统的开发将变得简单、高效：

- 原有的内部系统（比如ERP、CRM）无须开放API，只需写一个MCP服务器，开放关键的资源或功能函数即可；
- 新开发的对话产品，无须一个个对接内部系统，只需按照MCP的要求，实现MCP客户端必需的功能（连接MCP服务器、发起请求获取资源/提示词/工具、调用工具等操作）即可；
- 最终部署上线时，只需把对话产品和各个内部系统对应的MCP服务器部署在一起，通过本地网络通信即可。

通过这个例子，我们可以看到MCP在企业内部应用的价值：

- 通过协议层面的约定，让系统之间的开放与接入变得标准化，降低沟通与协作成本；
- 通过本地进程通信，保证内部系统交互的通信安全，满足对信息安全敏感的企业的要求。

1.4.2 在个人AI助手场景的应用

经常使用电脑办公的人，应该对启动器类应用不陌生，无论是在macOS、Windows还是Linux上，启动器类应用都能帮我们快速查找文件、打开软件等，从而

提高效率。macOS操作系统中自带的启动器应用叫作Spotlight，我日常使用的是一款名为uTools的第三方启动器应用，如图1-11所示。

每次通过Command+Space快捷键唤起uTools，输入关键词，即可进入启动器内置应用的操作界面，执行翻译、格式化、查找文件、联网检索等操作。

uTools内置了上百款应用，覆盖日常办公场景。而这些应用，是由uTools官方内置或者第三方开发者提供的。每个应用对应着一个用代码实现的插件，安装uTools启动器后即可使用。

参考启动器应用的设计思路，我们是不是可以基于MCP来开发个人AI助手？想象一下，通过快捷键唤起一个输入框，你可以提出以下问题：



图1-11 macOS中的第三方启动器应用uTools

❑

帮我查看一下微信中的AI产品交流群，看看大家在聊啥；

❑

查一下系统日历，看看我这周有哪些会议安排；

❑

打开浏览器帮我看看微博热搜，其中热搜排名前三的新闻是什么；

❑

帮我整理一下~/Documents目录下的文件，按月归类。

如果有一个AI助手常驻在你的电脑中，根据你的指令帮你完成此类需求，你会不会很期待？

你可能会问：这跟MCP有什么关系？ ChatGPT、豆包不都已经做了吗？

确实如此，ChatGPT、豆包等产品的团队也都在推广其桌面版，且提供了类似启动器应用的功能。从产品战略来看，大家都想抢占用户电脑端的上网入口。

但此类产品目前存在以下几个问题。

数据存储在云端

ChatGPT、豆包等桌面版产品的数据均保存在云端。也就是说，你在客户端的对话内容会被发送至其云服务器。例如，当你请求豆包分析某个聊天文件时，需要先将文件上传至云端，才能进行处理。在这一过程中，你可能会担心隐私泄露问题。

无法直接访问本地资源

这些桌面版产品通常无法直接读取你的本地资源。若需处理本地资源，必须先手动上传，才能进一步操作，使用体验相对烦琐。

功能扩展受限

ChatGPT、豆包等桌面版应用本质上是由开发商封闭维护的服务，用户无法自由扩展其功能。例如，用户若希望将自己与豆包的对话内容自动保存到本地笔记工具，除了复制/粘贴，几乎没有更自动化的方式可用了。

而基于MCP来实现个人AI助手，会有更大的优势和更好的体验。

- 有限提供私有数据，隐私安全。MCP服务器支持在本地指定范围内读取用户的私有数据，而无须用户把整个私有数据文件上传给个人AI助手，提高了用户的信任度。
- 可插拔架构，灵活度高。无须在个人AI助手内置大量的功能，而是使用可插拔架构，让用户自行选择使用哪些MCP服务器，从而让个人AI助手保持更高的灵活度。
- 无限扩展功能。用户可以自己开发，或者安装别人开发好的MCP服务器，来操作本地的各类资源。在个人AI助手上使用MCP服务器为大模型补充上下文，让大模型更好地理解和实现用户需求。

1.4.3 帮助资源方开放能力

诸如Perplexity之类提供数据或服务的资源方，通过开放API的方式，暴露自身的部分能力，让第三方应用能够接入使用，本质是为了提升ToB（企业端）收入。

而对接资源方的API，需要应用方投入开发成本，参照资源方的API说明，进行必要的调试和接入。

假设某AI应用为了实现联网检索功能，需要自行对接搜索服务商。在Perplexity和Tavily两家都提供联网检索功能的情况下，应用对接了其中一家，便大概率不会再对接另一家。这是因为每次对接，都需要重新调试一套新的API，有额外的时间成本。

有了MCP之后，Perplexity与Tavily都基于MCP实现了一个MCP服务器，通过MCP服务器开放了其联网检索工具。

对于应用而言，接入成本更低了——只需要实现一个MCP客户端，就可以一次性对接所有此类服务。毫无疑问，应用的对接意愿会更强。

就算应用自身不对接联网检索的MCP服务器，如果允许用户自行添加外部MCP服务器，那么Perplexity与Tavily开放的MCP服务器也有机会被用户添加到AI应用中。

MCP帮助资源方实现了“一次开放，随处运行”的目的。资源方的ToB调用量得以提升，ToB收入也会增长。随着MCP客户端的增多，这个价值会被进一步放大。

1.4.4 助力开发者高效创建智能体

在1.3节中，我们谈到了MCP的爆火，在一定程度上是由Manus带起来的。

根据网上有人对Manus的技术拆解，我们了解到，Manus在实现过程中集成了29个供大模型调用的工具：

- 12个浏览器操作工具（支持AI应用与浏览器、网站交互）；
- 5个shell操作工具（允许用户通过命令行界面与系统交互）；
- 5个文件操作工具（支持打开、读取、写入、创建文件等）；
- 3个部署工具（把生成的代码部署到容器，得到一个可公开访问的网址）；
- 2个消息交互工具（发送、暂停、删除、流式回复消息等）；
- 1个信息查询工具（联网检索）；
- 1个IDLE（Integrated Development and Learning Environment，集成开发和学习环境）工具。

Manus让很多人看到了智能体的价值，许多开发者都想复刻Manus，实现自己的智能体。

如果开发者自行实现上述的29个工具，毫无疑问要投入大量时间来开发与调试。而基于MCP及其生态，开发者可以从全世界开发者发布的MCP服务器中选择合适的工具（无须重复开发），以搭积木的形式进行拼装与整合，来实现自己的智能体产品。每一个MCP服务器中的每一个工具，都可以作为一项原子能力，帮助开发者更快、更好地创建智能体。

MCP通过连接全球开发者，大幅降低了创建智能体的技术门槛，展现出强大的生态价值。

1.5 小结

本章系统性介绍了MCP的工作原理与诞生背景：一开始通过高德地图服务器的例子演示了MCP的工作原理；然后从大模型、AI应用与技术设计的视角阐述了MCP解决的问题与其独特的价值；接下来，理顺了MCP爆发的过程，最后聊了聊MCP能做什么。通过对本章内容的学习，相信读者已经了解了MCP是如何发展起来的，有哪些具体的应用场景。接下来，我们来深入解析协议架构，让大家彻底搞懂MCP的设计。

第2章

MCP架构解析

在了解完MCP的工作原理、诞生背景和发展历程之后，我们对MCP有了初步了解，知道了MCP可以用于对接外部数据或服务，从而为大模型补充上下文。

本章将聚焦于MCP的架构解析，深入拆解MCP的核心架构、通信基础、连接生命周期、传输机制、能力、认证授权方式等，以探究MCP的设计理念。

2.1 核心架构：主机-客户端-服务器

2.1.1 架构简介

MCP在传统的客户端-服务器（C/S，client/server）架构的基础上，引入了“主机”（host）这一角色，构建出主机-客户端-服务器的基本架构，如图2-1所示。

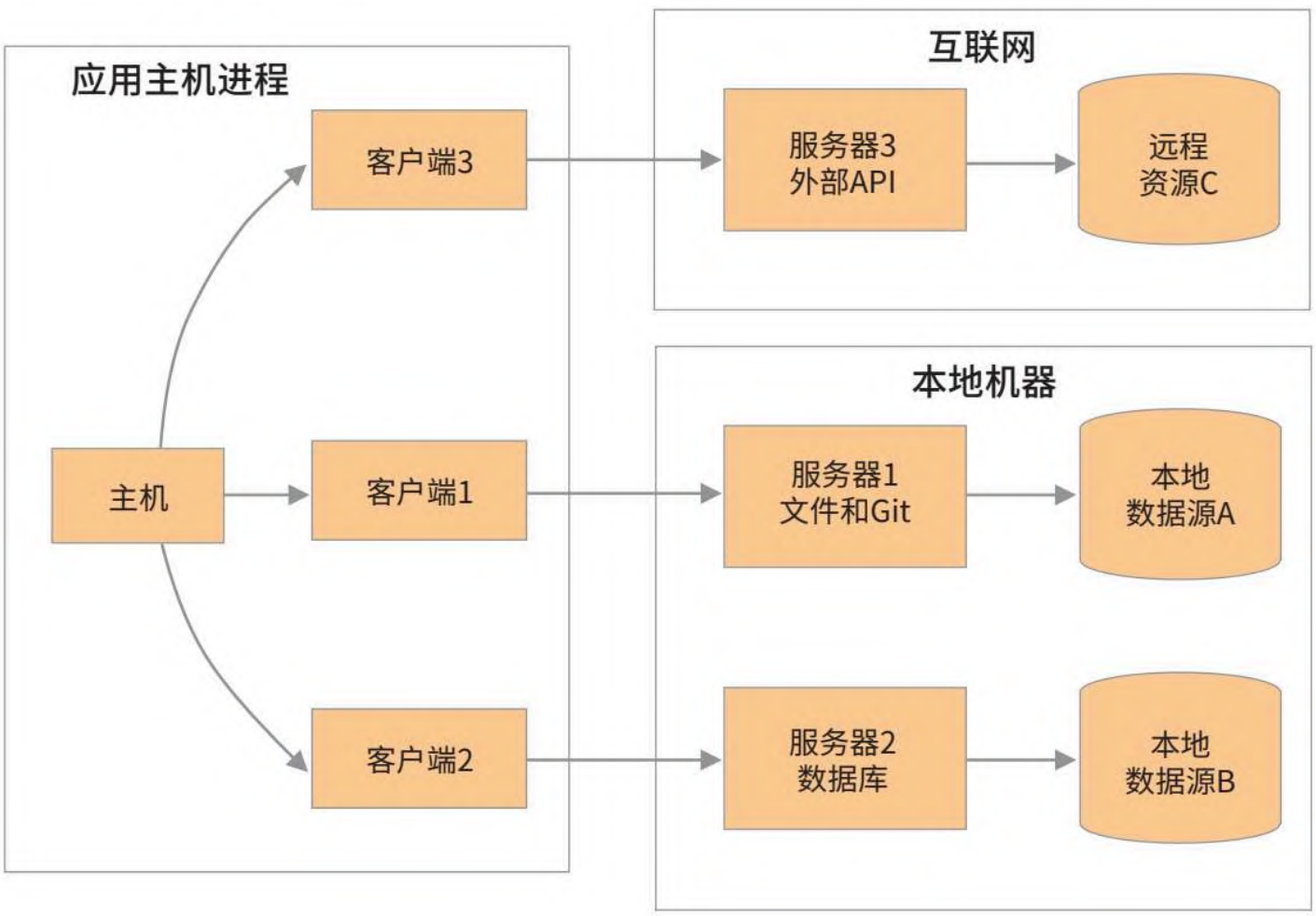


图2-1 MCP的主机-客户端-服务器架构

在1.1节中，我们已经介绍了MCP中主机、客户端、服务器的基本定义，并通过一个例子演示了它们是如何协同工作的。接下来，我们从功能职责的角度，详细介绍MCP架构的各个组成部分。

1. 主机

在MCP架构中，主机扮演着协调者的角色。它不仅负责创建和管理多个客户端进程，还掌控着这些客户端的连接权限与生命周期。同时，主机需要安全地维护客户端与服务器之间的连接，并在必要时请求用户授权。除此之外，主机还承担着大模

型的集成与采样任务，并负责跨客户端的上下文聚合管理，从而确保整个系统的高效协作与安全运行。

2. 客户端

在MCP架构中，我们可以将客户端看作主机内部的一个业务进程，它能够与服务器进程建立连接，实现数据的交互，从而帮助主机获取所需的外部资源。

客户端进程的核心职责包括：为每个服务器建立并维护有状态的会话，负责协议版本与能力协商，确保双方能够顺利沟通；同时还承担着协议消息的双向路由，保证信息能够及时传递；此外，客户端还需要管理订阅和通知。



什么是进程？

进程是具有一定独立功能的程序在特定数据集上的一次动态执行过程，是操作系统进行资源分配和调度的基本单位，也是应用程序运行的基本载体。

一个主机可以创建和管理多个客户端进程，每个客户端进程与一个特定的服务器进程维持1:1的独立连接。

3. 服务器

在MCP架构中，服务器是运行在主机之外的程序，专门用于向系统提供特定的资源和能力。它们通过MCP为主机和客户端提供提示词、资源、工具，并且以独立的方式运行。服务器可以通过客户端接口发起采样请求，同时必须严格遵循安全方面的各项限制。无论是作为本地进程还是远程服务，服务器都能灵活地为系统扩展功能。

2.1.2 设计原则

MCP的设计遵循一套贯穿架构和实现的核心原则。

❑

分离原则，MCP力求让服务器的构建变得简单。主机负责处理复杂的编排任务，服务器则专注于实现自身特定的能力，通过简洁的接口降低开发复杂度，通过分离原则提升代码的可维护性。

❑

模块化设计，MCP鼓励服务器具备高度的可组合性。每个服务器都专注于独立的能力模块，多个服务器可以无缝协作，通过共享协议实现互操作，并借助模块化设计支持系统的持续扩展。

❑

最小化暴露，在安全与隔离方面，MCP要求服务器仅访问必要的上下文信息，完整的对话历史由主机统一管理。服务器之间的连接是相互隔离的，跨服务器的交互由主机统一协调。主机进程同时划定安全边界，防止信息泄露和越权访问。

❑

渐进式扩展，核心协议只提供最基础的功能，服务器和客户端可以根据实际需求协商并添加额外的能力，二者能够独立演进。协议本身采用可扩展的设计，以便未来支持更多新特性，同时始终保持对旧版本的兼容性。

2.1.3 关键组成

MCP由多个关键组件协同构成，整体采用模块化设计，明确划分各个关注点，既提升了系统的可维护性，也为客户端与服务器之间的灵活交互奠定了基础。

❑

基础协议，MCP明确规定了客户端与服务器之间通信的编码方式（JSON RPC）、连接的完整生命周期、消息传输机制，以及双方通过能力协商确定可用能力的流程。

❑

能力模块，服务器和客户端都可以实现丰富的扩展。例如，服务器能够向客户端提供各类资源、提示词和工具，供用户或大模型调用；而客户端则可以为服务器提供采样等能力，并对服务器的可操作目录进行限制。

❑

额外工具，MCP支持配置管理、进度追踪、通信取消、错误报告和日志记录等功能，进一步丰富了客户端与服务器之间的交互手段。

❑

授权框架，MCP提供了基于HTTP的授权框架，适用于HTTP传输场景；而对于stdio传输，则通过环境变量获取凭据。同时，客户端和服务器也可以根据实际需求协商自定义的身份验证和授权策略。

2.1.4 能力协商机制

MCP采用能力协商机制，在初始化阶段，客户端与服务器分别通过capabilities字段声明自身支持的能力。这些声明决定了双方在整个会话期间的交互方式与能力边界。

例如，服务器可以声明支持提示词、资源、工具等能力；客户端则可能声明支持根（节点）、采样等能力。在整个会话过程中，双方需严格遵循已声明的能力执行逻辑。协议本身也支持协商添加更多扩展能力，实现渐进式演化。

每项能力声明都必须与实际实现相对应。例如：

❑

能力必须显式声明，服务器实现某能力，必须在capabilities字段中明确声明；

❑

权限受能力限制，未声明订阅能力的服务器将无法接收资源订阅通知；

❑

工具调用也需声明，服务器若支持工具能力，必须在能力列表中予以标注。如下所示为一个客户端在初始化请求中声明采样和根能力的示例：

```
{
"jsonrpc": "2.0",
"id": 1,
"method": "initialize",
"params": {
"protocolVersion": "2025-03-26",
"capabilities": {
"roots": {
"listChanged": true
},
"sampling": {}
},
"clientInfo": {
"name": "ExampleClient",
"version": "1.0.0"
}
}
}
```

若客户端支持采样能力，需在其能力声明中显式说明。这种能力协商机制不仅使通信双方对各自的能力边界有清晰认知，也为协议的可扩展性提供了机制保障。

2.1.5 通信安全

MCP赋予了系统强大的数据访问和代码执行能力，但这也带来了诸多安全与信任方面的挑战。因此，所有实现者都必须高度重视相关风险，确保协议的使用在可控、可信的范围内进行。

1. 核心安全原则

用户知情与自主

用户必须对所有的数据访问和操作有充分的知情权和决定权，能够自主选择哪些数据可以被共享、哪些操作可以被执行。协议的实现者应为用户提供清晰直观的界面，方便其审查和授权各项活动。

数据隐私保护

在主机向服务器披露用户数据前，必须获得用户的明确同意。未经授权，主机不得随意将资源数据传输到其他地方，所有用户数据都应受到严格的访问控制和保护。

工具调用安全

由于工具调用本质上涉及任意代码执行，因此必须格外谨慎。工具的行为描述（如注释）除非来自可信服务器，否则都不应被完全信任。主机在调用任何工具前，必须获得用户的明确授权，并确保用户在授权前充分了解每个工具的具体能力。

大模型采样权限管理

所有涉及大模型采样的请求均需用户显式批准。用户应自主决定是否允许采样、是否发送提示词，以及服务器可见的采样结果范围。协议本身也应限制服务器对提示内容的可见性，以降低潜在风险。

2. 实施建议

尽管MCP无法在协议层面强制执行所有安全原则，但开发者应主动承担安全责任，并采取以下措施：

- 在应用中构建完善的同意与授权机制；

提供清晰的安全风险说明文档；

❑

实施严格的访问控制和数据保护措施；

❑

遵循行业最佳实践并按标准进行集成；

❑

在功能设计阶段充分评估用户的隐私风险。

2.2 通信基础：JSON-RPC

MCP使用JSON-RPC作为客户端与服务器的通信基础。在介绍JSON-RPC之前，我们先来回顾一下JSON和RPC。

2.2.1 什么是JSON

JSON（JavaScript object notation，JavaScript对象表示法）是一种轻量级的数据交换格式，便于人类阅读和编写，也易于机器解析和生成。它是基于JavaScript编程语言的一个子集，但是独立于编程语言。

JSON主要有以下特点：

- 使用简单的文本格式，易于传输和存储；
- 支持嵌套数据结构，可以表示复杂的数据；
- 数据类型包括对象、数组、字符串、数字、布尔值和null；
- 已成为许多Web API和配置文件的标准格式。

JSON的语法简洁明了，使用键-值对组织数据，非常适合作为不同系统间的通用数据交换格式。一个用JSON表示的数据结构示例如下：

```
{
    "name": "张三",
    "age": 30,
    "isStudent": false,
    "hobbies": ["读书", "旅游", "编程"],
    "address": {
        "city": "北京",
        "postcode": "100000"
    },
```

```
    "phoneNumbers": [
        {
            "type": "家庭",
            "number": "010-12345678"
        },
        {
            "type": "工作",
            "number": "138XXXXXXXX"
        }
    ],
    "spouse": null
}
```


2.2.2 什么是RPC

RPC（remote procedure call，远程过程调用）是一种计算机通信协议，它允许程序能够像调用本地函数一样调用另一台计算机上的子程序，而无须编写底层网络通信代码。常见的RPC框架包括gRPC、Thrift、Dubbo等。

RPC主要有以下特点：

- 是构建分布式系统通信的基础机制；
- 让远程调用在代码层面表现得与本地调用无异；
- 隐藏了底层网络通信的复杂性；
- 支持不同编程语言和平台之间的互操作性。

典型的RPC系统包含两个关键组件：客户端和服务器。客户端发起请求调用远程服务器上的程序，服务器执行请求的操作并返回结果。RPC的通信流程如图2-2所示。

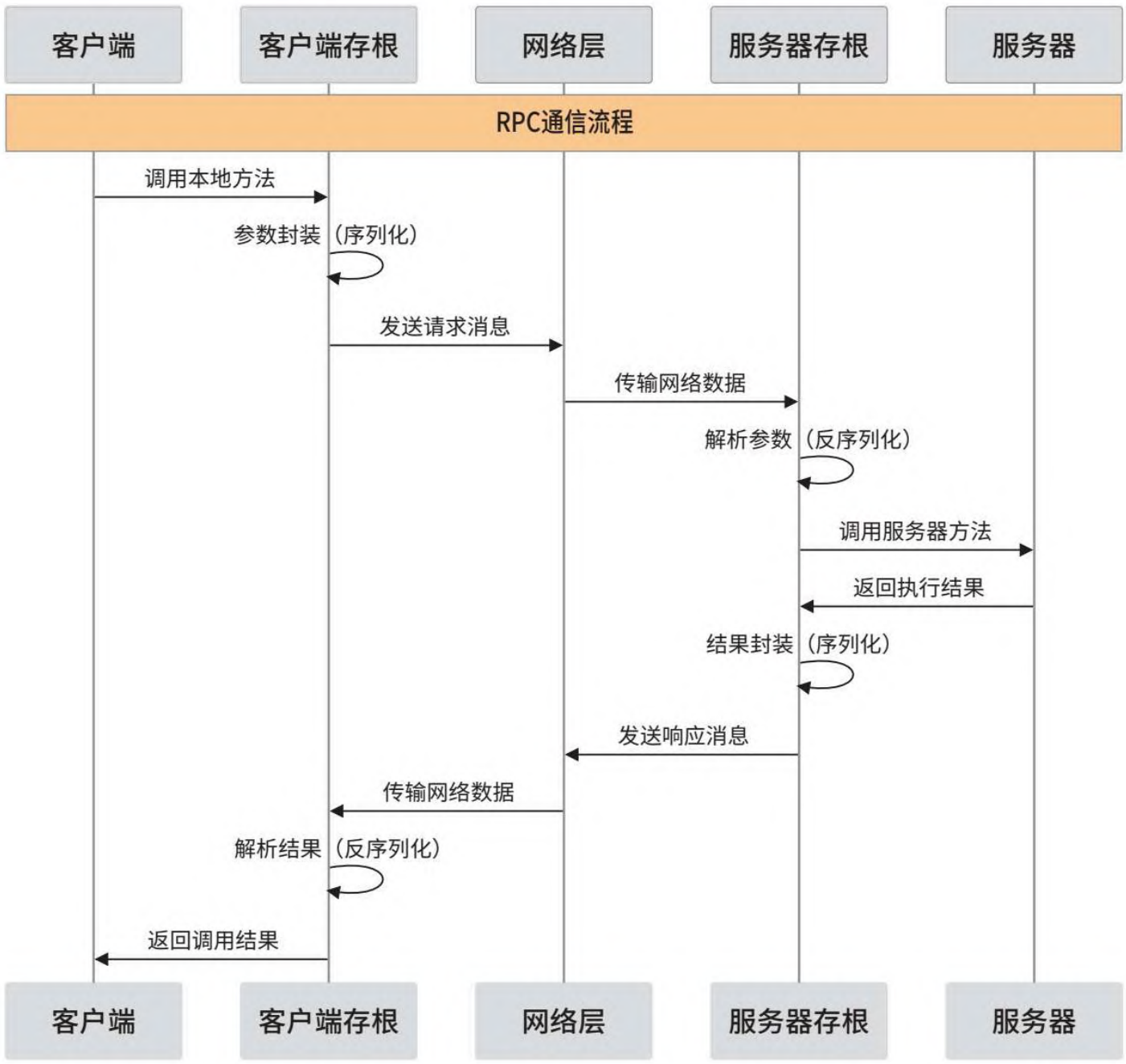


图2-2　RPC通信流程

可以看到，RPC通信主要做了两件事：

- 选择一种消息格式，对请求参数和响应参数进行序列化和反序列化；
- 选择一种传输协议，传递和接收数据。

2.2.3　什么是JSON-RPC

JSON-RPC是一种基于JSON的轻量级RPC协议。它定义了统一的数据格式和处理规则，用于在不同系统之间进行远程方法调用。

JSON-RPC的主要特点包括：

使用JSON作为消息格式；

- 支持标准的请求-响应通信模式；
- 简单易用，语言无关，易于跨平台实现；
- 可以通过HTTP/HTTPS或其他传输协议进行传输；

标准的JSON-RPC请求结构包括方法名、参数和请求ID；响应则包含结果或错误消息，以及与请求相匹配的ID。

当前，JSON-RPC的最新版本是2.0。MCP采用JSON-RPC 2.0作为客户端与服务器之间的通信基础。

2.2.4 JSON-RPC 2.0协议规范

1. 请求对象

JSON-RPC 2.0的请求对象包含以下字段：

- jsonrpc，字符串，指定JSON-RPC协议的版本，必须为2.0；
- method，字符串，表示要调用的方法名；
- params（可选），对象或数组，包含传递给方法的参数；
- id，字符串、数字或null，用于将请求与对应的响应关联起来。

请求对象主要有两类。

通知请求

请求方发起通知请求，不带id参数，不关心响应内容，只用于单方面传递消息，而无须等待和理解被请求方的响应内容。一个通知请求示例如下：

```
{
   "method": "notifications/initialized",
   "jsonrpc": "2.0"
```

```
}
```

待响应请求

请求方发起待响应请求，需要带上id参数。被请求方必须返回跟请求id匹配的响应内容。一个待响应请求的示例如下：

```
{
    "method": "initialize",
    "params": {
        "protocolVersion": "2024-11-05",
        "capabilities": {},
        "clientInfo": {
            "name": "mcp-client",
            "version": "1.0.0"
        }
    },
    "jsonrpc": "2.0",
    "id": 0
}
```

2. 响应对象

JSON-RPC 2.0的响应对象包含以下字段：

- jsonrpc，字符串，指定JSON-RPC协议的版本，必须精确地为2.0；
- result，响应成功时必须包含此成员，且包含方法调用的结果；
- error，响应失败时必须包含此成员，且包含错误消息；
- id，必须包含此成员，且必须与请求对象中的id值相同。

result和error成员是互斥的，响应中必须有且只能包含其中一个。

响应成功

被请求方处理请求成功后，会返回一个成功响应，且响应的id与请求的id保持一致。结果数据通过result字段返回。响应成功的示例如下：


```
{
    "jsonrpc": "2.0",
    "id": 0,
    "result": {
        "protocolVersion": "2024-11-05",
        "capabilities": {
            "tools": {}
        },
        "serverInfo": {
            "name": "mcp-server",
            "version": "0.0.3"
        }
    }
}
```


响应失败

被请求方处理请求失败后，会通过error字段返回错误对象，且返回的id也需要跟请求的id保持一致。响应失败的示例如下：


```
{
    "jsonrpc": "2.0",
    "error": {
        "code": -32602,
```

```
        "message": "Invalid params"
    },
    "id": 0
}
```

3. 错误对象

当被请求方处理请求遇到错误时，返回的响应对象必须包含一个error字段，其值是一个具有以下字段的对象：

- code，整数类型的错误码，用于表示错误类型；
- message，简短的错误描述，类型为字符串；
- data（可选），提供关于错误的附加信息。

JSON-RPC 2.0预定义了一批错误码，如表2-1所示。

表2-1 JSON-RPC 2.0预定义的错误码

错 误 码	消 息	含 义
-32700	Parse error	收到无效的 JSON
-32600	Invalid Request	发送的 JSON 不是有效的请求对象
-32601	Method not found	方法不存在或不可用
-32602	Invalid params	无效的请求参数
-32603	Internal error	JSON-RPC 内部错误
-32000 到 -32099	Server error	实现定义的服务器错误范围

4. 批处理

请求方可以同时发送多个请求对象，并将它们封装在一个数组中。被请求方应返回一个包含对应响应对象的数组，但对于通知类型的请求，不应返回任何响应。

一个批处理请求示例如下：

```
[
    { "jsonrpc": "2.0", "method": "sum", "params": [1, 2, 4], "id": "1" },
    { "jsonrpc": "2.0", "method": "notify_hello", "params": [7] },
```

```
    { "jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": "2" }

]
```

一个批处理响应示例如下：

```
[
    { "jsonrpc": "2.0", "result": 7, "id": "1" },
    { "jsonrpc": "2.0", "result": 19, "id": "2" }
]
```

2.3 连接生命周期

MCP定义了一个严格的生命周期（lifecycle），用于客户端-服务器连接，确保了通信双方能进行适当的状态管理和能力协商。

MCP连接生命周期主要分为三个阶段：

- 初始化阶段，客户端与服务器进行协议版本和能力协商；
- 操作阶段，客户端与服务器按照协议正常通信，交换消息；
- 关闭阶段，客户端与服务器各自优雅地终止连接。



MCP连接生命周期的设计，旨在最大限度地保障通信的可靠性。通信开始前，双方需先建立连接，以获取对方的身份信息、支持的协议版本以及已实现的能力。在此基础上，双方在协商确定的能力范围内进行通信，从而降低异常发生的可能性。通信结束后，连接将被及时关闭，以防资源泄露。

在MCP连接生命周期内，客户端与服务器的交互示例如图2-3所示。

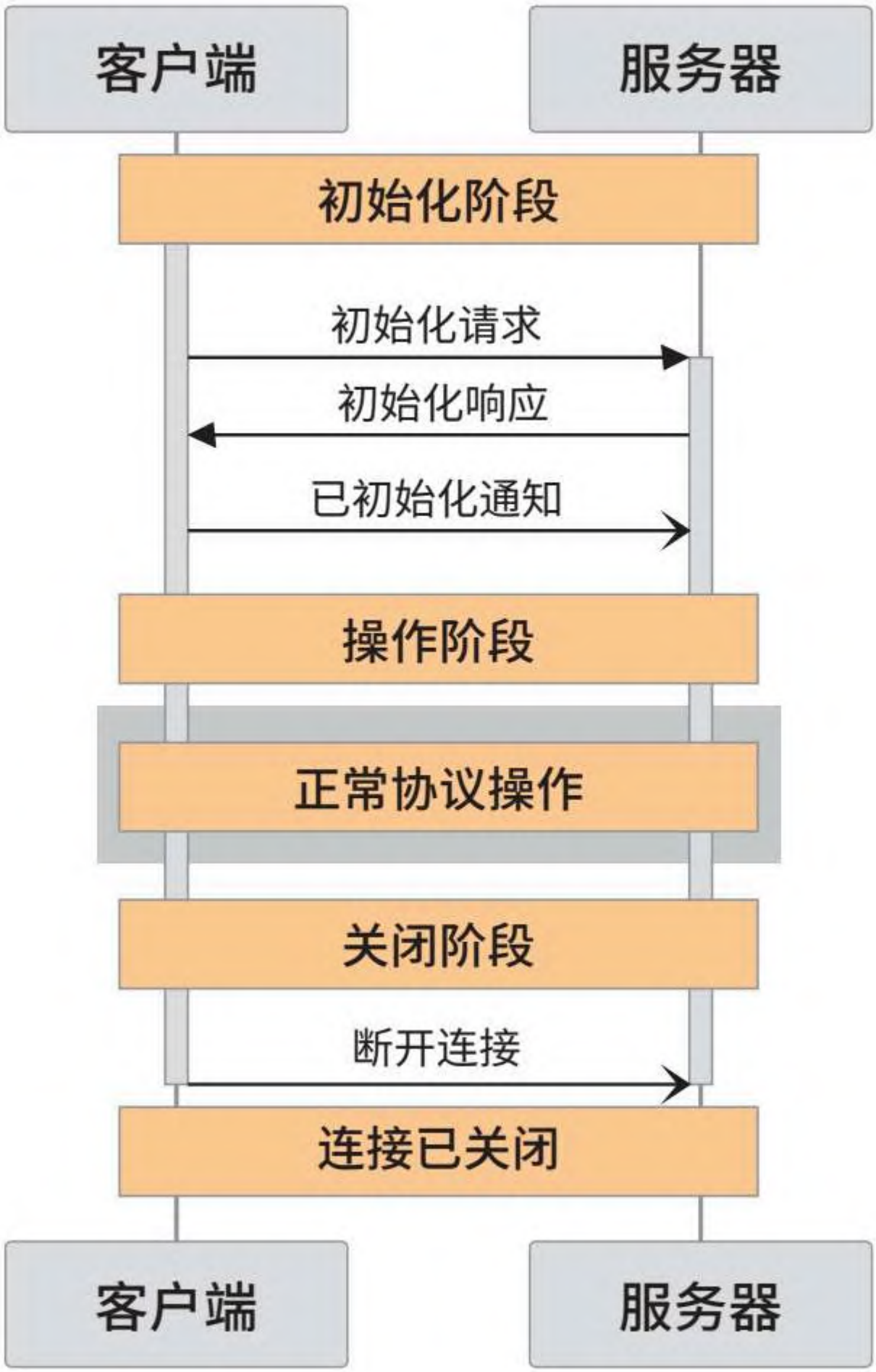


图2-3　在MCP接连生命周期内客户端与服务器的交互示例

2.3.1　初始化阶段

1. 建立连接

MCP连接生命周期的初始化阶段，是客户端与服务器的第一次交互。在此过程中，双方建立连接，协商通信使用的协议版本，各自声明支持的能力。

客户端发送初始化请求的消息示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "initialize",
    "params": {
        "protocolVersion": "2024-11-05",
        "capabilities": {
            "roots": {
                "listChanged": true
            },
            "sampling": {}
        },
        "clientInfo": {
            "name": "ExampleClient",
            "version": "1.0.0"
        }
    }
}
```

服务器在接收到客户端的初始化请求之后，必须响应其支持的能力和自身的基本信息。响应消息示例如下：

```
{
```

```
"jsonrpc": "2.0",
"id": 1,
"result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
        "logging": {},
        "prompts": {
            "listChanged": true
        },
        "resources": {
            "subscribe": true,
            "listChanged": true
        },
        "tools": {
            "listChanged": true
        }
    },
    "serverInfo": {
        "name": "ExampleServer",
        "version": "1.0.0"
    },
    "instructions": "Optional instructions for the client"
}
}
```

在收到服务器的响应之后，客户端必须发送一个已初始化通知，表明它已准备好开始正常操作。已初始化通知的消息示例如下：

```
{
    "jsonrpc":  "2.0",
    "method":  "notifications/initialized"
}
```

由以上步骤可以得知，客户端与服务器在初始化阶段建立连接的过程跟TCP三次握手过程类似，如图2-4所示。

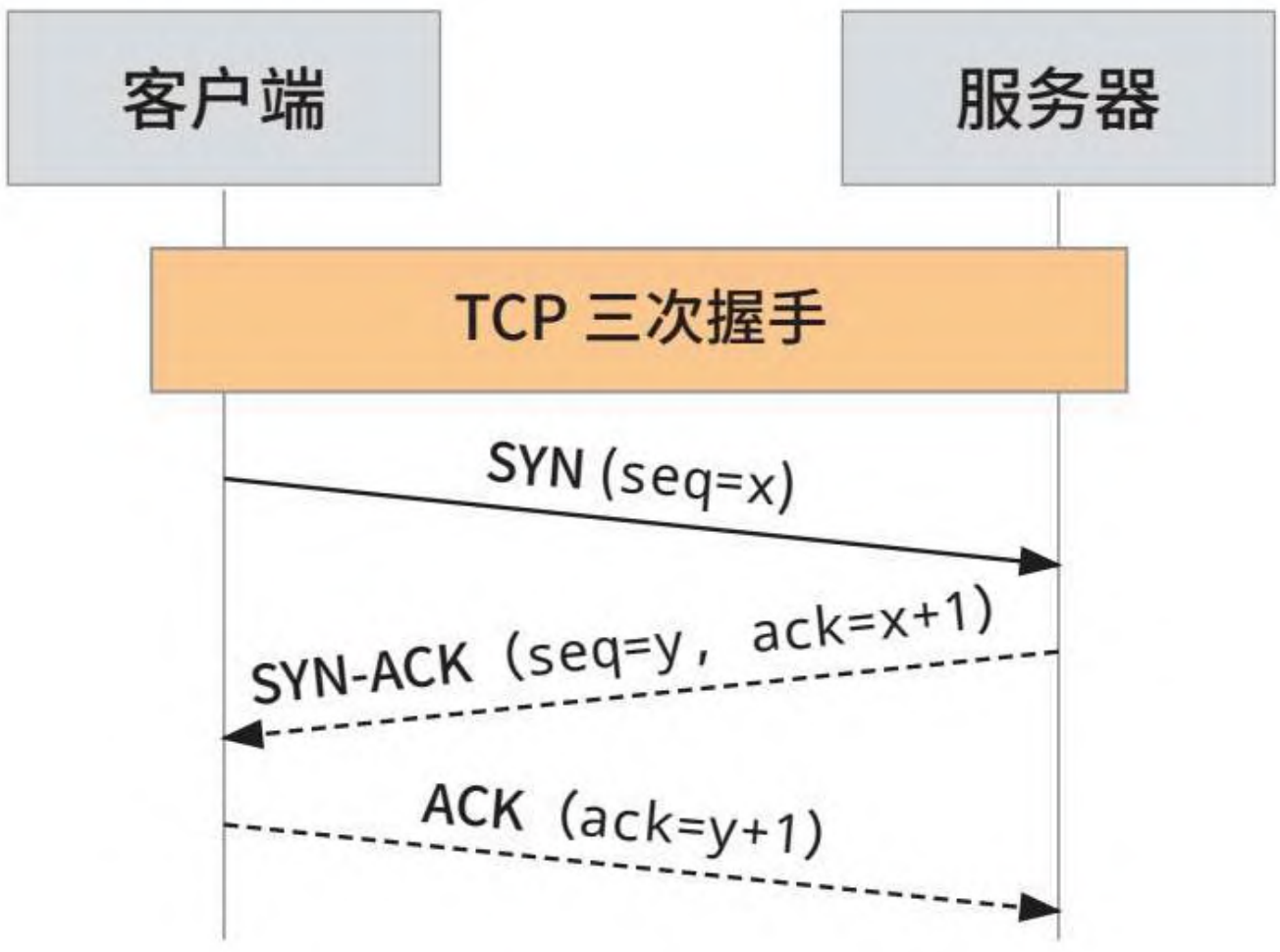


图2-4　TCP三次握手

2.　版本协商

在初始化请求过程中，客户端必须发送它支持的最新协议版本。

- ❑

如果服务器支持请求的协议版本，它必须以相同的版本响应。否则，服务器必须响应其支持的最新版本。

- ❑

如果客户端不支持服务器响应的版本，它应该断开连接。

3. 能力协商

在初始化阶段，客户端与服务器通过能力协商确定会话期间将用到哪些能力。客户端与服务器支持的能力如表2-2所示。

表2-2　客户端与服务器支持的能力

分　类	能　力	描　述
客户端	roots	提供文件系统根列表
客户端	sampling	支持大模型采样
客户端	experimental	支持非标准实验能力
服务器	prompts	提供提示词模板
服务器	resources	提供可读资源
服务器	tools	支持工具调用
服务器	logging	支持结构化日志
服务器	experimental	支持非标准实验能力

能力声明可以包含附加字段，例如：

- listChanged，支持列表变更通知（用于提示词、资源、工具、根等）；
- subscribe，支持订阅单个内容的变更通知（仅限资源）。

2.3.2　操作阶段

客户端与服务器在初始化阶段建立连接之后，进入操作阶段，开始正常通信。

在操作阶段，通信双方应该遵守以下原则：

- 尊重协商的协议版本；
- 仅使用协商的能力。

2.3.3　关闭阶段

客户端与服务器在关闭阶段断开连接。一方（通常是客户端）主动终止连接，另一方（通常是服务器）应使用底层传输机制来终止连接。

stdio

对于基于stdio传输通信的双方，应通过以下方式启动关闭流程：

1. 客户端关闭对子进程的输入流（服务器的stdin）；

2. 等待服务器退出，或者当服务器未在合理时间内退出时发送SIGTERM信号；

3. 在给服务器发送SIGTERM信号之后，如果服务器在合理时间内仍未退出，则继续给服务器发送SIGKILL信号；

4. 服务器通过关闭对客户端的输出流（stdout）来完成关闭流程。

HTTP

对于基于HTTP传输通信的双方，关闭HTTP连接即可。

2.3.4 超时机制

MCP通信双方应为所有发送的请求设置超时，以防止连接挂起和资源耗尽。当请求在超时期间未收到成功响应或收到错误响应时，发送者应发出对该请求的取消通知并停止等待响应。

SDK和其他中间件应支持为请求配置超时。

MCP通信双方还可以选择在收到与请求相关的进度通知时重置超时计时器，这表明工作实际上正在进行。但无论是否收到进度通知，系统都应该始终强制执行最大超时限制，以防通信过程中存在不遵守协议的一方对系统造成干扰。

在请求超时的情况下，请求方发送的取消请求通知示例如下：

```
{
    "jsonrpc": "2.0",
    "method": "notifications/cancelled",
    "params": {
        "requestId": 2,
```

```
        "reason": "Error: MCP error -32001: Request timed out"
    }
}
```


2.3.5 错误处理

MCP通信双方应准备处理以下错误情况：

- 协议版本不匹配；
- 无法协商所需能力；
- 请求超时。

被请求方响应的错误消息示例如下：


```
{
    "jsonrpc": "2.0",
    "id": 1,
    "error": {
        "code": -32602,
        "message": "Unsupported protocol version",
        "data": {
            "supported": ["2024-11-05"],
            "requested": "1.0.0"
        }
    }
}
```

2.4 传输机制：stdio/SSE/流式HTTP

MCP传输机制是客户端与服务器通信的桥梁，定义了客户端与服务器通信的细节，帮助客户端和服务器交换消息。

目前，MCP定义了三种传输机制用于客户端-服务器通信：

- stdio，通过标准输入和标准输出进行通信；
- SSE，通过服务器发送事件进行通信（协议版本自2024-11-05开始支持，即将废弃）；
- 流式HTTP（Streamable HTTP），通过HTTP进行通信，支持流式传输（协议版本自2025-03-26开始支持，用于替代SSE）。

MCP要求客户端应尽可能支持stdio。

MCP的传输机制是可插拔的，也就是说，客户端和服务器不局限于MCP定义的这几种传输机制，也可以通过自定义的传输机制来实现通信。

2.4.1 stdio传输

stdio即standard input & output（标准输入/输出），是MCP推荐使用的一种传输机制，主要用于本地进程通信。

1. stdio传输通信流程

基于stdio传输的通信流程如图2-5所示。

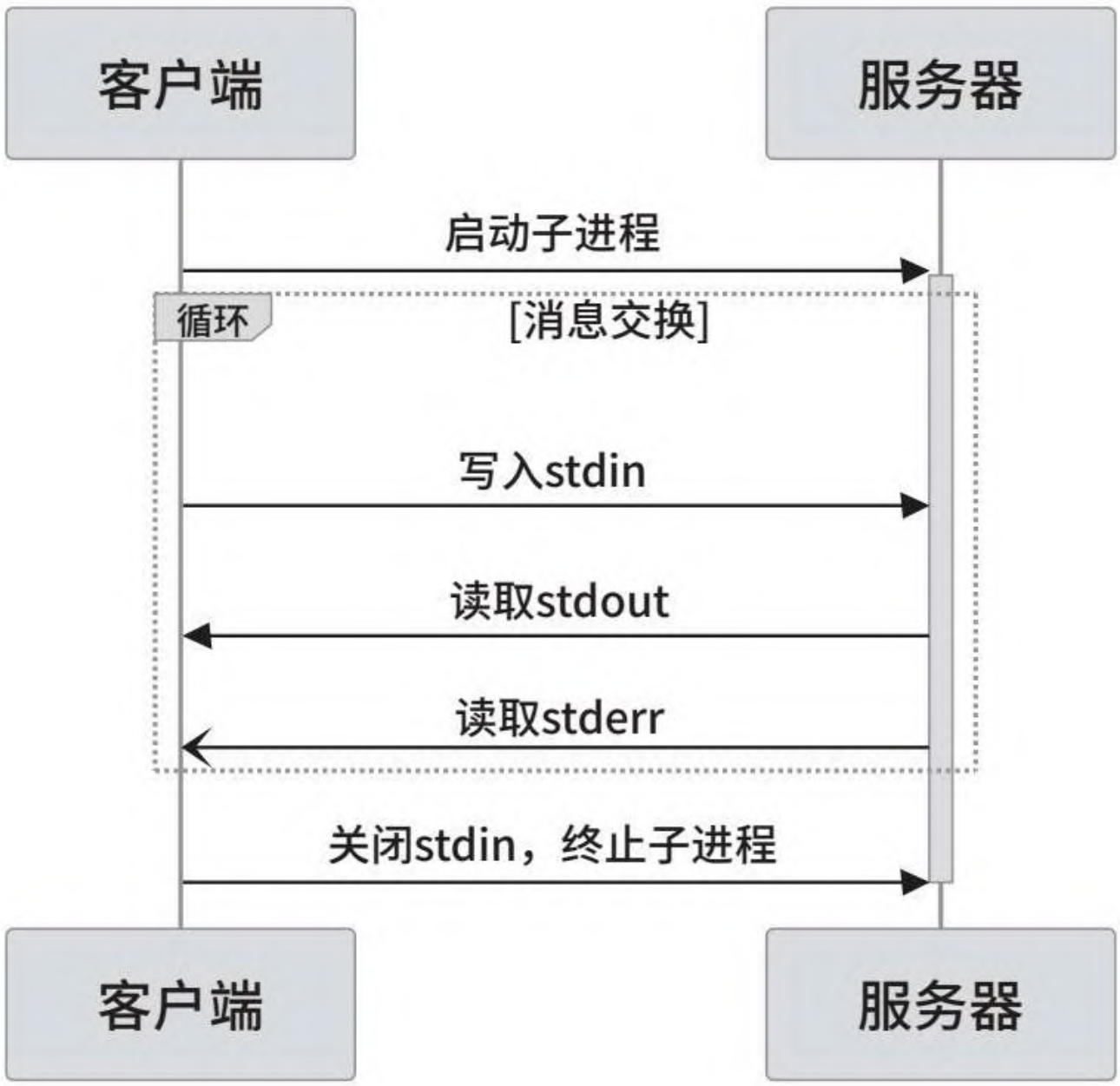


图2-5　stdio传输通信流程

通信步骤如下：

1. 客户端以子进程的方式启动服务器；
2. 客户端往服务器的stdin写入消息；
3. 服务器从自身的stdin读取消息；
4. 服务器往自身的stdout写入消息；
5. 客户端从服务器的stdout读取消息；
6. 客户端终止子进程，关闭服务器的stdin；
7. 服务器关闭自身的stdout。

2. stdio传输的实现

我们参考MCP官方的typescript-sdk来看一下stdio传输机制是如何实现的。

启动服务器

以命令行的方式在本地启动服务器：

```
npx -y mcp-server-time
```

创建stdio通信管道

服务器启动时会创建stdio通信管道（pipeline），用于跟客户端进行双向通信。在客户端发送关闭通知或者服务器因为异常退出之前，这个通信管道会一直保持，常驻进程。

stdio传输类StdioServerTransport实现了MCP的Transport接口，实现逻辑如下：

```
export class StdioServerTransport implements Transport {

    private _readBuffer: ReadBuffer = new ReadBuffer();
    private _started = false;
```

```
    constructor(
        private _stdin: Readable = process.stdin,
        private _stdout: Writable = process.stdout
    ) {}

    onclose?: () => void;
    onerror?: (error: Error) => void;
    onmessage?: (message: JSONRPCMessage) => void;
}
```

从标准输入stdin读取请求消息

客户端把消息发送到通信管道，服务器通过标准输入stdin读取客户端发送的消息，以换行符\n作为读取完成标识。

服务器读取消息的实现逻辑如下（读取的消息最终以JSON-RPC编码的结构体形式返回）：

```
readMessage(): JSONRPCMessage | null {
        if (!this._buffer) {
            return null;
        }

        const index = this._buffer.indexOf("\n");
        if (index === -1) {
```

```
        return  null;

    }
```

```
        const line = this._buffer.toString("utf8", 0, index).replace(/\r$/, '');

        this._buffer = this._buffer.subarray(index + 1);
        return deserializeMessage(line);
    }
```

把响应消息写入stdout

服务器运行完内部逻辑之后，需要向客户端发送响应消息。具体实现流程为：服务器先用JSON-RPC编码消息，再把消息写入标准输出stdout。实现逻辑如下：

```
send(message: JSONRPCMessage): Promise<void> {
        return new Promise((resolve) => {
            const json = serializeMessage(message);
            if (this._stdout.write(json)) {
                resolve();
            } else {
                this._stdout.once("drain", resolve);
            }
        });
    }
}
```



上面的后两个步骤演示了客户端请求服务器单向通信的过程：客户端往stdin写入消息，服务器从stdin读取消息；服务器往stdout写入消息，客户端从stdout读取消息。同理，如果是服务器给客户端发送通知的单向通信场景，这个步骤应该反

过来，变成：服务器往stdin写入消息，客户端从stdin读取消息；客户端往stdout写入消息，服务器从stdout读取消息。

关闭stdio通信管道

当客户端退出时，会向服务器发送关闭通知。服务器在通过stdio通信管道收到客户端发送的关闭通知或者内部运行错误之后，将主动关闭stdio通信管道。

一旦通信管道关闭，客户端与服务器之间便不能再相互发送消息，除非再次建立stdio通信管道。以下是关闭通信管道的实现逻辑：

```
async close(): Promise<void> {
        // 首先移除data事件监听器
        this._stdin.off("data", this._ondata);
        this._stdin.off("error", this._onerror);


        // 检查剩余的data事件监听器
        const remainingDataListeners = this._stdin.listenerCount('data');
        if (remainingDataListeners === 0) {
            // 仅在无剩余data事件监听器的情况下暂停stdin
            // 避免干扰其他可能正在使用stdin的程序
            this._stdin.pause();
        }
```

```
        //  清空读取缓冲区并触发关闭回调
        this._readBuffer.clear();
        this.onclose?.();
}
```

3. stdio传输的利弊与适用场景

stdio传输机制依靠本地进程通信实现，其主要优势是：

- 无外部依赖，实现简单；
- 无网络传输，通信速度快；
- 本地通信，安全性高，无网络攻击风险。

但也存在一些局限性，主要体现在：

- 单进程通信，无法并行处理多个客户端请求；
- 进程通信的资源开销大，很难在本地运行多个服务器。

stdio传输适用于操作本地资源，且不希望暴露给外部访问的场景。例如，你可能希望通过AI对话助手总结微信消息，而这些消息文件存储在本地电脑上，既无法也不应该被外部访问。

若需要访问远程服务器上的资源，虽然仍可使用stdio传输，但实现会更加复杂，一般需要经过以下两步：

- 在远程服务器上部署一个API服务，用于操作资源并提供公网访问能力；
- 编写一个MCP服务器对接该API，再通过stdio传输与客户端进行本地通信。

既然使用stdio传输访问远程资源如此烦琐，那是否存在更合适的传输机制？

答案是肯定的——可以使用SSE传输。

2.4.2　SSE传输

MCP使用SSE（Server-Sent　Events，服务器发送事件）传输来解决远程资源访问的问题。底层基于HTTP通信，通过类似API的方式，让客户端直接访问远程资源，而不用通过stdio传输做中转。

在SSE传输中，服务器作为一个独立进程运行，可以处理多个客户端连接。服务器必须提供两个端点，分别是：

- SSE端点，供客户端与服务器建立双向通信连接（GET请求）；
- 消息端点，供客户端向服务器发送消息（POST请求）。

1.　SSE传输通信流程

基于SSE传输的通信流程如图2-6所示。

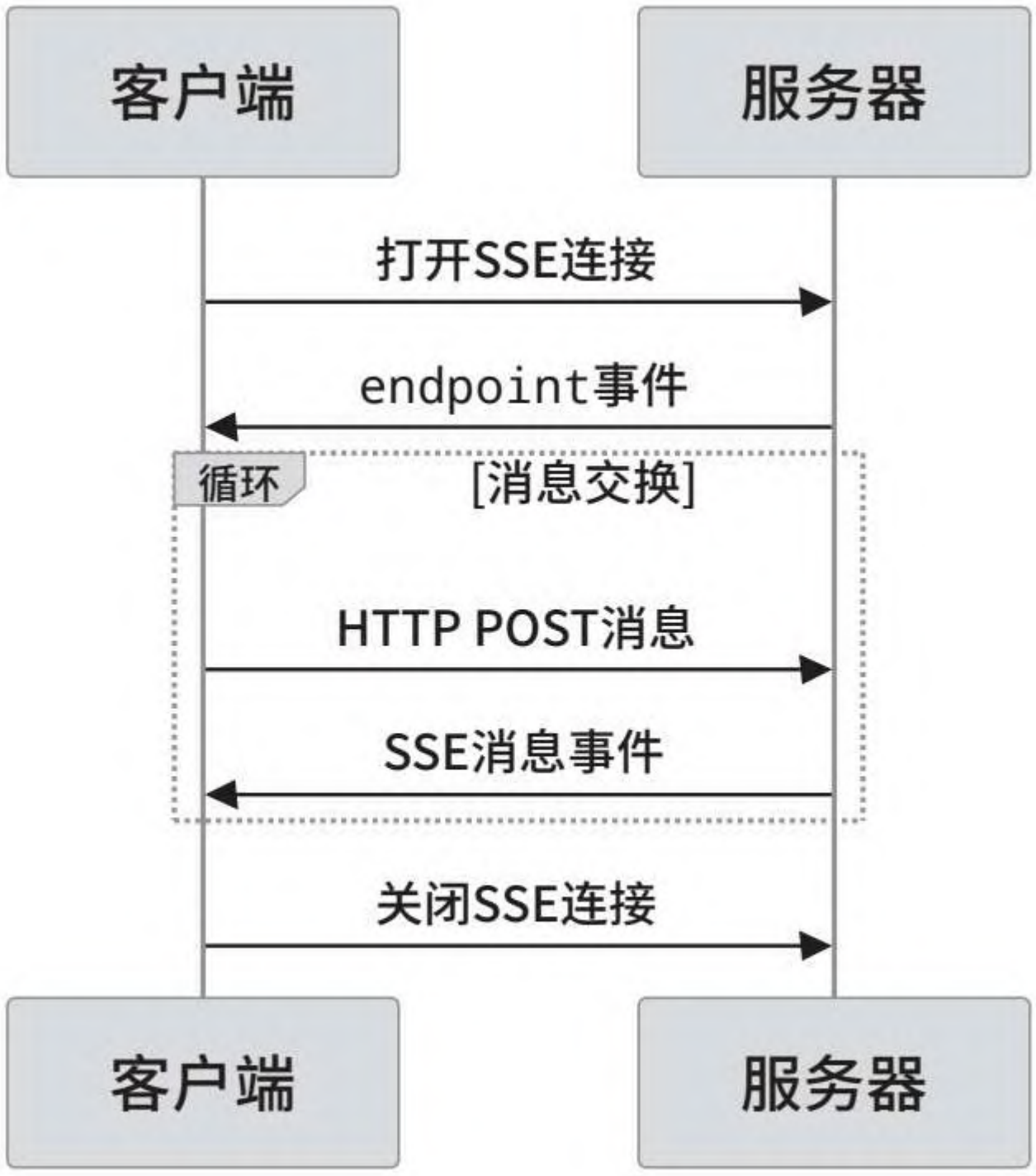


图2-6　SSE传输通信流程

通信步骤如下：

1. 客户端向服务器的/sse端点发送请求（一般是GET请求），建立SSE连接；
2. 服务器通过该连接发送事件消息，告知客户端发送所需要的端点地址；
3. 客户端根据地址向服务器发送消息请求；
4. 服务器返回状态码，确认已接收消息；
5. 服务器通过SSE连接向客户端推送处理结果或其他事件消息；

6. 客户端从SSE流中持续接收服务器发送的事件消息；

7. 通信结束之后，客户端关闭SSE连接。

2. SSE传输的实现

我们参考MCP官方的typescript-sdk来看一下SSE传输机制是如何实现的。

启动服务器

通常以命令行方式启动服务器（一般部署在远程服务器上），实际上是启动一个HTTP服务，通过监听指定端口对外提供HTTP接口。服务器定义与启动的主要实现逻辑如下：

```
const server = new McpServer({
    name: "example-server",
    version: "1.0.0",
});

const app = express();
```

```
app.get("/sse", async (_: Request, res: Response) => {
    const transport = new SSEServerTransport("/messages", res);

    await server.connect(transport);
});

app.post("/messages", async (req: Request, res: Response) => {
    await transport.handlePostMessage(req, res);
});
app.listen(3001);
```

这个示例使用express框架启动了一个HTTP服务，监听端口为3001，并对外暴露了两个接口端点：

- /sse，GET请求，用于建立SSE连接；
- /messages，POST请求，用于接收客户端发送的消息。

服务器启动后，需要通过DNS解析将其绑定到一个可公开访问的域名，比如example.com。

建立SSE连接

客户端请求服务器的SSE端点地址https://example.com:3001/sse，与服务器建立双向通信连接；服务器在建立连接之后，给客户端发送一条事件消息，包含消息端点地址，实现逻辑如下：

```
res.writeHead(200,  {
    "Content-Type":  "text/event-stream",
    "Cache-Control":  "no-cache,  no-transform",
    Connection:  "keep-alive",
});


const  messagesUrl  =  "https://example.com:3001/messages?sessionId=xxx";


res.write(`event:  endpoint\ndata:  ${messagesUrl}\n\n`);
```

客户端从服务器返回的endpoint事件中读取消息端点地址，与服务器成功建立SSE连接。

消息交互

客户端与服务器建立SSE连接之后，开始给消息端点地址发送消息。

MCP中的SSE传输是双通道响应机制，也就是说，服务器在接收到客户端的请求消息之后，既要给当前的请求回复一条响应消息，也要给之前建立的SSE连接发送一条事件消息（通知类型消息除外）。

举个例子，客户端与服务器建立SSE连接之后，给服务器发送的第一条消息，用于初始化阶段做能力协商。

客户端给消息端点发送的请求消息示例如下：

```
curl  -X  POST  https://example.com/messages?sessionId=xxx  \
-H  "Content-Type:  application/json"  \
-d  '{
   "jsonrpc":  "2.0",
   "id":  "1",
   "method":  "initialize",
```

```
    "params": {
        "protocolVersion": "1.0",
        "capabilities": {},
        "clientInfo": {
            "name": "mcp-client",
            "version": "1.0.0"
        }
    }
}'
```

服务器从HTTP请求体里读取客户端发送的消息，在执行完内部逻辑之后，给客户端响应202状态码（无响应体），表示请求已收到，实现逻辑如下：

```
async handlePostMessage(
    req: IncomingMessage,
    res: ServerResponse,
    parsedBody?: unknown,
): Promise<void> {
    if (!this._sseResponse) {
        const message = "SSE connection not established";
        res.writeHead(500).end(message);
        throw new Error(message);
    }


    let body: string | unknown;
    try {
```

```
    const ct = contentType.parse(req.headers["content-type"] ?? "");
    if (ct.type !== "application/json") {
        throw new Error(`Unsupported content-type: ${ct}`);
    }

    body = parsedBody ?? await getRawBody(req, {
        limit: MAXIMUM_MESSAGE_SIZE,
        encoding: ct.parameters.charset ?? "utf-8",
    });
} catch (error) {
    res.writeHead(400).end(String(error));
    this.onerror?.(error as Error);
    return;
}
```

```
        try {
            await this.handleMessage(typeof body === 'string' ? JSON.parse(body) :
body);
        } catch {
            res.writeHead(400).end(`Invalid message: ${body}`);
            return;
        }

        res.writeHead(202).end("Accepted");
    }
```

然后，服务器把响应给客户端的消息内容，通过之前建立的SSE连接，以事件消息的形式发送，消息内容使用JSON-RPC编码，实现逻辑如下：

```
    async send(message: JSONRPCMessage): Promise<void> {
        if (!this._sseResponse) {
            throw new Error("Not connected");
        }

        this._sseResponse.write(
            `event: message\ndata: ${JSON.stringify(message)}\n\n`,
        );
    }
```

客户端根据服务器同步响应的202状态码，判断服务器已经收到请求，开始从之前建立的SSE连接中读取服务器发送的响应内容。



客户端与服务器之间的SSE连接应保持一一对应关系，以避免数据混淆。在建立连接阶段，服务器应为每个SSE连接分配一个唯一标识符sessionId，并将该标识符附加在消息端点地址中，例如/messages?sessionId=xxx。在后续的消息交互过程中，服务器可通过请求参数中的sessionId准确匹配对应的SSE连接，并仅向该连接推送事件消息，确保数据的隔离性与一致性。

断开SSE连接

服务器和客户端任一方都可能主动断开SSE连接。为了防止连接异常中断而资源未被释放，仍保持连接的一方应实现连接检测与超时关闭机制。例如，可以通过SSE传输通道定期发送心跳消息以检测对方状态；若连续多次未收到响应，可视为对方已断开连接，这时候可以主动关闭SSE连接，以避免资源泄露。一个用Go实现的心跳检测和超时关闭示例如下：

```
//  设置心跳间隔定时器

heartbeatInterval  :=  30  *  time.Second

heartbeatTicker  :=  time.NewTicker(heartbeatInterval)

defer  heartbeatTicker.Stop()



//  设置空闲超时时间

idleTimeout  :=  5  *  time.Minute

idleTimer  :=  time.NewTimer(idleTimeout)

defer  idleTimer.Stop()



go  func()  {

    for  {

        select  {
```

```
case <-session.Done():
    return
case <-heartbeatTicker.C:
    // 发送心跳
    if err := writer.SendHeartbeat(); err != nil {
        // 心跳无响应，关闭连接
        session.Close()
        return
    }
case <-idleTimer.C:
    // 超时不活跃，关闭连接
    session.Close()
            return
        }
    }
}()
```

3. SSE传输的利弊与适用场景

SSE传输适用于客户端和服务器不在同一网络下的通信场景。例如，你希望通过AI对话助手查询自己云服务器上的数据库，可以在云服务器上部署一个MCP服务器，读取数据库内容，并通过SSE URL的方式在AI对话助手中进行配置和接入。

实际上，使用SSE传输的服务器，理论上也可以通过“本地stdio传输+API中转”的方式实现相同的功能。两者的主要区别在于：

❑

使用SSE传输时，客户端可以直接与服务器通信，而不用通过本地的stdio传输调用API进行中转；

❑

SSE接入只需配置一个URL，对本地环境没有要求，不需要在本地运行服务器，用户接入门槛更低。

SSE传输的主要优势如下：

- 支持远程资源访问，解决了stdio传输仅适用于本地资源的问题；
- 基于HTTP实现，兼容性好，便于与现有Web基础设施集成；
- 服务器可作为独立进程运行，支持处理多个客户端连接；
- 相比WebSocket，实现更简单，因为它是HTTP的扩展，不需要协议升级。

SSE传输的主要劣势如下：

- 复杂的双通道响应机制，SSE传输要求服务器在接收客户端消息后，既要给当前请求响应，也要给之前建立的SSE连接发送事件消息，实现复杂；
- 连接不稳定，在无服务器环境中，SSE连接会随机、频繁断开，不能提供可靠连接；
- 不方便持久连接，无服务器架构通常自动扩缩容，不适合长时间连接，而SSE需要维持持久连接；
- 需要大量会话管理，SSE传输需要为每个SSE连接分配一个唯一的标识符（sessionId）来防止数据混淆，这意味着实现方必须承担额外的会话管理工作，增加了系统的实现复杂度；
- 额外实现成本，需要实现心跳检测和超时机制来避免资源泄露。

正因为这些问题，MCP在新的协议（2025-03-26版本）中引入了流式HTTP传输机制来替代SSE传输，并计划将其废弃。新的传输机制保留了HTTP的基础，但支持更灵活的连接方式，更适合现代云原生架构和无服务器环境。

2.4.3　流式HTTP传输

基于流式HTTP传输的通信流程如图2-7所示。

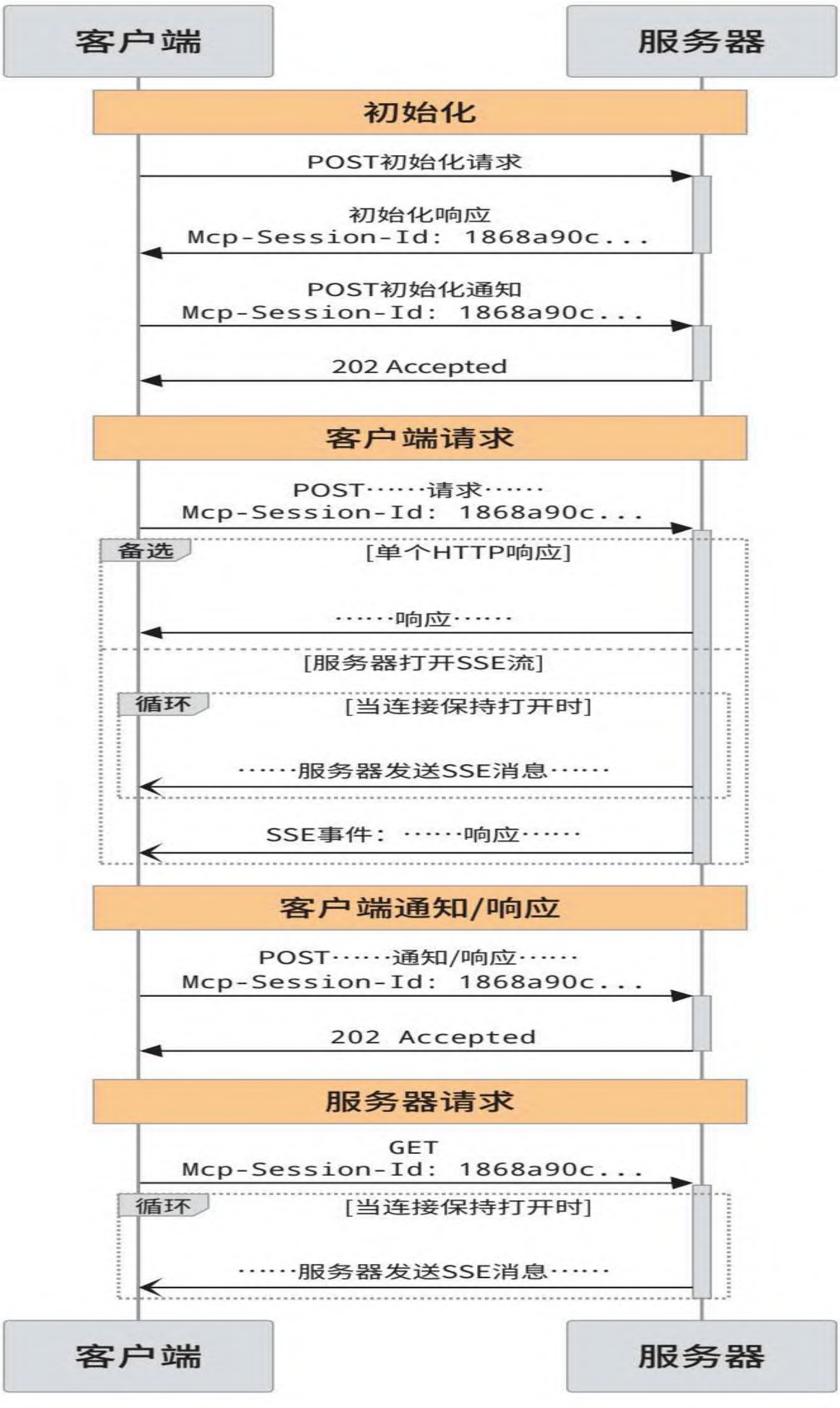

客户端
服务器
初始化
POST初始化请求
初始化响应
Mcp-Session-Id: 1868a90c...
POST初始化通知
Mcp-Session-Id: 1868a90c...
202 Accepted
客户端请求
POST·····请求·····
Mcp-Session-Id: 1868a90c...
备选
[单个HTTP响应]
·····响应·····
[服务器打开SSE流]
循环
[当连接保持打开时]
·····服务器发送SSE消息·····
SSE事件: ·····响应·····
客户端通知/响应
POST·····通知/响应·····
Mcp-Session-Id: 1868a90c...
202 Accepted
服务器请求
GET
Mcp-Session-Id: 1868a90c...
循环
[当连接保持打开时]
·····服务器发送SSE消息·····
客户端
服务器

图2-7　流式HTTP传输通信流程

1.　流式HTTP通信流程

在流式HTTP传输中，服务器必须提供一个同时支持POST请求和GET请求的HTTP端点。流式HTTP传输通信流程的主要步骤如下：

1.　客户端向服务器的通信端点发送消息；

2.　服务器返回响应；

3.　客户端继续向服务器发送下一条消息；

4.　服务器继续响应客户端消息。

跟SSE传输的双通道响应机制不同，在流式HTTP传输中，客户端与服务器的消息交互基本上是“一来一回”的单通道响应，即每个请求只能对应一个响应。不过，该响应可以同步返回，也可以通过SSE实现异步推送。此外，流式HTTP传输支持有状态和无状态两种交互方式：有状态交互需要通过Mcp-Session-Id维护会话状态，而无状态交互则意味着每次请求都是独立的，互不依赖。

2.　流式HTTP传输的实现

我们参考MCP官方的typescript-sdk来看一下流式HTTP传输机制是如何实现的。

启动服务器

跟SSE传输机制一样，流式HTTP传输本质上也基于HTTP通信，需要先通过命令行运行服务器，启动HTTP服务。服务器定义与启动的主要实现逻辑如下：

```
const server = new McpServer({
    name: "example-server",
    version: "1.0.0",
});



const app = express();
```

```
app.all("/mcp", async (req: Request, res: Response) => {
    const transport = new StreamableHTTPServerTransport();
```

```
        await  server.connect(transport);



        await  transport.handleMessage(req,  res);
    });
    app.listen(3002);
```

这个示例使用express框架启动了一个HTTP服务，监听端口为3002，且对外暴露了一个端点：

- /mcp，接收客户端建立连接、交换消息的请求。

服务器启动后，需要通过DNS解析将其绑定到一个可公开访问的域名，比如example.com。



跟SSE传输需要暴露两个端点（SSE端点与消息端点）不同，流式HTTP传输只需要暴露一个端点，来接收各种类型的客户端请求（GET/POST/DELETE）。

消息交互

在服务器启动成功之后，客户端可以直接给服务器的端点地址发消息，而无须先建立连接。

客户端可以通过GET或者POST方式向服务器发送消息，每个请求必须在请求头中设置Accept，传递以下两个值：

- application/json，用于接收服务器响应的JSON-RPC编码消息；
- text/event-stream，用于开启服务器的流式传输通道，客户端通过该通道持续接收事件消息。

客户端请求示例如下：

```
curl -X POST https://example.com/mcp \
-H "Content-Type: application/json" \
-H "Accept: application/json, text/event-stream" \
-d '{
    "jsonrpc": "2.0",
    "id": "1",
    "method": "initialize",
    "params": {
        "protocolVersion": "1.0",
        "capabilities": {},
        "clientInfo": {
            "name": "mcp-client",
            "version": "1.0.0"
        }
    }
}'
```

在流式HTTP传输机制下，客户端与服务器通信有以下几个要点。

❑

客户端可以给服务器发送不包含请求体的GET请求，用于建立SSE连接；服务器可以主动给客户端先发送消息。

❑

对于客户端给服务器发送JSON-RPC消息的情况，必须使用POST请求，并将请求头中的Accept设置为application/json, text/event-stream。

❑

服务器接收到客户端的GET请求时，要么返回Content-Type: text/event stream开启SSE连接，要么返回HTTP 405状态码，表示不支持SSE连接。

❑

服务器接收到客户端的POST请求时，从请求体读取JSON-RPC消息：如果是通知消息，就响应HTTP 202状态码，表示消息已收到；如果是非通知消息，服务器可以选择返回Content-Type: text/event-stream开启SSE传输，或者返回Content-Type: application/json同步响应一条JSON-RPC消息。

会话保持

流式HTTP传输既支持无状态交互——每一次请求都是独立的，不需要记录状态；也支持有状态交互——新的请求可能需要依赖之前的请求或者响应信息作为上下文，这种机制被称为**会话保持**。

对于会话保持的情况，服务器与客户端之间的交互应该遵守以下原则：

❑

使用 流式HTTP传输的服务器可以在初始化时，通过Mcp-Session-Id响应头返回会话ID；

❑

如果服务器在初始化时返回了Mcp-Session-Id，客户端必须在所有后续请求中通过Mcp-Session-Id请求头带上会话ID；

❑

服务器可以随时终止会话，之后它必须使用HTTP 404 Not Found响应包含该会话ID的请求；

❑

当客户端收到对包含Mcp-Session-Id请求的HTTP 404响应时，它必须通过发送一个不带会话ID的新的InitializeRequest来启动新会话（可选）；

❑

不再需要特定会话的客户端应该发送一个带有Mcp-Session-Id请求头的HTTP DELETE请求到服务器，以显式终止会话（可选，但推荐）。

服务器验证会话的实现逻辑如下：

```
/**
 * 校验非初始化请求的会话ID
 * 如果会话合法，返回true；否则返回false
 */
private validateSession(req: IncomingMessage, res: ServerResponse): boolean {
    if (!this._initialized) {
```

```
    //  如果服务器尚未初始化，拒绝所有请求
    res.writeHead(400).end(JSON.stringify({
        jsonrpc:  "2.0",
        error:  {
            code:  -32000,
            message:  "Bad  Request:  Server  not  initialized"
        },
        id:  null
    }));
    return  false;
}
if  (this.sessionId  ===  undefined)  {
    //如果没有设置sessionId，说明未启用会话管理，
    //  此时无须校验会话ID
    return  true;
}
const  sessionId  =  req.headers["mcp-session-id"];
```

```
if (!sessionId) {
    // 非初始化请求必须带有会话ID，否则返回400错误
    res.writeHead(400).end(JSON.stringify({
        jsonrpc: "2.0",
        error: {
            code: -32000,
            message: "Bad Request: Mcp-Session-Id header is required"
        },
        id: null
    }));
            return false;
        } else if (Array.isArray(sessionId)) {
            res.writeHead(400).end(JSON.stringify({
                jsonrpc: "2.0",
                error: {
                    code: -32000,
                    message: "Bad Request: Mcp-Session-
Id header must be a single value"
                },
                id: null
            }));
            return false;
```

```
}
else if (sessionId !== this.sessionId) {
    // 会话ID不匹配时，返回404，表示未找到会话
    res.writeHead(404).end(JSON.stringify({
        jsonrpc: "2.0",
        error: {
            code: -32001,
            message: "Session not found"
        },
        id: null
    }));
    return false;
}


    return true;
}
```

连接断开与重连

在流式HTTP传输中，如果客户端与服务器使用SSE连接通信，断开连接的方式跟SSE传输断开连接的方式一致：可以由连接的任意一方主动断开连接，仍然保持连接的一方则需要实现心跳检测和超时机制，以便能及时关闭连接，避免资源泄露。

相比SSE传输，流式HTTP传输引入了连接恢复机制，支持在连接中断后重新发送可能丢失的消息：

服务器可在SSE事件中附加一个ID字段，该ID在所有事件流中必须全局唯一。若客户端希望在连接断开后恢复连接，应重新发起HTTP GET请求，并通过Last-Event-ID请求头传递上次成功接收的事件ID。服务器可据此重放自该ID之后的事件，实现事件流的恢复。

3. 流式HTTP传输的利弊与适用场景

流式HTTP传输机制结合了SSE传输的远程访问能力和无状态HTTP的灵活性，同时解决了SSE传输中的许多问题。

它的主要优势如下：

- 兼容无服务器环境，可以在短连接模式下工作；
- 灵活的连接模式，支持简单的请求-响应和流式传输；
- 会话管理更加标准化和清晰；
- 支持断开连接恢复和消息重传，比SSE传输更加可靠；
- 保留了SSE的流式传输能力，同时解决了其稳定性问题。

它的主要劣势如下：

- 状态判断过多，实现复杂度高；
- 处理连接断开和恢复的逻辑复杂，实现成本高；
- 会话管理需要服务器引入额外的组件（比如用Redis来存储会话）。

流式HTTP传输适用于以下场景：

- 需要远程访问服务，特别是云原生和无服务器架构的场景；
- 需要支持流式输出的AI服务；
- 需要服务器主动推送消息给客户端的场景；
- 面向大规模部署，要求具备高可用性和可扩展性的服务；
- 需要在不稳定的网络环境中保持可靠通信的场景。

与SSE传输相比，流式HTTP传输是一个更全面、更灵活的解决方案，也是MCP在访问远程资源场景时主推的传输机制。

2.5　服务器能力：提示词/资源/工具

MCP为服务器的实现提供提示词（Prompts）、资源（Resources）、工具（Tools）三大核心能力，如表2-3所示。

表2-3　三大核心能力：提示词、资源、工具

特　　性	控制方式	描　　述	示　　例
提示词	用户控制	由用户选择调用的提示词模板	斜杠命令、菜单选项
资源	应用控制	由客户端附加和管理的上下文数据	文件内容、数据库表结构
工具	模型控制	暴露给大模型用于执行操作的函数	API 请求、文件写入

我们可以把服务器的三大核心能力理解为“积木”：虽然每个能力（积木）本身很简单，但通过组合这些能力可以构建出复杂的功能（就像用积木搭建城堡）。每个能力都有其特定的用途，不能被简化为更基础的操作。

2.5.1　提示词及其交互示例

MCP为服务器向客户端暴露提示词提供了一种标准方式。提示词允许服务器提供与大模型交互的结构化消息和指令。客户端可以发现可用的提示词，检索其内容，并提供自定义参数来使用它们。

提示词被设计为由用户控制，这意味着它们从服务器暴露给客户端之后，由用户来进行选择和使用。通常，在客户端的UI界面中，用户通过输入命令来触发提示词，用户可以自然地发现和调用可用的提示词。

例如，用户可以输入斜杠命令来调出可用的提示词，如图2-8所示。

提示词交互示例

服务器声明提示词能力

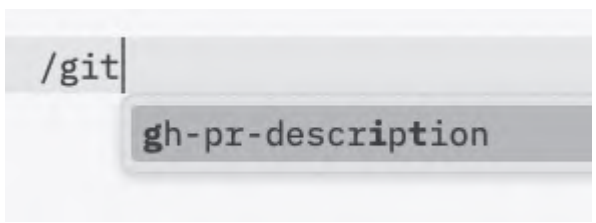


图2-8　用户输入斜杠命令调用提示词

支持提示词能力的服务器需要在服务器定义时声明此项能力：

```
{
    "capabilities": {
        "prompts": {
            "listChanged": true
        }
    }
}
```

"listChanged": true表示服务器支持在提示词列表发生变化时发送通知。

客户端获取提示词列表

客户端给服务器发送一个prompts/list请求来获取服务器定义的提示词列表，请求示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "prompts/list",
    "params": {
        "cursor": "optional-cursor-value"
    }
}
```

服务器收到请求后，返回提示词列表，响应示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
```

```
        "prompts": [
            {
                "name": "code_review",
                "description": "Asks the LLM to analyze code quality and suggest improvements",
                "arguments": [
                    {
                        "name": "code",
                        "description": "The code to review",
                        "required": true
                    }
                ]
            }
        ],
        "nextCursor": "next-page-cursor"
    }
}
```


如果服务器定义的提示词过多，希望客户端通过分页参数来分批次查询，可以在响应提示词列表时，通过result.nextCursor字段来控制分页。

客户端在下一次请求中，带上服务器上一次返回的result.nextCursor值，作为新请求的params.cursor值来继续获取服务器的下一批提示词列表。

客户端获取单个提示词

客户端给服务器发送一个prompts/get请求来获取特定提示词的内容，请求示例如下：


```
{
```

```
    "jsonrpc": "2.0",
    "id": 2,
    "method": "prompts/get",
    "params": {
        "name": "code_review",
        "arguments": {
            "code": "def hello():\n        print('world')"
        }
    }
}
```


服务器收到请求后，根据请求参数返回提示词内容，响应示例如下：


```
{
    "jsonrpc": "2.0",
    "id": 2,
    "result": {
        "description": "Code review prompt",
        "messages": [
            {
                "role": "user",
                "content": {
                    "type": "text",
                    "text": "Please review this Python code:\ndef hello():\n
print('world')"
```

```
                }
            }
        ]
    }
}
```

服务器发送提示词列表变更通知

服务器在提示词列表发生变化时，向客户端发送提示词列表变更通知，通知示例如下：

```
{
    "jsonrpc": "2.0",
    "method": "notifications/prompts/list_changed"
}
```

客户端在接收到通知后，应该重新请求服务器获取提示词列表。

提示词交互示例

客户端与服务器关于提示词的交互示例如图2-9所示。

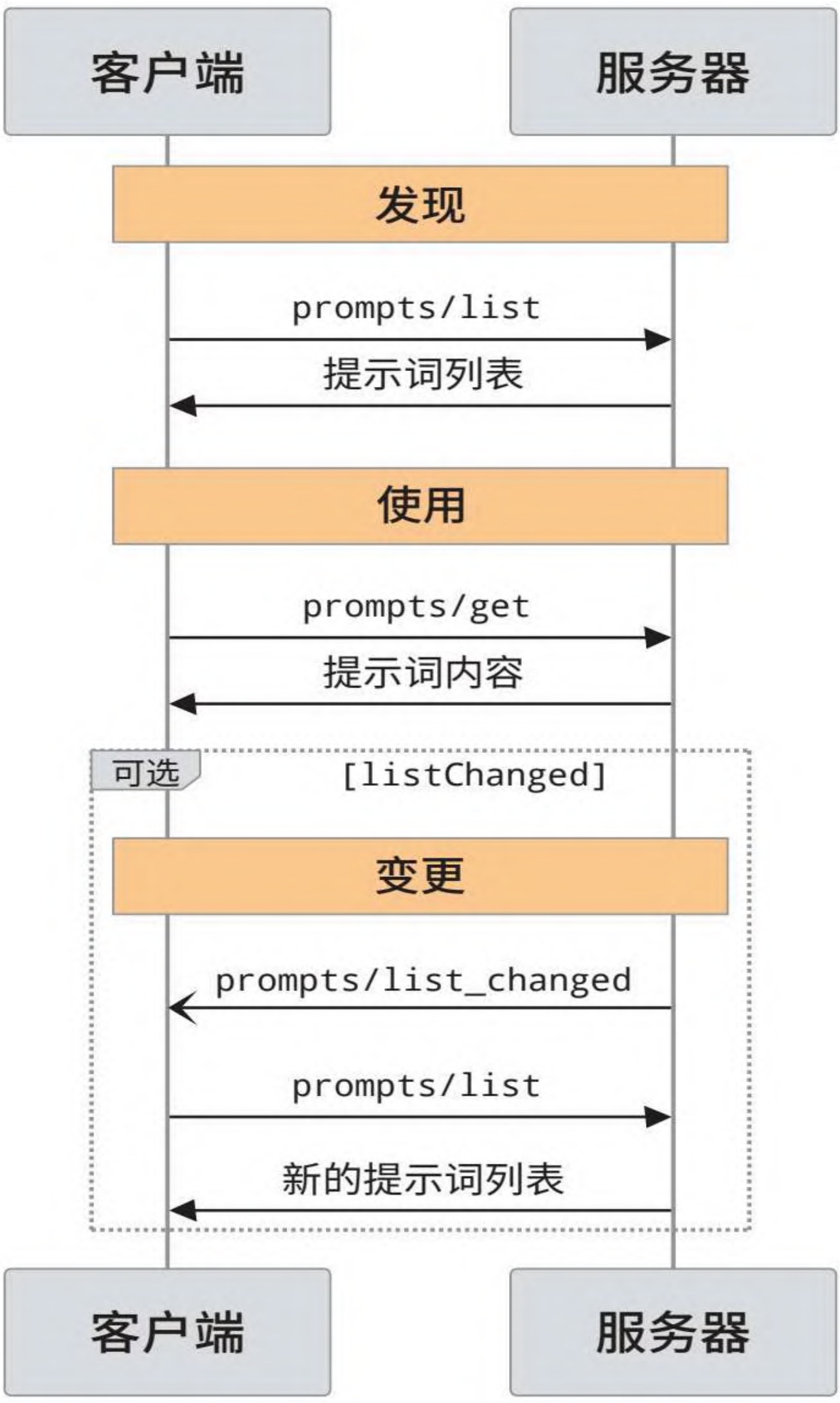

客户端
服务器
发现
prompts/list
提示词列表
使用
prompts/get
提示词内容
可选
[listChanged]
变更
prompts/list_changed
prompts/list
新的提示词列表
客户端
服务器

图2-9　客户端与服务器关于提示词的交互示例

2.5.2　资源及其交互示例

MCP为服务器向客户端暴露资源提供了一种标准方式。资源允许服务器共享为大模型提供上下文的数据，例如文件、数据库结构或特定的应用程序信息。每个资源由一个URI（Uniform Resource Identifier，统一资源标识符）进行唯一标识。

MCP中的资源被设计为由应用驱动，也就是说由主机来根据其自身的需求，将服务器的资源融入其上下文。例如，主机可以这样实现：

- 通过一个列表或者目录树，显示可用的资源；
- 允许用户搜索和筛选可用的资源；
- 将资源整合到上下文中。

如图2-10所示，用户在Claude中选择由服务器提供的资源。



图2-10　用户选择资源

资源交互示例

服务器声明资源能力

支持暴露资源的服务器需要在服务器定义时声明此项能力：

```
{
  "capabilities": {
    "resources": {
      "subscribe": true,
      "listChanged": true
```

```
        }
    }
}
```

此项能力支持两个附加字段：

- subscribe，是否允许客户端订阅和接收单个资源的变更通知；
- listChanged，当资源列表发生变化时，是否允许服务器通知客户端。

subscribe和listChanged都是可选的，服务器可以支持其中任何一个，或者两个都支持。

客户端获取资源列表

客户端给服务器发送一个resources/list请求来获取服务器定义的资源列表，请求示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "resources/list",
    "params": {
        "cursor": "optional-cursor-value"
    }
}
```

服务器收到请求后，返回资源列表，响应示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
```

```
        "resources":  [
            {
                "uri":  "file:///project/src/main.rs",
                "name":  "main.rs",
                "description":  "Primary  application  entry  point",
                "mimeType":  "text/x-rust"
            }
        ],
        "nextCursor":  "next-page-cursor"
    }
}
```

跟获取提示词列表的逻辑一致，获取资源列表也支持分页控制。

客户端读取单个资源

客户端给服务器发送一个resources/read请求来获取特定资源的内容，请求示例如下：

```
{
    "jsonrpc":  "2.0",
    "id":  2,
    "method":  "resources/read",
    "params":  {
        "uri":  "file:///project/src/main.rs"
    }
}
```

服务器收到请求后，根据资源定位符，返回对应的资源内容，响应示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 2,
    "result": {
        "contents": [
            {
                "uri": "file:///project/src/main.rs",
                "mimeType": "text/x-rust",
                "text": "fn main() {\n    println!(\"Hello world!\");\n}"
            }
        ]
    }
}
```


客户端获取资源模板列表

MCP允许客户端通过资源模板动态获取服务器的资源。

客户端给服务器发送一个resources/templates/list请求来获取服务器定义的资源模板列表，请求示例如下：


```
{
    "jsonrpc": "2.0",
    "id": 3,
    "method": "resources/templates/list"
}
```


服务器收到请求后，返回资源模板列表，响应示例如下：


```
{
```

```
"jsonrpc": "2.0",
"id": 3,
"result": {
    "resourceTemplates": [
        {
            "uriTemplate": "file:///{path}",
            "name": "Project Files",
            "description": "Access files in the project directory",
            "mimeType": "application/octet-stream"
        }
    ]
}
```


客户端可以根据需求，拼凑uriTemplate的参数来动态获取服务器的资源。

服务器发送资源列表变更通知

当服务器资源列表发生变化时，声明了listChanged的服务器需要给客户端发送通知，通知示例如下：


```
{
    "jsonrpc": "2.0",
    "method": "notifications/resources/list_changed"
}
```


客户端在收到服务器的资源列表变更通知时，应该重新请求服务器获取资源列表。

客户端订阅资源更新通知

声明了subscribe的服务器，允许客户端订阅对某个资源的变更通知。客户端订阅资源变更通知的请求示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 4,
    "method": "resources/subscribe",
    "params": {
        "uri": "file:///project/src/main.rs"
    }
}
```

服务器在接到客户端对某个资源的订阅通知请求时，应该把客户端订阅的资源的URI缓存下来，并给客户端返回一条订阅成功的消息，响应示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 4,
    "result": {}
}
```

客户端发送资源更新通知

当客户端订阅的资源内容发生变化时，服务器应该给客户端发送资源变更通知，通知示例如下：

```
{
    "jsonrpc": "2.0",
    "method": "notifications/resources/updated",
    "params": {
        "uri": "file:///project/src/main.rs"
```

```
        }
    }
```

客户端在接到服务器对某个资源的变更通知时，应该使用该资源的URI，再次请求服务器获取该资源的最新内容。

资源交互示例

客户端与服务器关于资源的交互示例如图2-11所示。

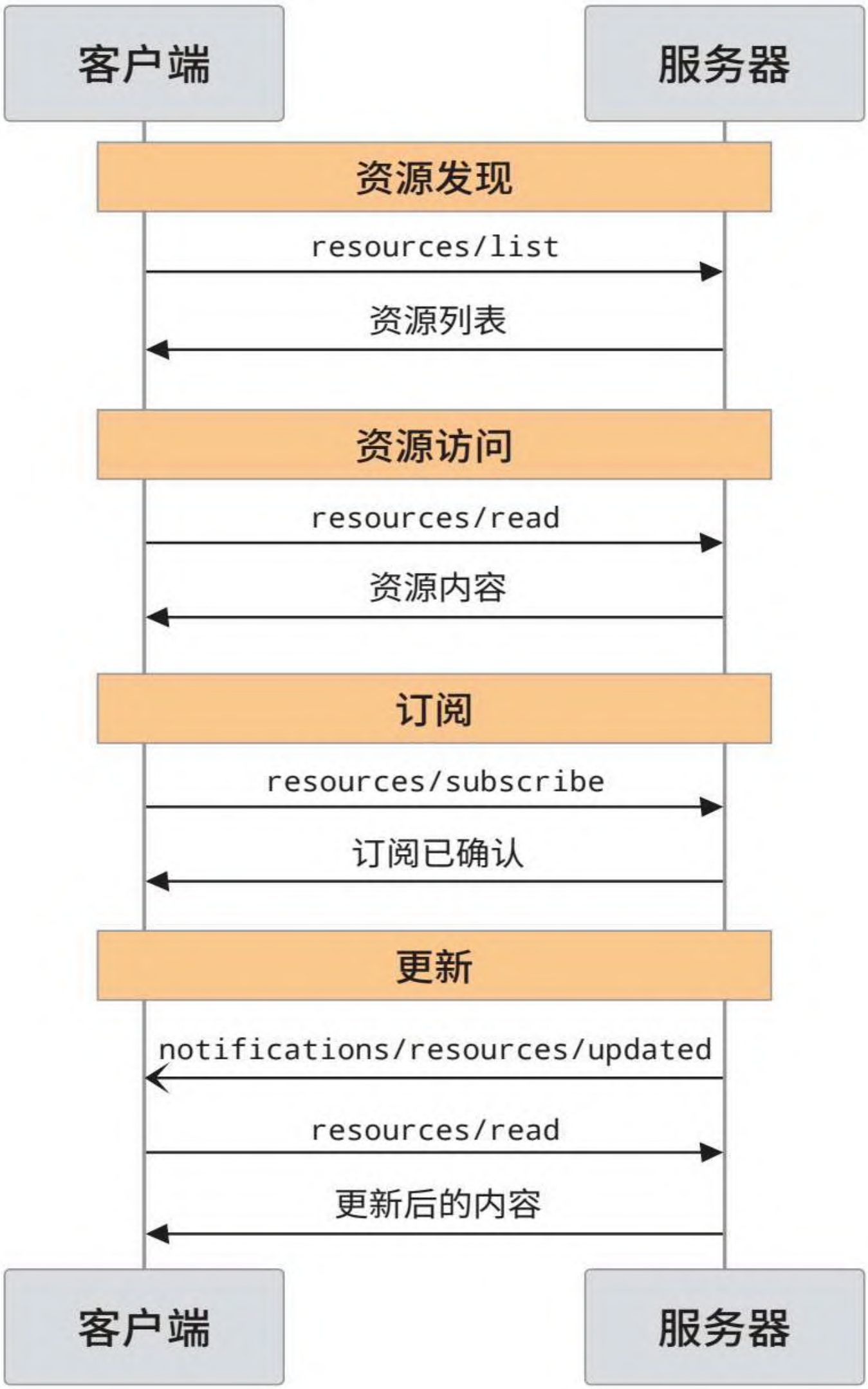

客户端
服务器
资源发现
resources/list
资源列表
资源访问
resources/read
资源内容
订阅
resources/subscribe
订阅已确认
更新
notifications/resources/updated
resources/read
更新后的内容
客户端
服务器

图2-11　客户端与服务器关于资源的交互示例

2.5.3　工具及其交互示例

MCP允许服务器暴露可供大模型调用的各类工具，支持与外部系统交互，比如数据库查询、API调用、数据处理等。每一个工具必须定义唯一的标识符（name）和对其功能的描述（description）。

MCP中的工具设计为由模型控制，这意味着大模型可以根据其对上下文的理解和用户输入的内容自动发现和选择工具。

从信任和安全角度考虑，工具调用应该让用户感知，并且用户有拒绝调用工具的权利。实现工具调用的应用应该：

- 提供清晰的UI，以明确展示哪些工具被暴露给了大模型；
- 当工具被调用时，视觉呈现上要有明确的提示；
- 设置弹框用来提示用户确认对工具的调用操作。

工具交互示例

服务器声明工具能力

支持工具能力的服务器需要在服务器定义时声明此项能力：

```
{
    "capabilities":  {
        "tools":  {
            "listChanged":  true
        }
    }
}
```

"listChanged": true表示服务器支持在工具列表发生变化时发送通知。

客户端获取工具列表

客户端给服务器发送一个tools/list请求来获取服务器定义的工具列表，请求示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/list",
    "params": {
        "cursor": "optional-cursor-value"
    }
}
```

服务器收到请求后，返回工具列表，响应示例如下：

```
{
"jsonrpc": "2.0",
"id": 1,
"result": {
    "tools": [
        {
            "name": "get_weather",
            "description": "Get current weather information for a location",
            "inputSchema": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
```

```
                        "description": "City name or zip code"
                    }
                },
                "required": ["location"]
            }
        }
    ],
    "nextCursor": "next-page-cursor"
  }
}
```

获取工具列表跟获取提示词列表一样，支持分页控制。

客户端调用工具

客户端给服务器发送一个tools/call请求来调用某个工具，请求示例如下：

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}
```

服务器收到请求后，根据工具名称和参数调用工具，并返回工具调用结果，响应示例如下：

```
{
    "jsonrpc": "2.0",
    "id": 2,
    "result": {
        "content": [
            {
                "type": "text",
                "text": "Current weather in New York:\nTemperature: 72° F\nConditions:
Partly cloudy"
            }
        ],
        "isError": false
    }
}
```

服务器发送工具列表变更通知

当服务器可用的工具列表发生变化时，服务器需要给客户端发送工具列表变更通知，通知示例如下：

```
{
    "jsonrpc": "2.0",
    "method": "notifications/tools/list_changed"
}
```

客户端在接到服务器的工具列表变更通知时，应该重新请求服务器获取工具列表。

工具交互示例

客户端与服务器关于工具的交互示例如图2-12所示。

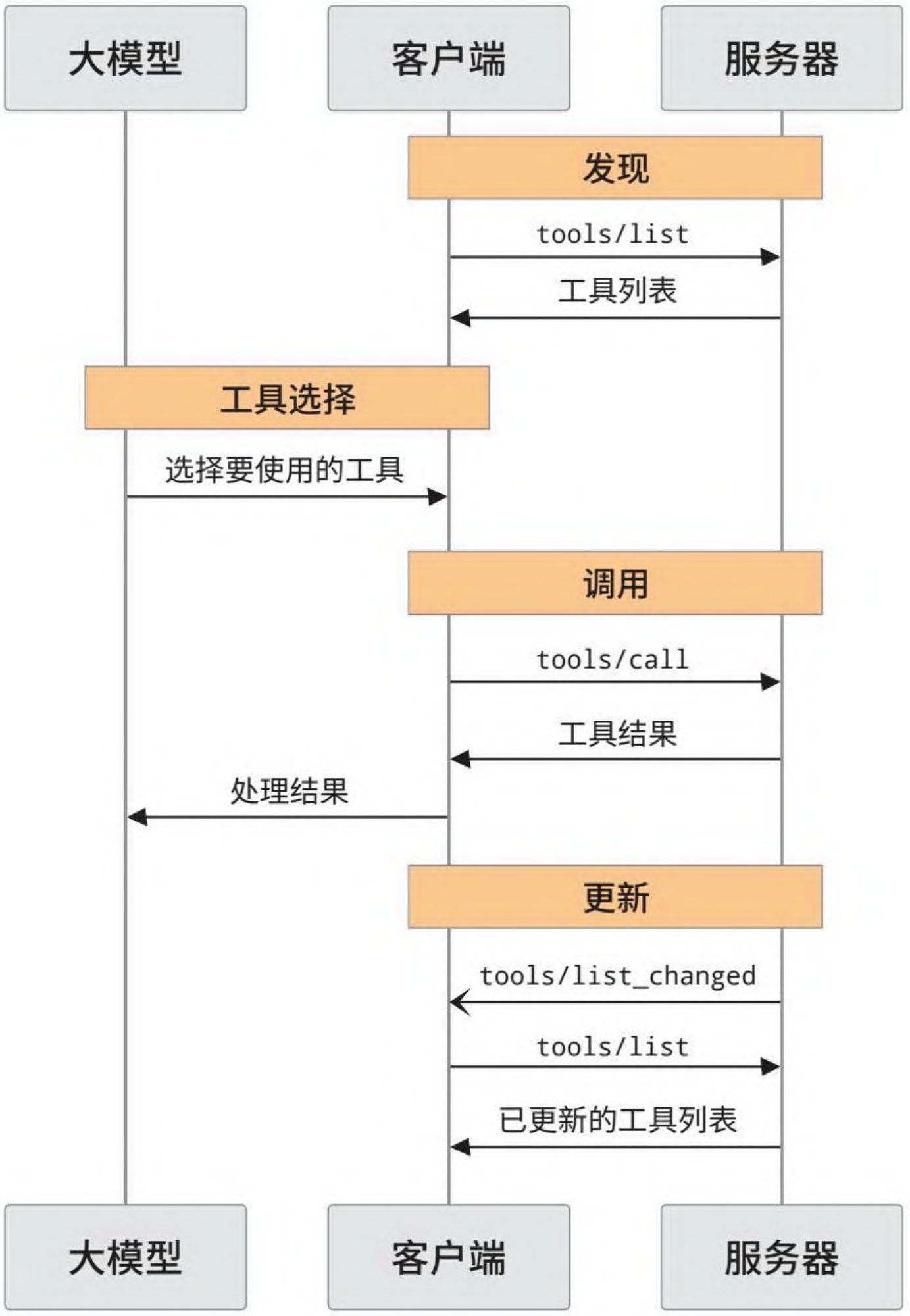


图2-12　客户端与服务器关于工具的交互示例

2.6 客户端能力：根/采样

MCP为客户端的实现提供了两个能力：

- 根（root）
- 采样（sampling）

基于这两个能力，客户端可以为服务器提供文件系统访问控制和大模型代理服务。

2.6.1 根及其交互示例

MCP为客户端提供了一种标准方式，使其能够向服务器暴露文件系统的“根”。根定义了服务器在文件系统中可以操作的边界，使它们能够理解自己有权访问哪些目录和文件。服务器可以从支持的客户端请求根列表，并在该列表发生变化时接收通知。

MCP中的根通常通过工作区或项目配置界面暴露。例如，主机可以提供工作区/项目选择器，允许用户选择服务器有权访问的目录和文件，也可以通过版本控制系统或打开的文件自动检测工作区。

根本质上是一种针对文件系统的访问控制机制。客户端根能力的设计，主要出于以下几方面的考虑：

- **安全边界控制**，为提升安全性，应限制服务器只能访问指定目录和文件，避免接触敏感或未授权的内容；
- **最小权限原则**，服务器只有在需要操作文件系统时，才向客户端申请根列表，减少权限过大带来的安全风险；
- **用户控制和透明度**，服务器操作文件系统时需要向客户端申请根列表，由用户选择同意或拒绝，操作过程对用户透明；
- **多项目隔离**，服务器在操作不同项目时需要申请不同的根列表，实现多项目隔离。

根交互示例

客户端声明根能力

支持根能力的客户端需要在客户端定义时声明此项能力：

```
{
"capabilities": {
"roots": {
"listChanged": true
}
}
}
```

"listChanged": true表示客户端支持在根发生变化时发送通知。

服务器获取根列表

当服务器需要操作文件系统时，会向客户端请求根列表，请求示例如下：

```
{
"jsonrpc": "2.0",
"id": 1,
"method": "roots/list"
}
```

客户端收到请求后返回根列表，响应示例如下：

```
{
"jsonrpc": "2.0",
"id": 1,
"result": {
"roots": [
{
```

```
"uri": "file:///home/user/projects/myproject",
"name": "My Project"
}
]
}
}
```


客户端发送根变更通知

在根发生变化时，客户端会向服务器发送通知，通知示例如下：


```
{
"jsonrpc": "2.0",
"method": "notifications/roots/list_changed"
}
```


服务器在接到通知后，应该重新请求客户端获取根列表。

根交互示例

服务器与客户端关于根的交互示例如图2-13所示。

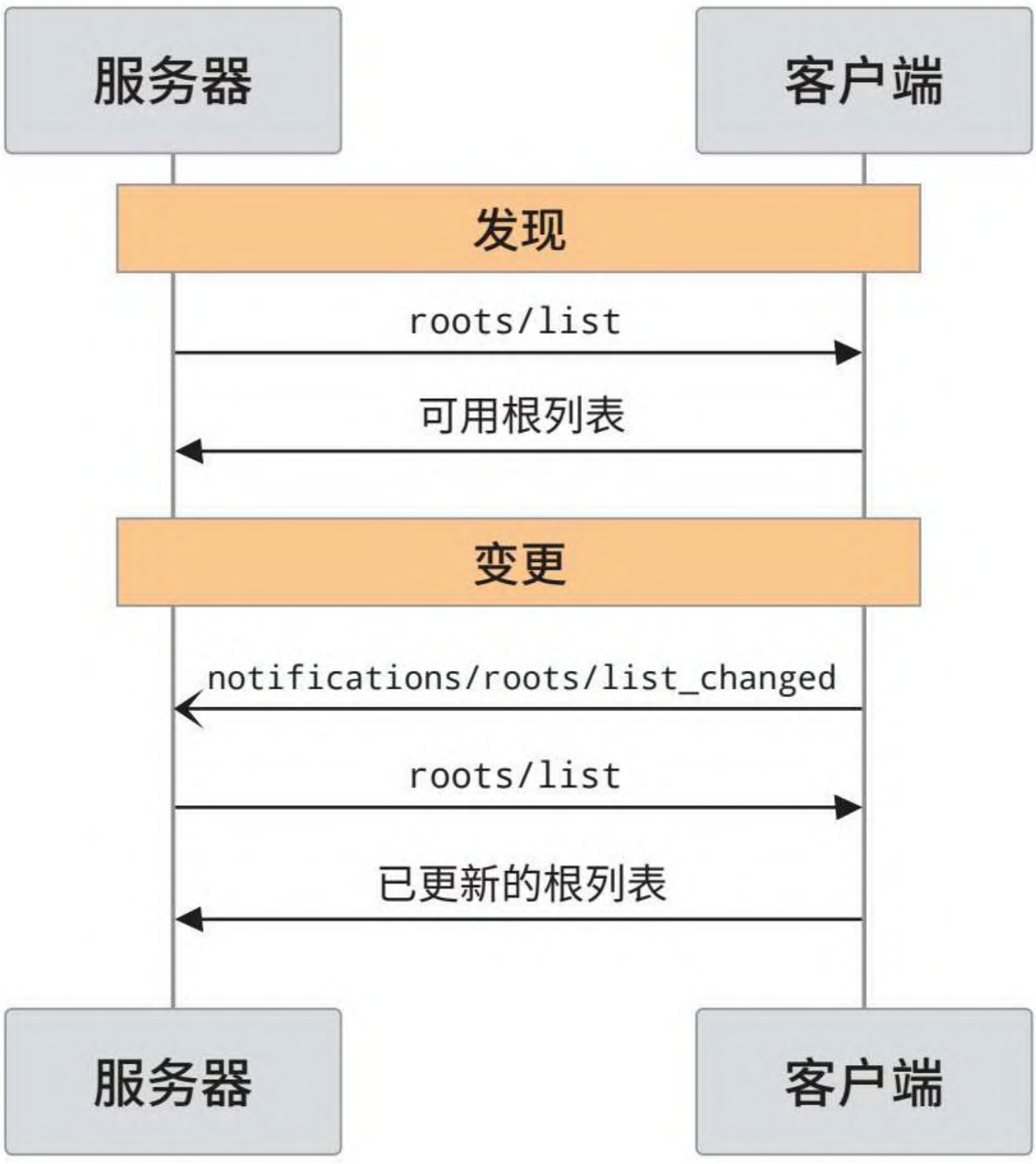


图2-13 服务器和客户端关于根的交互示例

2.6.2 采样及其交互示例

MCP为服务器提供了一种标准方式，通过客户端请求大模型采样（补全或生成内容）。此流程允许客户端保持对模型访问、选择和权限的控制，同时使服务器能够利用大模型的能力，而无须在服务器端设置大模型的API密钥。

采样的本质是客户端为服务器调用大模型提供代理服务。客户端采样能力的设计，主要出于以下几方面的考虑。

❑

控制权分离与安全性。由客户端控制大模型的访问、选择和权限，服务器只关注功能实现，无须关注大模型的接入问题。大模型的API密钥由客户端统一管理，以提高安全性。

❑

实现智能代理行为。服务器可以在处理复杂任务时通过客户端代理，动态调用大模型的能力。

❑

灵活的模型选择机制。采样允许服务器建议特定模型，并由客户端做最终决定。

❑

支持多模态。采样支持文本、音频和图像交互，为复杂的AI应用场景提供基础。

采样交互示例

客户端声明采样能力

支持采样能力的客户端需要在客户端定义时声明此项能力：

```
{
"capabilities": {
"sampling": {}
}
}
```

服务器获取采样结果

在运行过程中，服务器若需用到大模型的能力，会向客户端发起采样请求，请求示例如下：

```
{
"jsonrpc": "2.0",
"id": 1,
"method": "sampling/createMessage",
"params": {
```

```
"messages": [
{
"role": "user",
"content": {
"type": "text",
"text": "What is the capital of France?"
}
}
],
"modelPreferences": {
"hints": [
{
"name": "claude-3-sonnet"
}
],
"intelligencePriority": 0.8,
"speedPriority": 0.5
},
"systemPrompt": "You are a helpful assistant.",
"maxTokens": 100
}
}
```


客户端收到采样请求后，根据采样的消息内容，请求大模型生成回复，并给服务器回复采样结果，响应示例如下：

```
{
"jsonrpc": "2.0",
"id": 1,
"result": {
"role": "assistant",
"content": {
"type": "text",
"text": "The capital of France is Paris."
},
"model": "claude-3-sonnet-20240307",
"stopReason": "endTurn"
}
}
```


采样交互示例

服务器与客户端关于采样的交互示例如图2-14所示。

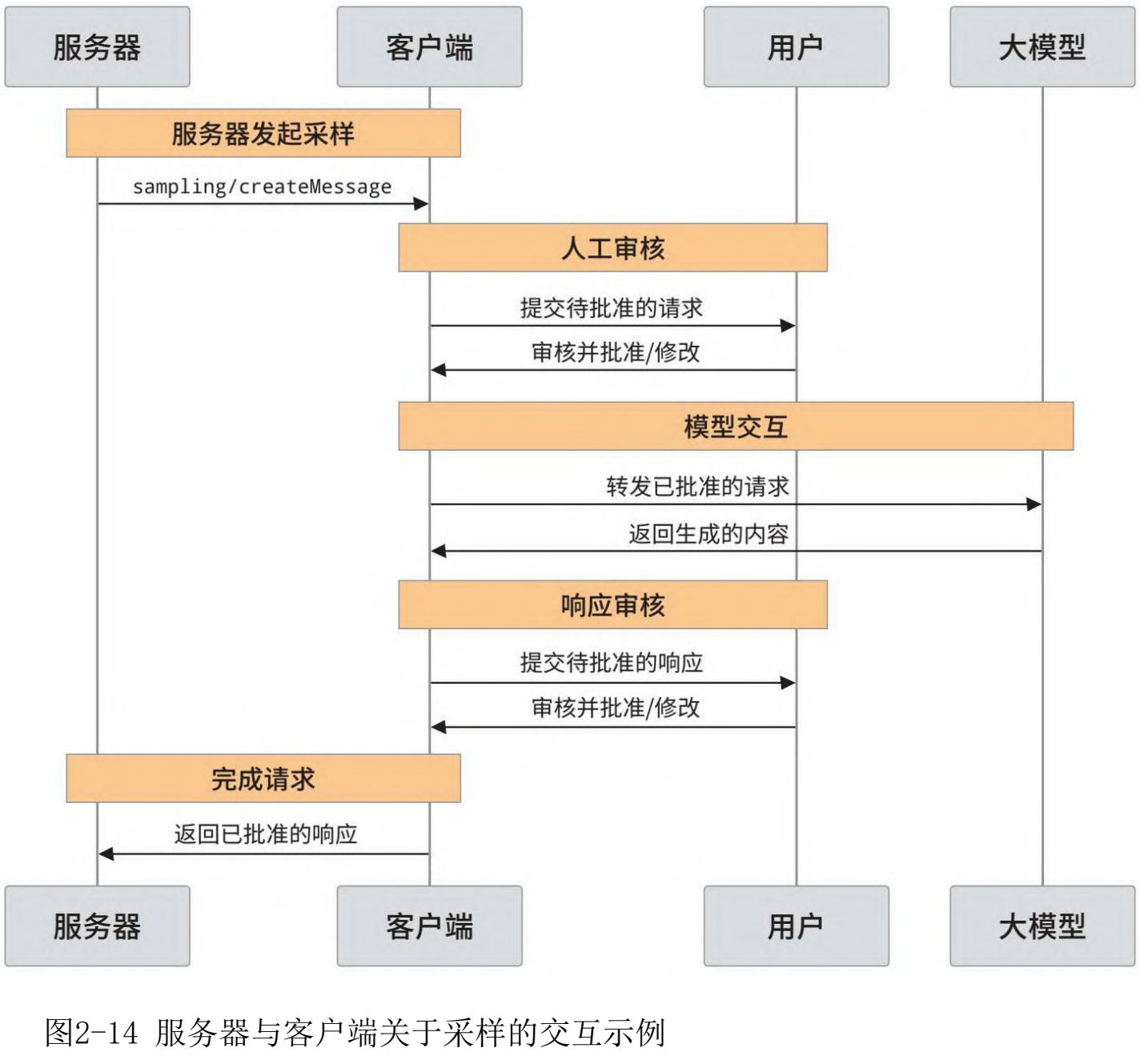


图2-14 服务器与客户端关于采样的交互示例

2.7 授权机制

通过授权机制，MCP为客户端安全访问服务器的受保护资源提供了标准方式，其核心是资源所有者授权客户端代表其向服务器请求受保护资源。

按照MCP的要求，基于stdio传输实现的服务器应该从环境变量读取授权凭证；而基于HTTP传输实现的服务器，应该基于OAuth 2.1实现授权机制。

2.7.1 OAuth 2.1

在介绍OAuth 2.1之前，我们先来简单了解一下OAuth 2.0 。

OAuth 2.0是一个开放标准的授权框架，允许第三方应用在不获取用户密码的情况下访问用户的受保护资源。OAuth 2.0为现代互联网应用提供了标准化的授权解决方案，平衡了安全性和易用性，是目前最主流的授权框架。OAuth 2.0广泛应用于社交账号（Google、GitHub、微信等）登录、API访问控制、第三方应用集成。

OAuth 2.1是OAuth 2.0授权框架的最新版本，进一步简化了授权流程，增强了安全性。

OAuth 2.1的核心概念和角色如下：

- 资源所有者（resource owner），通常是用户；
- 客户端（client），请求访问资源的应用；
- 资源服务器（resource server），存储受保护资源的服务器；
- 授权服务器（authorization server），验证身份并颁发访问令牌的服务器。

OAuth 2.1支持的令牌类型如下：

- 访问令牌（access token），用于访问受保护资源；
- 刷新令牌（refresh token），用于获取新的访问令牌；
- 授权码（authorization code），用于获取访问令牌。

OAuth 2.1最主要同时也是最推荐的授权方式是：授权码流程+PKCE（Proof Key for Code Exchange，代码交换证明密钥），其步骤如下：

1. 客户端生成代码验证器（code_verifier）和代码质询（code_challenge）；

2. 将用户重定向到授权服务器；

3. 用户授权后获得授权码；

4. 客户端使用授权码和代码验证器交换访问令牌。

2.7.2 MCP授权基本流程

MCP授权的基本流程如图2-15所示。

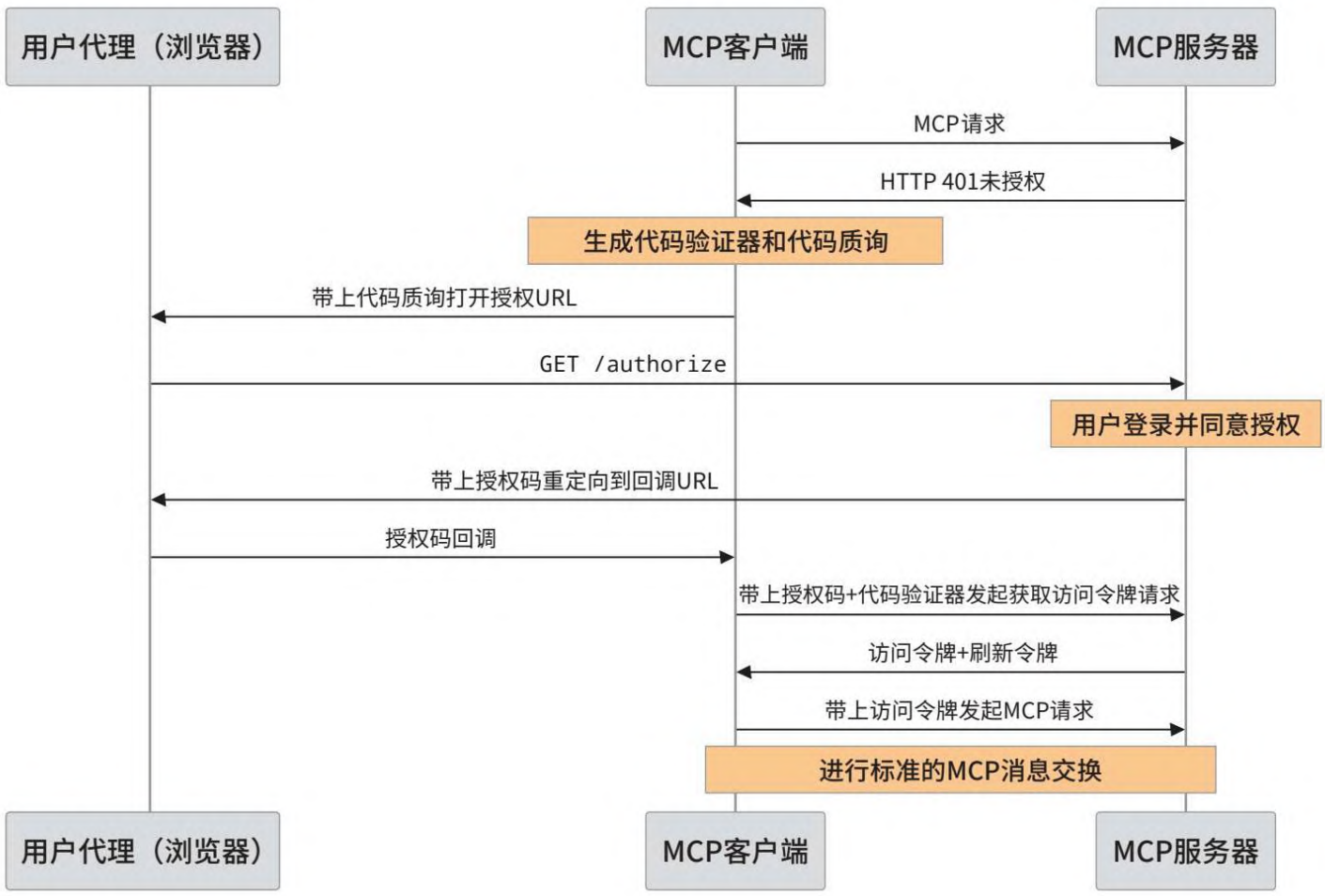


图2-15 MCP授权基本流程

该流程的核心步骤简单描述如下：

1. 初始请求阶段，MCP客户端向MCP服务器发送请求，收到401未授权响应；

2. PKCE准备，MCP客户端生成代码验证器和代码质询；

3. 用户授权，通过浏览器进行用户授权流程；

4. 令牌交换，使用授权码换取访问令牌；

5. 正常通信，使用访问令牌进行标准的MCP消息交换。

MCP授权的最终目的是获取访问MCP服务器资源所需的令牌。在授权流程中，多个步骤依赖URL跳转来传递授权凭证，因此需要特别注意URL参数的拼接方式，避免在跳转过程中丢失关键凭证信息。

2.7.3 授权服务器元数据发现

在上述MCP授权基本流程中，默认把MCP服务器也看作授权服务器，并使用默认的服务器端点完成了授权流程。

但在实际使用中，授权服务器可能独立于MCP服务器存在，授权端点也可能不同。因此，MCP客户端需要实现OAuth 2.0规定的授权服务器元数据发现流程，并根据元数据定义的授权端点完成授权流程。

授权服务器元数据发现流程如图2-16所示。

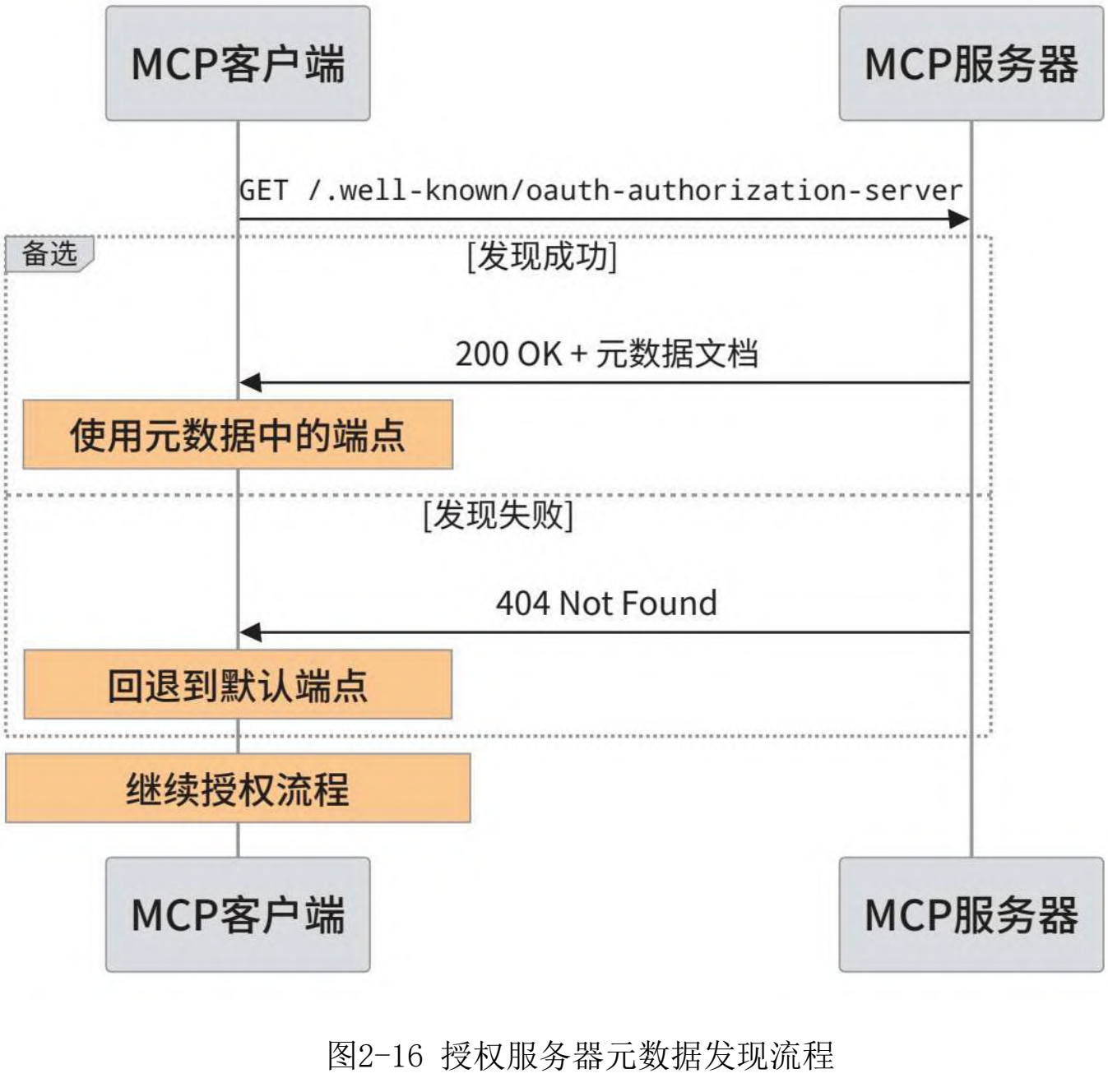


图2-16 授权服务器元数据发现流程

该流程的核心步骤简单描述如下：

1. MCP客户端向MCP服务器发送请求，获取授权服务器元数据；

2. 如果MCP服务器返回200响应，表示它实现了授权服务器元数据发现，MCP客户端就可以根据元数据中的授权端点继续后面的授权流程；

3. 如果MCP服务器返回404响应，表示它没有实现授权服务器元数据发现，这时MCP客户端使用默认的授权端点完成授权流程。

对于没有实现授权服务器元数据发现的MCP服务器，MCP客户端必须使用表2-4中所示的相对于授权服务器的基础URL的默认端点路径。

表2-4 默认端点路径

端　　点	默认路径	描　　述
授权端点	/authorize	用于授权请求
令牌端点	/token	用于令牌交换和刷新
注册端点	/register	用于动态客户端注册

2.7.4 动态客户端注册

在上述授权服务器元数据发现流程中，MCP客户端从MCP服务器元数据中获取了授权端点，比如/authorize。

按照2.7.2节介绍的基本授权流程，客户端需要将用户重定向到授权端点，并携带客户端参数。在常见的Web应用OAuth授权场景中，客户端参数需要通过提前在服务器注册获得（比如在GitHub创建应用后得到client_id和client_secret）。

然而，MCP服务器是海量的，如果MCP客户端需要为每个MCP服务器提前申请授权参数，成本会非常高。因此，MCP客户端和MCP服务器需要支持OAuth 2.0规定的动态客户端注册流程。该流程的核心步骤简单描述如下：

1. MCP客户端首先通过元数据发现获取注册端点，比如/register；

2. MCP客户端向注册端点发送请求，提交客户端信息（名称、版本、回调地址等）；

3. MCP服务器完成客户端注册，并返回授权参数（client_id和client_secret等）。



在动态客户端注册场景中，MCP服务器应严格验证客户端提交的元数据信息，包括重定向URI、授权类型、作用域等。同时，应加强对注册端点的保护，防止恶意客户端随意注册。常见的保护措施包括频率限制、白名单机制等。

加入授权服务器元数据发现和动态客户端注册流程后，完整的MCP授权流程如图2-17所示。

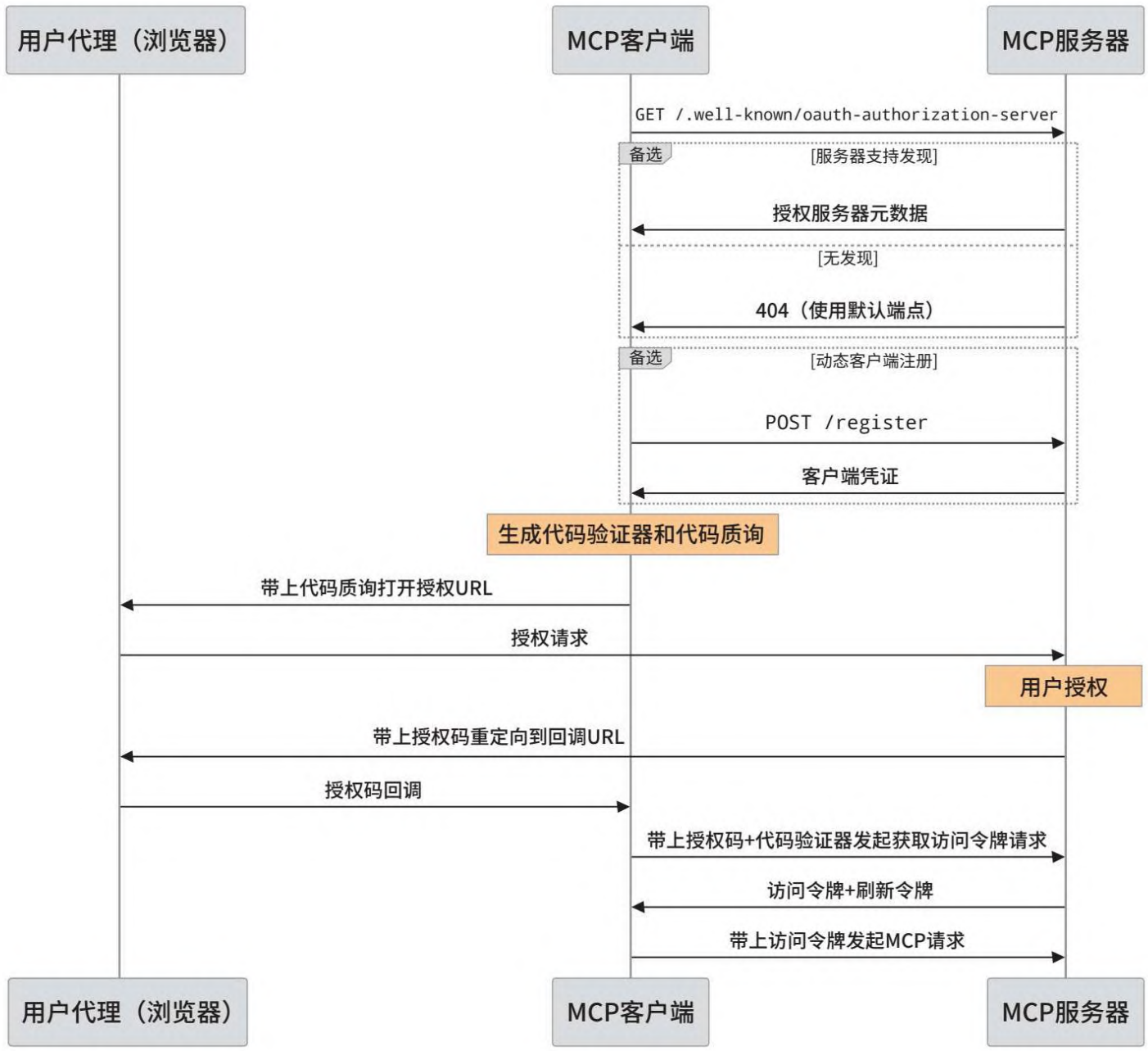


图2-17 完整的MCP授权流程

2.7.5 访问令牌使用

在通过授权流程得到访问令牌后，MCP客户端在访问MCP服务器的受保护资源时，需要携带访问令牌。发送的HTTP请求示例如下：

```
GET /v1/contexts HTTP/1.1

Host: mcp.example.com

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

MCP服务器需要验证MCP客户端访问令牌的合法性，并根据访问令牌的权限决定是否返回受保护资源。如果MCP客户端请求的令牌无效或已过期，MCP服务器必须返回401未授权响应。

2.7.6 第三方授权流程

MCP服务器还可以通过第三方授权服务器支持委托授权。在这种情况下，MCP服务器同时充当OAuth客户端（之于第三方授权服务器）和OAuth授权服务器（之于MCP客户端）。

第三方授权流程如图2-18所示。

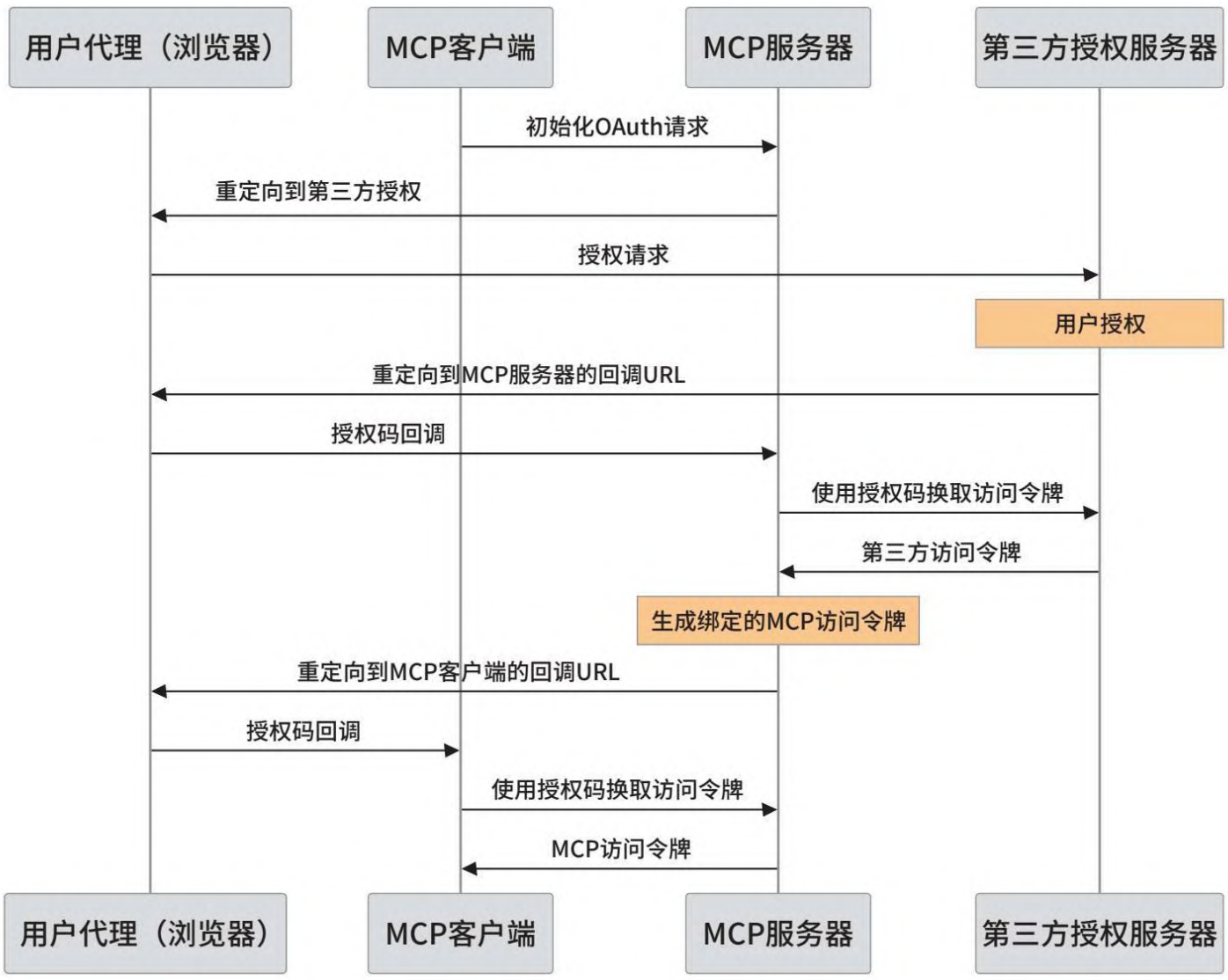


图2-18 第三方授权流程

该流程的核心步骤简单描述如下：

1. MCP客户端与MCP服务器启动标准OAuth流程；
2. MCP服务器将用户重定向到第三方授权服务器；
3. 用户通过第三方服务器进行授权；
4. 第三方服务器携带授权码重定向到MCP服务器；
5. MCP服务器用授权码交换第三方访问令牌；

6. MCP服务器生成自身绑定到第三方会话的访问令牌；

7. MCP服务器与MCP客户端完成授权流程。

2.8 小结

本章主要介绍了MCP的架构、通信基础、生命周期、传输机制和授权机制，旨在让读者系统性了解MCP的协议内容和设计原则。

本章为后续几章提供了理论支撑。相信通过本章的学习，读者已经掌握了MCP的底层原理，为接下来开发MCP服务器和MCP客户端做好了准备。

第3章

MCP服务器开发

在了解了MCP的运作原理和协议架构之后，你是不是已经跃跃欲试，想要开发一个MCP服务器了？

在本章中，我们计划通过两个实际的案例来演示MCP服务器开发的完整流程。第一个案例开发一个记笔记MCP服务器，支持通过对话的方式记笔记；第二个案例开发一个虚拟试衣MCP服务器，支持通过大模型客户端快速试衣。

3.1　MCP服务器开发流程

在开始案例讲解之前，我们先来准备开发MCP服务器所需的环境和调试用的大模型客户端，并简要梳理开发的核心流程。

3.1.1　前置准备

1.　选择开发语言

MCP支持使用任意编程语言开发MCP服务器。MCP官方目前开放了多种编程语言的SDK（Software　Development　Kit，软件开发工具包），包括：

- Python　SDK
- TypeScript　SDK
- Java　SDK
- Kotlin　SDK
- C#　SDK
- Rust　SDK

我们可以选择其中一种SDK，使用对应的编程语言来开发MCP服务器。除此之外，MCP还支持使用其他编程语言来开发MCP服务器，比如以下是一些第三方的SDK。

- mark3labs/mcp-go：使用Go语言开发MCP服务器的SDK。
- goplus/mcp：使用Go+语言开发MCP服务器的SDK。
- logiscape/mcp-sdk-php：使用PHP语言开发MCP服务器的SDK。

MCP发布之后，开源社区的开发者积极贡献各类开发工具，让MCP服务器的开发变得越来越简单。我们只需选择自己擅长的编程语言和对应的SDK，即可快速上手MCP服务器开发。

在本章中，我们选择TypeScript作为开发语言。

2. 初始化开发环境

根据选择的开发语言，我们在自己的电脑上初始化对应的开发环境。这里选择的是TypeScript，因此在进行MCP服务器开发之前，我们需要安装Node.js及其包管理工具npm。

安装fnm

fnm是一款Node.js版本管理工具。进入fnm在GitHub上的官方仓库，按照fnm的说明文档，根据操作系统类型选择对应的安装方式，并在终端软件执行命令。

等待fnm安装完之后，在终端软件执行命令，查看fnm的版本以验证fnm是否安装成功：

```
#  查看fnm安装路径

which  fnm

#  查看fnm版本

fnm  --version

#  查看fnm使用说明

fnm  --help
```

安装Node.js

在终端软件执行命令，通过fnm安装指定版本的Node.js：

```
#  查看所有可安装的Node.js版本

fnm  ls-remote

#  选择一个版本进行安装

fnm  install  v22.2.0
```

按照MCP官方的要求，Node.js的版本不低于16，比如我们可以选择安装v22.2.0。

等待Node.js安装完成，在终端软件执行命令，启用指定版本的Node.js、查看Node.js的版本以验证Node.js是否安装成功：

```
#  查看所有已安装的Node.js版本

fnm  list

#  设置默认的Node.js版本

fnm  use  v22.2.0

#  查看Node.js安装路径

which  node

#  查看Node.js版本

node  --version
```

安装npm

npm是Node.js官方集成的一个包管理工具，用于安装Node.js第三方库。

npx也是Node.js官方集成的一个工具，可以直接运行npm包中的命令，而不需要全局或本地安装npm包。

在终端软件执行命令以验证npm和npx是否可用：

```
#  查看npm安装路径

which  npm

#  查看npm版本

npm  --version

#  查看npx安装路径

which  npx

#  查看npx版本

npx  --version
```

npm和npx都安装成功之后，接下来我们就可以使用TypeScript开发MCP服务器了。

3. 安装大模型客户端

为了在开发MCP服务器的过程中以对话的方式调试MCP服务器的功能，你需要安装一些（至少一个）大模型客户端应用，这些应用需要支持MCP，即能够配置MCP服务器。

我推荐的大模型客户端应用包括：

- 来自MCP官方的Claude（桌面客户端）；
- VS Code、Cursor、Windsurf等AI编辑器；
- ChatMCP、ChatWise、Cherry Studio、DeepChat等对话客户端。

本章设计的两个MCP服务器开发案例，分别使用Cursor和Claude作为大模型客户端，用于MCP服务器的功能调试。读者可以进入Cursor和Claude官网自行安装。

3.1.2 核心步骤

MCP服务器开发流程包括以下几个核心步骤。

1. 创建MCP服务器项目

MCP官方提供了一个叫作create-server的命令行工具，通过执行该命令可以快速创建MCP服务器项目，自动生成组织良好的代码结构，在此基础上，我们可以快速启动MCP服务器的开发工作。假设我们想开发一个查询天气的MCP服务器，取名为weather-mcp，就可以在终端软件执行以下命令创建MCP服务器：

```
npx @modelcontextprotocol/create-server weather-mcp
```

2. 实现MCP服务器的业务逻辑

使用官方的create-server工具创建MCP服务器项目时，会自动安装TypeScript SDK：@modelcontextprotocol/sdk。项目的核心逻辑位于src/index.ts文件，我们只需修改该文件即可实现具体的业务功能。

3. 调试MCP服务器

MCP官方提供了一个名为MCP Inspector的调试工具，可以帮助开发者调试MCP服务器内部实现的功能。

在MCP服务器的根目录下执行以下命令，快速启动MCP Inspector：

```
npx @modelcontextprotocol/inspector node build/index.js
```

终端会输出一个本地运行的地址，点击该地址即可进入MCP Inspector，连接MCP服务器成功之后即可在面板中调试MCP服务器内部实现的功能。



请注意，在MCP服务器开发过程中，开发者通常会边开发边调试。因此，一般是在MCP服务器项目创建成功后就启动MCP Inspector，而不是等到MCP服务器内部功能实现完成后才启动。在本章后面的案例中，我们会详细演示。另外，我们也会在大模型客户端中配置正在开发的MCP服务器，以测试其内部实现的功能。

4. 发布MCP服务器

将开发完成的MCP服务器公开发布，不仅能让更多用户在更多场景中使用，也能为开发者带来成就感。

通常我们会优先将MCP服务器源代码上传至GitHub，以开源形式发布；此外，也可以将其发布到npm平台，便于用户通过npx命令快速配置与运行；还可以提交到第三方MCP应用市场（如MCP.so），以便更多用户发现和使用。在后续案例中，我们将详细讲解这几种发布方式。

MCP.so是全球知名的第三方MCP应用市场，收录了全世界开发者开发的优质的MCP应用（包括MCP服务器和MCP客户端）。MCP.so的网站主页如图3-1所示。

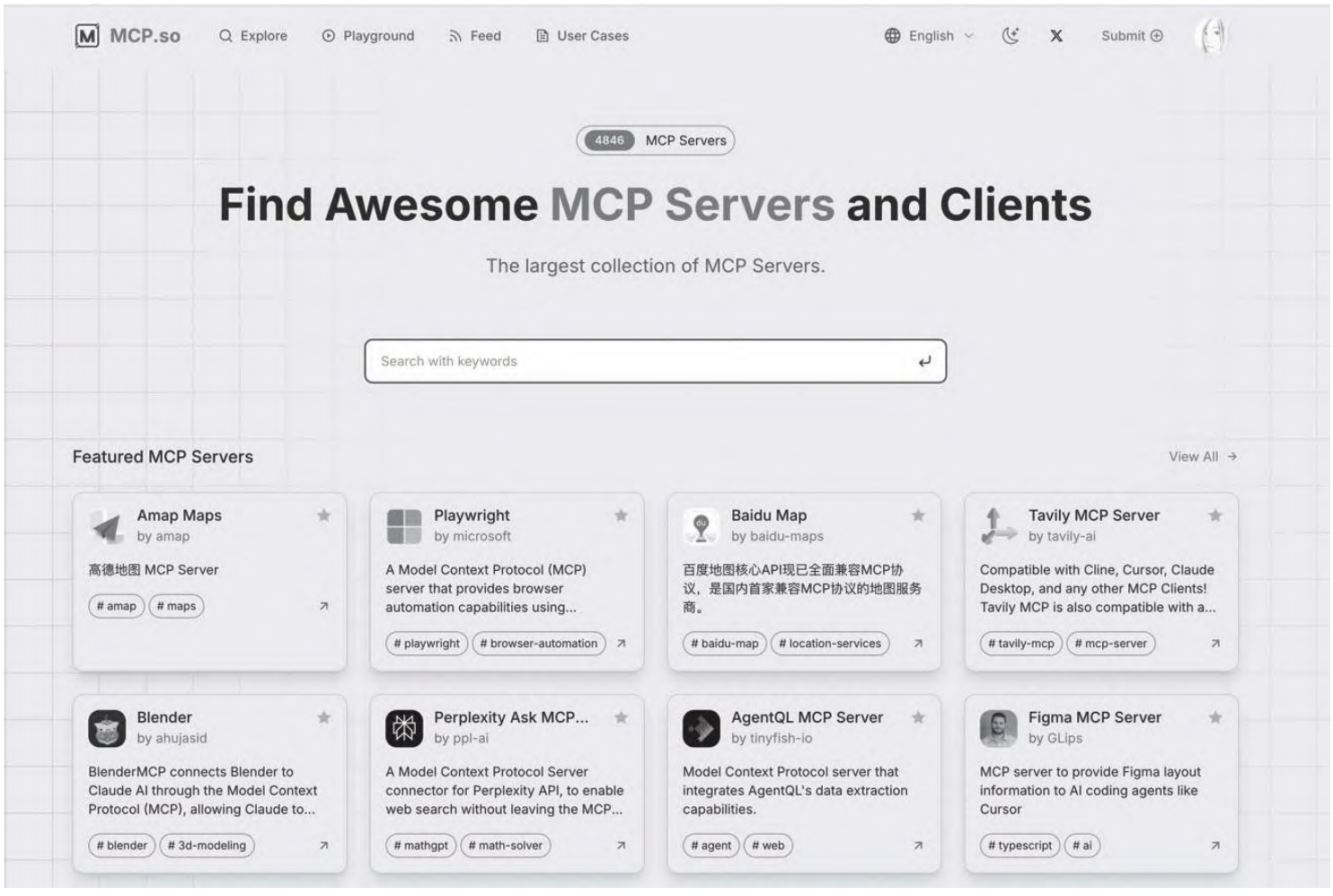


图3-1 MCP.so的网站主页

在MCP.so网站上，用户可以输入关键词，搜索目标MCP服务器，并查看MCP服务器内部的工具列表；用户也可以点击某个工具进入调试面板，对工具实现的功能进行调试。

在准备好MCP服务器开发环境，了解了MCP服务器开发的核心流程之后，接下来，我们就通过具体的案例来讲解MCP服务器开发的完整流程。

3.2　案例1：开发flomo　MCP服务器，高效记笔记

3.2.1　开发目标

在本节中，我们计划开发一个名为flomo-mcp的MCP服务器，对接flomo浮墨笔记（简称flomo）的API，实现在任意大模型客户端，以对话的形式将笔记内容写入flomo，从而大大提升碎片化知识管理的效率。

flomo是一款专注于碎片化知识记录的轻量级笔记工具，由中国团队开发。它的设计摒弃了传统笔记软件的复杂功能，强调用最简单的方式捕捉灵感、想法和日常思考。flomo产品官网如图3-2所示。



图3-2　flomo产品官网

本案例旨在讲解如何开发以API对接为主的MCP服务器。举一反三，如果你习惯使用Notion、Obsidian等笔记软件，也可以参考此案例，实现此类支持对话式记笔记的MCP服务器。

3.2.2　前置准备

为实现MCP服务器与flomo　API的稳定对接，需提前完成账号权限配置与API功能验证。请按顺序完成以下操作。

1. 在个人主页左上角，点击个人账号下拉列表中的“扩展中心 & API”进入“记录API”控制台，如图3-3所示。



图3-3 flomo记录API控制台

2. flomo记录API是会员专享功能，你需要先升级成PRO会员（注册即可领取14天的免费会员）。

3. 升级成PRO会员之后，复制你的专属记录API链接。

4. 打开终端软件，通过curl命令请求API：

```
curl -X POST https://flomoapp.com/iwh/xxx/xxxxxxxxxxxxxxx/ \
-H 'Content-Type: application/json' \
-d '{"content": "从现在开始，我要学习MCP Server开发"}'
```

回到flomo笔记列表（个人主页），可以看到刚刚通过终端调用API写入的笔记，如图3-4所示。



图3-4　查看通过终端调用API写入的笔记

接下来，我们使用Node.js来实现flomo-mcp。

3.2.3　创建flomo　MCP服务器项目

使用MCP官方提供的命令行工具create-server，通过以下命令创建项目：

```
npx  @modelcontextprotocol/create-server  flomo-mcp
```

随后，命令行会提示输入MCP服务器的名称、描述信息，并询问是否为Claude.app安装该服务器。确认信息后，系统将自动完成项目的初始化并给出后续的操作指引，如图3-5所示。

```
~/code/all-in-aigc/chatmcp (30.793s)
npx @modelcontextprotocol/create-server flomo-mcp

? What is the name of your MCP server? flomo-mcp
? What is the description of your server? write notes to flomo
? Would you like to install this server for Claude.app? No
✔ MCP server created successfully!

Next steps:
  cd flomo-mcp
  npm install
  npm run build  # or: npm run watch
  npm link       # optional, to make available globally
```

图3-5　创建flomo　MCP服务器项目



虽然MCP官方文档提供了手动创建服务器项目的示例，但对于初次上手MCP服务器开发的朋友，我更建议使用官方SDK快速生成项目模板，并在此基础上进行开

发，这样能大幅提升开发效率。

项目创建完成后，按照提示进入flomo-mcp的目录并安装项目依赖：

```
cd  flomo-mcp

npm  install
```

用代码编辑器打开创建好的flomo-mcp，可以看到默认生成的项目代码结构，如图3-6所示。

❑

src/index.ts是flomo-mcp的主源代码文件，实现flomo-mcp的具体业务逻辑；

❑

build/index.js是flomo-mcp源代码编译后的可执行文件，调试和发布上线都会用到。

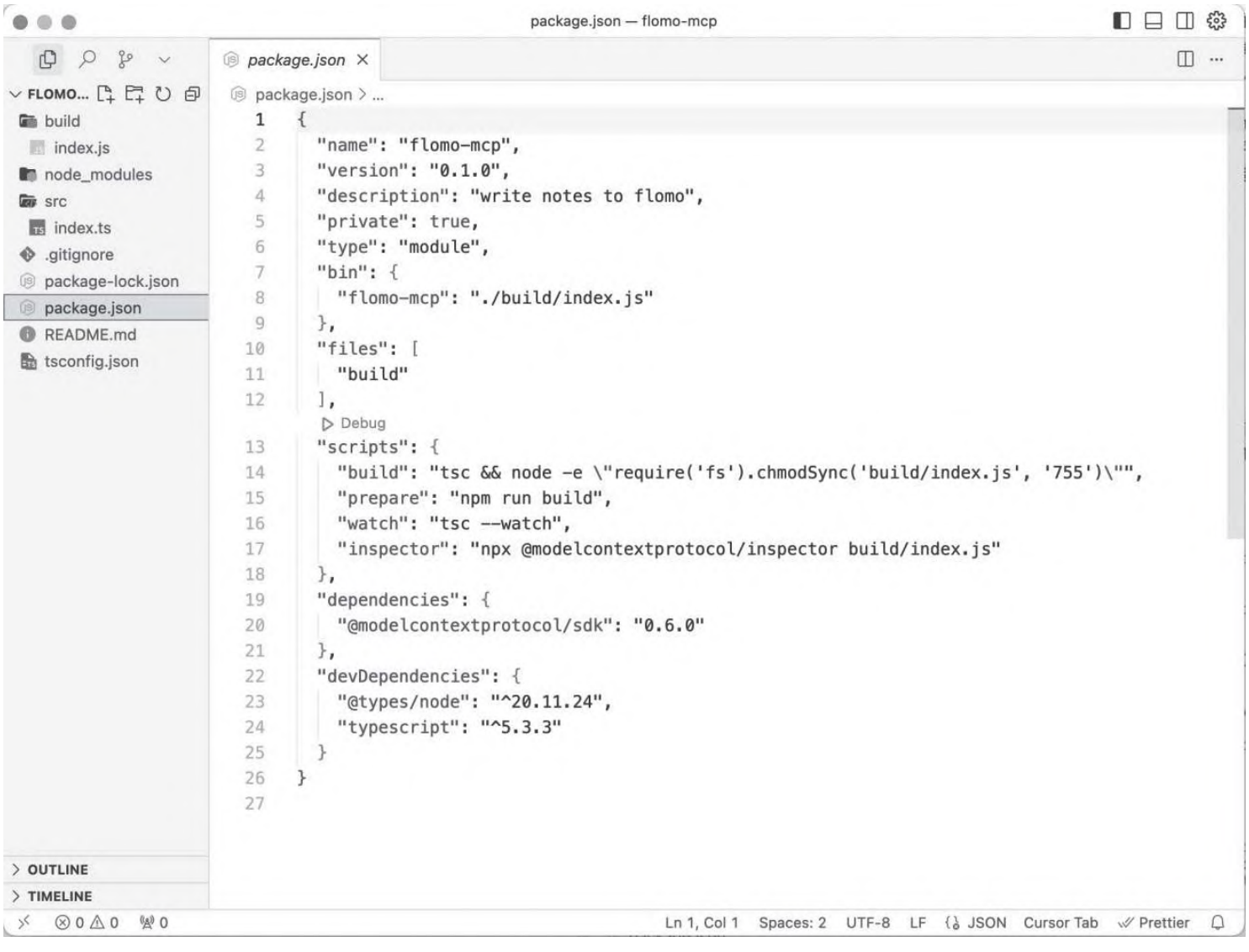


图3-6　MCP服务器flomo-mcp的项目代码结构

3.2.4　调试flomo　MCP服务器

在flomo-mcp项目目录下，使用以下两个命令进行调试。

npm run watch

运行npm run watch启动一个监听服务，监听src/index.ts源代码文件的变动，并实时将其编译成build/index.js可执行文件，用于后续运行和调试。

npm run inspector

运行npm run inspector实际执行的命令是：

```
npx @modelcontextprotocol/inspector build/index.js
```

该命令调用了MCP官方提供的调试工具MCP Inspector，如图3-7所示。

```
~/code/all-in-aigc/chatmcp/flomo-mcp
npm run inspector

> flomo-mcp@0.1.0 inspector
> npx @modelcontextprotocol/inspector build/index.js

Starting MCP inspector...
⚙️ Proxy server listening on port 6277
🔍 MCP Inspector is up and running at http://127.0.0.1:6274 🚀
```

图3-7 启动并连接MCP Inspector

运行npm run inspector命令后，终端会输出一个本地运行的地址（见图3-7最后一行），点击该地址即可进入MCP Inspector。

在MCP Inspector中，左侧菜单栏中flomo-mcp的相关参数已经默认加载，点击Connect可以连接到flomo-mcp进行调试，显示Connected表示连接成功，如图3-8所示。

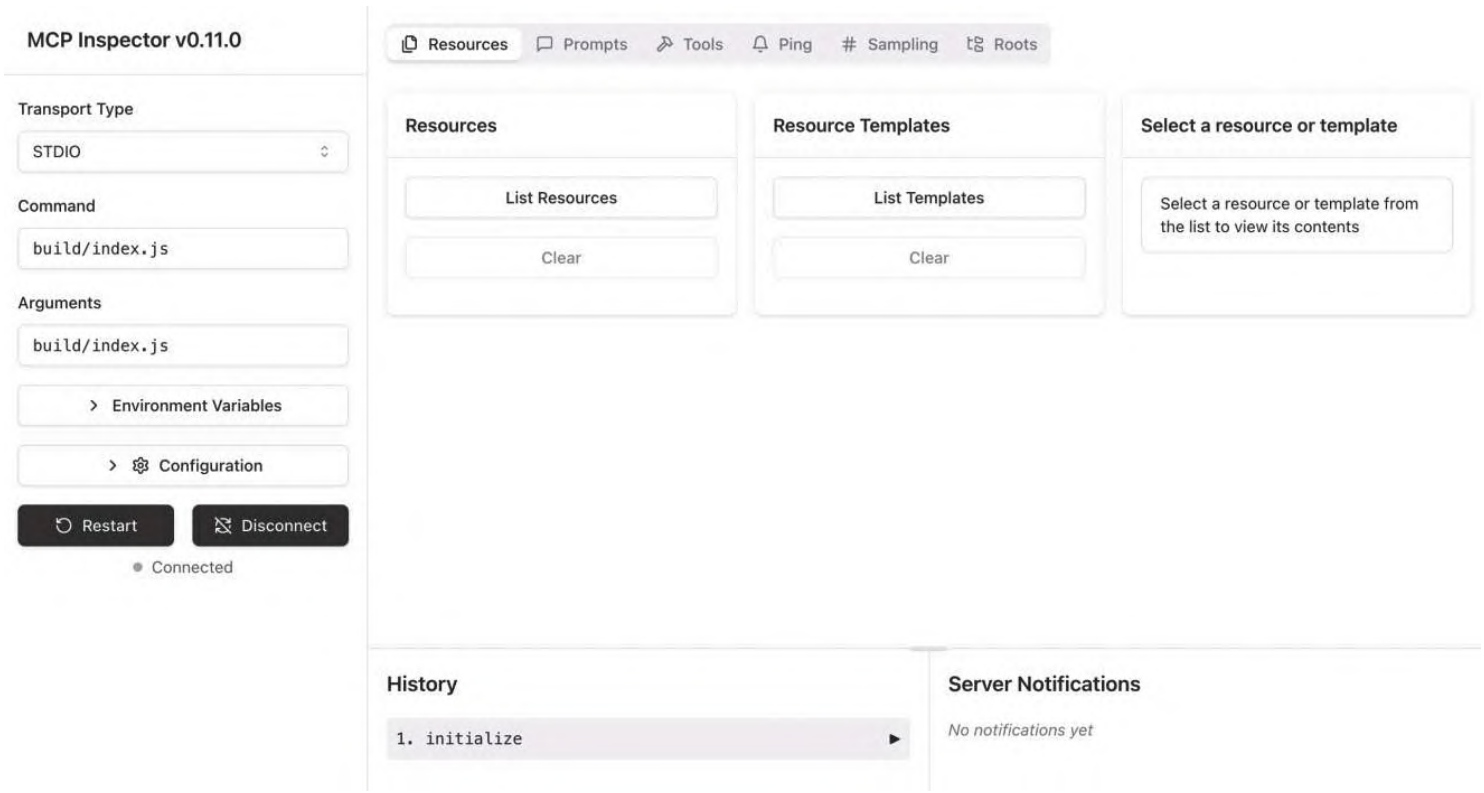


图3-8　在MCP　Inspector中调试MCP服务器

我们可以在MCP　Inspector中设置MCP服务器的启动参数Arguments和环境变量Environment　Variables。两者的参数值可以在MCP服务器的实现逻辑中读取。

在MCP　Inspector的右侧区域，可以对MCP服务器内部实现的三大能力进行调试。

- Resources（资源）：MCP服务器内部定义的资源。
- Prompts（提示词）：MCP服务器内部定义的提示词。
- Tools（工具）：MCP服务器内部定义的工具。

要查看MCP服务器内部定义的所有工具，可以在右侧的Tools选项卡中点击List　Tools。请注意，通过官方命令行工具创建的MCP服务器，默认会生成一个create_note工具。

选择Tools下的某个工具（比如create_note）并填写请求参数，随后点击Run　Tool即可给此工具发送请求，得到响应数据，如图3-9所示。

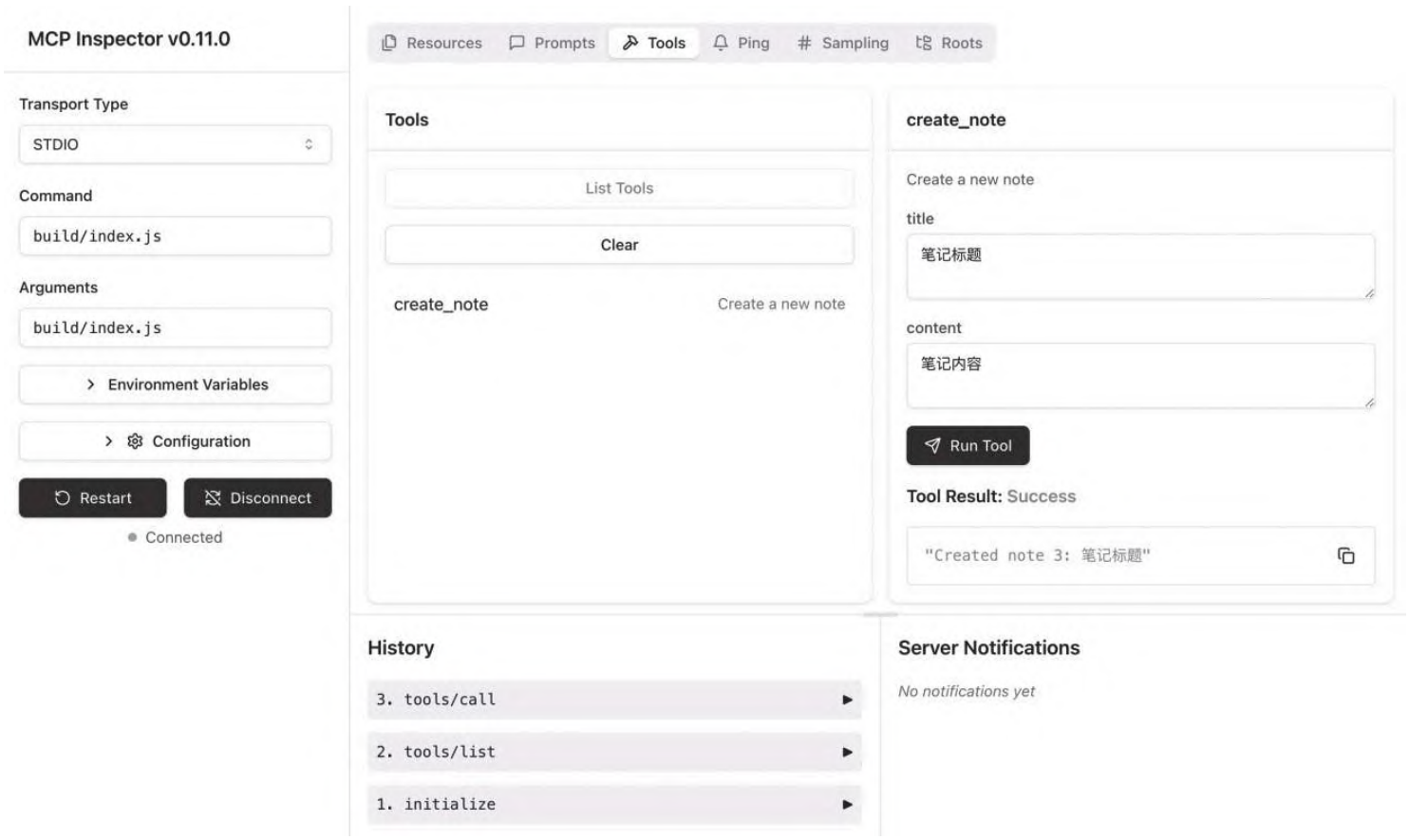


图3-9　在MCP　Inspector中调用工具

3.2.5　实现flomo　MCP服务器的业务逻辑

在3.2.2节中，我们准备好了一个flomo记录API的专属链接，并且通过命令行调用API进行了调试，跑通了记笔记的流程。

接下来，我们在flomo-mcp中实现一个工具，通过该工具调用flomo记录API记笔记。

1.　定义flomo-mcp的信息

打开src/index.ts文件，可以看到默认生成的代码通过new　Server创建了一个Server。

```
const  server  =  new  Server(
    {
        name:  "flomo-mcp",
        version:  "0.1.0",
    },
    {
```

```
        capabilities:  {
            resources:  {},
            tools:  {},
            prompts:  {},
        },
    }
);
```

鉴于flomo-mcp只需要实现一个记笔记的工具，不会实现其他功能，我们可以修改代码，定义flomo-mcp只有Tools能力。我们还可以自定义版本号，比如从0.0.1开始。

```
const  server  =  new  Server(
    {
        name:  "flomo-mcp",
        version:  "0.0.1",
    },
    {
        capabilities:  {
            tools:  {},
        },
    }
);
```

2. 定义flomo-mcp的工具列表

修改src/index.ts文件中默认生成的获取工具列表（ListTools）的逻辑，定义一个write_note方法，功能描述使用英文撰写，说明这个工具的主要作用是把笔记发送到flomo。

write_note只接受一个参数：content（字符串，必填），即笔记的正文内容需要是Markdown格式的文本。

ListTools的定义如下：

```
server.setRequestHandler(ListToolsRequestSchema,  async  ()  =>  {
    return  {
        tools:  [
            {
                name:  "write_note",
                description:  "Write  note  to  flomo",
                inputSchema:  {
                    type:  "object",
                    properties:  {
                        content:  {
                            type:  "string",
                            description:  "Text  content  of  the  note  with  Markdown  f
ormat",
                        },
                    },
                    required:  ["content"],
                },
            },
        ],
    };
});
```

3. 实现flomo-mcp的工具逻辑

先创建一个新的文件src/flomo.ts，定义一个对接flomo API的FlomoClient类。该类通过构造方法接收外部传入的apiUrl，并实现一个writeNote方法，用于将指定内容作为笔记写入flomo：

```
/**
  *  对接flomo  API的客户端类
  */
export  class  FlomoClient  {
    private  readonly  apiUrl:  string;
```

```
/**
  *  构造函数
  *  @param  apiUrl  -  专属记录API
  */
constructor({  apiUrl  }:  {  apiUrl:  string  })  {
    this.apiUrl  =  apiUrl;
}


/**
  *  写笔记方法
  *  @param  content  -  笔记内容
  *  @returns  记录API响应内容
  */
async  writeNote({  content  }:  {  content:  string  })  {
    try  {
        if  (!content)  {
            throw  new  Error("invalid  content");
        }

        const  req  =  {
```

```
    content,
};


const resp = await fetch(this.apiUrl, {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
    },
    body: JSON.stringify(req),
});

if (!resp.ok) {
    throw new Error(`request failed with status ${resp.statusText}`);
}
```

```
            return  resp.json();
        }  catch  (e)  {
            throw  e;
        }
    }
}
```

然后，我们修改CallTool的逻辑：接到write_note工具调用请求时，先获取传递的参数content，再调用FlomoClient的writeNote方法将笔记发送到flomo：

```
import  {  FlomoClient  }  from  "./flomo.js";

server.setRequestHandler(CallToolRequestSchema,  async  (request)  =>  {
    switch  (request.params.name)  {
        case  "write_note":  {
            const  content  =  String(request.params.arguments?.content);
            if  (!content)  {
                throw  new  Error("Content  is  required");
            }

            const  apiUrl  =
                "https://flomoapp.com/iwh/xxx/xxxxxxxxxxxxxxxx/";
```

```
const flomo = new FlomoClient({ apiUrl });

const result = await flomo.writeNote({ content });
```

```
                return {
                    content: [
                        {
                            type: "text",
                            text: `Write note to flomo success: ${JSON.stringify(result
)}`,
                        },
                    ],
                };
            }
        default:
            throw new Error("Unknown tool");
    }
```

4. 调试flomo-mcp的工具

在MCP Inspector中，点开flomo-mcp的Tools选项卡，选择write_note工具，填入content参数，点击Run Tool，查看工具的调用结果，如图3-10所示。

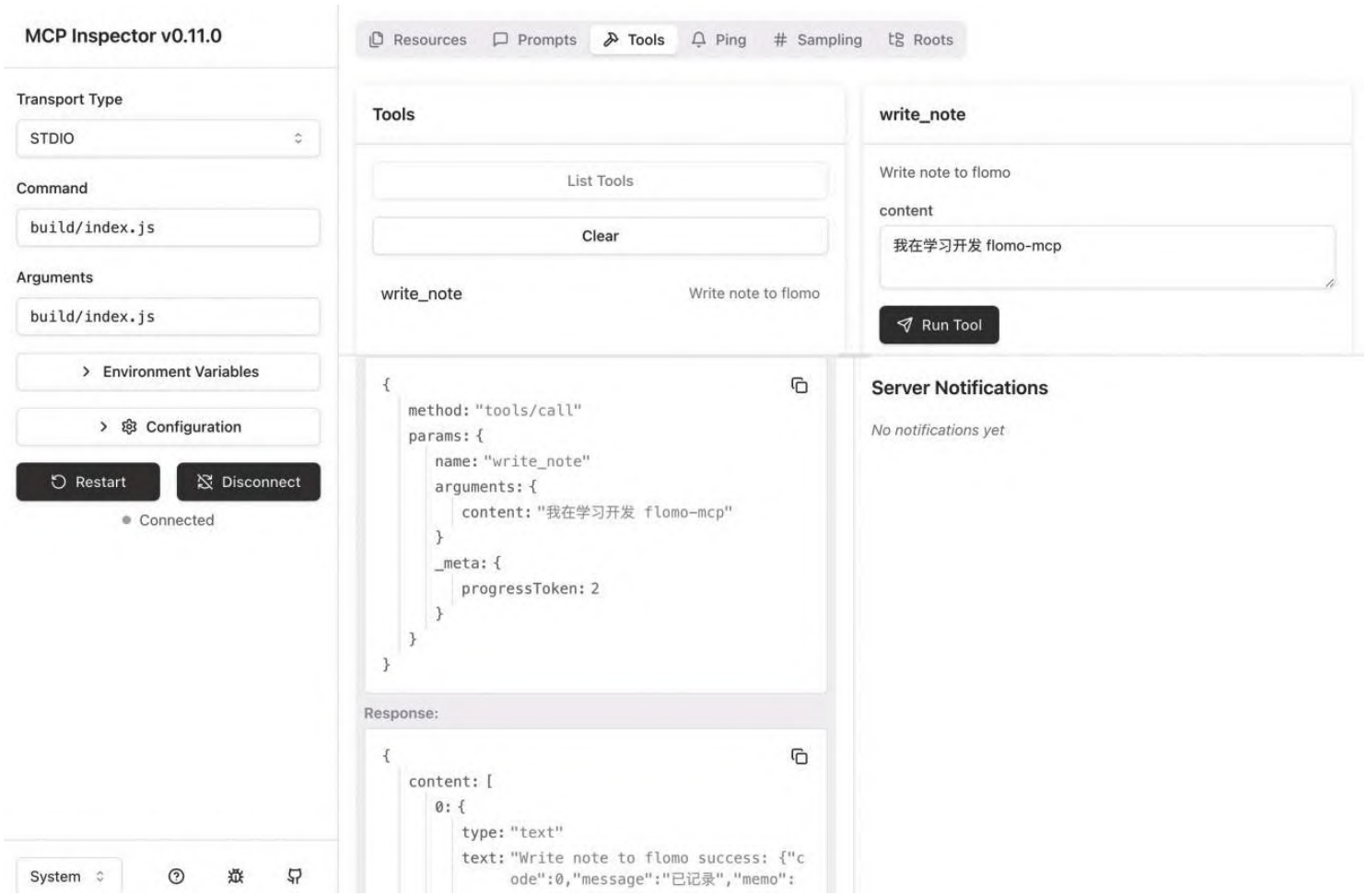


图3-10　调试flomo-mcp的write_note工具

可以看到，flomo记录API返回了code：　0，　message：　"已记录"，表示笔记已记录到flomo中。

登录flomo后台，可以看到笔记确实已记录，如图3-11所示。



图3-11　在flomo后台查看笔记

3.2.6　优化flomo　MCP服务器

通过前面的步骤，我们在flomo-mcp中实现了基本的业务逻辑，在发布上线之前，还可以针对一些小问题进行优化。

1. 删除冗余代码

由于flomo-mcp的核心逻辑只有write_note这一个工具，因此我们可以把src/index.ts文件中默认生成的跟工具无关的代码都删掉，包括以下几部分的逻辑：

- ListResources
- ReadResource
- ListPrompts
- GetPrompt

2. 修改响应内容

在3.2.4节中，通过调用create_note写入笔记成功后，我们把API的响应数据通过JSON编码后直接返回，这种格式的响应数据对于前端调用方不够友好。我们可以改成返回flomo笔记详情页的链接，格式类似于：

```
https://v.flomoapp.com/mine/?memo_id=xxx
```

因此，修改一下

```
server.setRequestHandler(CallToolRequestSchema,  async(request)  =>  {
    //  …
})
```

里响应数据的逻辑：

```
const  flomo  =  new  FlomoClient({  apiUrl  });
const  result  =  await  flomo.writeNote({  content  });



if  (!result.memo  ||  !result.memo.slug)  {
    throw  new  Error(
        `Failed  to  write  note  to  flomo:  ${result?.message  ||  "unknown  error"}`
```

```
        );

    }
```

```
const flomoUrl = `https://v.flomoapp.com/mine/?memo_id=${result.memo.slug}`;



return {
    content: [
        {
            type: "text",
            text: `Write note to flomo success, view it at: ${flomoUrl}`,
        },
    ],
};
```

3. 修改鉴权参数

在前面CallTool的实现逻辑中，创建FlomoClient时传递的apiUrl参数是硬编码的。由于不同的用户使用flomo-mcp时，需要使用自己的flomo记录API，因此apiUrl的设置需要改成动态传参。

有两种方法可以实现动态传参。

通过MCP服务器启动命令传递参数

实现一个parseArgs函数，在启动flomo-mcp的时候，从命令行参数中读取flomo_api_url：

```
/**
 * 解析命令行参数
 * 示例: node index.js --flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/
 */
```

```
function parseArgs() {
    const args: Record<string, string> = {};
    process.argv.slice(2).forEach((arg) => {
        if (arg.startsWith("--")) {
            const [key, value] = arg.slice(2).split("=");
            args[key] = value;
        }
    });
    return args;
}


const args = parseArgs();
const apiUrl = args.flomo_api_url || "";
```

修改CallTool的内部实现逻辑，判断apiUrl是否传递，动态传入apiUrl参数，用于记笔记：

```
if (!apiUrl) {
    throw new Error("flomo API URL not set");
}
```

```
const content = String(request.params.arguments?.content);
if (!content) {
    throw new Error("Content is required");
}

const flomo = new FlomoClient({ apiUrl });
const result = await flomo.writeNote({ content });
```

在MCP Inspector的左侧区域，将Arguments输入框内的参数设置为：--flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/。

之后按照前面描述的方法请求write_note工具，写入笔记成功，如图3-12所示。



图3-12　通过MCP服务器启动命令传递参数

对应的MCP服务器的启动命令是：

```
node build/index.js --flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/
```

通过环境变量传递参数

也可以通过环境变量设置动态参数，作为命令行读取参数的一个补充。

修改动态参数的读取逻辑，优先读取命令行参数。如果命令行参数未传递，就从环境变量读取参数：

```
const args = parseArgs();

const apiUrl = args.flomo_api_url || process.env.FLOMO_API_URL || "";
```

在MCP Inspector中，删掉Arguments，添加环境变量FLOMO_API_URL并设置响应的值。

请求write_note工具，写入笔记成功，如图3-13所示。

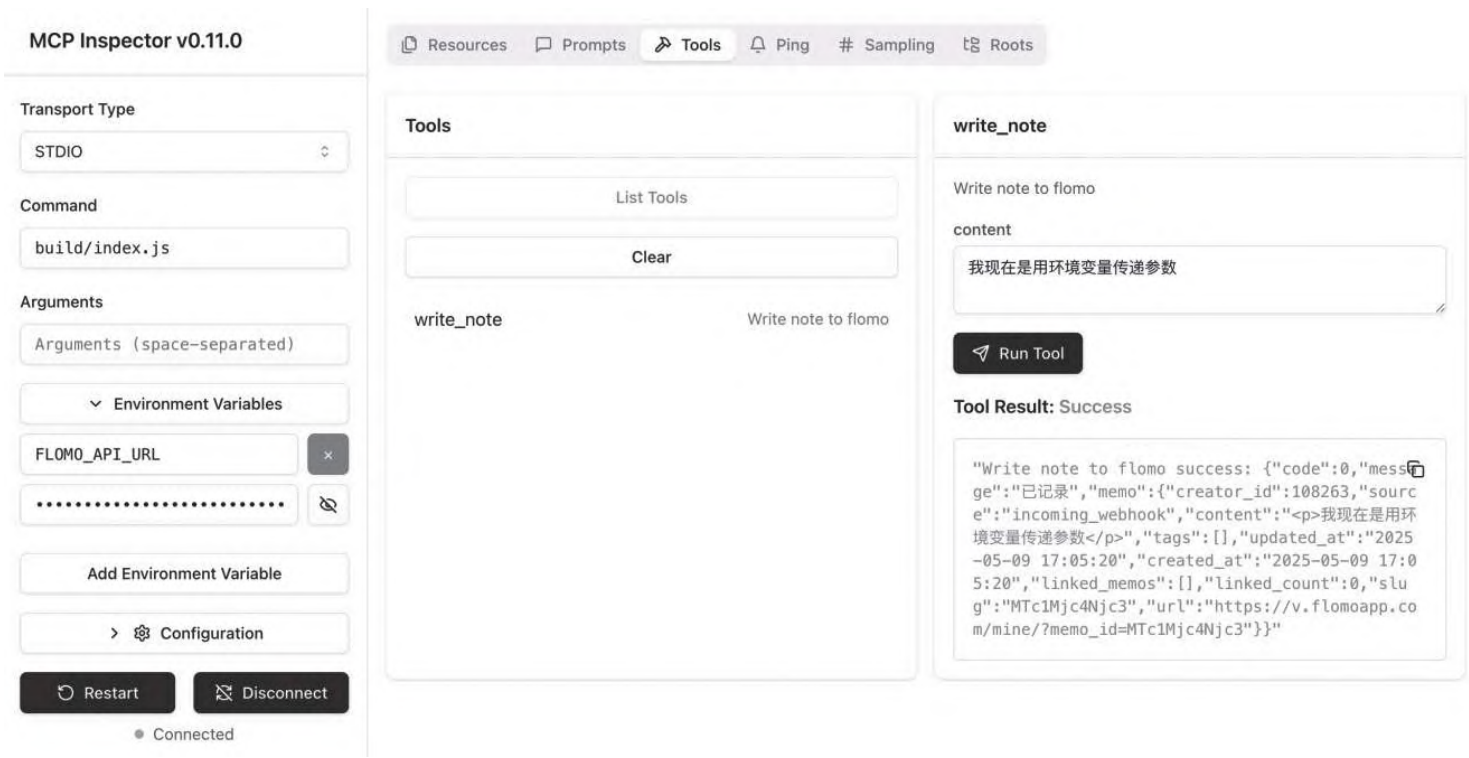


图3-13　通过环境变量传递参数

对应的MCP服务器的启动命令是：

```
FLOMO_API_URL=https://flomoapp.com/iwh/xxx/xxx/  node  build/index.js
```

3.2.7　在大模型客户端测试flomo　MCP服务器

我们选择Cursor作为大模型客户端，来测试我们开发的flomo-mcp。

首先，打开Cursor的MCP服务器配置文件（一般位于~/.cursor/mcp.json），写入flomo-mcp的配置：

```
{
    "mcpServers": {
        "flomo-mcp": {
            "command": "node",
            "args": [
                "/Users/idoubi/code/all-in-aigc/chatmcp/flomo-mcp/build/index.js"
            ],
            "env": {
                "FLOMO_API_URL": "https://flomoapp.com/iwh/xxx/xxx/"
            }
        }
    }
}
```

通过环境变量传递FLOMO_API_URL，args里面填写flomo-mcp编译后文件的绝对地址。

在Cursor的MCP配置面板中，可以看到flomo-mcp已经成功运行，并且获取了可用的工具。

打开Cursor AI对话面板，选择Agent模式，输入以下或类似的内容：

```
我今天开发了一个MCP服务器，感觉很开心。帮我记录一下。
```

Cursor会加载所有已配置MCP服务器中的可用工具请求大模型进行意图识别。然后根据大模型返回的参数调用对应的工具，此处调用了flomo-mcp的write_note工具，写入笔记到flomo，如图3-14所示。

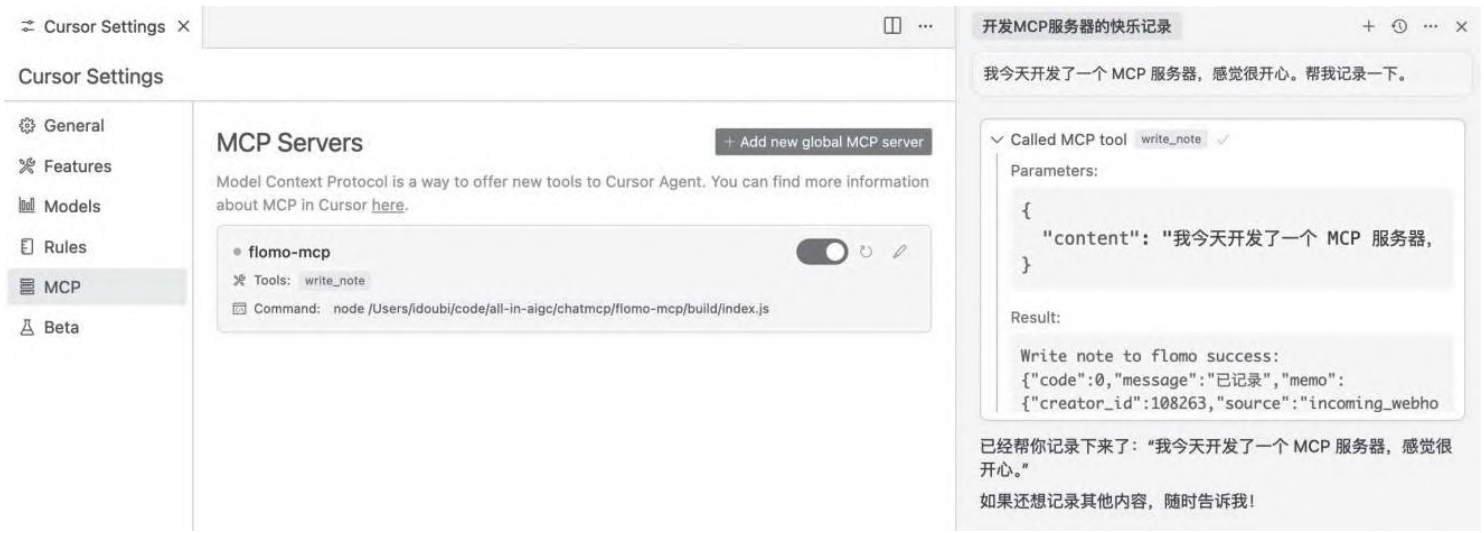


图3-14　在Cursor中调用flomo-mcp

3.2.8　发布flomo　MCP服务器

在Cursor中测试成功证明我们开发的flomo-mcp功能正常，接下来就可以发布给其他人使用了。

1.　发布到GitHub平台

我们可以把新开发的flomo-mcp的代码发布到GitHub，开源给所有用户使用。

首先，更新README.md文件，写入flomo-mcp的示例配置和使用说明。

在GitHub创建代码仓库，并提交代码：

```
cd  flomo-mcp

git  init

git  remote  add  origin  git@github.com:chatmcp/flomo-mcp.git

git  add  .

git  commit  -m  "first  version"

git  push  origin  main
```

flomo-mcp在GitHub的代码仓库如图3-15所示。

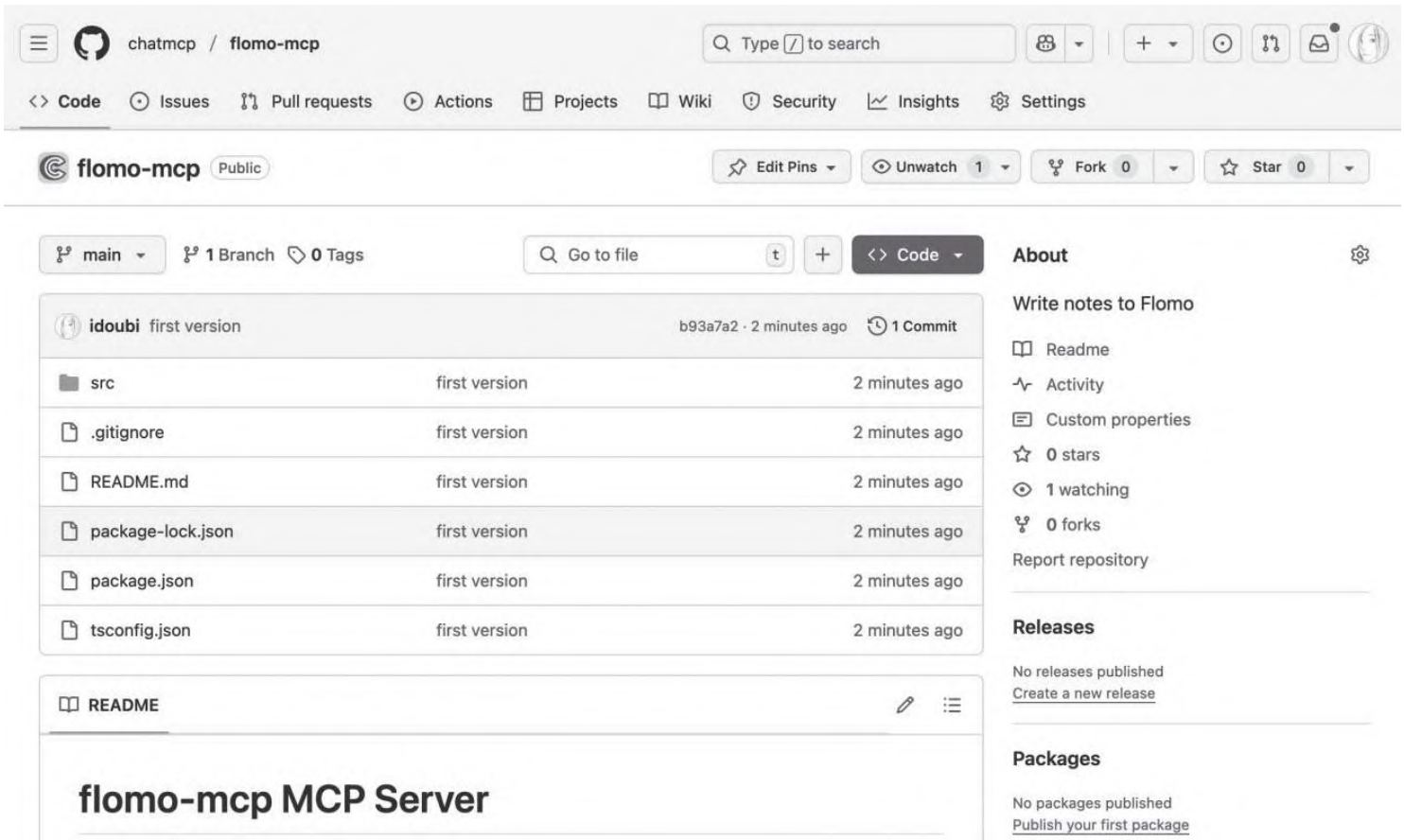


图3-15　flomo-mcp的代码仓库

2.　发布到npm平台

我们希望其他用户简单配置即可使用flomo-mcp，无须拉取源代码。由此，我们可以把flomo-mcp作为一个npm包，发布到npm应用市场。

首先，修改flomo-mcp项目的package.json，作为一个公开类型的npm包，发布到指定的组织，比如@chatmcp（这是我创建的一个组织，你可以在npm管理后台创建自己的组织）。

```
{
    "name": "@chatmcp/flomo-mcp",
    "version": "0.0.1",
    "description": "write notes to flomo.",
    "private": false,
    "type": "module",
    "homepage": "https://github.com/chatmcp/flomo-mcp",
    "bin": {
```

```
        "flomo-mcp": "./build/index.js"
    },
    "files": ["build"],
    "scripts": {
        "build": "tsc && node -
e \"require('fs').chmodSync('build/index.js', '755')\"",
        "prepare": "npm run build",
        "watch": "tsc --watch",
        "inspector": "npx @modelcontextprotocol/inspector build/index.js"
    },
    "dependencies": {
        "@modelcontextprotocol/sdk": "0.6.0"
    },
    "devDependencies": {
        "@types/node": "^20.11.24",
        "typescript": "^5.3.3"
    }
}
```

然后，执行命令，发布npm包：

```
cd flomo-mcp
npm install
npm run build
npm login # 打开登录连接，在浏览器中登录npm平台
npm publish --access public # 将包以公开形式发布到npm
```

发布成功后，打开npm包的访问地址，可以看到 @chatmcp/flomo-mcp已经上线，如图3-16所示。

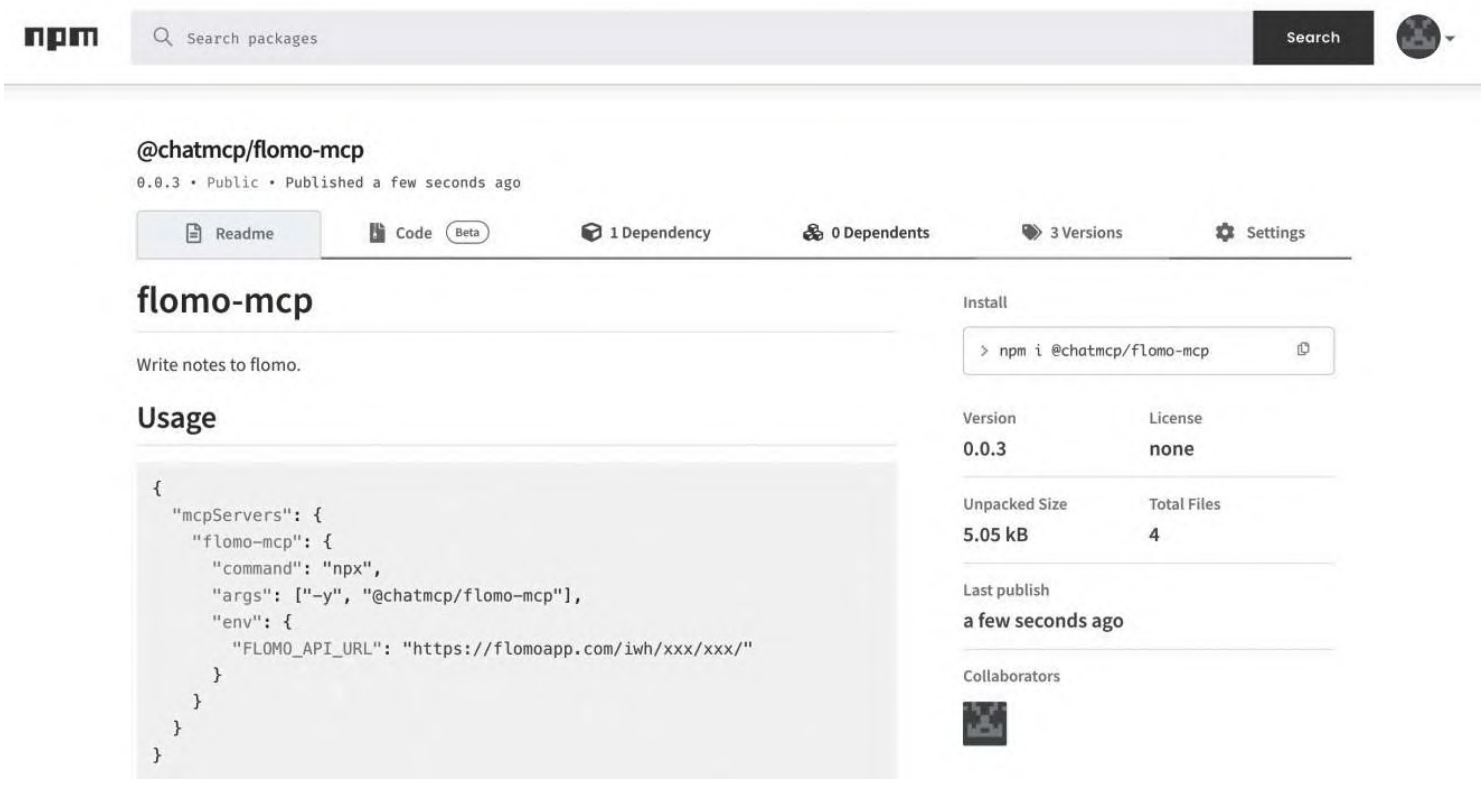


图3-16 flomo-mcp的npm包信息页

修改Cursor的配置信息，使用npx作为命令启动 @chatmcp/flomo-mcp，并传入环境变量配置：

```
{
    "mcpServers": {
        "flomo-mcp": {
            "command": "npx",
            "args": ["-y", "@chatmcp/flomo-mcp"],
            "env": {
                "FLOMO_API_URL": "https://flomoapp.com/iwh/xxx/xxx/"
            }
        }
    }
}
```

在Cursor中继续使用flomo-mcp，可以看到该MCP服务器已经按照新的配置方式成功运行。我们可以把日常开发中遇到的问题，一句话快速记录到flomo中，如图3-17所示。

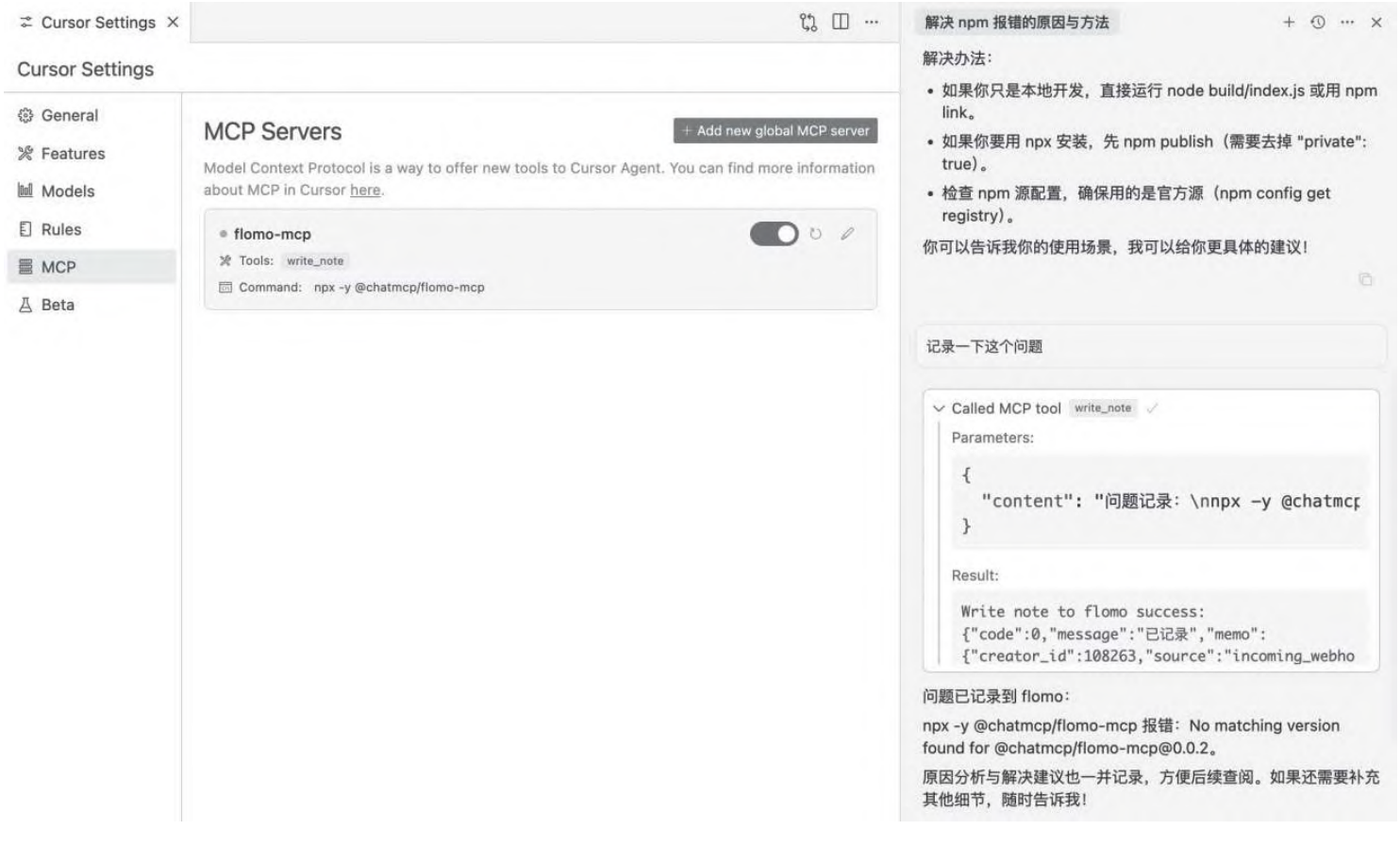


图3-17　在Cursor中记录开发问题

Cursor作为大模型客户端，能够自动拼接上下文，调用大模型整理笔记内容，并通过flomo-mcp实现记笔记。有了这个MCP服务器，我们就可以在编码过程中一边提问、一边记录，从而提升整体工作效率。

这样，我们就可以在flomo中查找和回顾日常记录的一些问题，如图3-18所示。



图3-18　在flomo中查看笔记

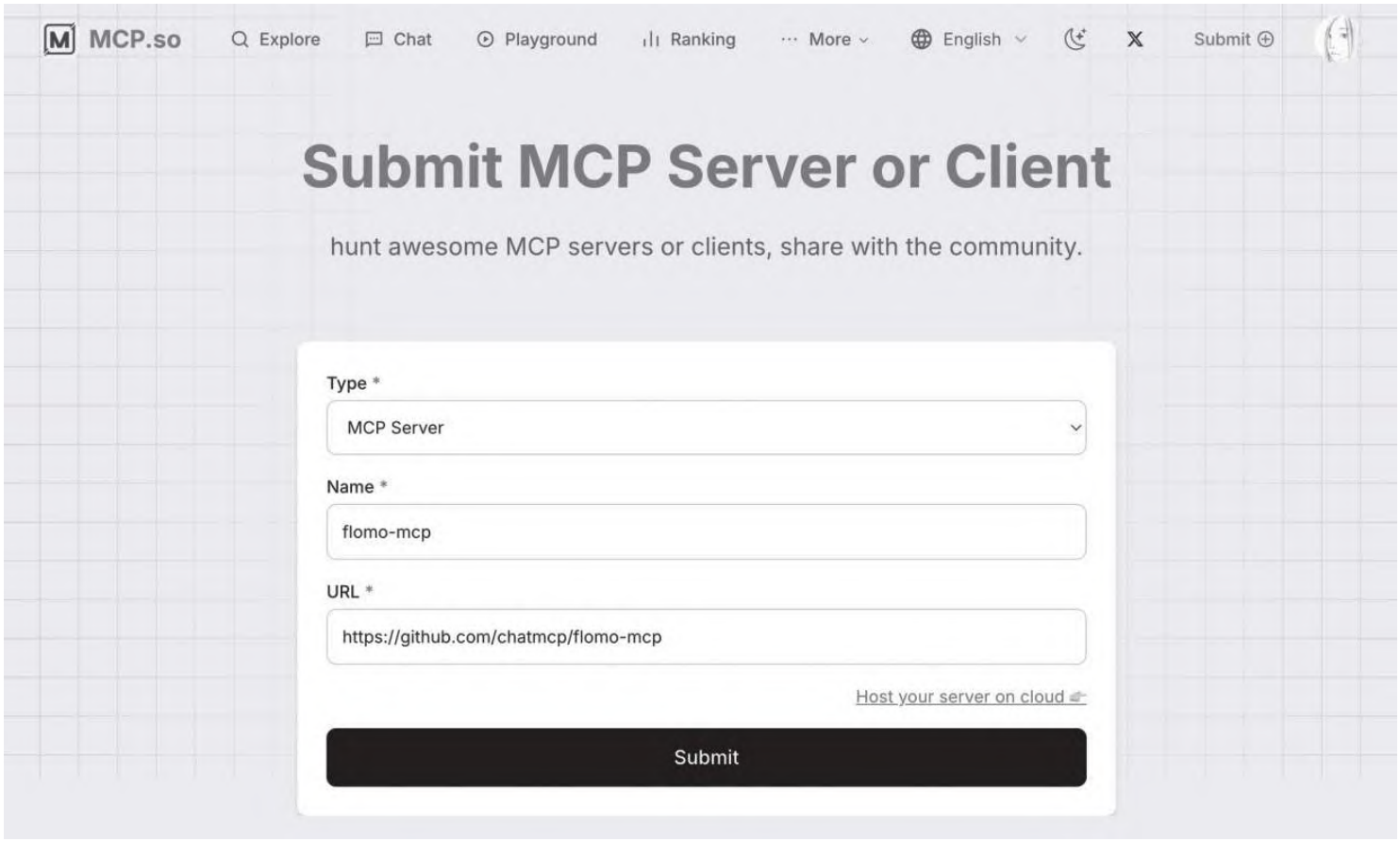
从这个例子可以看出，将MCP服务器挂载到大模型客户端，不仅能够顺利接入flomo这样的外部工具，还能将日常工作中的问题、思考与产出内容及时记录并进行结构化沉淀。通过这种方式，大模型客户端在知识积累与信息回溯上的能力得以增强，进而极大提升个人或团队的工作效率。

3.　发布到第三方MCP应用市场

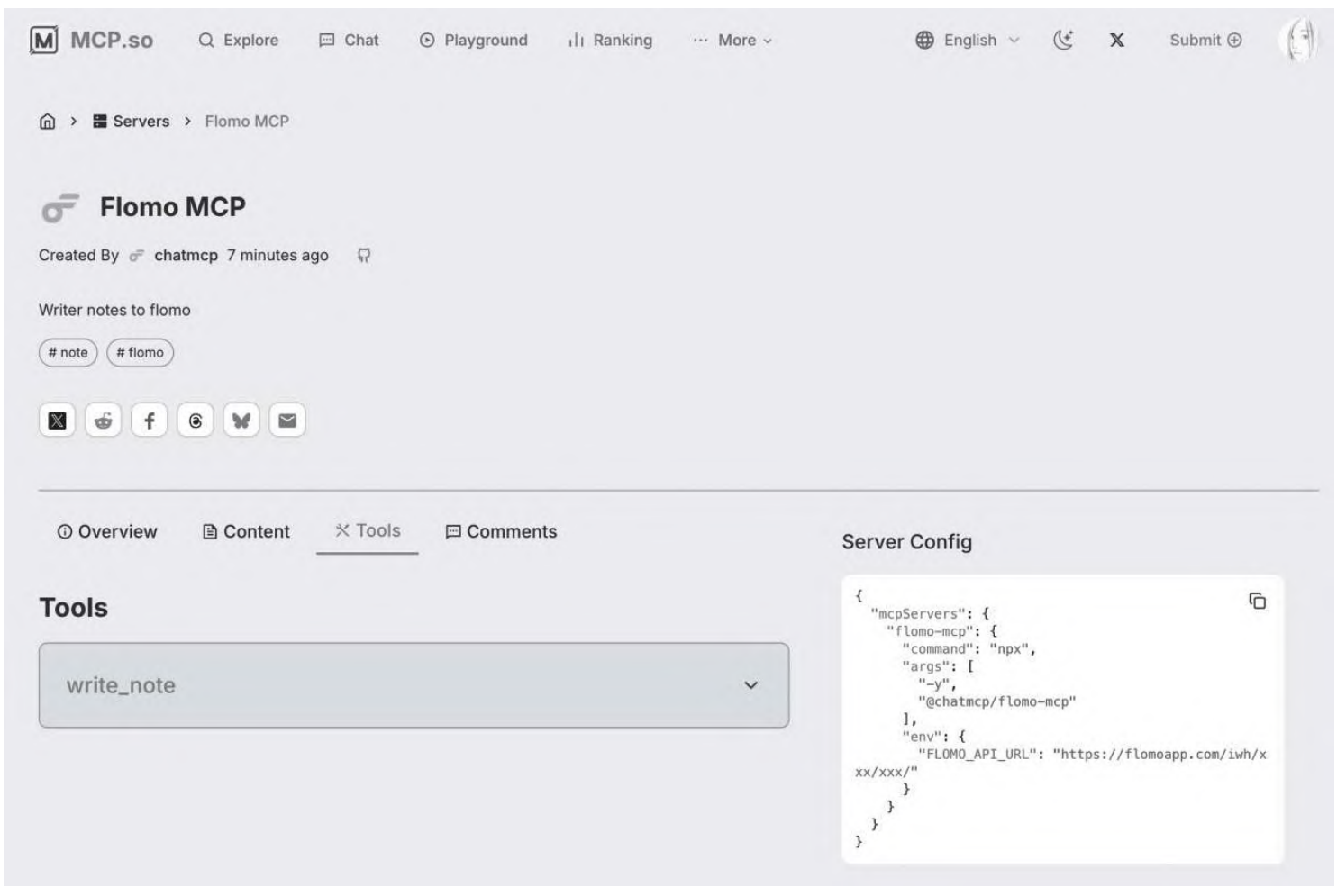
我们可以把flomo-mcp发布到第三方MCP应用市场，比如MCP.so。

点击进入Submit页面，填写MCP服务器信息，把flomo-mcp提交到MCP.so应用市场，如图3-19所示。

提交成功之后，你就可以在MCP.so应用市场看到自己的MCP服务器了。用户在应用详情页面，可以查看此MCP服务器的配置信息和内部实现的工具，如图3-20所示。



▲图3-19　将flomo-mcp发布到MCP.so应用市场



▲图3-20　在应用市场查看flomo　MCP服务器

3.2.9　案例1小结

在本节中，我们开发了一个可以对接flomo记录API的MCP服务器flomo-mcp，实现了在大模型客户端高效记笔记的功能。

我们详细介绍了MCP服务器的开发、调试和发布流程，并且演示了在MCP服务器中定义工具和实现工具的流程。

以此为例，我们可以开发更多的MCP服务器，以API的形式对接各类数据和资源。

3.3　案例2：开发HeyBeauty　MCP服务器，快速虚拟试衣

在前面的MCP服务器开发案例中，我们主要是在MCP服务器接收文本输入，并通过工具调用将文本内容返回给客户端以供展示。

在本节中，我们计划开发一个图片处理类型的MCP服务器，通过一个虚拟试衣的例子，演示如何接收客户端发送的图片，经过处理之后再将图片结果返回给客户端。此外，上一个案例主要讲解了在MCP服务器定义和实现工具能力。在此案例中，我们会完整演示在一个MCP服务器内实现工具（Tools）、资源（Resources）、提示词（Prompts）三大能力。目前市面上的大多数MCP服务器，主要实现的是工具能力。工具能力通常由大模型调度，侧重获取动态内容或对接外部服务；而资源和提示词则更侧重于由主机或用户选择，一般用于提供静态内容。在一个案例中实现三大能力有助于读者理解三者在实现与交互上的差异，为大家构建更强大的MCP服务器打好基础。

3.3.1　开发目标

在本节中，我们计划开发一个名为heybeauty-mcp的MCP服务器，接入HeyBeauty虚拟试衣平台的API，让用户能够在任意支持MCP服务器的客户端，以对话的方式体验虚拟试衣功能。这不仅降低了试衣门槛，也为AI驱动的个性化时尚体验打开了新的交互入口。

2024年4月，我开发了一个叫作HeyBeauty的虚拟试衣产品（heybeauty.ai），核心功能是让用户在网页端上传自己的照片，以及想要试穿的衣服的图片，HeyBeauty通过自研的试衣模型生成用户试穿衣服后的效果图，如图3-21所示。

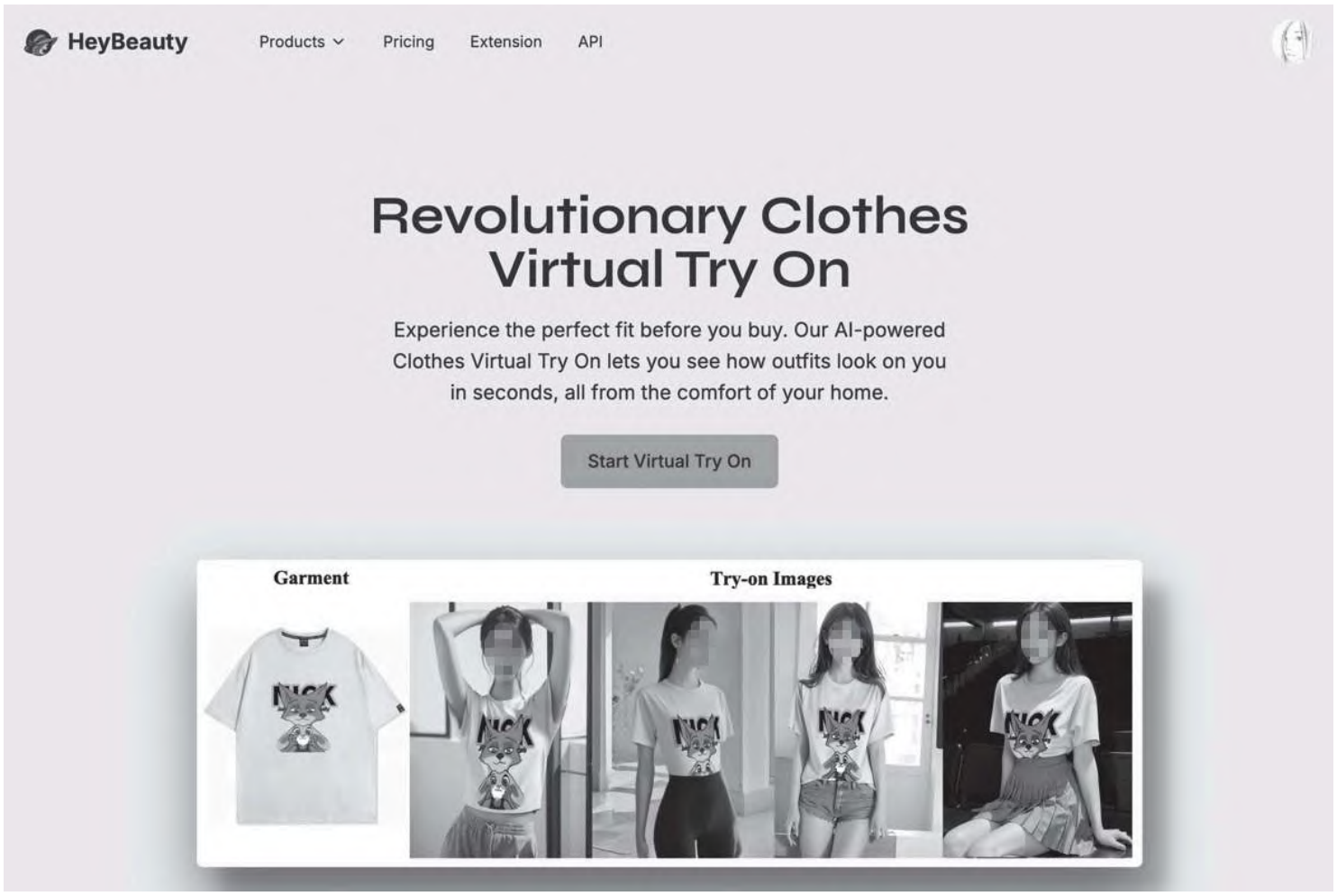


图3-21　HeyBeauty产品官网

用户试衣的流程如下所示：

1. 用户打开网页，进入HeyBeauty试衣控制台，如图3-22所示；

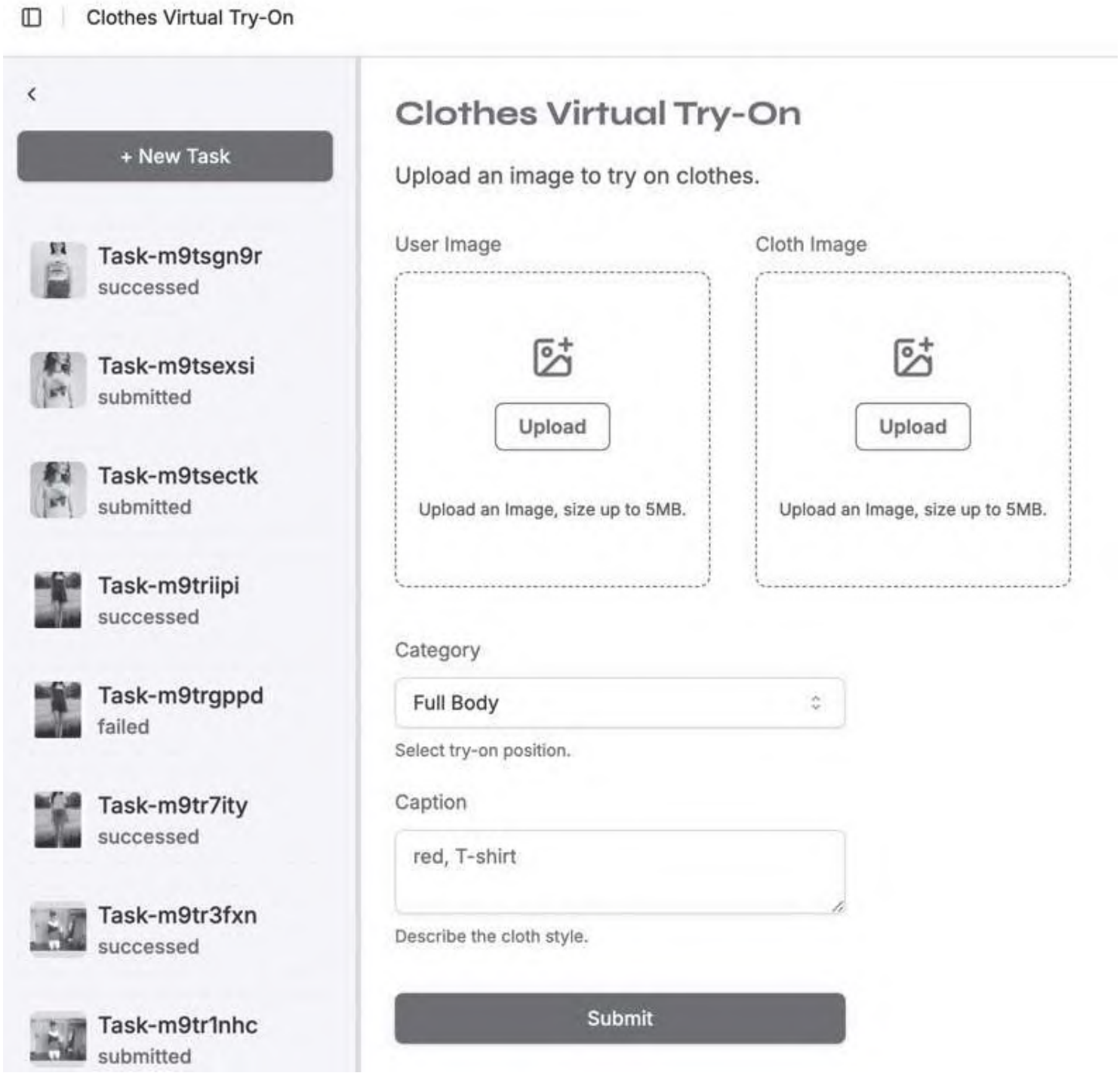


图3-22　HeyBeauty试衣控制台

2.　用户上传一张自己的照片（User　Image），和一张想要试穿的衣服的图片（Cloth　Image）；

3.　用户提交（Submit）试衣任务，HeyBeauty开始识别图片并运行试衣程序；

4.　用户查看试衣结果（Query　Task）。

为了让更多人用上HeyBeauty的试衣功能，我们开放了API，允许第三方应用接入API，帮助用户在更多终端场景进行虚拟试衣。

在过去一年，HeyBeauty对接的终端应用有微信小程序、移动App、Shopify插件、试衣镜硬件等，在不同的场景为用户提供了虚拟试衣的功能。

3.3.2　前置准备与开发思路

在开发HeyBeauty　MCP服务器之前，我们先熟悉一下HeyBeauty开放的接口。

从HeyBeauty接口文档（doc.heybeauty.ai）可以看到，HeyBeauty开放了虚拟试衣接口（Try-On　API），使用方式如下：

1.　从HeyBeauty平台申请一个API密钥；

2.　请求接口创建试衣任务（Create　Try-On　Task）；

3.　上传试衣任务需要用到的图片（Upload　Images）；

4.　查询试衣结果（Query　Try-On　Result）。

HeyBeauty试衣接口文档如图3-23所示。

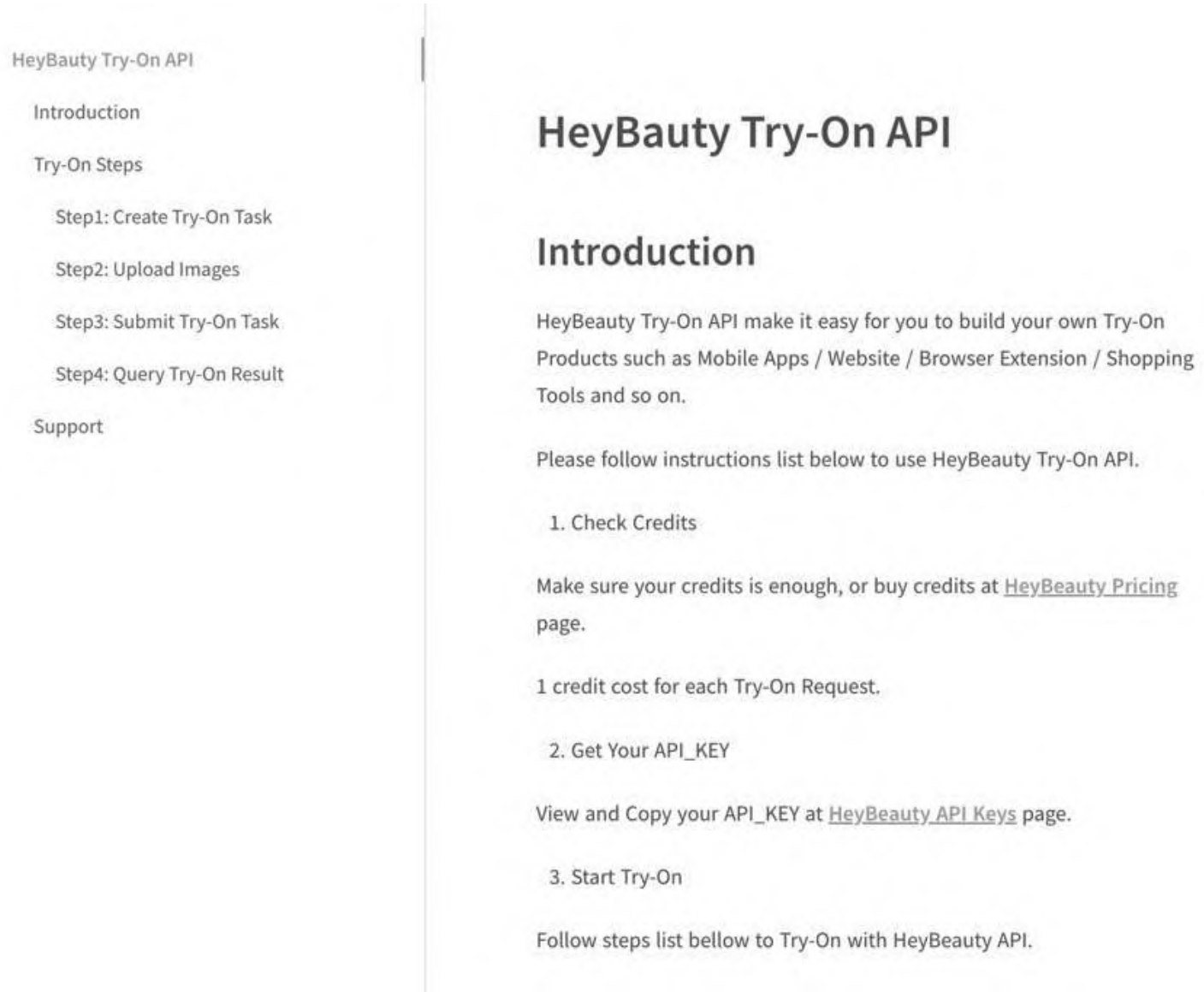


图3-23　HeyBeauty试衣接口文档

申请API密钥的流程不做过多阐述，用户在HeyBeauty官网自助申请即可。

根据HeyBeauty接口文档的试衣流程，我们先来梳理一下接下来要开发的HeyBeauty MCP服务器，其用户侧主要的交互流程如下：

1. 大模型客户端在启动时，请求HeyBeauty MCP服务器，拿到一批衣服图片，提供给用户选择；

2. 用户上传自己的照片，并选择其中一张衣服图片进行试衣；

3. 大模型客户端上传用户照片，得到两个图片地址（user_image_url和cloth_image_url）；

4. 大模型客户端请求大模型挑选可以实现试衣需求的工具；

5. 大模型客户端根据大模型返回的工具和参数，请求HeyBeauty MCP服务器，发送两个图片地址；

6. HeyBeauty MCP服务器请求HeyBeauty接口，提交试衣任务；

7. 大模型客户端请求HeyBeauty MCP服务器查询试衣结果。

在第2步，用户可以不选择MCP服务器返回的衣服图片，而是主动发送衣服图片，这样在第3步，大模型客户端就需要同时上传用户照片和衣服图片，得到两个图片的地址。

在整个交互流程中，大模型客户端必须实现图片转换的逻辑，也就是说，用户上传的图片不能直接以原始文件（二进制内容）的形式发送给大模型或者MCP服务器，而应该先将图片上传到第三方图床，得到图片的URL。



虽然按照MCP的约定，客户端可以将图片的内容以base64编码的方式传给MCP服务器，也可以接收MCP服务器返回的base64编码的图片数据，但在实际流程中存在一些问题：客户端向哪个MCP服务器发起请求以及传递什么参数，是由大模型进行调度的，也就是说需要先把图片传给大模型，大模型再告诉客户端，应该调用的MCP服务器和需要请求的图片内容。在这个过程中，如果图片都以base64的格式进行传输，不仅会大量占用token，带来额外的成本开销，还可能因数据过大或编码不完整，导致图片解析失败或传输异常。因此，更推荐的做法是将图片上传至图床，仅传递图片的URL。

理解这一逻辑后，在开发HeyBeauty MCP服务器时，我们不再在服务器内部处理图片的格式转换。凡涉及图片处理的地方，统一使用图片的URL，而非base64编码的图片内容。

接下来，我们就来实现HeyBeauty MCP服务器。

3.3.3　创建HeyBeauty　MCP服务器项目

1.　创建项目

跟前面的案例一样，我们先通过MCP官方提供的命令行工具来创建要开发的MCP服务器项目，取名为heybeauty-mcp：

```
npx　@modelcontextprotocol/create-server　heybeauty-mcp
```

进入项目根目录，安装项目依赖，启动监听服务：

```
npm　install

npm　run　watch
```

这样，当你修改项目代码时，监听程序会自动重新编译和运行，方便实时调试开发。

创建HeyBeauty　MCP服务器并启动监听服务的实现过程如图3-24所示。

```
~/code/all-in-aigc/chatmcp (2m 11.38s)
npx @modelcontextprotocol/create-server heybeauty-mcp

? What is the name of your MCP server? heybeauty-mcp
? What is the description of your server? HeyBeauty Virtual TryOn
? Would you like to install this server for Claude.app? No
✔ MCP server created successfully!

Next steps:
  cd heybeauty-mcp
  npm install
  npm run build  # or: npm run watch
  npm link       # optional, to make available globally
```

```
~/code/all-in-aigc/chatmcp (0.036s)
cd heybeauty-mcp
```

```
~/code/all-in-aigc/chatmcp/heybeauty-mcp (6.546s)
npm install

> heybeauty-mcp@0.1.0 prepare
> npm run build


> heybeauty-mcp@0.1.0 build
> tsc && node -e "require('fs').chmodSync('build/index.js', '755')"


added 17 packages, and audited 18 packages in 6s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
~/code/all-in-aigc/chatmcp/heybeauty-mcp
npm run watch
[11:25:34 PM] Starting compilation in watch mode...

[11:25:35 PM] Found 0 errors. Watching for file changes.
```

图3-24　通过命令行创建MCP服务器并启动监听服务

2.　启动MCP　Inspector

在HeyBeauty　MCP服务器根目录下执行以下命令快速启动MCP　Inspector：

```
npx　@modelcontextprotocol/inspector　build/index.js
```

点击上述命令运行成功后输出的本地地址，进入MCP　Inspector，随后连接HeyBeauty　MCP服务器，如图3-25所示。

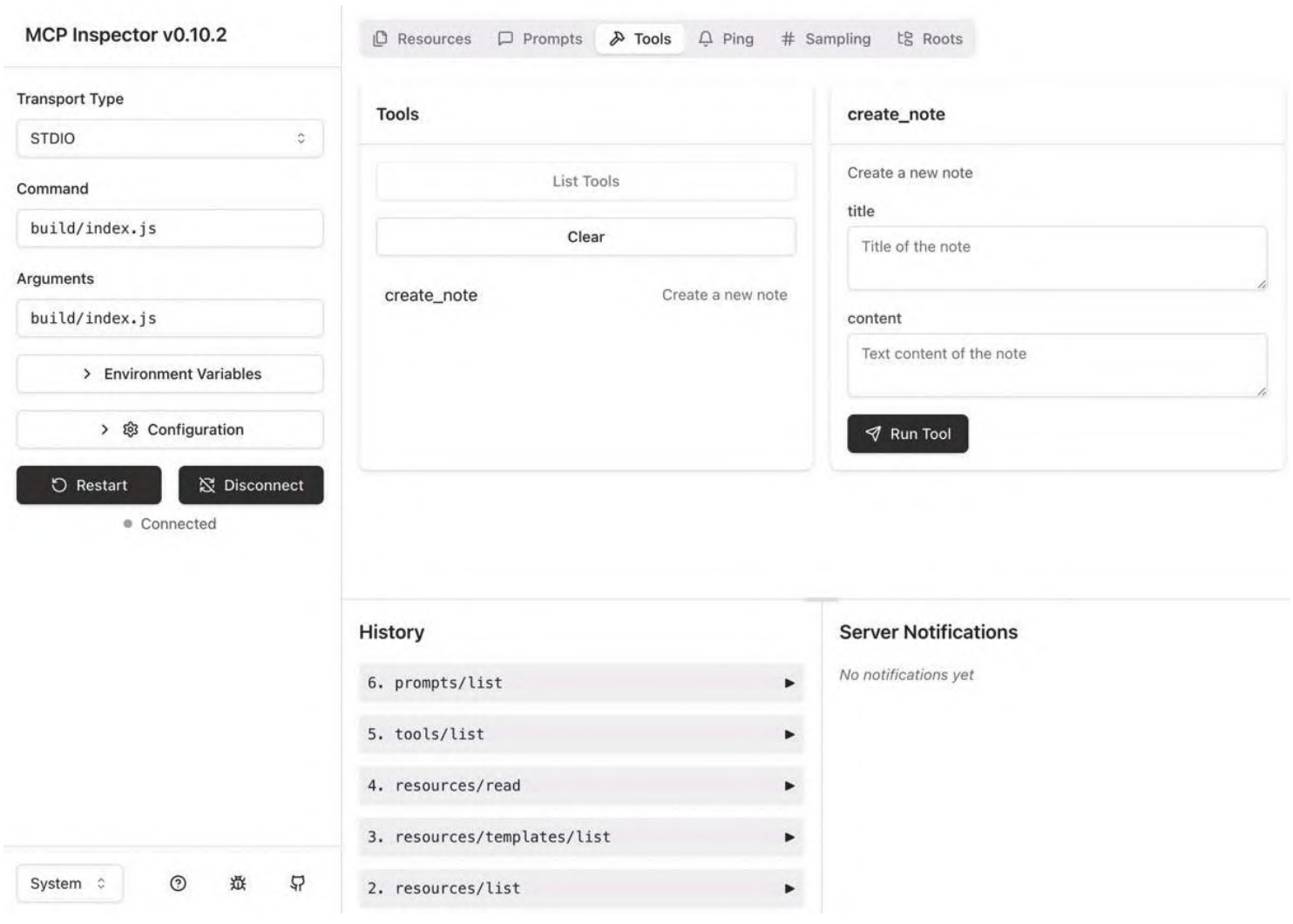


图3-25　在MCP　Inspector中调试HeyBeauty　MCP服务器

从图3-25中可以看到，用命令行工具创建的HeyBeauty　MCP服务器，默认实现了Resources、Prompts和Tools三大能力。

Inspector是一款让MCP服务器的调试模块化、系统化、可视化的工作台，聚焦资源、提示词、工具三大模块，支持本地调试，免安装快速接入，并可视化配置环境变量和运行参数，显著提升了调试效率。

接下来，我们就开始在HeyBeauty　MCP服务器内部实现与虚拟试衣相关的功能。

3.3.4　实现HeyBeauty　MCP服务器的业务逻辑

1.　实现与HeyBeauty　API进行交互的客户端类

实现调用HeyBeauty　API的客户端类

在开始开发HeyBeauty　MCP服务器之前，我们首先要设计一个专门与HeyBeauty　API进行交互的客户端类。这个类将封装与HeyBeauty　API通信的细

节，为MCP服务器的其他功能模块提供一个简洁、易用的接口。我们将其命名为HeyBeautyClient，并新建文件src/heybeauty.ts来实现该类，具体实现如下所示：

```
/**
 * HeyBeauty API 客户端类
 * 用于请求heybeauty.ai提供的接口
 */
export class HeyBeautyClient {
    private readonly apiBaseUrl: string = "https://heybeauty.ai/api";
    private readonly apiKey: string;
```

```
    /**
     * 构造函数
     * @param apiKey - 从heybeauty.ai申请的API密钥
     * @param apiBaseUrl - API 基础地址（可选）
     */
    constructor({ apiKey, apiBaseUrl }: { apiKey: string; apiBaseUrl?: string }
) {
        this.apiKey = apiKey;
        this.apiBaseUrl = apiBaseUrl || this.apiBaseUrl;
    }
```

```
/**
 * 获取可试穿的衣服列表
 * @returns 衣服列表
 */
async getClothes() {
    try {
        const req = {
            page: 1,
            limit: 10,
        };
```

```
const resp = await fetch(`${this.apiBaseUrl}/get-clothes`, {
    method: "POST",
    headers: {
        Authorization: `Bearer ${this.apiKey}`,
    },
    body: JSON.stringify(req),
});


if (!resp.ok) {
    throw new Error("request failed with status " + resp.status);
}
const { code, message, data } = await resp.json();
            if (code !== 0) {
                throw new Error(message);
            }


            return data;
        } catch (error) {
            throw error;
        }
    }
}
```

在这个客户端类里，我们实现了一个getClothes方法，用于请求HeyBeauty的get-clothes接口，获取一批可以用于虚拟试衣的衣服图片。

调试客户端类，调用getClothes方法

在实现完HeyBeautyClient类之后，我们需要对其进行调试，以确保它能够正确地与HeyBeauty API进行交互，并返回预期的数据。为此，我们可以在src/index.ts文件中的启动函数里创建该客户端的实例，并调用其方法进行简单测试。具体实现如下所示：

```
/**
 * 使用stdio传输启动服务器
 * 通过标准输入/输出通信
 */
async function main() {
    const apiKey = "xxxxxx";
    const client = new HeyBeautyClient({ apiKey });
    const clothes = await client.getClothes();
    console.log(clothes);


    const transport = new StdioServerTransport();
    await server.connect(transport);
}
```

```
main().catch((error)  =>  {
    console.error("Server  error:",  error);
    process.exit(1);
});
```

打开终端，运行node build/index.js命令，可以看到在main函数调用客户端获取HeyBeauty衣服列表的结果。我们主要关心以下4个数据字段。

- title：衣服的名称。
- description：衣服的描述信息。
- cloth_id：衣服的编号。
- cloth_img_url：衣服的图片地址。



HeyBeautyClient客户端类的设计主要遵循以下几个关键点：

- **职责单一**，专注于封装HeyBeauty API的请求逻辑；
- **可配置性强**，支持通过构造函数传入自定义API地址；
- **错误处理机制**，对HTTP层和业务层异常都进行了判断和错误抛出；
- **可复用性高**，可以在多个MCP工具模块中共享调用；
- **易于扩展**，后续可以方便地新增更多接口方法，满足业务需求。

2. 实现与资源（Resources）相关的功能

实现获取资源列表的逻辑

修改src/index.ts中获取资源列表的逻辑，传入apiKey创建HeyBeautyClient，把调用getClothes得到的衣服列表作为MCP服务器的资源列表返回。

```
/**
  *  以资源的形式返回衣服列表
  *  每件衣服作为一个资源
  *  -  以cloth://URI作为资源的唯一标识符
  *  -  资源是纯文本类型
  *  -  资源包含name字段和description字段
  */
server.setRequestHandler(ListResourcesRequestSchema,  async  ()  =>  {
    try  {
        const  apiKey  =  "xxxxxx";
        const  client  =  new  HeyBeautyClient({  apiKey  });

        const  clothes  =  await  client.getClothes();


        return  {
            resources:  clothes.map((clothe:  any)  =>  ({
                uri:  `cloth:///${clothe.cloth_id}`,
                mimeType:  "text/plain",
                name:  clothe.title,
                description:  `${clothe.description}`,
            })),
        };
    }  catch  (error:  any)  {
```

```
            throw new Error("get resources failed: " + error.message);
        }
    });
```

在MCP Inspector中调用List Resources以获取服务器返回的资源列表，如图3-26所示。

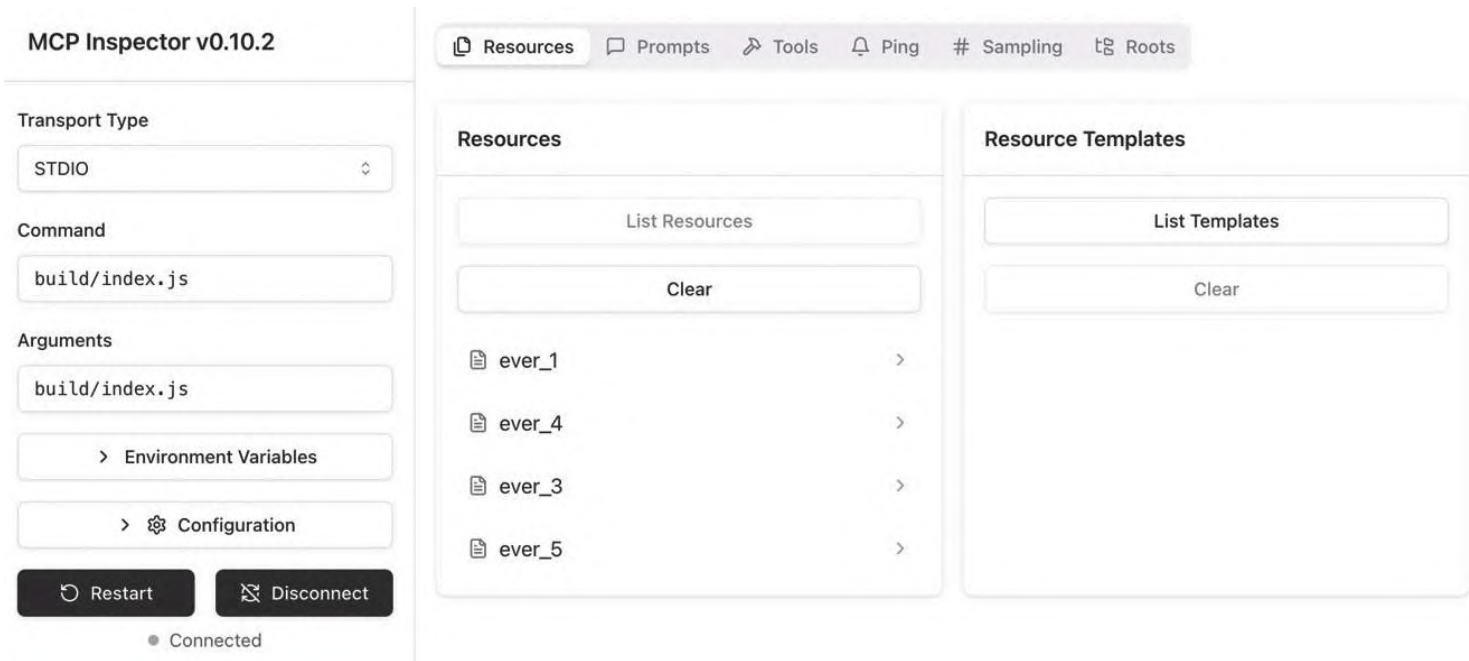


图3-26 调用List Resources获取资源列表

实现读取资源的逻辑

修改src/index.ts中读取资源（read resource）内容的逻辑，传入apiKey创建HeyBeautyClient，调用getClothes得到衣服列表，再根据请求的衣服ID，获取对应衣服的图片地址。

```
//**
 * 读取资源内容
 * 接收一个cloth://URI，以纯文本的形式返回资源（衣服的图片地址）
 */
server.setRequestHandler(ReadResourceRequestSchema, async (request) => {
    try {
        const url = new URL(request.params.uri);
        const id = url.pathname.replace(/^\//, "");
```

```
const apiKey = "xxxxxx";

const client = new HeyBeautyClient({ apiKey });



const clothes = await client.getClothes();
```

```
const cloth = clothes.find((clothe: any) => clothe.cloth_id == id);

if (!cloth) {
    throw new Error(`Cloth ${id} not found`);
}

return {
    contents: [
        {
            uri: request.params.uri,
            mimeType: "text/plain",
            text: cloth.cloth_img_url,
        },
    ],
};
} catch (error: any) {
    throw new Error("read resource failed: " + error.message);
}
});
```

在MCP Inspector中选择List Resources返回的资源之一，触发读取资源请求，可以看到服务器返回的资源内容，如图3-27所示。

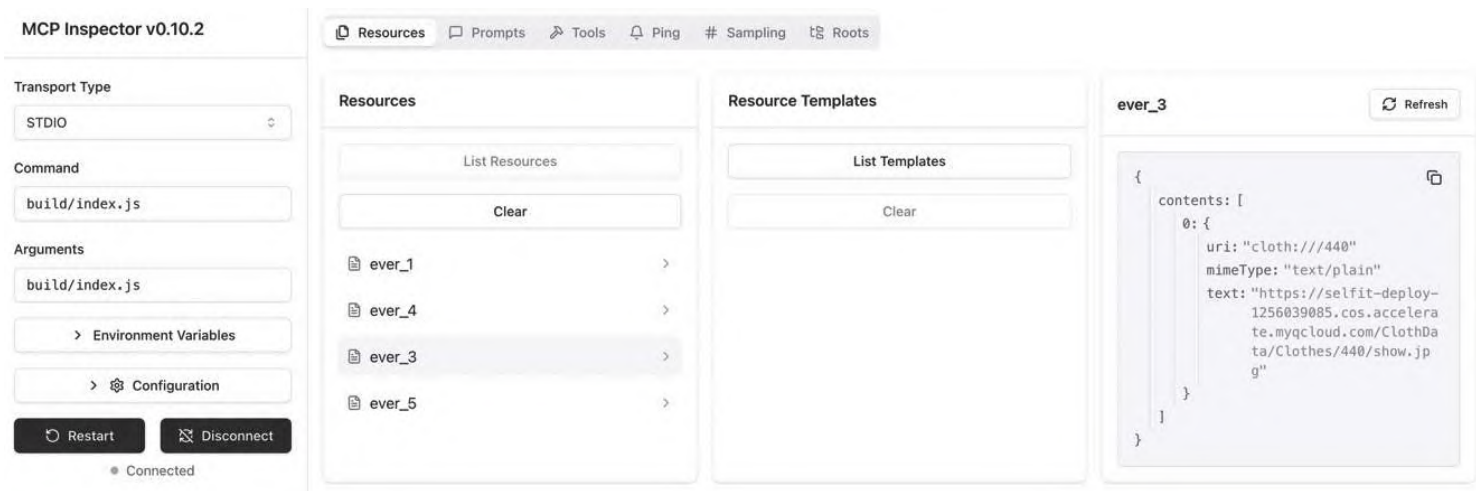


图3-27　在MCP　Inspector中读取资源

在大模型客户端测试资源

前面我们已经看到，通过MCP　Inspector获取资源列表和读取资源都没有问题，接下来，我们再在大模型客户端继续测试，看看如何在大模型客户端使用服务器暴露的资源。

在Claude中配置HeyBeauty　MCP服务器：

```
{
    "mcpServers":  {
        "heybeauty-mcp":  {
            "command":  "node",
            "args":  [
                "/Users/idoubi/code/all-in-aigc/chatmcp/heybeauty-mcp/build/index.js"
            ],
            "env":  {
                "HEYBEAUTY_API_KEY":  "xxxxxx"
            }
        }
    }
}
```

运行以下命令监听Claude的输出日志：

```
tail  -f~/Library/Logs/Claude/mcp*.log
```

启动Claude，如图3-28所示，日志输出的内容显示：客户端给HeyBeauty MCP服务器发送了resources/list请求，获得了MCP服务器返回的资源列表。



图3-28 Claude读取HeyBeauty MCP服务器资源列表的日志

用户在Claude中点击输入框最右侧的图标，可以看到HeyBeauty MCP服务器返回的资源列表（支持试穿的衣服列表），如图3-29所示。

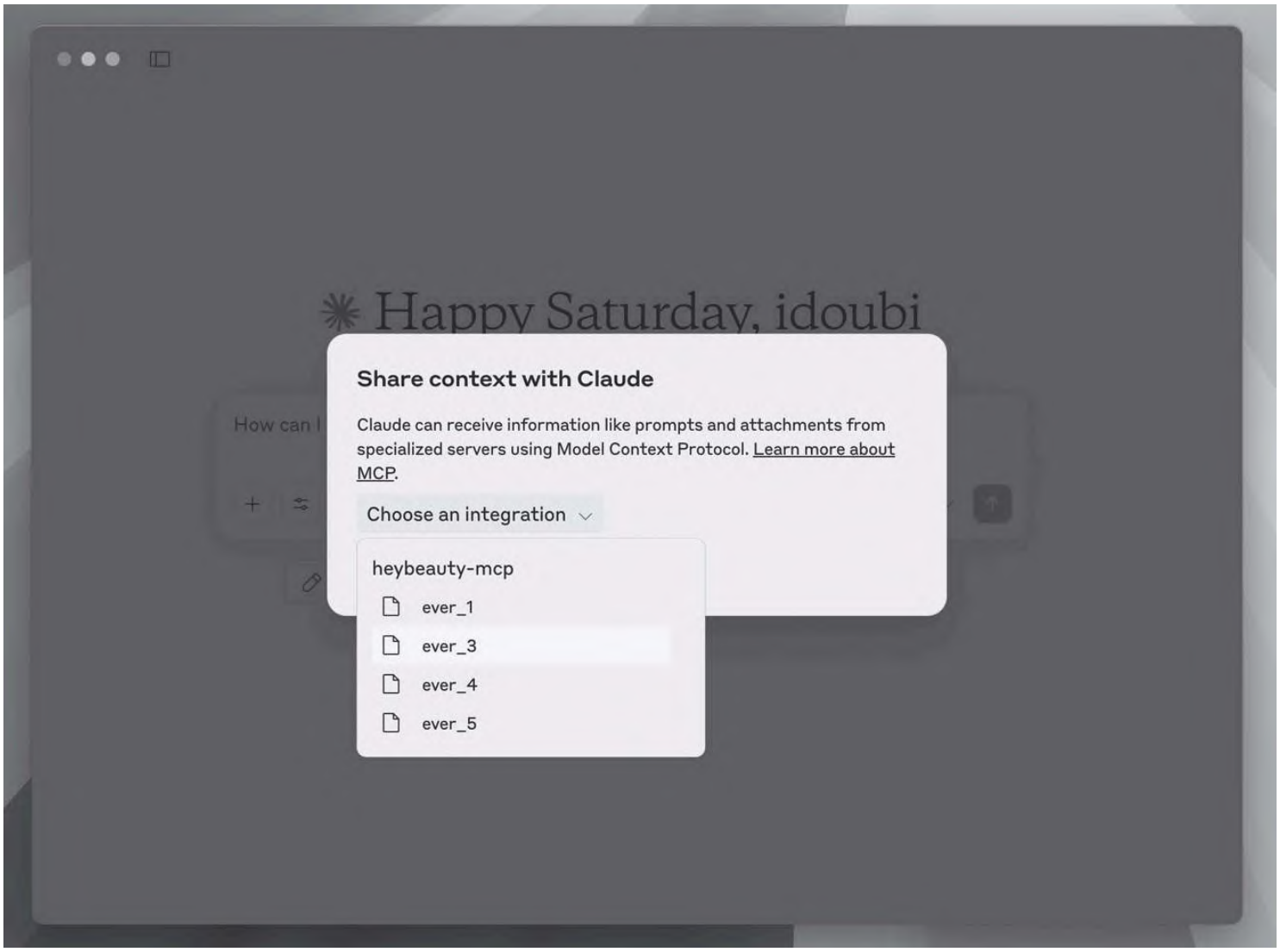


图3-29　HeyBeauty　MCP服务器返回的资源列表（支持试穿的衣服列表）

用户点击选择其中一个资源，比如ever_3，通过Claude输出的日志可以看到，客户端向HeyBeauty　MCP服务器发送了resources/read请求，传递的参数是资源的ID（cloth_id），HeyBeauty　MCP服务器返回了资源的内容（支持试穿的衣服的图片地址），如图3-30所示。

```
~/code/all-in-aigc/chatmcp/heybeauty-mcp
tail -f ~/Library/Logs/Claude/mcp*.log | grep cloth

2025-04-26T02:40:08.181Z [info] [heybeauty-mcp] Message from client: {"method":"
resources/read","params":{"uri":"cloth:///440"},"jsonrpc":"2.0","id":76}
2025-04-26T02:40:09.104Z [heybeauty-mcp] [info] Message from server: {"jsonrpc":
"2.0","id":76,"result":{"contents":[{"uri":"cloth:///440","mimeType":"text/plain
","text":"https://selfit-deploy-1256039085.cos.accelerate.myqcloud.com/ClothData
/Clothes/440/show.jpg"}]}}
```

图3-30　HeyBeauty　MCP服务器返回了资源的内容

Claude会在输入框下方显示用户选择的资源，用户点击资源可以查看资源的内容。在这个场景里，资源的内容就是HeyBeauty　MCP服务器返回的一件衣服的图片地址，如图3-31所示。

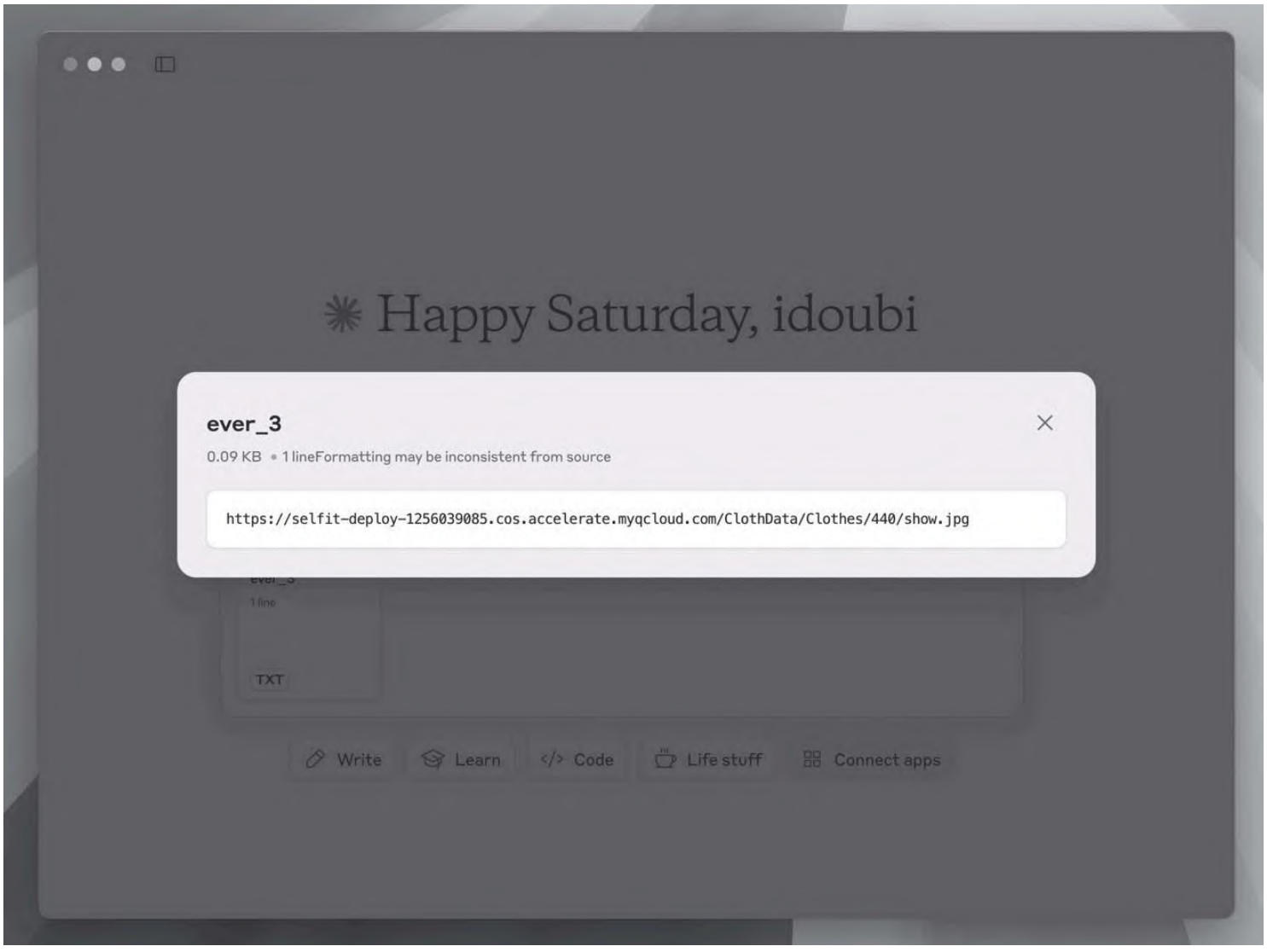


图3-31　HeyBeauty　MCP服务器返回的一件衣服的图片地址

通过上面的示例，我们大致了解到，在大模型客户端，获取MCP资源列表以及用户选择资源、查看资源的流程。

接下来我们继续实现HeyBeauty　MCP服务器的业务逻辑。

3.　完善与HeyBeauty　API进行交互的客户端类

我们继续完善src/heybeauty.ts的逻辑，在HeyBeautyClient类里添加submitTask和queryTask两个方法，分别调用HeyBeauty的API提交试衣任务和查询试衣结果。

submitTask（提交试衣任务）

该方法调用HeyBeauty的异步试衣接口/api/mcp-vton，提交一项虚拟试衣任务，其实现逻辑如下：

```
async  function  submitTask({

    user_img_url,
```

```
    cloth_img_url,
    cloth_id,
    cloth_description,
}: {
    user_img_url: string;
    cloth_img_url: string;
    cloth_id?: string;
    cloth_description?: string;
}) {
    try {
        if (!user_img_url || !cloth_img_url) {
            throw new Error("user_img_url and cloth_img_url are required");
        }

const req = {
    user_img_url,
    cloth_img_url,
    cloth_id,
    caption: cloth_description || "",
    category: "1",
    is_sync: "0",
};
const resp = await fetch(`${this.apiBaseUrl}/mcp-vton`, {
```

```
    method: "POST",
    headers: {
        Authorization: `Bearer ${this.apiKey}`,
    },
    body: JSON.stringify(req),
});

if (!resp.ok) {
    throw new Error("request failed with status " + resp.status);
}

const { code, message, data } = await resp.json();
if (code !== 0) {
    throw new Error(message);
}
```

```
    return  {
        task_id:  data.uuid,
        created_at:  data.created_at,
        updated_at:  data.updated_at,
        status:  data.status,
        tryon_img_url:  data.tryon_img_url  ||  "",
    };
}  catch  (error)  {
    throw  error;
}
}
```

该方法包括如下请求参数。

- 必填项：用户照片地址和衣服图片地址。
- 选填项：衣服ID和衣服描述。
- 其他参数：category参数固定为1；is_sync设置为0，表示这是一个异步任务。

接口响应task_id作为任务标识，后续可用于查询试衣结果。

queryTask（查询试衣结果）

该方法用提交试衣任务得到的task_id作为请求参数，调用HeyBeauty的/api/get-task-info查询试衣任务的结果，其实现逻辑如下：

```
async  function  queryTask({  task_id  }:  {  task_id:  string  })  {
    try  {
```

```
const uri = `${this.apiBaseUrl}/get-task-info`;
const req = {
    task_uuid: task_id,
};


const resp = await fetch(uri, {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${this.apiKey}`,
    },
    body: JSON.stringify(req),
});
```

```
if (!resp.ok) {
    throw new Error("request failed with status: " + resp.status);
}

const { code, message, data } = await resp.json();

if (code !== 0) {
    throw new Error(message);
}
```

```
    return  {
        task_id:  task_id,
        created_at:  data.created_at,
        updated_at:  data.updated_at,
        status:  data.status,
        tryon_img_url:  data.tryon_img_url  ||  "",
    };
}  catch  (err)  {
    throw  err;
}
```

接口返回试衣状态status，其值可能是：

- successed，表示试衣成功；
- failed，表示试衣失败；
- processing，表示任务正在处理中。

如果试衣成功，那么接口返回的字段tryon_img_url中会包含一张图片的地址，这张图片就是用户“试穿”完成后的效果图。

4. 实现与工具（Tools）相关的功能

定义工具列表

我们在src/index.ts里面定义两个工具：submit_tryon_task、query_tryon_task，分别对应HeyBeautyClient里实现的submitTask和queryTask方法。它们的实现逻辑如下：

```
/**
 *  列出服务器实现的工具列表
 *  暴露一个submit_tryon_task工具用于提交试衣任务
 *  暴露一个query_tryon_task工具用于查询试衣结果
 */
server.setRequestHandler(ListToolsRequestSchema,  async  ()  =>  {
    return  {
        tools:  [
            {
                name:  "submit_tryon_task",
                description:
                    "Submit  a  tryon  task  with  user  image  url  and  cloth  image  u
rl",
                inputSchema:  {
                    type:  "object",
                    properties:  {
                        user_img_url:  {
                            type:  "string",
                            description:  "User  image  url,  should  be  a  url  of  a  p
icture",
                        },
                        cloth_img_url:  {
                            type:  "string",
                            description:
```

```
                    "Cloth image url, should be a url of a picture, user input or get
    from the selected cloth resource",
                },
                cloth_id: {
                    type: "string",
                    description: "Cloth id, get from the selected cloth resource",
                },
                cloth_description: {
                    type: "string",
                    description:
                        "Cloth description, user input or get from the selected cloth
    resource" ,
                    },
                },
                required: ["user_img_url", "cloth_img_url"],
            },
        },
        {
            name: "query_tryon_task",
            description: "Query a tryon task with task id",
            inputSchema: {
                type: "object",
```

```
                properties: {
                    task_id: {
                        type: "string",
                        description: "Task id, get from the submit_tryon_task
tool",
                    },
                },
                required: ["task_id"],
            },
        },
    ],
};
});
```

请注意，在定义工具时，应尽可能在description字段中详细说明工具的用途，帮助大模型准确理解并提高调度效率。同时，还需合理设置每个字段的类型和说明，以便模型在识别工具后能够返回正确的调用参数。

通过MCP Inspector可以获取HeyBeauty MCP服务器定义的工具列表，如图3-32所示。

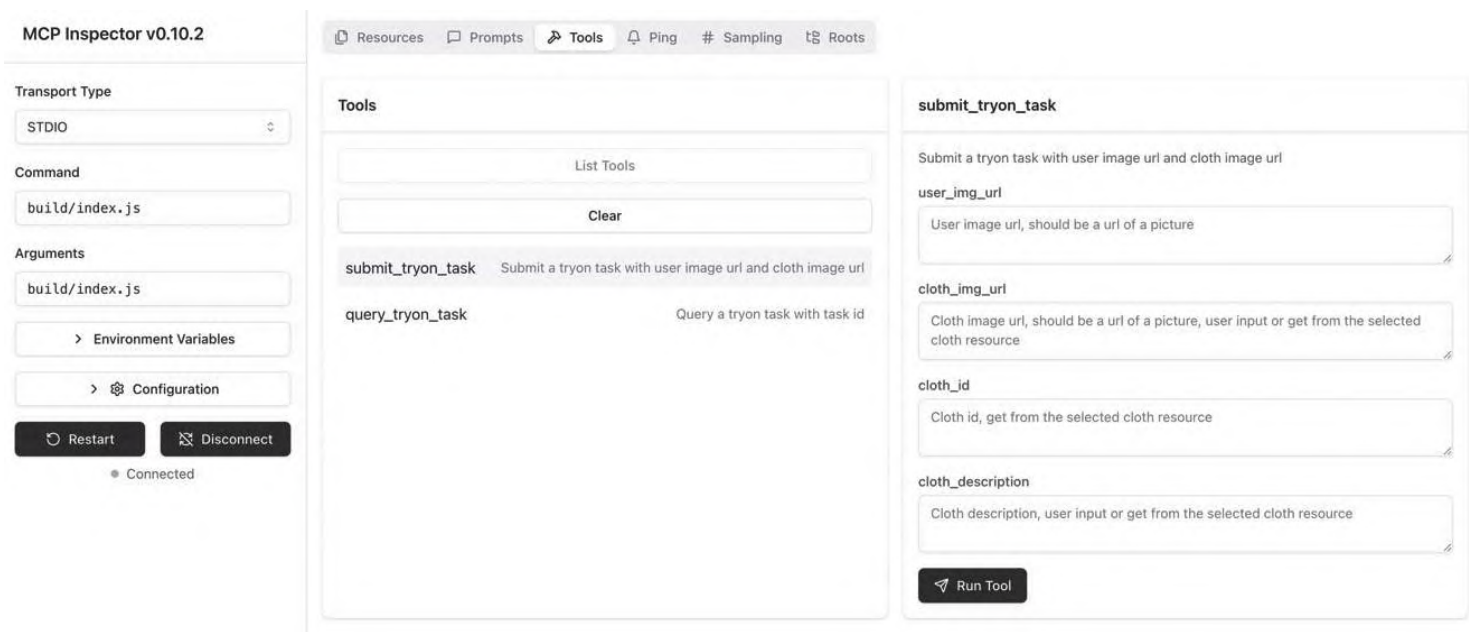


图3-32　通过MCP　Inspector获取工具列表

实现调用工具的逻辑

在src/index.ts中实现调用试衣工具的核心逻辑（主要包含两个工具的处理）：

```
/**
 * 实现submit_tryon_task和query_tryon_task两个工具的逻辑
 */
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  try {
    const apiKey = "xxxxxx";
    const client = new HeyBeautyClient({ apiKey });
    switch (request.params.name) {
      case "submit_tryon_task": {
        const user_img_url = String(request.params.arguments?.user_img_url);
        const cloth_img_url = String(request.params.arguments?.cloth_img_url)
;
        const cloth_id = String(request.params.arguments?.cloth_id);
        const cloth_description = String(
          request.params.arguments?.cloth_description
        );
        if (!user_img_url) {
```

```
    throw new Error("user image is required");
}


if (!cloth_img_url) {
    throw new Error("cloth image is required");
}


const res = await client.submitTask({
    user_img_url,
    cloth_img_url,
    cloth_id,
    cloth_description,
});


return {
    content: [
        {
            type: "text",
            text: JSON.stringify(res),
        },
    ],
};
```

```
}

case "query_tryon_task": {
    const task_id = String(request.params.arguments?.task_id);
    if (!task_id) {
        throw new Error("task id is required");
    }

    const res = await client.queryTask({ task_id });
    return {
        content: [
            {
                type: "text",
                text: JSON.stringify(res),
            },
        ],
    };
}
```

```
            default:
                throw new Error("Unknown tool");
        }
    } catch (error: any) {
        throw new Error("call tool failed: " + error.message);
    }
});
```

❑

submit_tryon_task

接到submit_tryon_task工具的调用请求时，从请求参数中读取用户照片地址、衣服图片地址、衣服ID、衣服描述等信息，通过HeyBeautyClient调用submitTask方法，提交试衣任务，得到试衣任务的task_id。

❑

query_tryon_task

接到query_tryon_task工具的调用请求时，从请求参数中读取task_id，调用HeyBeautyClient的queryTask方法，查询试衣结果。

调试工具

❑

调试submit_tryon_task接口

在MCP Inspector中输入请求参数，对submit_tryon_task工具进行调试，工具响应成功会返回试衣任务的task_id，如图3-33所示。

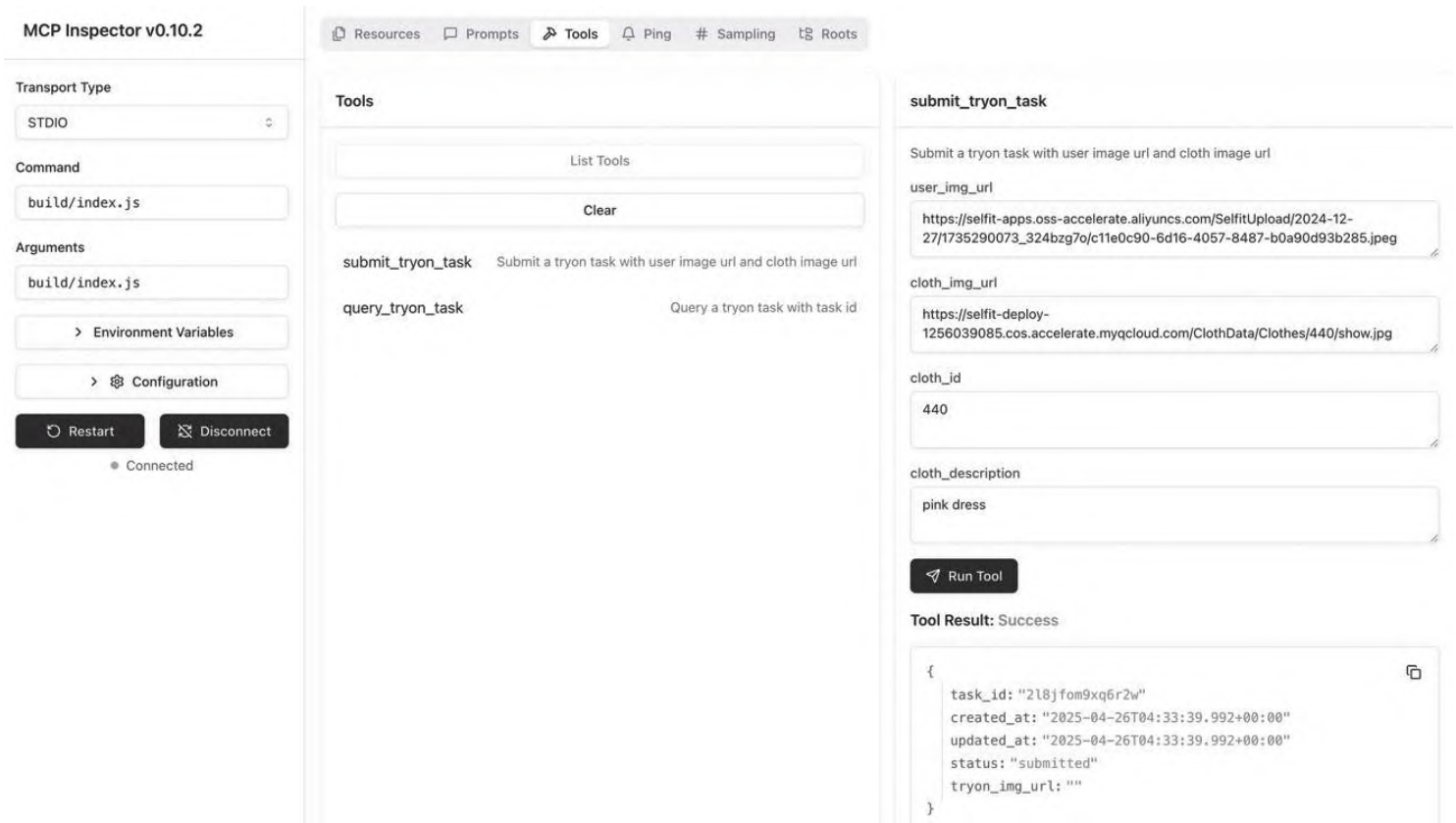


图3-33　对submit_tryon_task工具进行调试

调试query_tryon_task接口

在MCP　Inspector中，输入请求参数，对query_tryon_task工具进行调试，工具响应成功会返回试衣任务的结果信息，如图3-34所示。

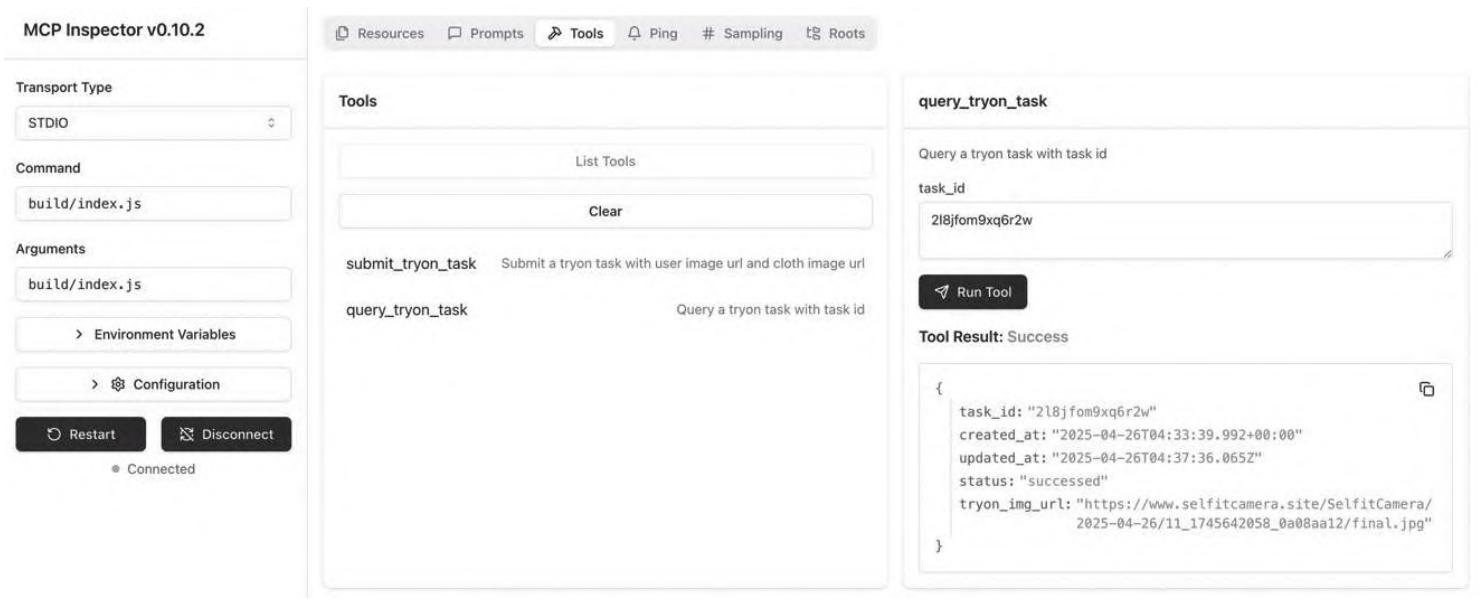


图3-34　对query_tryon_task工具进行调试

当任务状态为successed时，表示试衣流程已成功完成，tryon_img_url字段中返回的是用户试穿衣服后的照片地址。

5.　在大模型客户端测试

重启Claude，选择一张衣服图片，输入一张照片的地址，在输入框中写明试衣需求，如图3-35所示。



图3-35　通过Claude虚拟试衣

Claude解析用户选择的图片地址和输入的内容，请求大模型调度，大模型返回要调用的工具名称和请求参数；客户端调用HeyBeauty　MCP服务器的submit_tryon_task工具，得到响应的task_id，表明试衣任务已经成功创建，如图3-36所示。



图3-36　试衣任务已成功创建

接下来，继续由Claude请求大模型调度，根据大模型的调度指示，调用HeyBeauty　MCP服务器的query_tryon_task工具，查询试衣任务结果。当查到的任务状态为successed的时候，不再请求查询任务工具，而是由大模型根据查询试衣任务的结果，将最终的试衣效果图回复给用户。用户打开链接即可看到试衣效果图。在这里，用户通过一次对话完成了试穿衣服的需求，如图3-37所示。



图3-37　用户通过一次对话完成了试穿衣服的需求

我把这个任务中涉及的图片——用户输入的照片、选择的衣服图片和最终试衣的效果图——放在了同一张图里，更加直观地展示试衣的效果，如图3-38所示。

至此，HeyBeauty　MCP服务器的核心功能就开发完成了。

用户可以在任意支持Resources的客户端，选择一张衣服图片，再发送一张照片进行试穿。



图3-38　HeyBeauty试衣效果展示

如果客户端不支持Resources，那就需要用户输入两张图片的地址，比如在Cursor中使用HeyBeauty　MCP时，需要如图3-39（右侧区域）所示这样发送提示词。

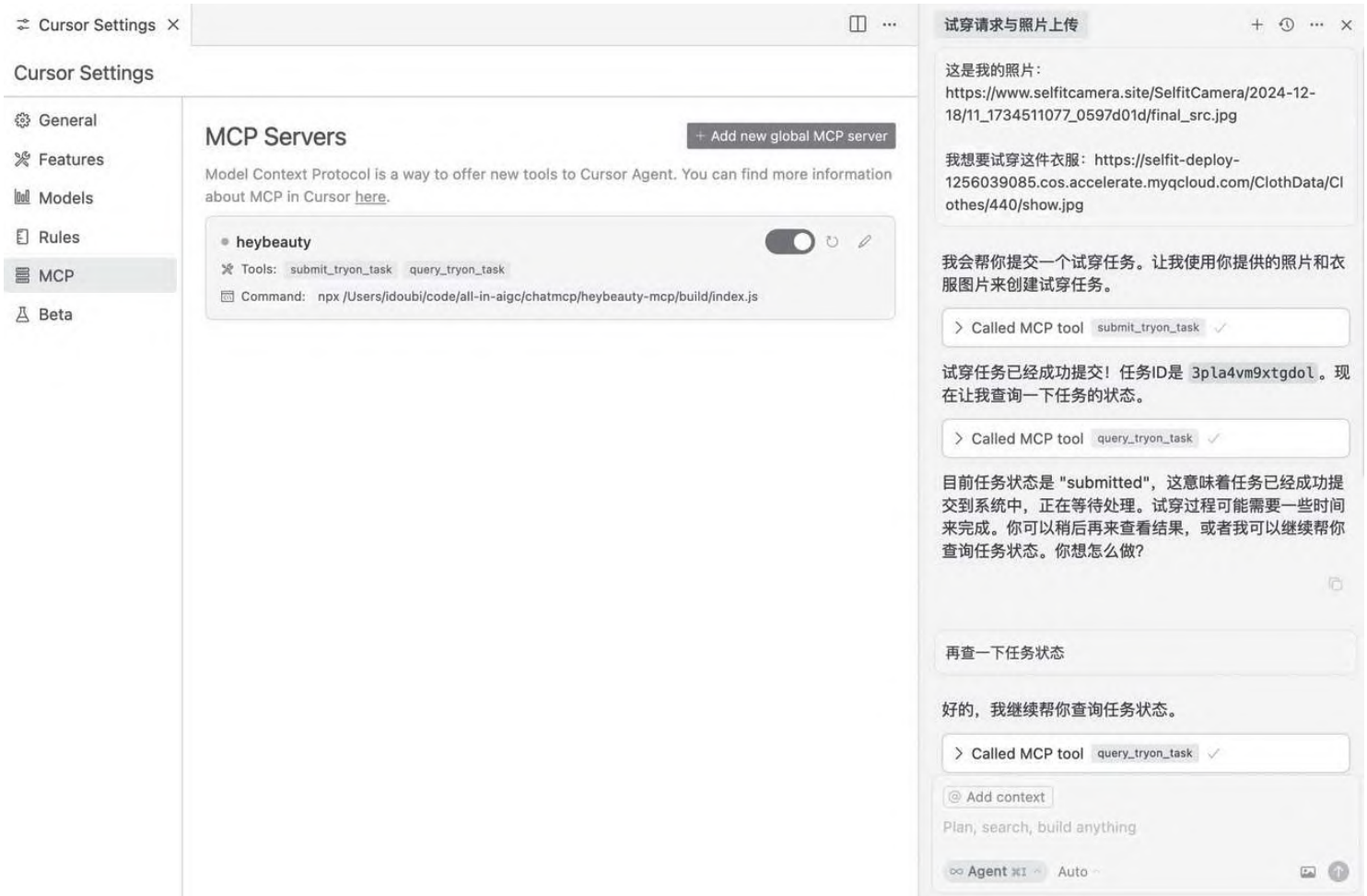


图3-39　在Cursor中使用HeyBeauty　MCP服务器

6. 实现与提示词（Prompts）相关的功能

为HeyBeauty MCP服务器内置提示词

通过前面的步骤，我们实现了HeyBeauty MCP服务器，可以实现试衣需求。但是用户需要在大模型客户端明确告诉大模型自己要试衣，如果只发图片地址，大模型可能不会调度HeyBeauty MCP服务器。另外，在成功提交试衣任务之后，如何轮询试衣任务状态，以及多久轮询一次，不同的模型调度结果会有比较大的差别。

我们可以在HeyBeauty MCP服务器通过提示词模板来提高HeyBeauty MCP服务器的调度准确性，明确告知大模型查询试衣任务的调度频率，减少不必要的资源消耗。

修改src/index.ts中返回提示词列表和获取提示词内容的方法，自定义一个tryon_cloth的提示词模板，指导大模型在接到用户发送的图片地址后，自动调度HeyBeauty MCP服务器的试衣工具。

```
/**
 * 列出服务器实现的提示词列表
 * 暴露一个tryon_cloth提示词（使用用户照片和衣服图片试衣）
 */
server.setRequestHandler(ListPromptsRequestSchema, async () => {
    return {
        prompts: [
            {
                name: "tryon_cloth",
                description: "Tryon with user image and cloth image",
            },
        ],
    };
});
```

```
/**
  *  返回tryon_cloth提示词的内容
  */
server.setRequestHandler(GetPromptRequestSchema,  async  (request)  =>  {
    if  (request.params.name  !==  "tryon_cloth")  {
        throw  new  Error("Unknown  prompt");
    }
```

```
        return {
            messages: [
                {
                    role: "user",
                    content: {
                        type: "text",
                        text: `You are a helpful assistant that helps users try on clothes
    virtually. When a user provides their photo URL and either:
    1. A cloth image URL they want to try on, or
    2. Selects a cloth item from the available resources
```

Here's how to handle each case:

1. If the user provides their own cloth image URL:
   - Use the submit_tryon_task tool with:
     - user_img_url: The URL of the user's photo
     - cloth_img_url: The URL of the cloth image provided by the user
   - cloth_id and cloth_description can be left empty

2. If the user selects a cloth item from resources:

   - Use the submit_tryon_task tool with:

     - user_img_url: The URL of the user's photo

     - cloth_img_url: The URL from the selected cloth resource

     - cloth_id: The ID from the selected cloth resource

     - cloth_description: The description from the selected cloth resource

After submitting the task:

- Get the task_id from the response

- Use the query_tryon_task tool every 5 seconds to check the task status

- Continue checking until the task status is either "succeeded" or "failed"

- If successful, display the tryon_img_url in markdown format: ![Try-on Result]

(tryon_img_url)

- If failed, inform the user about the failure

Throughout the process:

- Keep the user informed about the current status
- Be patient and friendly
- Handle any errors gracefully

```
Here is the user's photo URL and either their cloth image URL or the selected cloth
item:`,
                },
            },
        ],
    };
});
```

在MCP Inspector中调用提示词列表，查询提示词的内容，如图3-40所示。

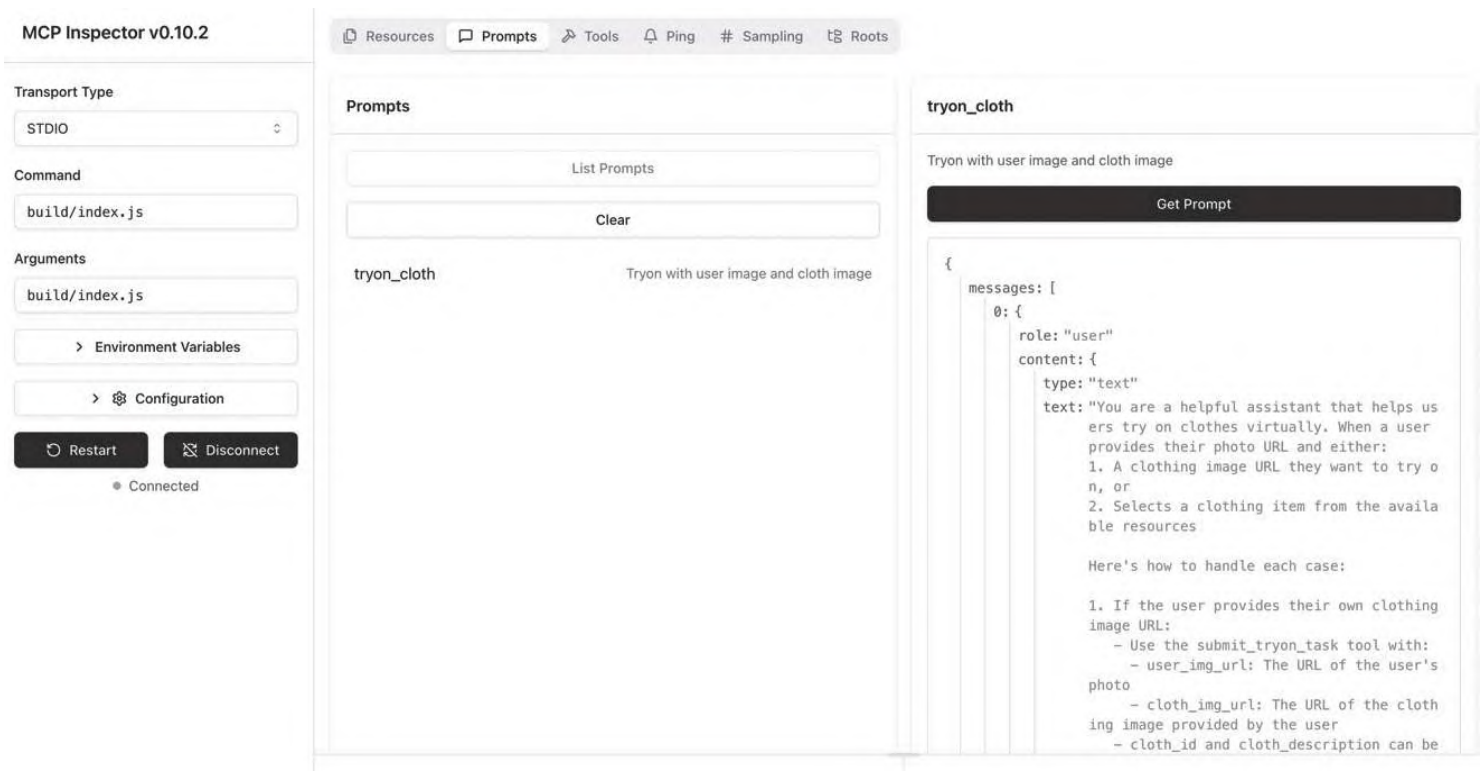


图3-40 在MCP Inspector中查询提示词的内容

在Claude中使用提示词

重启Claude，点击右下角的连接图标，可以看到HeyBeauty MCP服务器提供的提示词列表和资源列表，如图3-41所示。

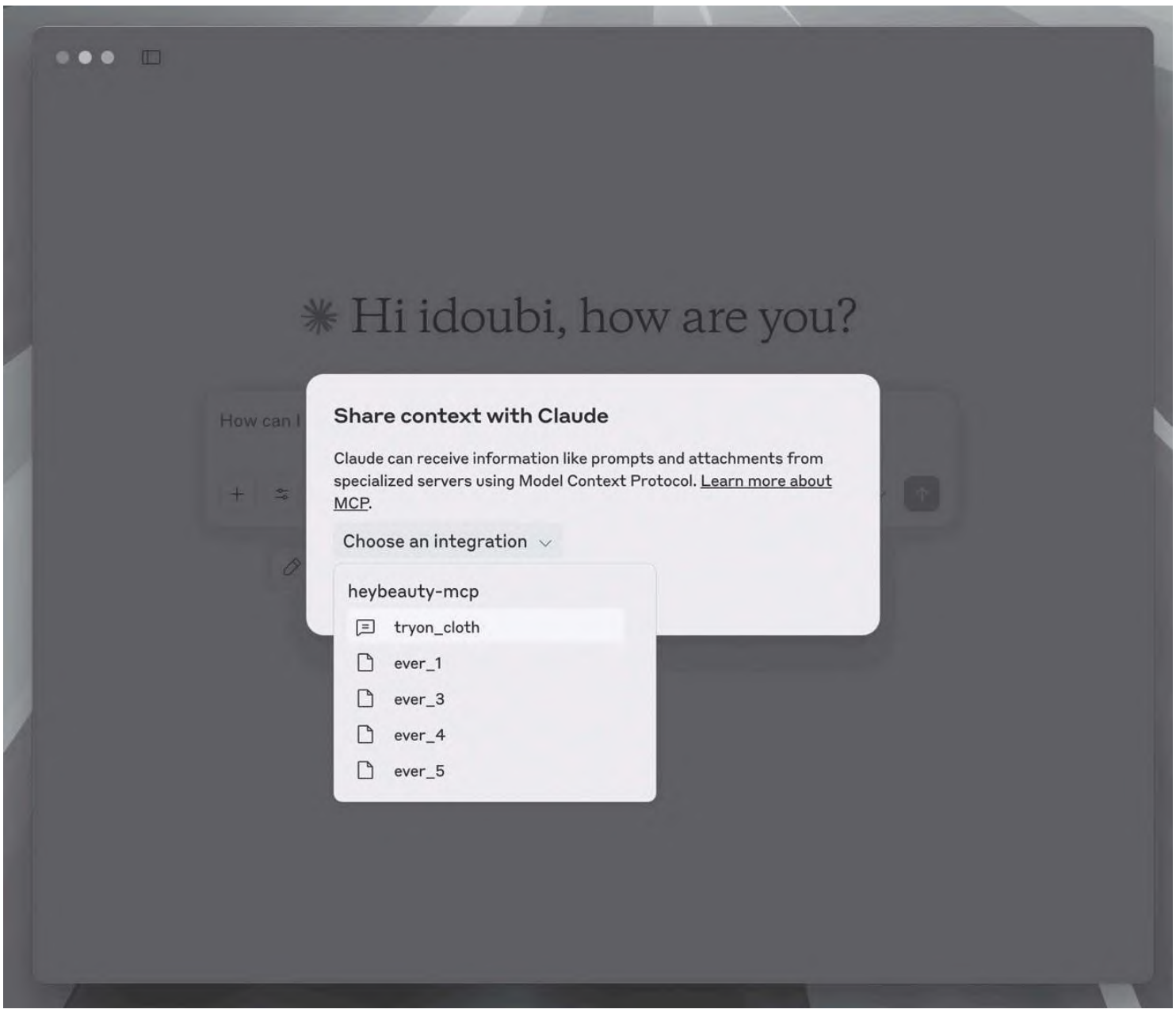


图3-41　HeyBeauty　MCP服务器提供的提示词列表和资源列表

在MCP中，工具通常由大模型自动选择，而资源和提示词则由主机或用户主动选择。在Claude的实现中，从MCP服务器获取的资源和提示词会以列表形式展示给用户，用户可通过点击进行选择。这种设计将主动权交给用户，交互直观，体验良好。用户可以一次性选择多个资源或提示词。选择完成后，Claude会向MCP服务器请求相应的内容，并将其与用户输入的问题一并发送给大模型进行处理。

选择提示词tryon_cloth和一张衣服图片，用户只需要在输入框中输入照片地址，不需要任何文字说明，大模型即可识别HeyBeauty　MCP服务器的试衣工具并进行调度，如图3-42所示。

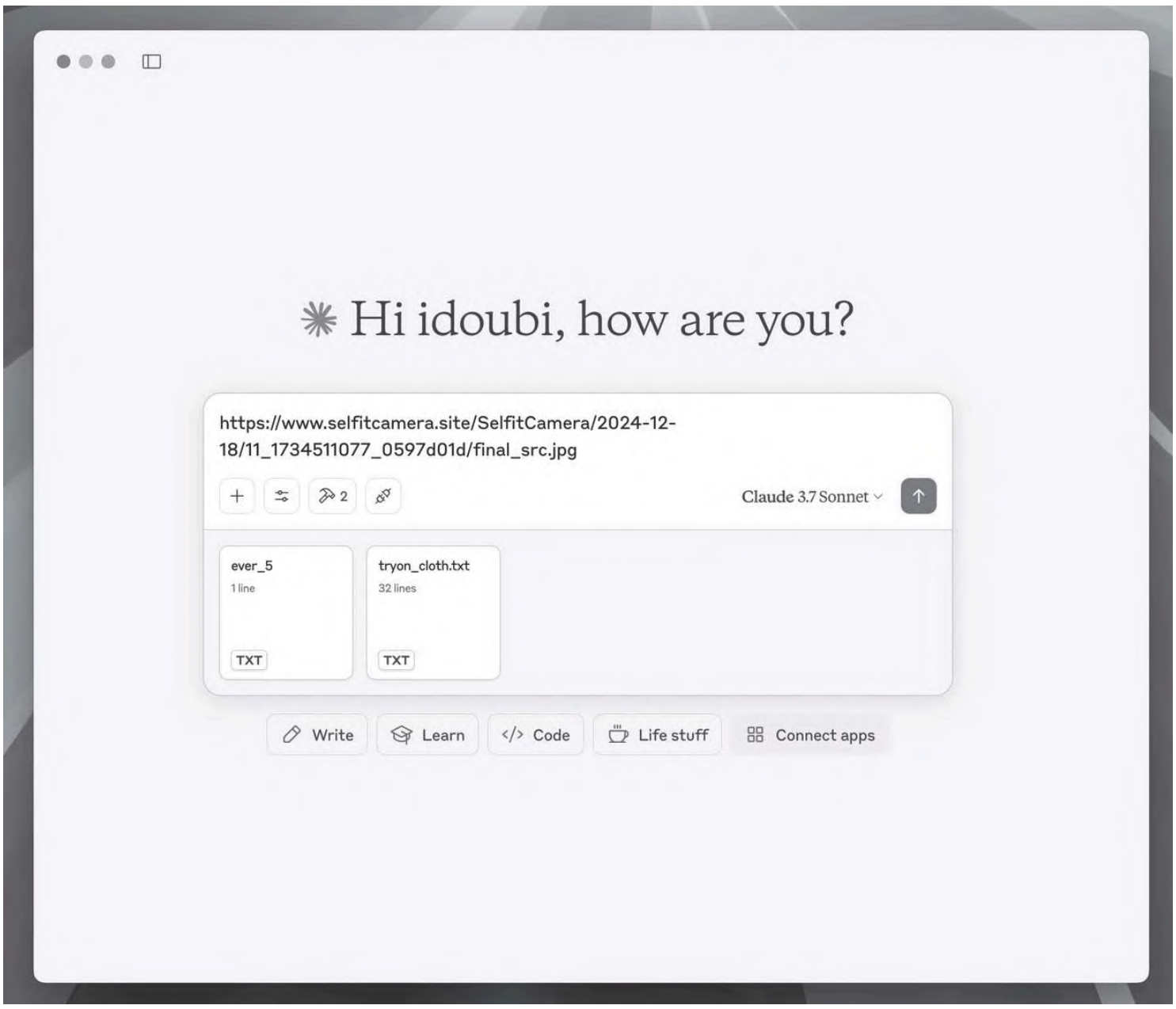


图3-42　使用MCP服务器内置的提示词和资源

在该交互中，提示词的作用尤为关键，它不仅定义了任务触发的条件（如检测用户输入的图片地址），还详细规定了后续工具的调用流程，包括参数格式、任务提交方式及状态查询逻辑。这种通过提示词模板驱动的设计，极大地降低了用户操作的复杂度，并提升了大模型对任务意图理解的准确率。对于开发者而言，合理撰写提示词内容，尤其是清晰描述工具用途和参数约束，是确保模型正确调度工具的基础，也是实现低代码甚至零代码集成的核心路径。

客户端调用HeyBeauty　MCP服务器的submit_tryon_task工具提交试衣任务之后，拿到task_id开始查询任务状态，如图3-43所示。

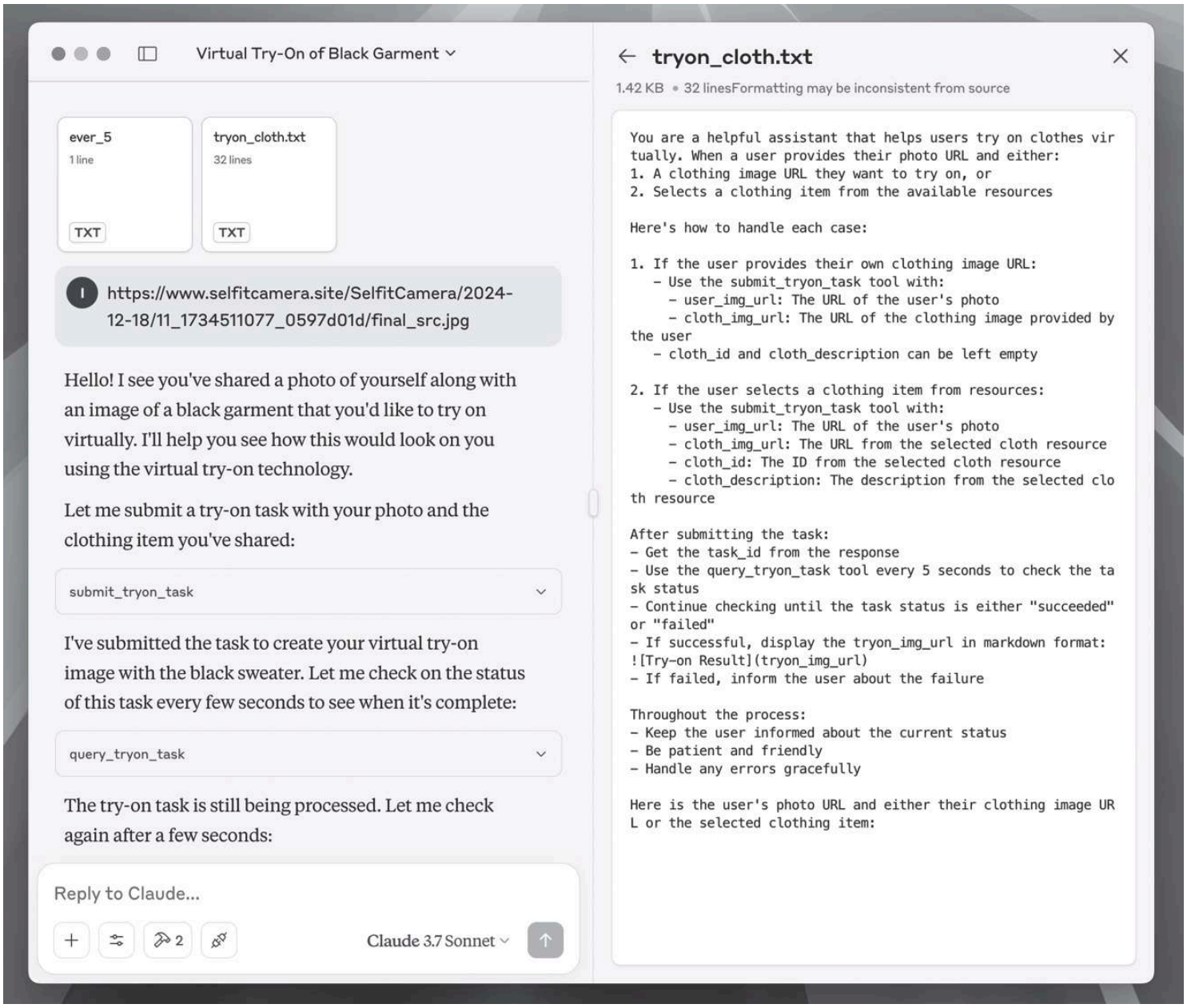


图3-43　客户端开始查询任务状态

试衣任务提交成功后，进入状态查询流程。虽然这一过程由提示词定义，但其运行逻辑完全交由大模型客户端自动控制。这种“提示词定义+客户端执行”的分离式结构，使得系统具备高度灵活性，同时也对提示词的准确性提出更高的要求。

值得注意的是，提示词中定义的轮询策略（如每5秒查询一次）并非强约束，不同客户端和模型对它的解释可能存在差异。因此，在实际部署中，开发者需关注任务延迟、资源占用与用户体验的平衡，适当调整提示词策略，以确保执行过程既稳定又高效。

每5秒调用query_tryon_task工具查询一次任务状态，大概查了10次之后，客户端得到了试衣结果，如图3-44所示。

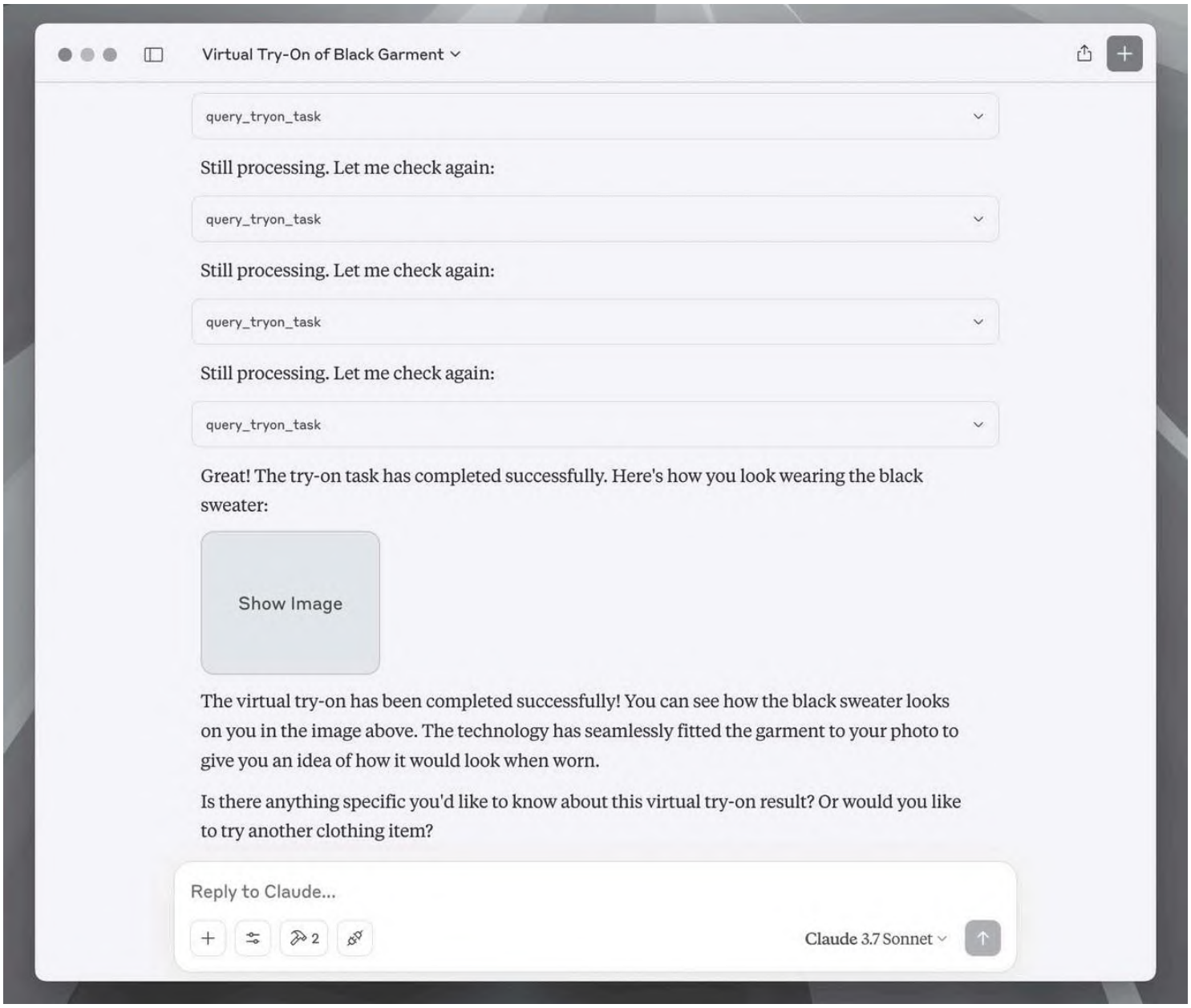


图3-44　客户端得到试衣结果

以上过程由Claude自主控制循环调度工具，根据提示词中的轮训间隔，调用query_tryon_task工具查询试衣任务状态。开发者可根据实际情况调整查询试衣任务的轮训间隔。

需要注意的是，轮训间隔的设置是由提示词控制的，并不是通过程序逻辑设定的，因此，在不同的大模型客户端中调用不同的大模型时，实际执行情况可能不同，很可能跟你的预期不一致。

把用户输入的照片地址、选择的衣服图片、试衣结果图片放在一起，可以看到本次试衣任务完成得很不错，如图3-45所示。而用户只需在大模型客户端输入一张照片的地址，选择一张衣服图片，选择一个内置的提示词即可，交互非常简单。



图3-45　试衣效果图

3.3.5　发布HeyBeauty　MCP服务器

通过前面的步骤，我们已经实现了一个功能完整的HeyBeauty　MCP服务器，帮助用户进行虚拟试衣。

在真正发布MCP服务器之前，我们需要先检查一遍实现代码，优化业务逻辑。为简单起见，这里就不展开了，优化思路可以参考flomo　MCP的案例。

1.　发布到GitHub平台

登录GitHub，创建代码仓库并提交HeyBeauty　MCP服务器的代码：

```
git  init

git  remote  add  origin  git@github.com:chatmcp/heybeauty-mcp.git

git  add  .

git  commit  -m  "first  version"

git  push  origin  main
```

2.　发布到npm平台

修改package.json，设置为公有npm包：

```
{
  "name": "heybeauty-mcp",
  "version": "0.1.0",
  "description": "HeyBeauty Virtual TryOn",
  "private": false,
  "type": "module",
  "bin": {
    "heybeauty-mcp": "./build/index.js"
  },
  "files": ["build"],
  "scripts": {
    "build": "tsc && node -e \"require('fs').chmodSync('build/index.js', '755')\"",
    "prepare": "npm run build",
    "watch": "tsc --watch",
    "inspector": "npx @modelcontextprotocol/inspector build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "0.6.0"
  },
  "devDependencies": {
    "@types/node": "^20.11.24",
    "typescript": "^5.3.3"
  }
```

```
}
```

执行命令将其发布到npm平台：

```
npm  publish  --access=public
```

发布成功后，在npm官网可以看到这个包。然后其他人就可以通过以下配置使用HeyBeauty MCP服务器了：

```
{
   "mcpServers":  {
      "heybeauty-mcp":  {
         "command":  "npx",
         "args":  ["-y",  "heybeauty-mcp"],
         "env":  {
            "HEYBEAUTY_API_KEY":  "xxxxxx"
         }
      }
   }
}
```

3. 发布到第三方MCP应用市场

可以把HeyBeauty MCP服务器提交到MCP.so之类的第三方应用市场，以方便更多人发现和使用，如图3-46所示。

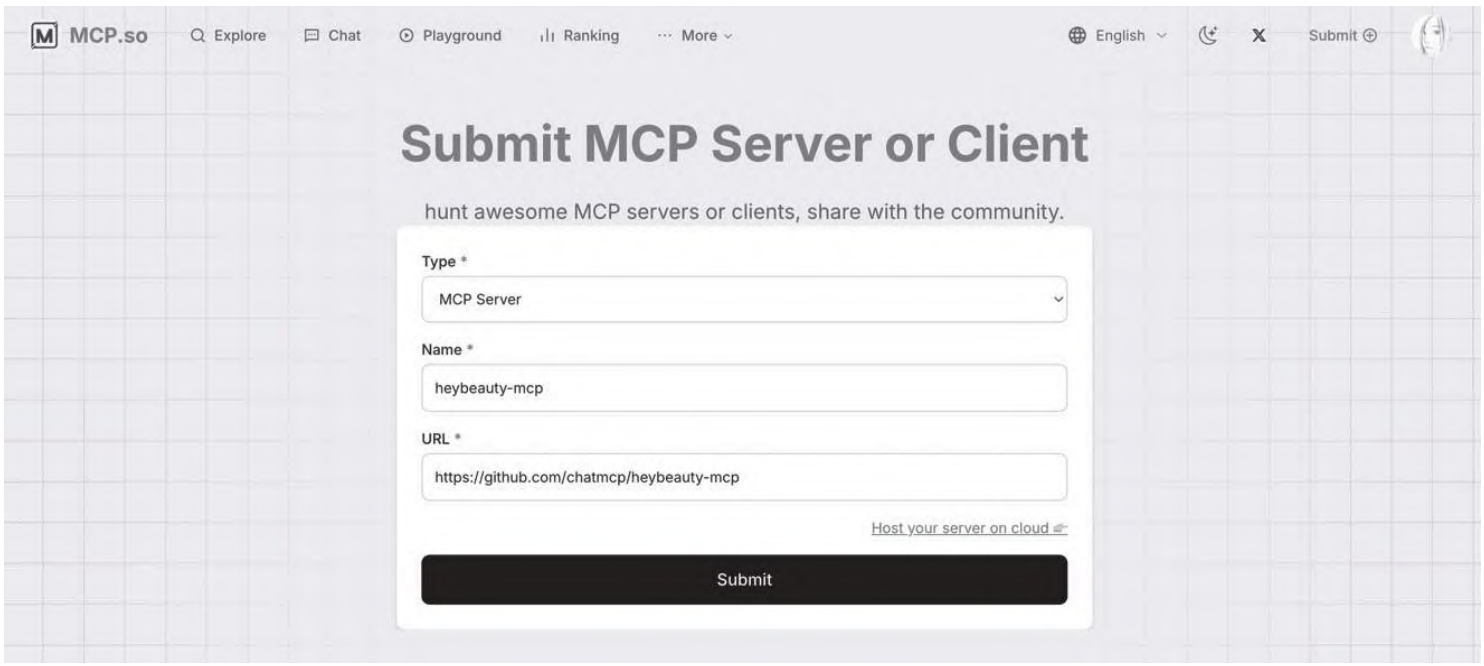


图3-46　将HeyBeauty　MCP服务器发布到第三方应用市场MCP.so

MCP.so应用市场展示的HeyBeauty　MCP服务器的详情如图3-47所示。

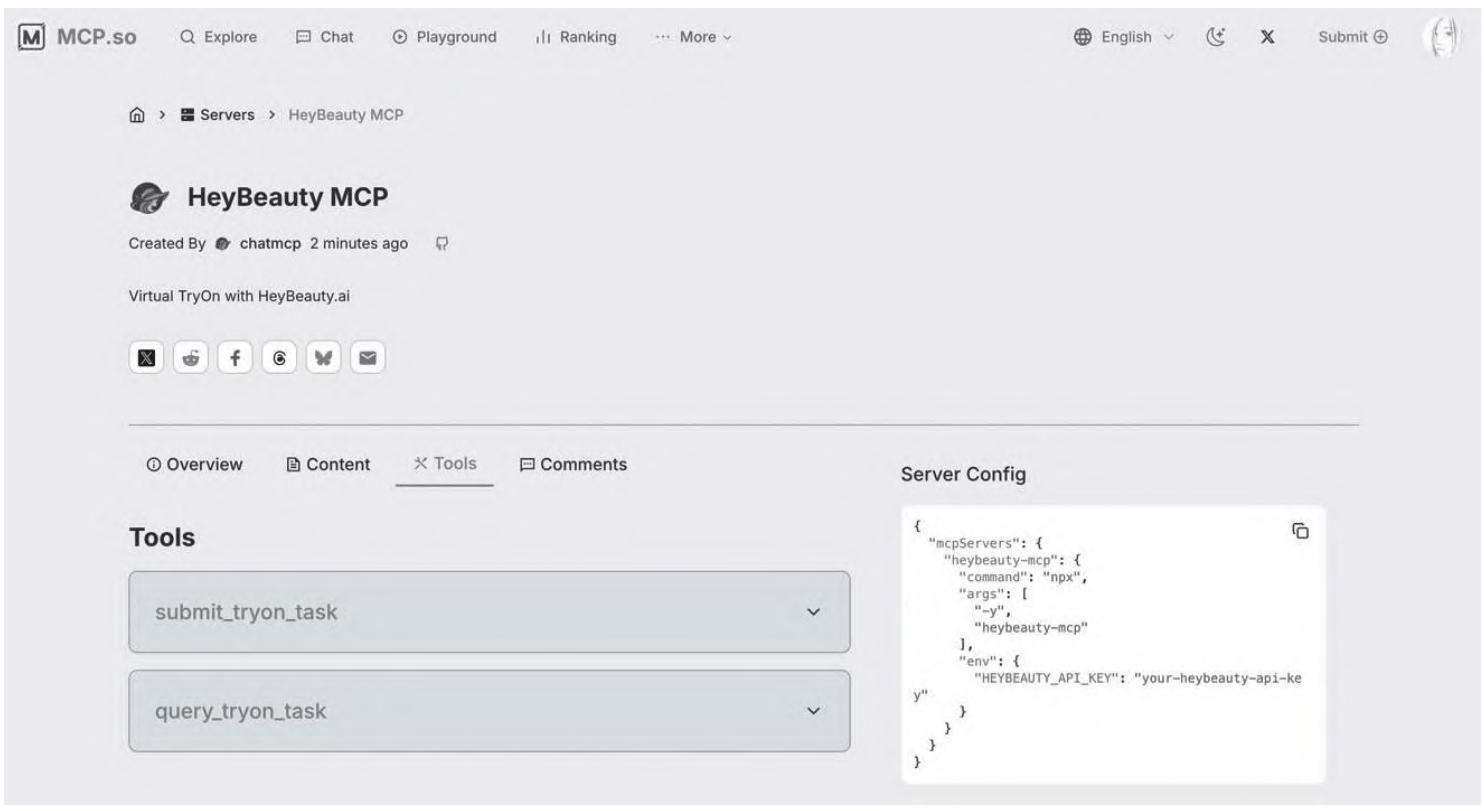


图3-47　MCP.so应用市场展示的HeyBeauty　MCP服务器

3.3.6　案例2小结

本节以HeyBeauty　MCP服务器为例，展示了一个支持图片处理的MCP服务器的完整开发流程，主要实现了以下三大能力。

❑

Resources（资源）

通过ListResources和ReadResource接口，向客户端提供HeyBeauty平台上的衣服资源，支持用户选图试衣。

❑

Tools（工具）

实现submit_tryon_task和query_tryon_task两个工具，接入HeyBeauty API，实现虚拟试衣任务的提交与查询。

❑

Prompts（提示词）

内置tryon_cloth提示词模板，引导大模型正确调用试衣工具，同时控制查询频率，优化资源使用。

通过该案例，我们学习了如何开发支持图片处理的MCP服务器，以及如何整合资源、工具和提示词，从而提升用户体验。

与此同时，这个案例展示了MCP在图像处理场景中的实际应用，希望能为大家今后开发如图像编辑、风格迁移等更丰富的图像类MCP服务器提供参考。

3.4 小结

本章通过两个案例展示了MCP服务器开发的完整流程，帮助读者掌握创建、调试与发布MCP服务器的基本方法，从而快速上手实际开发。

本章重点展示了MCP服务器中最核心的能力——工具能力的实现方式，并进一步介绍了资源能力与提示词能力的开发方法，使读者对MCP服务器的功能体系有更全面的认识，从而在实际应用中构建出功能完善、可扩展的MCP服务器。

第4章

MCP客户端开发

在第2章讲解MCP架构时，我们了解到MCP定义了主机、客户端、服务器三个角色。MCP客户端是“寄生”在主机中的子进程，用于与MCP服务器通信。而主机也就是我们常说的大模型客户端，主要用于与大模型交互。

我们在第3章通过两个实际的案例演示了MCP服务器开发的完整流程。用户在大模型客户端配置后即可使用我们开发的MCP服务器。

为了跟MCP服务器开发对应，本章的主题叫作“ MCP客户端开发”，但实际讲述的是大模型客户端的开发流程。在本章中，我们统一使用“MCP客户端”这一表述，后续不再额外说明。

本章计划通过两个实际的案例，演示MCP客户端开发的完整流程。第一个案例是开发一个AI对话助手，读取用户本地配置的MCP服务器列表，由大模型调度工具增强生成；第二个案例是开发一个AI搜索智能体，通过程序预置的MCP服务器配置，用特定的工具列表编排工作流，实现RAG（检索增强生成）。

4.1　MCP客户端开发流程

MCP客户端通常是大型项目，不同类型的应用对应不同的技术栈。例如，在开发跨平台的桌面端对话类应用（比如Claude、Cursor等）时，常会使用React Native、Flutter、Electron或Tauri；而智能体类项目（比如Manus、Lovart等）多以Web形态为主，常用的技术栈包括Next.js、Nuxt等。鉴于篇幅所限，本章的两个案例将重点讲解后台业务的核心实现逻辑，前端/UI部分不作展开。

在进入具体案例前，我们先梳理一下MCP客户端开发的基本流程，聚焦核心步骤与实现思路，具体细节将在后文案例中详述。

核心步骤与实现思路

1.　安装MCP客户端SDK

在创建MCP客户端项目之后，需要安装对应编程语言的MCP客户端SDK，主要用于创建MCP客户端实例，连接MCP服务器，执行获取工具列表、调用工具等操作。

以TypeScript开发为例，安装对应的MCP客户端SDK的命令如下：

```
npm install @modelcontextprotocol/sdk
```

2.　读取MCP服务器列表

MCP客户端需要读取用户配置的MCP服务器列表，以获取可用的工具并提供给大模型调度使用。因此在开发MCP客户端时，需要先实现读取MCP服务器列表的逻辑。对于不同类型的MCP客户端，其配置方式有所不同。

❑

如果是桌面端对话类MCP客户端，MCP服务器列表一般配置在本地，由用户根据自身需求填入MCP服务器配置。例如在macOS操作系统下，Claude的MCP服务器配置文件为~/Library/Application Support/Claude/claude_desktop_config.json，Cursor编辑器的MCP服务器配置文件为~/.cursor/mcp.json。

❑

如果是智能体类MCP客户端，MCP服务器列表一般配置在项目文件中，由开发者根据业务需求预置MCP服务器配置。

3.　从MCP服务器获取工具列表

MCP客户端在获取MCP服务器列表之后，需要连接MCP服务器以获取其内部定义的工具列表。此处需要用到第1步的MCP客户端SDK，创建MCP客户端实例，连接MCP服务

器，并发送listTools请求，以获取该服务器定义的工具列表。

4. 将工具列表发送给大模型，由其选择调用

MCP客户端可过滤和组合从用户配置的MCP服务器列表获取的全部工具，并设置提示词请求大模型挑选工具，由大模型选择合适的工具并返回工具调用信息（工具名称、参数）。

5. 解析大模型返回的工具调用信息

大模型的回复为纯文本，其中的工具调用信息被特殊标签包裹。MCP客户端需从中提取有效的调用数据，以便后续调用工具。

6. 调用工具

MCP客户端根据上一步解析出来的工具调用信息，向目标MCP服务器发送callTool请求，并获取工具调用结果。

7. 总结输出或进入下一轮工具调用

MCP客户端可将工具调用结果再次提交给大模型，进入下一轮工具调用流程（步骤4）。若大模型不再返回包含工具调用信息的回复，或调用次数已达设定上限，则MCP客户端终止循环，输出最终结果。

可以用一幅图概括MCP客户端的交互流程（请注意，这里跟上述的核心步骤并非一一对应），如图4-1所示。

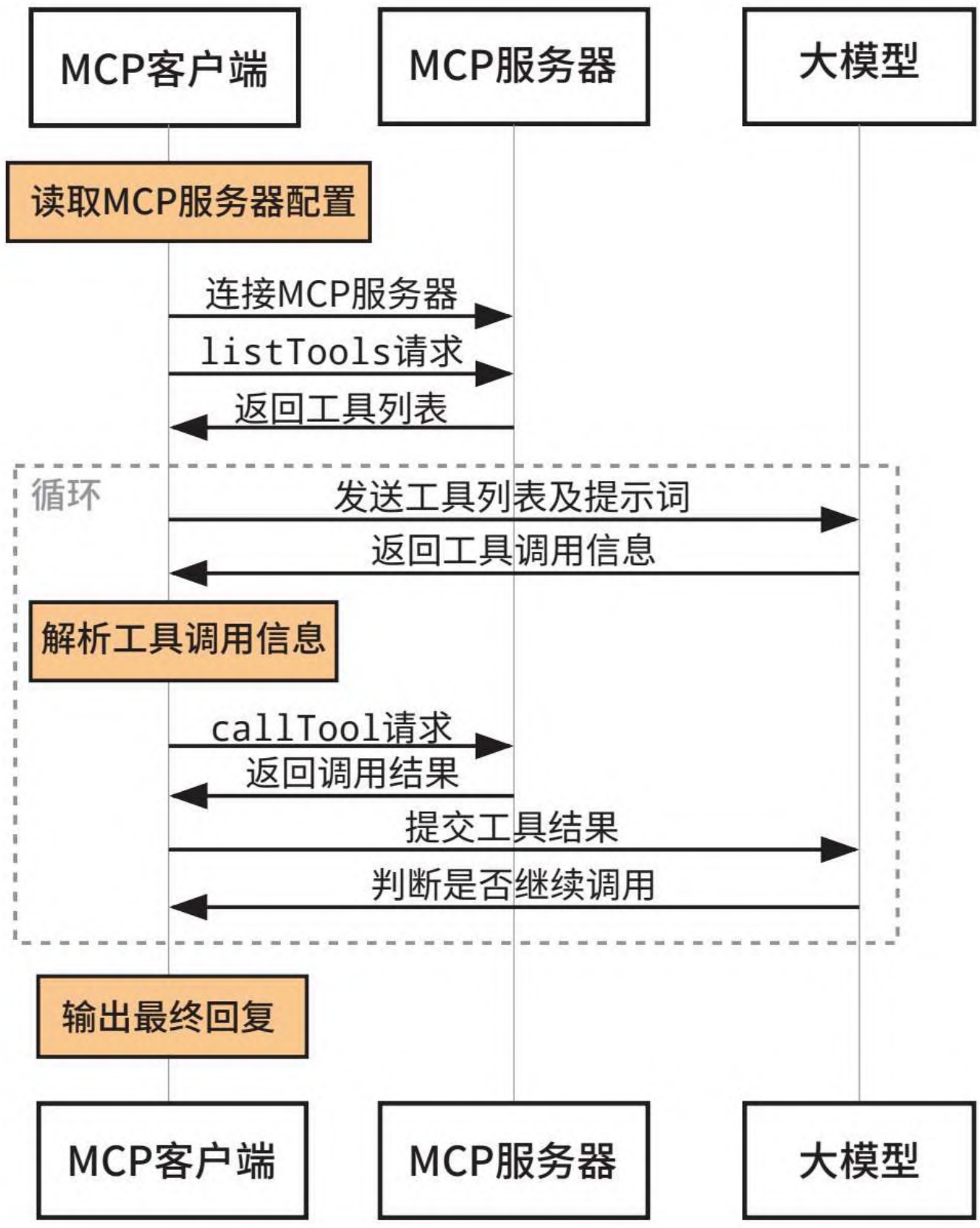


图4-1　MCP客户端的交互流程

接下来，我们就通过具体的案例来讲解MCP客户端的开发。

4.2　案例1：开发AI对话助手

在本节中，我们计划开发一个名为ChatMCP的AI对话助手，通过这个案例来演示对话助手类MCP客户端开发的具体流程。

4.2.1　开发目标

本案例旨在开发一个通过MCP服务器增强大模型生成能力的通用型AI对话助手，其中，外部功能的扩展由用户配置MCP服务器实现。

按照“测试用例驱动开发”的思路，我们先来设计一个测试用例，梳理用户使用ChatMCP与大模型对话的流程。

1.　用户打开ChatMCP的配置文件，写入MCP服务器配置。

2.　用户启动ChatMCP，ChatMCP从配置文件中读取MCP服务器列表，并获取所有MCP服务器提供的所有工具。

3.　用户在对话框中输入提问，ChatMCP带上支持的工具列表，请求大模型调度。

4.　大模型挑选工具，返回工具名称、调用参数和工具所属的服务器名称。

5.　ChatMCP调用工具，获得结果。

6.　ChatMCP带上工具列表、工具调用结果，继续请求大模型调度。

7.　ChatMCP循环步骤4~6，直到大模型的回复内容不包含工具调用信息，或者达到程序设置的最大循环次数时退出循环。

8.　ChatMCP给用户输出最终回复。

用已经在mcp.so/chat上线的ChatMCP网页版演示用户配置MCP服务器与大模型对话并获得回复的效果，如图4-2所示。

在这个演示案例中，用户配置了一个高德地图MCP服务器（amap-maps）。在用户输入提问后，ChatMCP将maps_geo、maps_direction_driving等工具传递给大模型。根据大模型返回的工具调用信息，ChatMCP执行了三次工具调用，最终实现了用户的行程计划需求。

此演示案例旨在从用户视角直观展示ChatMCP网页版的交互流程，参考这个流程，我们来实现ChatMCP客户端的交互逻辑。



图4-2　ChatMCP网页版用户对话演示

4.2.2　前置准备

在实现ChatMCP的交互逻辑之前，我们做一些前置准备，安装需要用到的外部依赖，实现通用的功能函数。

1. 安装SDK

我们选择用TypeScript来开发ChatMCP，所以需要先安装TypeScript版本的MCP客户端SDK：

```
npm install @modelcontextprotocol/sdk
```

2. 实现获取工具列表的函数

我们来实现一个listTools函数，使用SDK创建MCP客户端实例，获取MCP服务器内部定义的工具列表：

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";

import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";



async function listTools({
    command,
    args,
    env = {},
}: {
    command: string;
    args: string[];
    env?: Record<string, string>;
}) {
    const transport = new StdioClientTransport({
        command,
        args,
        env: {
```

```
        ...(process.env as Record<string, string>),
        ...env,
    },
});

const client = new Client({
    name: "chatmcp",
    version: "1.0.0",
});

await client.connect(transport);
        const tools = await client.listTools();

        return tools;
    }
```

使用第3章开发的天气查询MCP服务器配置，测试listTools函数，获得此服务器内部定义的工具列表，如图4-3所示。



图4-3　测试获取MCP服务器工具列表的函数

3.　实现调用工具的函数

我们来实现一个callTool函数，使用SDK创建MCP客户端实例，调用MCP服务器内部实现的工具：

```
async function callTool({
    command,
    args,
    env = {},
    name,
    params,
}: {
    command: string;
    args: string[];
    env?: Record<string, string>;
    name: string;
```

```
    params?: Record<string, unknown>;
}) {
    const transport = new StdioClientTransport({
        command,
        args,
        env: {
            ...(process.env as Record<string, string>),
            ...env,
        },
    });
    const client = new Client({
        name: "chatmcp",
        version: "1.0.0",
    });
```

```
await  client.connect(transport);
```

```
    const  result  =  await  client.callTool({
        name,
        arguments:  params,
    });


    return  result;
}
```

使用上一步的天气查询MCP服务器配置、工具名称和请求参数，测试callTool函数，获得目标工具调用结果，如图4-4所示。



图4-4　测试调用MCP服务器工具的函数

在准备好这两个功能函数之后，接下来就可以实现ChatMCP的交互逻辑了。

4.2.3　读取用户配置的MCP服务器列表

ChatMCP需要设置一个本地文件来保存用户配置的MCP服务器列表。在ChatMCP启动时，从此文件中读取MCP服务器列表。

为了方便实现，我们把Cursor的MCP配置文件作为ChatMCP的配置文件。

macOS操作系统下的配置文件路径是：/Users/$USER/.cursor/mcp.json。

用户在此文件内配置要使用的MCP服务器列表，比如把我们在第3章开发的两个MCP服务器配置进来，配置内容如下：

```
{
    "mcpServers": {
        "weather-mcp": {
            "command": "npx",
            "args": ["-y", "@chatmcp/weather-mcp"],
            "env": {
                "WEATHER_API_KEY": "xxx"
            }
        },
        "flomo-mcp": {
            "command": "npx",
            "args": ["-y", "@chatmcp/flomo-mcp"],
            "env": {
                "FLOMO_API_URL": "https://flomoapp.com/iwh/xxx/xxxxxx/"
            }
        }
    }
}
```

其中，WEATHER_API_KEY和FLOMO_API_URL需要根据实际情况填写，此处采用脱敏处理，用占位符替代。

在开发ChatMCP时，按以下步骤实现读取用户配置的MCP服务器列表的逻辑。

❑

定义MCP服务器数据类型：

```
interface McpServer {
    name: string;
    command: string;
    args?: string[];
    env?: Record<string, string>;
}
```

❑

实现一个函数，从给定的配置内容中解析MCP服务器列表：

```
async function getMcpServers(
    config: string
): Promise<Record<string, McpServer>> {
    const mcpConfig = JSON.parse(config);

    const mcpServers = Object.entries(mcpConfig.mcpServers).reduce(
        (acc, [key, value]) => {
            acc[key] = {
                name: key,
                command: (value as any).command,
                args: (value as any).args,
                env: (value as any).env,
```

```
            };
            return  acc;
        },
        {}  as  Record<string,  McpServer>
    );


    return  mcpServers;
}
```

此函数返回一个key-value对象，其中key是MCP服务器的名称，value是步骤1定义的MCP服务器信息。

❑

读取配置文件，从配置内容中获取MCP服务器列表：

```
const  configFile  =  `/Users/idoubi/.cursor/mcp.json`;
const  config  =  await  fs.readFile(configFile,  "utf-8");
const  mcpServers  =  await  getMcpServers(config);
```

此处的configFile需要开发者根据实际情况设置为用户本地的ChatMCP配置文件地址。

❑

调试接口，输出配置的MCP服务器列表，如图4-5所示。

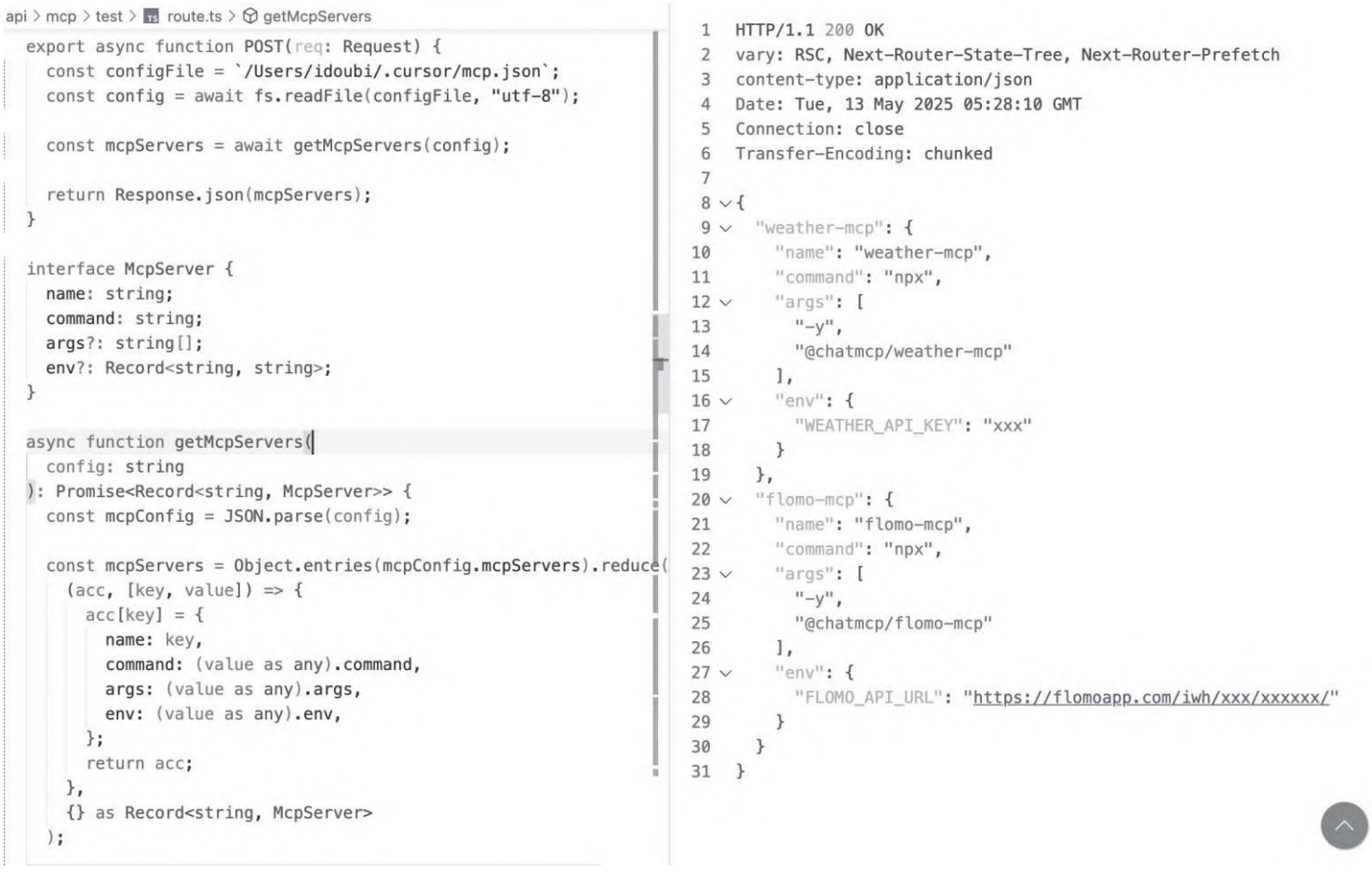


图4-5　调试接口，输出配置的MCP服务器列表

4.2.4　从MCP服务器获取工具列表

ChatMCP启动时，会从配置文件中读取用户配置的MCP服务器列表，并通过SDK与每个MCP服务器建立连接，请求获取每个MCP服务器内部定义的工具列表。

在开发ChatMCP时，按以下步骤实现获取MCP服务器工具列表的逻辑。

1.　定义工具数据类型：

```
interface  McpTool  {
    server_name:  string;
    name:  string;
    description:  string;
    inputSchema:  Record<string,  unknown>;
}
```

2.　实现一个函数，从MCP服务器列表中获取工具列表：

```
async  function  getMcpTools(
```

```
    mcpServers: Record<string, McpServer>
): Promise<McpTool[]> {
    const allTools = await Promise.all(
        Object.entries(mcpServers).map(async ([name, server]) => {
            const tools = await listTools({
                command: server.command,
                args: server.args || [],
                env: server.env || {},
            });

            return tools.tools.map((tool) => ({
                server_name: name,
                name: tool.name,
                description: tool.description || "",
                inputSchema: tool.inputSchema,
            }));
        })
    );

    return allTools.flat();
}
```

此函数遍历MCP服务器列表，并行调用前面步骤实现的listTools函数，获取每个MCP服务器内部定义的工具列表，最后合并返回所有工具列表。

3. 读取配置文件，解析MCP服务器列表，获取所有工具：

```
const configFile = `/Users/idoubi/.cursor/mcp.json`;

const config = await fs.readFile(configFile, "utf-8");

const mcpServers = await getMcpServers(config);

const mcpTools = await getMcpTools(mcpServers);
```

此处的configFile需要开发者根据实际情况设置为用户本地的ChatMCP配置文件地址。

4. 调试接口，输出获取到的所有工具列表，如图4-6所示。

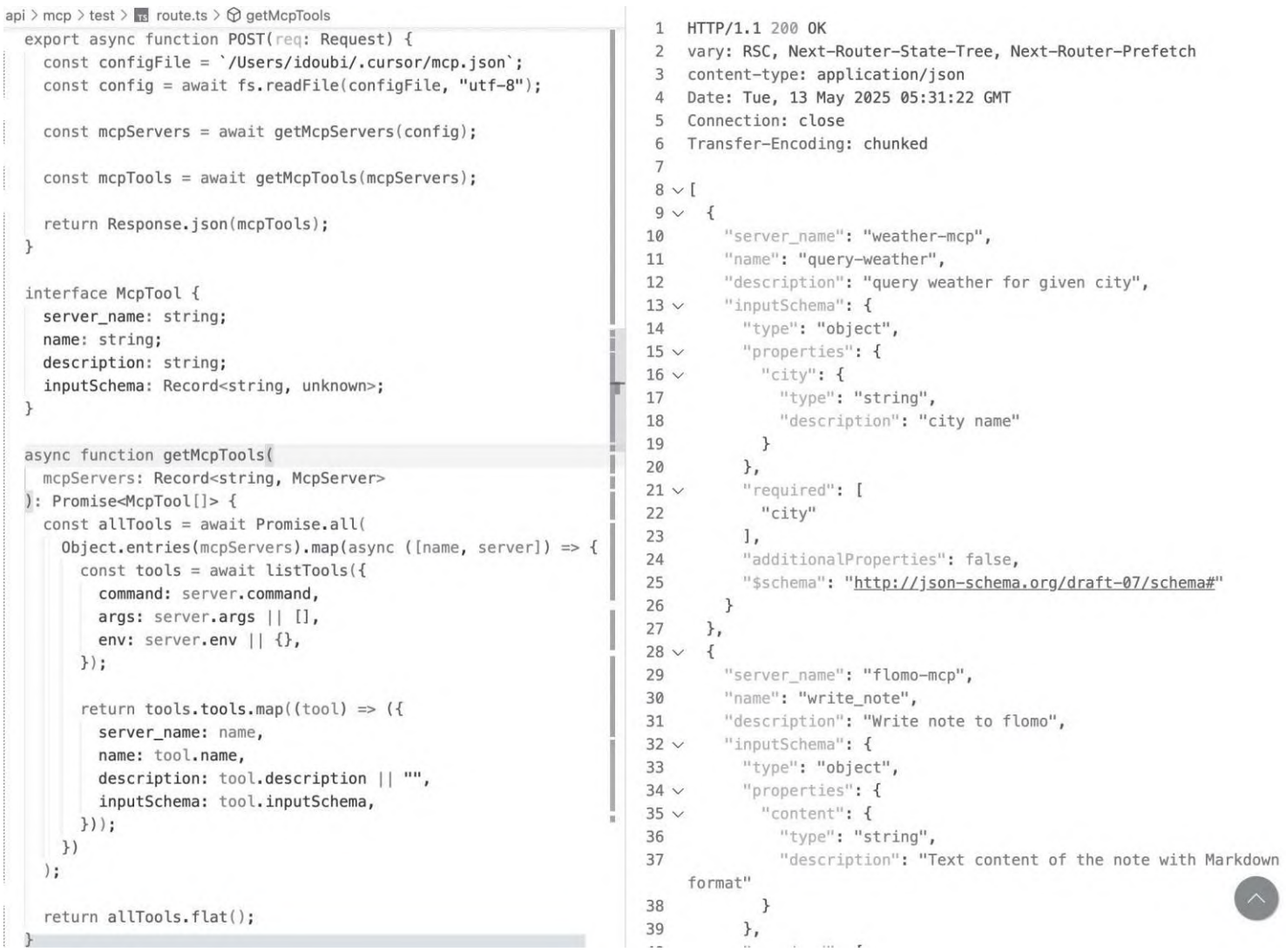


图4-6 调试接口，输出获取到的所有工具列表

4.2.5 请求大模型挑选工具

ChatMCP启动时，获取了用户配置的MCP服务器列表，并通过与每个MCP服务器连接，获取到了所有可用的工具。

用户在ChatMCP的对话框中输入问题后，ChatMCP会根据用户勾选启用的MCP服务器，从所有工具中过滤出部分工具，并将这些工具跟用户的提问一起发送给大模型，由大模型挑选最适合的工具进行调用。

在开发ChatMCP时，按以下步骤实现请求大模型挑选工具的逻辑。

1. 设置系统提示词

为了让大模型更好地理解用户意图，在请求大模型挑选工具时，需要设置系统提示词。

本案例设计的系统提示词如下：

```
你是ChatMCP，由ThinkAny  AI开发的AI对话助手。



#  通用指令
```

请结合可用的上下文信息（CONTEXT_MESSAGES）和上一次工具调用结果（PREVIOUS_TOOL_RE-SULTS），针对用户的查询（USER_QUERY），写出准确、详细且全面的回复。

- 回答应精确、高质量、专业且公正。
- 回答所用的语言必须与提问所用的语言一致。
- 禁止使用带有主观判断倾向或模棱两可的语言，避免如下表达：
  - ″重要的是……″
  - ″不适当的是……″
  - ″……是主观的″

格式要求

- 使用Markdown格式化段落、列表、表格和引用。
- 用二级标题、三级标题分隔内容，如 "## 标题"，但**不要**以标题开头。
- 列表项之间用单个换行符，段落之间用两个换行符分隔。
- 如有图片，使用Markdown渲染。
- 不要写URL 或链接。

工具调用

如需调用工具，请遵循以下流程。

1. 判断是否需要工具，判断依据是：当前用户查询、已有工具结果和可用工具列表（AVAILABLE_TOOLS）。
2. 返回的tool_name仅能使用可用工具列表中明示的工具，tool_params中的参数必须跟in-putSchema完全匹配。
3. 返回的server_name 必须跟tool_name完全匹配。
4. 工具调用格式如下，且必须放在回复的最后一行：

<<tool-start>>

{

"server_name": "提供工具的服务器名称",

```
    "tool_name": "工具名称",
    "tool_params": {
        "参数1": "参数1的值",
        "参数2": "参数2的值"
    }
}
<<tool-end>>
```

- 一次只能调用一个工具，不能提前给出答案。
- 工具调用后等待结果，再继续回复。
- 工具调用失败两次后，不再重试，直接基于已有信息作答并说明原因。

如果不需要工具，直接回复内容，不要包含工具调用区块。

以下为本次请求的变量信息：

用户查询：USER_QUERY={USER_QUERY}

上下文：CONTEXT_MESSAGES={CONTEXT_MESSAGES}

上一次工具调用结果：PREVIOUS_TOOL_RESULTS={PREVIOUS_TOOL_RESULTS}

可用工具列表：AVAILABLE_TOOLS={AVAILABLE_TOOLS}

请用你的推理能力，分析信息，生成最有帮助的回复。

此提示词模板的核心逻辑是：把可用的工具列表通过参数AVAILABLE_TOOLS传递给大模型，让大模型从中选择应该调用的工具来补充上下文。把历史消息和上一次工具调用结果通过CONTEXT_MESSAGES、PREVIOUS_TOOL_RESULTS传递给大模型，让大模型在多轮对话或连续调用多个工具时，有足够的信息作为参考。

如果大模型判断需要调用工具来补充上下文，会返回应该调用的工具信息。上面的提示词约定用特殊标签<<tool-start>>和<<tool-end>>包裹需要调用的工具信息。

2. 实现与大模型对话的函数

在开发ChatMCP时，通过AI SDK库实现与大模型对话的逻辑，并使用OpenRouter作为大模型接口供应商。安装相关依赖的命令如下：

```
npm install ai

npm install @openrouter/ai-sdk-provider
```

然后我们来实现一个chatWithLLM函数，通过此函数请求大模型，并获得大模型的响应内容。chatWithLLM函数的实现逻辑如下：

```
import { openrouter } from "@openrouter/ai-sdk-provider";

import { streamText } from "ai";

async function chatWithLLM({

    query,

    contextMessages,

    tools,

    toolResults,

}: {

    query: string;

    contextMessages?: string;

    tools?: string;

    toolResults?: string;

}) {

    const prompt = mcpPrompt
```

```
    .replace("{USER_QUERY}", query)
    .replace("{CONTEXT_MESSAGES}", contextMessages || "")
    .replace("{AVAILABLE_TOOLS}", tools || "")
    .replace("{PREVIOUS_TOOL_RESULTS}", toolResults || "");

const result = await streamText({
    model: openrouter("anthropic/claude-3.5-sonnet"),
    prompt,
});

return result;
}
```

在此函数中，mcpPrompt是前面步骤定义的系统提示词，query是用户输入的问题，contextMessages是历史对话消息，tools是可用工具列表，toolResults是上一次工具调用结果，作为上下文的一部分与其他内容一同填入提示词的对应位置。函数最终调用AI SDK库的streamText函数，将构造后的提示词发送给大模型，并获取大模型的流式响应内容。



chatWithLLM的实现较为简单，主要通过字符串替换的方式，将系统提示词中的占位符替换为实际内容，从而生成用于请求大模型的提示词。streamText函数封装了调用大模型所需的参数，我们只需传入模型名称和提示词即可。此处通过大模型接口供应商OpenRouter调用anthropic/claude-3.5-sonnet，该模型在识别传入的工具列表（tools）方面表现更优，具有更高的调用准确率。

3. 调试大模型输出结果

把ChatMCP启动时获取到的工具列表作为tools参数，跟用户的query一同传入chatWithLLM函数，调试大模型的输出结果，如图4-7所示。



图4-7　调试大模型的输出结果

可以看到，大模型返回了需要调用的工具信息，并将其包裹在预定义的标签中。

4.2.6　解析大模型响应的工具信息

在4.2.5节中，ChatMCP请求大模型挑选工具，大模型响应了需要调用的工具信息，并将其包裹在特殊标签中。接下来，ChatMCP需要从大模型的响应内容中，解析需要调用的工具信息。按以下步骤实现解析工具信息的逻辑。

1.　定义混合内容数据类型

大模型响应的内容是在文本中包裹了特殊标签，因此我们可以定义一个混合内容数据类型，把文本内容和特殊标签包裹的工具信息区分开：

```
interface MixContent {
    type: "text" | "tool";
    text?: string;
```

```
    tool?:  {
        server_name:  string;
        tool_name:  string;
        tool_params?:  Record<string,  unknown>;
    };
}
```

2. 实现一个函数，解析混合内容

此函数通过正则表达式，从文本内容中提取特殊标签包裹的工具信息，并返回一个混合内容数组。

```
function  parseMixContents(input:  string):  MixContent[]  {
    const  result:  MixContent[]  =  [];
    //  提取工具信息的正则表达式
    const  regex  =  /<<tool-start>>\s*([\s\S]*?)\s*<<tool-end>>/g;
    let  lastIndex  =  0;
    let  match:  RegExpExecArray  |  null;


    while  ((match  =  regex.exec(input))  !==  null)  {
        //  处理当前tool块之前的普通文本
        if  (match.index  >  lastIndex)  {
            const  text  =  input.slice(lastIndex,  match.index).trim();
            if  (text)  {
                result.push({  type:  "text",  text  });
            }
        }
```

```
        //  解析并处理tool块中的JSON内容
        const  toolJson  =  match[1].trim();
        try  {
            const  tool  =  JSON.parse(toolJson);
            result.push({  type:  "tool",  tool  });
        }  catch  (e)  {
            //  如果遇到解析失败的情况，跳过这个错误继续执行
        }
        lastIndex  =  regex.lastIndex;
    }
    //  处理最后一个tool块之后剩余的普通文本
    if  (lastIndex  <  input.length)  {
        const  text  =  input.slice(lastIndex).trim();
        if  (text)  {
            result.push({  type:  "text",  text  });
        }
    }
    //  返回解析后的混合内容
    return  result;
}
```

3. 调试解析工具信息

将大模型返回的包含工具调用信息的内容作为输入，调用parseMixContents函数解析，得到一个由文本片段和特殊标签包裹的工具信息组成的混合内容数组。通过调试接口查看输出结果，如图4-8所示。

api > mcp > test > route.ts > parseMixContents

```
export async function POST(req: Request) {
  const pickToolResults = `
为了查询广州的天气情况，我需要使用天气查询工具。

<<tool-start>>
{
  "server_name": "weather-mcp",
  "tool_name": "query-weather",
  "tool_params": {
    "city": "广州"
  }
}
<<tool-end>>
`;

  const result = parseMixContents(pickToolResults);

  return Response.json(result);
}

interface MixContent {
  type: "text" | "tool";
  text?: string;
  tool?: {
    server_name: string;
    tool_name: string;
    tool_params?: Record<string, unknown>;
  };
}

function parseMixContents(input: string): MixContent[] {
  const result: MixContent[] = [];
  const regex = /<<tool-start>>\s*([\s\S]*?)\s*<<tool-end>>/g;
  let lastIndex = 0;
  let match: RegExpExecArray | null;

  while ((match = regex.exec(input)) !== null) {
    // 前面的文本
    if (match.index > lastIndex) {
```

```
1  HTTP/1.1 200 OK
2  vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
3  content-type: application/json
4  Date: Tue, 13 May 2025 06:06:04 GMT
5  Connection: close
6  Transfer-Encoding: chunked
7
8  [
9    {
10     "type": "text",
11     "text": "为了查询广州的天气情况，我需要使用天气查询工具。"
12   },
13   {
14     "type": "tool",
15     "tool": {
16       "server_name": "weather-mcp",
17       "tool_name": "query-weather",
18       "tool_params": {
19         "city": "广州"
20       }
21     }
22   }
23 ]
```

图4-8　调试解析工具信息

可以看到，混合内容数组中的文本内容和工具信息已经被成功区分。在后续的步骤中，我们可以分别处理这两类内容。



调用parseMixContents函数返回的数组可能包含多个元素，后续代码应遍历该数组，根据每个元素的type字段执行相应的操作。具体来说，当type为tool时，提取参数并调用对应的工具；当type为text时，则直接显示文本内容。

4.2.7　调用工具

在ChatMCP解析的大模型返回的混合内容数组中，包含了需要调用的工具信息：

```
{
    "type":  "tool",
    "tool":  {
```

```
        "server_name": "weather-mcp",
        "tool_name": "query-weather",
        "tool_params": {
            "city": "广州"
        }
    }
}
```

我们先将解析出来的工具信息作为callTool函数的固定参数，跟前面读取MCP服务器列表的逻辑结合起来，实现调用工具的逻辑，如下所示：

```
const configFile = `/Users/idoubi/.cursor/mcp.json`;
const config = await fs.readFile(configFile, "utf-8");
```

```
const mcpServers = await getMcpServers(config);

const mcpTools = await getMcpTools(mcpServers);

const mixContents = parseMixContents(pickToolResults);

const callToolResult = await callTool({
    command: mcpServers["weather-mcp"].command,
    args: mcpServers["weather-mcp"].args || [],
    env: mcpServers["weather-mcp"].env || {},
    name: "query-weather",
    params: {
        city: "广州",
    },
});
```

然后把上一次模型响应的内容作为上下文，带上工具调用结果，再次请求大模型回答用户最初的问题。实现逻辑如下：

```
const result = await chatWithLLM({
    query: "广州在下雨吗？",
    contextMessages: JSON.stringify(mixContents),
```

```
    tools:  JSON.stringify(mcpTools),
    toolResults:  JSON.stringify(callToolResult),
  });
```

调试接口，输出结果如图4-9所示。



```
api > mcp > test > route.ts > chatWithLLM
export async function POST(req: Request) {
  const configFile = `/Users/idoubi/.cursor/mcp.json`;
  const config = await fs.readFile(configFile, "utf-8");

  const mcpServers = await getMcpServers(config);

  const mcpTools = await getMcpTools(mcpServers);

  const mixContents = parseMixContents(pickToolResults);

  const callToolResult = await callTool({
    command: mcpServers["weather-mcp"].command,
    args: mcpServers["weather-mcp"].args || [],
    env: mcpServers["weather-mcp"].env || {},
    name: "query-weather",
    params: {
      city: "广州",
    },
  });

  const result = await chatWithLLM({
    query: "广州在下雨吗？",
    contextMessages: JSON.stringify(mixContents),
    tools: JSON.stringify(mcpTools),
    toolResults: JSON.stringify(callToolResult),
  });

  return result.toTextStreamResponse();
}
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: text/plain; charset=utf-8
Date: Tue, 13 May 2025 06:31:05 GMT
Connection: close
Transfer-Encoding: chunked

根据天气查询结果，广州目前天气状况为多云，没有下雨。具体天气情况如下：

## 实时天气
- 温度：28°C
- 湿度：55%
- 天气：多云
- 风向：南风
- 风力：3级
- 空气质量指数：62

## 未来天气预报
- 5月14日：多云转阵雨，气温21-32°C
- 5月15日：阵雨转雷阵雨，气温23-31°C
- 5月16日：雷阵雨，气温23-30°C
- 5月17日：雷阵雨转中雨，气温24-31°C

虽然现在没有下雨，但未来几天有降雨趋势，建议出门携带雨具。
```

图4-9　调试大模型回复结果

可以看到，大模型有了工具调用结果作为上下文，回复内容包含了实时信息。

至此，ChatMCP的核心逻辑基本开发完成。ChatMCP可以根据用户的提问和配置的MCP服务器，请求大模型调度工具，由客户端执行工具调用，实现为大模型补充上下文的目的，最终让大模型的回复内容更加准确和实时。

4.2.8　优化交互逻辑

在前面的内容中，我们拆解了几个步骤，实现了ChatMCP读取MCP服务器工具列表，与大模型交互的基本流程。在此基础上，我们继续优化交互逻辑，把接口层的逻辑写完整。

1.　实现接口，处理用户请求

我们在ChatMCP中实现一个POST接口，包含以下几部分逻辑：

接收用户输入的query参数，获取用户配置的MCP服务器提供的工具列表，请求大模型挑选一个工具；

❑

解析工具信息，调用工具，获取工具调用结果，再次请求大模型回答用户的问题；

❑

输出接口响应内容。

此接口的实现逻辑如下：

```
export async function POST(req: Request) {
    let { query } = await req.json();
    let contextMessages: MixContent[] = [];


    //  加载MCP配置文件，获取MCP服务器列表及其提供的工具
    const configFile = `/Users/idoubi/.cursor/mcp.json`;
    const config = await fs.readFile(configFile, "utf-8");
    const mcpServers = await getMcpServers(config);
    const mcpTools = await getMcpTools(mcpServers);


    //  向大模型发送用户问题和工具列表，请求其选择需要调用的工具
    const pickToolResult = await chatWithLLM({
        query,
        contextMessages: JSON.stringify(contextMessages),
        tools: JSON.stringify(mcpTools),
        toolResults: "",
    });
```

```
let content = "";
for await (const chunk of pickToolResult.textStream) {
    content += chunk;
}
// 解析大模型响应的混合内容
const mixContents = parseMixContents(content);
// 解析需要调用的工具信息
let callToolParams = null;
for (const mixContent of mixContents) {
    if (mixContent.type === "tool") {
        const tool = mixContent.tool;
        if (
            tool &&
            tool.tool_name &&
            tool.server_name &&
            mcpServers[tool.server_name] &&
            mcpServers[tool.server_name].command
        ) {
            callToolParams = {
                command: mcpServers[tool.server_name].command,
                args: mcpServers[tool.server_name].args || [],
                env: mcpServers[tool.server_name].env || {},
```

```
                name:  tool.tool_name,
                params:  tool.tool_params,
            };
            break;
        }
    }
}
//  需要调用工具
if  (callToolParams)  {
    //  调用工具
    const  callToolResult  =  await  callTool(callToolParams);
    //  带上工具调用结果请求大模型回复
    const  result  =  await  chatWithLLM({
        query,
        contextMessages:  JSON.stringify(mixContents),
        tools:  JSON.stringify(mcpTools),
        toolResults:  JSON.stringify(callToolResult),
});
```

```
        return  result.toTextStreamResponse();
    }
    //  无须调用工具
    return  pickToolResult.toTextStreamResponse();
}
```

模拟用户输入一个问题，并将其作为query参数传入调试接口，以查看大模型的响应内容，输出结果如图4-10所示。

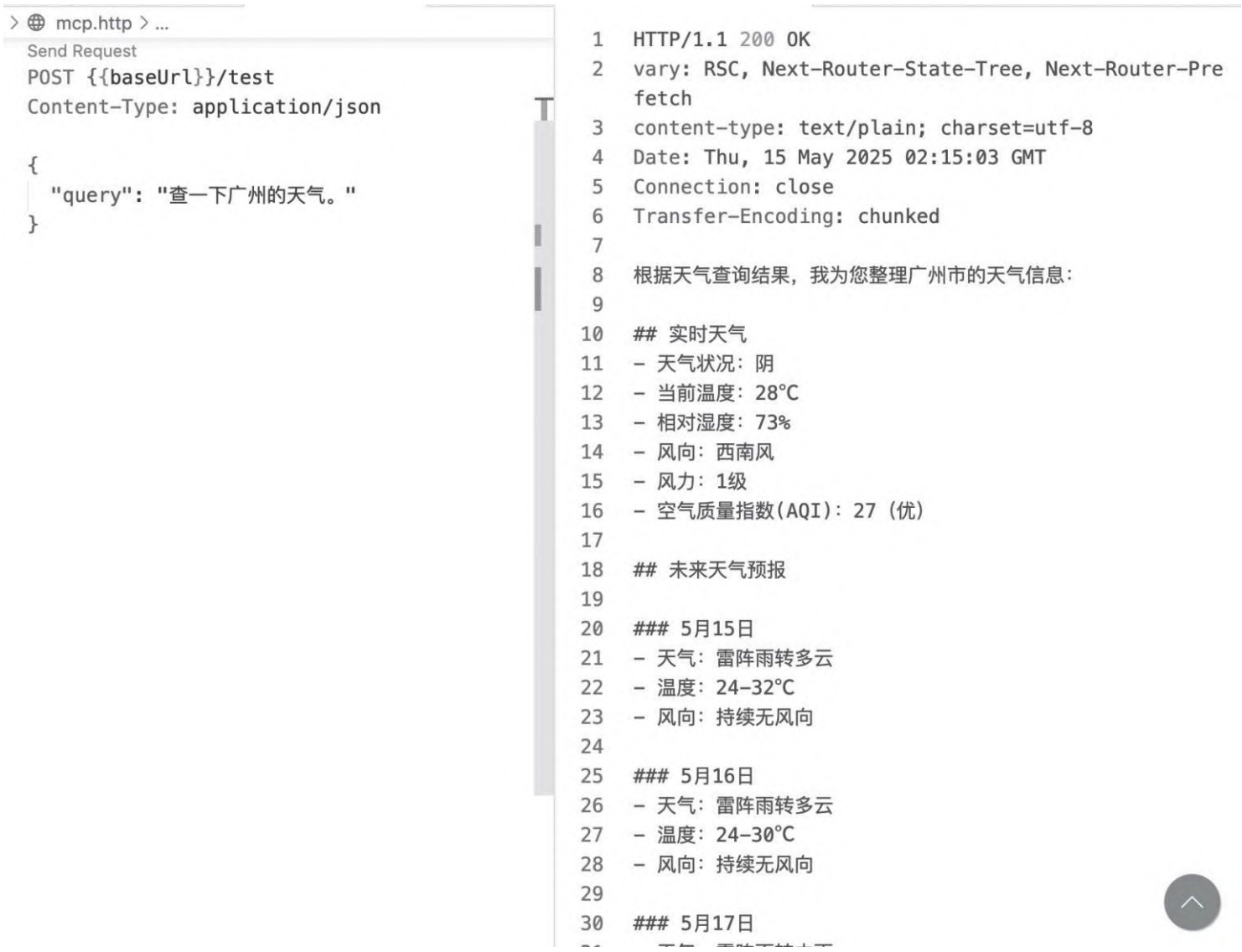


图4-10　调试接口响应内容

可以看到，接口响应符合预期。ChatMCP调用天气查询MCP服务器内部的工具，为大模型补充了实时天气信息，给用户响应了正确的内容。

2. 循环调用多个工具，实现自动工作流

上面实现的接口逻辑最多只能调用一次工具，在把第一次工具调用结果传给大模型之后，就算大模型继续返回工具调用信息，ChatMCP也不会再调用工具，而是直接回复用户了。

然而，实际情况中，用户的提问有时候会很复杂，需要调用多个工具才能补充足够的信息。因此我们可以继续优化接口，实现循环调用工具的逻辑，让大模型根据传递的工具列表和用户查询的问题，自动编排工作流。

设置最大循环次数为10次。当大模型的响应内容不包含工具调用信息或者循环次数达到最大次数时，退出循环。

优化后的接口逻辑如下：

```
export async function POST(req: Request) {

    let { query } = await req.json();
```

```
//  加载MCP配置文件，获取MCP服务器列表和可用工具
const  configFile  =  `/Users/idoubi/.cursor/mcp.json`;
const  config  =  await  fs.readFile(configFile,  "utf-8");
const  mcpServers  =  await  getMcpServers(config);
const  mcpTools  =  await  getMcpTools(mcpServers);


let  contextMessages:  MixContent[]  =  [];
let  toolResults  =  "";
let  reply  =  "";
```

```
//最多循环10次，用于连续调用工具的多轮推理
for (let i = 0; i < 10; i++) {
    //  向大模型发送请求，由其判断是否需要调用工具
    const pickToolResult = await chatWithLLM({
        query,
        contextMessages: JSON.stringify(contextMessages),
        tools: JSON.stringify(mcpTools),
        toolResults: toolResults,
    });
```

```
//  解析大模型返回的内容（文本  +  工具信息）
let  content  =  "";
for  await  (const  chunk  of  pickToolResult.textStream)  {
    content  +=  chunk;
}
const  mixContents  =  parseMixContents(content);
contextMessages.push(...mixContents);
reply  +=  content;
```

```
//  从返回内容中提取工具调用请求参数
let  callToolParams  =  null;
for  (const  mixContent  of  mixContents)  {
    if  (mixContent.type  ===  "tool")  {
        const  tool  =  mixContent.tool;
        if  (
            tool  &&
            tool.tool_name  &&
            tool.server_name  &&
            mcpServers[tool.server_name]  &&
            mcpServers[tool.server_name].command
        )  {
            callToolParams  =  {
                command:  mcpServers[tool.server_name].command,
                args:  mcpServers[tool.server_name].args  ||  [],
                env:  mcpServers[tool.server_name].env  ||  {},
                name:  tool.tool_name,
                params:  tool.tool_params,
            };
            break;
        }
    }
```

```
}


//  如果需要调用工具，则执行调用，并将结果更新为下一轮上下文
if  (callToolParams)  {
    const  callToolResult  =  await  callTool(callToolParams);
    toolResults  =  JSON.stringify(callToolResult);


    reply  +=  `\n\n${toolResults}\n\n`;
```

```
            continue;
        }


        //  如果不需要调用工具，则跳出循环
        break;
    }
```

```
        return  new  Response(reply);
    }
```

我们以一个新的问题作为query参数，通过调试接口查看大模型的响应内容，输出结果如图4-11所示。

可以看到，当用户的提问意图涉及多个工具调用时，大模型会自动编排工作流，依次返回多个工具的调用信息。每次由客户端执行工具调用，并把工具调用结果作为上下文补充给大模型，大模型再返回下一个工具的调用信息。

mcp.http > ...

```
POST {{baseUrl}}/test
Content-Type: application/json

{
  "query": "查一下广州的天气，保存到笔记中。"
}
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: text/plain;charset=UTF-8
Date: Thu, 15 May 2025 02:23:05 GMT
Connection: close
Transfer-Encoding: chunked

我需要分两步完成这个任务：
1. 先查询广州天气
2. 然后将天气信息保存到笔记中

让我们先查询广州的天气：

<<tool-start>>
{
  "server_name": "weather-mcp",
  "tool_name": "query-weather",
  "tool_params": {
    "city": "广州"
  }
}
<<tool-end>>

我看到已经获取到了广州天气信息，现在需要将这些信息整理后保存到笔记中。我会将天气信息格式化为易读的笔记格式。

<<tool-start>>
{
  "server_name": "flomo-mcp",
  "tool_name": "write_note",
  "tool_params": {
    "content": "# 广州天气预报\n\n## 实时天气\n- 温度：28°C\n- 湿度：73%\n- 天气：阴\n- 风向：西南风\n- 风力：1级\n- 空气质量指数：27\n\n## 未来天气预报\n\n### 5月15日\n- 温度：24-32°C\n- 天气：雷阵雨转多云\n- 风向：持续无风向\n\n### 5月16日\n- 温度：24-30°C\n- 天气：雷阵雨转多云\n- 风向：持续无风向\n\n### 5月17日\n- 温度：24-31°C\n- 天气：雷阵雨转中雨\n- 风向：持续无风向\n\n### 5月18日\n- 温度：24-31°C\n- 天气：中雨\n- 风向：持续无风向\n\n### 5月19日\n- 温度：24-30°C\n- 天气：中雨\n- 风向：持续无风向"
  }
}
```

图4-11　调试循环调用多个工具的接口

4.2.9　案例1小结

本节通过一个AI对话助手的例子，讲解了开发MCP客户端的基本流程和核心逻辑。

通过这个例子，我们演示了如何读取用户配置的MCP服务器列表，如何获取MCP服务器提供的工具，如何设置提示词让大模型挑选合适的工具，如何调用工具为大模型补充上下文等知识。

在实际工程实践中，要开发一个面向用户的MCP客户端，还涉及前端UI交互、流式数据解析、工具调用错误处理、失败重试、调度流程优化等内容，鉴于篇幅有限，本书就不一一展开了。

理解了本节内容，大家就可以开始动手开发自己的MCP客户端了。

4.3　案例2：开发AI搜索智能体

在上一节内容中，我们学习了如何开发对话助手类的MCP客户端，调用用户配置的MCP服务器与大模型交互。在本节中，我们继续学习如何开发智能体类的MCP客户端。在开始讲解具体的案例之前，先来介绍一下我在2024年做的一款AI搜索引擎产品——ThinkAny。

2024年3月，我发布了ThinkAny，其核心功能是根据用户输入的问题，先联网检索信息，再请求大模型回答用户的问题。它的核心原理是通过RAG技术，将传统搜索引擎的搜索结果作为上下文补充给大模型，旨在减少大模型的幻觉，为用户提供更准确的回答。ThinkAny的一个问答示例如图4-12所示。

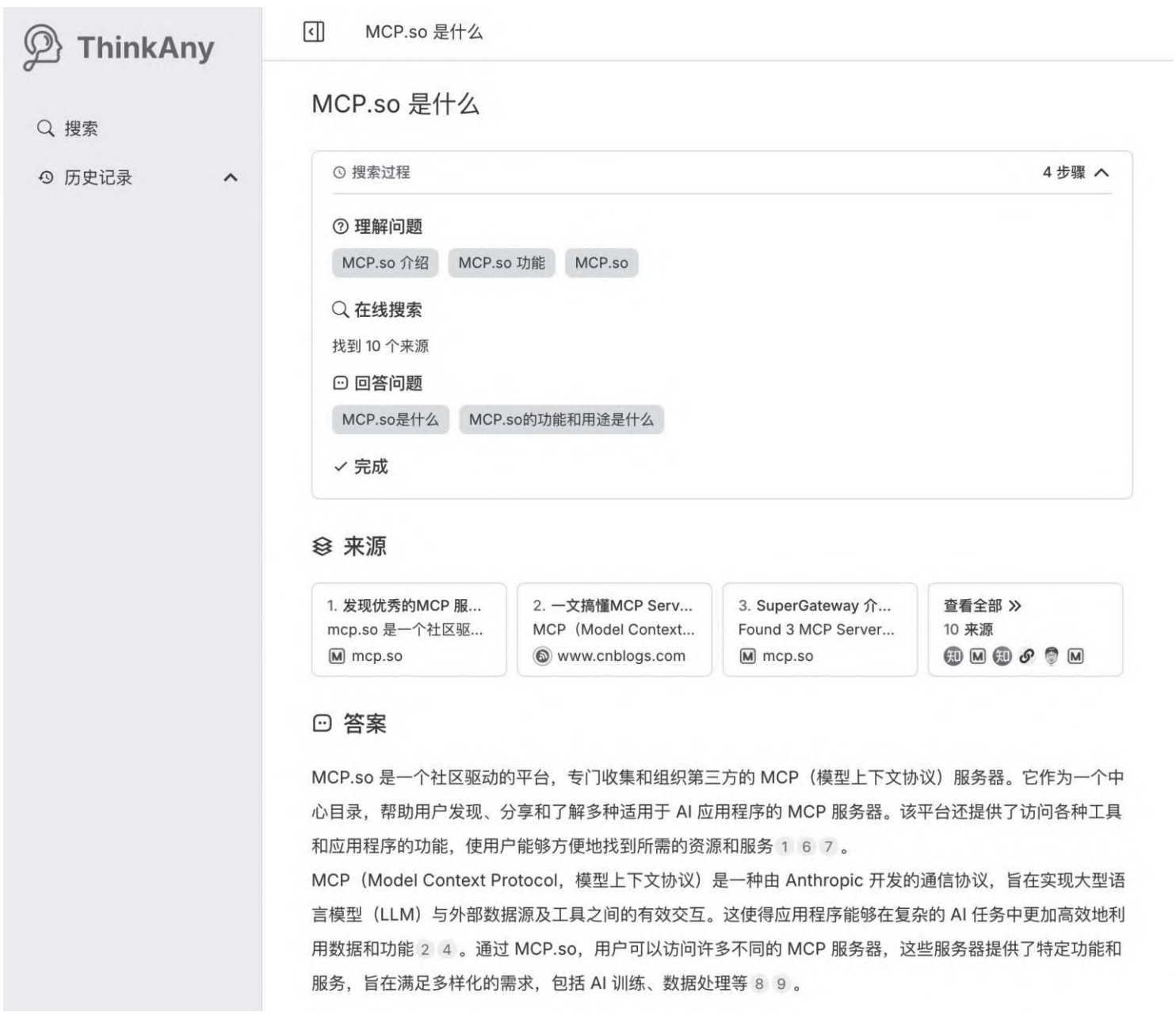


图4-12　ThinkAny的一个问答示例

在本节内容中，我们计划开发一个与ThinkAny同名的AI搜索智能体，通过这个案例来演示智能体类MCP客户端开发的具体流程。

4.3.1　开发目标

本案例旨在开发一个支持AI搜索功能的智能体，完成问题改写、联网检索、读取网页内容，且以上三步均通过MCP工具实现，由提示词自动编排工具调用流程，构建基础的搜索问答能力。

我梳理了AI搜索引擎类产品的核心流程，如图4-13所示。

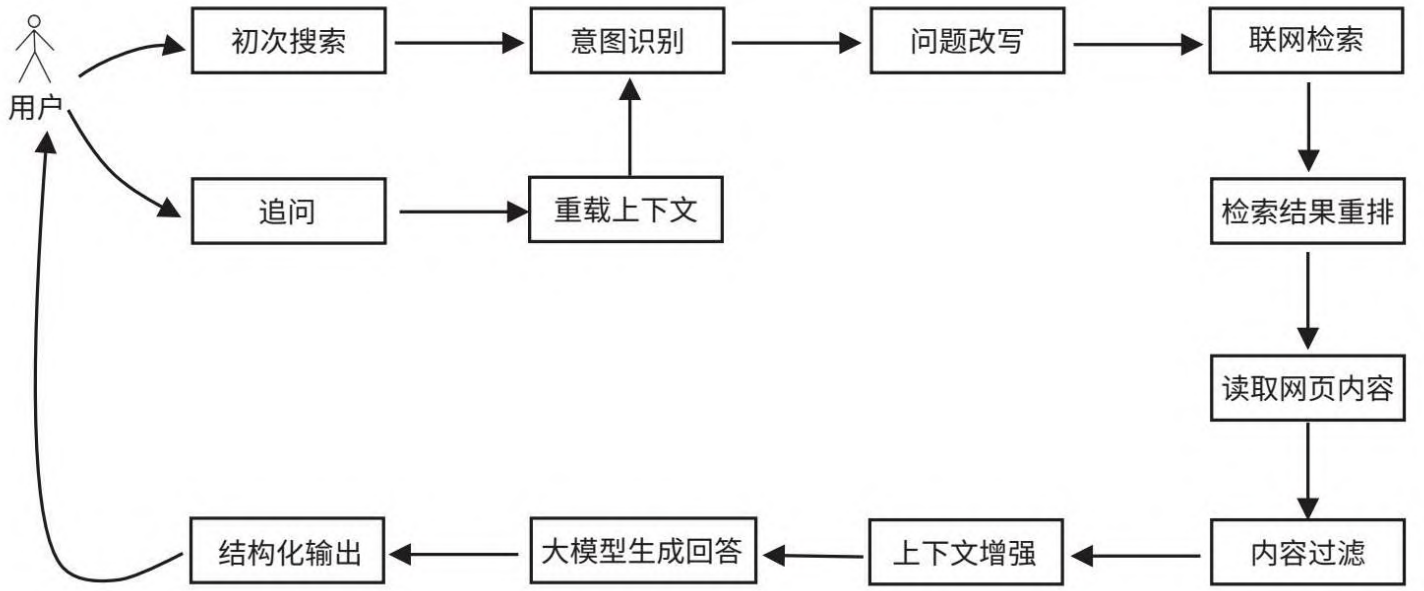


图4-13　AI搜索引擎类产品的核心流程

上面提到，此类产品背后的核心技术原理是RAG（retrieval-augmented generation，检索增强生成），其中的检索步骤，是通过调用传统搜索引擎查询接口来实现的。为了得到更丰富的参考内容，一般需要对用户的原始问题进行改写，并对检索得到的部分网页读取详细内容。

ThinkAny网页版完整实现了上述流程。鉴于篇幅有限，在讲解AI搜索智能体开发时，本节只聚焦实现三个核心步骤：

- 问题改写（用于生成更符合查询习惯的关键词，从而获得更精准、匹配用户意图的检索结果）；
- 联网检索（是RAG的基础，主要获取一批相关网页的链接和摘要）；
- 读取网页内容（进一步提取目标网页中的详细信息，以丰富后续生成的答案）。

4.3.2 前置准备

我们先来为开发目标中所述的三个步骤选择合适的MCP服务器并进行调试。

1. 获取问题改写MCP服务器的工具列表

用户输入的问题可能是某个关键词，也可能是某个句子。为了获得更好的搜索结果，往往需要对用户的原始问题进行改写，然后用改写后的内容作为查询关键词联网检索。

这里选择一个叫作query-rewrite-mcp的MCP服务器，其提供一个rewrite_query工具，可以对用户输入的问题进行改写。

配置此MCP服务器的运行信息，通过listTools函数获取此MCP服务器内部定义的工具列表，调试输出结果，如图4-14所示。



图4-14 调试query-rewrite-mcp工具列表

调试逻辑里面用到的listTools函数，我们在4.2节已详细讲解过其实现逻辑，这里不再赘述。调试此MCP服务器需要填写的OPENROUTER_API_KEY在OpenRouter平台获取；REWRITE_MODEL是用于改写问题的模型，开发者可自行指定，推荐使用Claude系列模型，会有更好的改写效果。

2. 调用问题改写工具

根据问题改写MCP服务器输出的工具列表，通过callTool函数调用rewrite_query工具，调试输出结果，如图4-15所示。

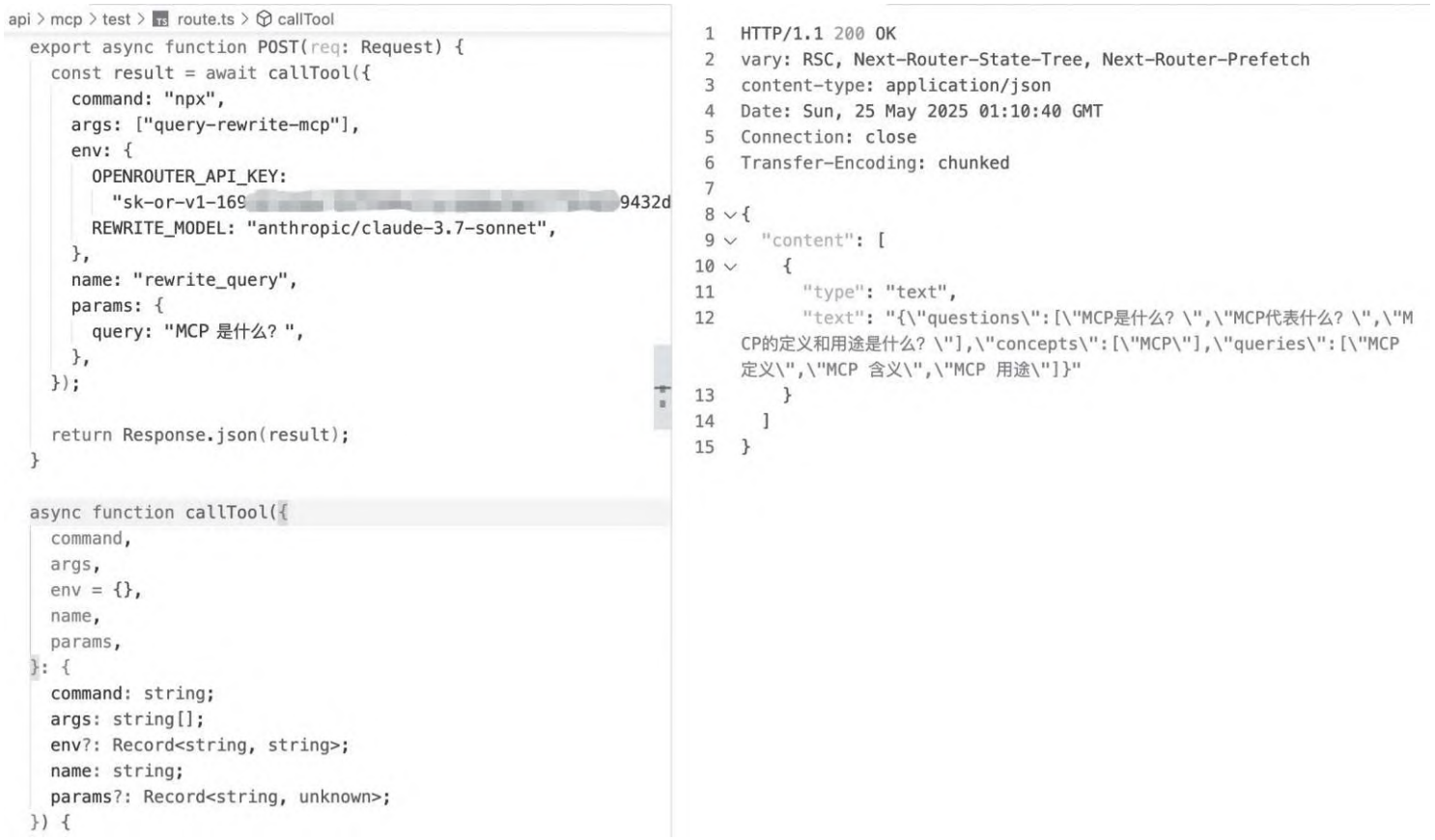


图4-15　调试rewrite_query工具

通过上述调试输出可以得知，rewrite_query工具可以改写用户的原始问题，返回更加丰富的改写内容。比如输入的原始问题是：

```
{
    "query": "MCP 是什么？"
}
```

改写后的内容是：

```
{
    "questions": ["MCP 是什么？", "MCP 代表什么？", "MCP 的定义和用途是什么？"],
    "concepts": ["MCP"],
    "queries": ["MCP 定义", "MCP 含义", "MCP 用途"]
}
```

以上代码中三个字段的含义说明如下：

questions，基于原始问题生成的扩展问题，帮助大模型从不同的维度构建回答；

❑

concepts，从原始问题提取的相关概念或术语，用于百科词条检索；

❑

queries，根据原始问题生成的查询语句，用于搜索引擎检索。

3. 获取联网检索MCP服务器的工具列表

这里选择一个叫作serper-mcp-server的MCP服务器，其提供一个google_search工具，可以模拟Google搜索引擎查询得到搜索结果。

配置此MCP服务器的运行信息，通过listTools函数获取此MCP服务器内部定义的工具列表，调试输出结果，如图4-16所示。



图4-16 调试serper-mcp-server工具列表

调试此MCP服务器需要填写的SERPER_API_KEY在Serper平台获取。

4. 调用联网检索工具

根据联网检索MCP服务器输出的工具列表，通过callTool函数调用google_search工具，调试输出结果，如图4-17所示。



图4-17　调试google_search工具

通过上述的调试输出得知，google_search工具可以模拟Google搜索引擎查询得到搜索结果。比如输入的查询参数为：

```
{
    "q": "mcp"
}
```

此工具返回搜索结果列表，每个结果包含标题、链接、描述、来源等信息，其中的一条搜索结果示例如下：

```
{
    "title": "Model Context Protocol: Introduction",
    "link": "https://modelcontextprotocol.io/introduction",
    "snippet": "MCP is an open protocol that standardizes how applications provide
context to LLMs. Think of MCP like a USB-C port for AI applications."
}
```

5.　获取读取网页内容MCP服务器的工具列表

这里选择一个叫作jina-mcp-tools的MCP服务器，其提供一个jina_reader工具，可以读取网页内容。配置此MCP服务器的运行信息，通过listTools函数获取此MCP服务器内部定义的工具列表，调试输出结果，如图4-18所示。

调试此MCP服务器需要填写的JINA_API_KEY在Jina平台获取。

api > mcp > test > route.ts > listTools

```
export async function POST(req: Request) {
  const tools = await listTools({
    command: "npx",
    args: ["jina-mcp-tools"],
    env: {
      JINA_API_KEY:
        "jina_f9[redacted]gKwE
    },
  });

  return Response.json(tools);
}

async function listTools({
  command,
  args,
  env = {},
}: {
  command: string;
  args: string[];
  env?: Record<string, string>;
}) {
  const transport = new StdioClientTransport({
    command,
    args,
    env: {
      ...(process.env as Record<string, string>),
      ...env,
    },
  });
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: application/json
Date: Sat, 24 May 2025 12:20:52 GMT
Connection: close
Transfer-Encoding: chunked

{
  "tools": [
    {
      "name": "jina_reader",
      "description": "Read and extract content from web pages using Jina AI's powerful web reader",
      "inputSchema": {
        "type": "object",
        "properties": {
          "url": {
            "type": "string",
            "format": "uri",
            "description": "URL of the webpage to read and extract content from"
          },
          "format": {
            "type": "string",
            "enum": [
              "Default",
              "Markdown",
              "HTML",
              "Text",
              "Screenshot",
              "Pageshot"
```

图4-18　调试jina-mcp-tools工具列表

6.　调用读取网页内容的工具

根据读取网页内容MCP服务器输出的工具列表，通过callTool函数调用jina_reader工具，调试输出结果，如图4-19所示。



图4-19　调试jina_reader工具

从调试结果得知，此工具接收网页URL，默认通过Markdown格式返回读取到的网页内容。

至此，我们已经准备好了实现AI搜索智能体需要用到的全部MCP服务器及其内部定义的工具。接下来就可以专注于实现AI搜索智能体与大模型交互的逻辑了。

4.3.3　定义MCP服务器配置

跟上一个案例从配置文件读取用户配置的MCP服务器不同，在此案例中，AI搜索智能体需要用到的MCP服务器和工具是固定的，因此可以在程序逻辑中直接定义。MCP服务器配置内容如下：

```
{
    "mcpServers":  {
        "query-rewrite-mcp":  {
            "command":  "npx",
            "args":  ["query-rewrite-mcp"],
            "env":  {
                "OPENROUTER_API_KEY":  "xxx",
```

```
            "REWRITE_MODEL": "anthropic/claude-3.7-sonnet"
        }
    },
    "serper-search-mcp": {
        "command": "uvx",
        "args": ["serper-mcp-server"],
        "env": {
            "SERPER_API_KEY": "xxx"
        }
    },
    "jina-reader-mcp": {
        "command": "npx",
        "args": ["jina-mcp-tools"],
        "env": {
            "JINA_API_KEY": "xxx"
        }
    }
}
```

以上代码中的配置内容做了脱敏处理，每个MCP服务器用到的参数使用了占位符，大家可根据实际情况自行填写。

4.3.4 过滤MCP服务器工具列表

跟上一个案例把用户配置的MCP服务器内部定义的全部工具传给大模型不同，在此案例中，AI搜索智能体用到的工具是固定的，因此可以定义一个工具列表，对获取到的MCP服务器工具列表进行过滤，只选择实现AI搜索智能体需要用到的工具。

过滤MCP服务器工具列表的逻辑如下：

```
const mcpServers = await getMcpServers(config);
const mcpTools = await getMcpTools(mcpServers);



const filterTools = ["rewrite_query", "google_search", "jina_reader"];
const filteredTools = mcpTools.filter((tool) =>
    filterTools.includes(tool.name)
);
```

此逻辑中，config是在程序中定义的MCP服务器配置内容。关于getMcpServers和getMcpTools函数，我们在4.2节已详细讲解其实现逻辑，这里不再赘述。

调用接口获取过滤后的工具列表，调试输出结果，如图4-20所示。

```
api > mcp > test > route.ts > getMcpTools
export async function POST(req: Request) {
  const mcpServers = await getMcpServers(config);
  const mcpTools = await getMcpTools(mcpServers);

  const filterTools = ["rewrite_query", "google_search", "jina_reader"];
  const filteredTools = mcpTools.filter((tool) =>
    filterTools.includes(tool.name)
  );

  return Response.json(filteredTools);
}

async function getMcpTools(
  mcpServers: Record<string, McpServer>
): Promise<McpTool[]> {
  const allTools = await Promise.all(
    Object.entries(mcpServers).map(async ([name, server]) => {
      const tools = await listTools({
        command: server.command,
        args: server.args || [],
        env: server.env || {},
      });

      return tools.tools.map((tool) => ({
        server_name: name,
        name: tool.name,
        description: tool.description || "",
        inputSchema: tool.inputSchema,
      }));
    })
  );

  return allTools.flat();
}
```

```
1   HTTP/1.1 200 OK
2   vary: RSC, Next-Router-State-Tree, Next-Router-Prefe
    tch
3   content-type: application/json
4   Date: Sun, 25 May 2025 01:14:58 GMT
5   Connection: close
6   Transfer-Encoding: chunked
7
8   [
9     {
10      "server_name": "query-rewrite-mcp",
11      "name": "rewrite_query",
12      "description": "Rewrite a query to make it more
    clear and concise",
13    "inputSchema": {…
32      }
33    },
34    {
35      "server_name": "serper-search-mcp",
36      "name": "google_search",
37      "description": "Search Google for results",
38    "inputSchema": {…
124     }
125   },
126   {
127     "server_name": "jina-reader-mcp",
128     "name": "jina_reader",
129     "description": "Read and extract content from we
    b pages using Jina AI's powerful web reader",
130   "inputSchema": {…
167     }
168   }
169 ]
```

图4-20　获取过滤后的MCP服务器工具列表

从调试结果得到，在接口逻辑中获得了AI搜索智能体需要用到的三个工具及其对应的MCP服务器：

- rewrite_query工具，对应query-rewrite-mcp　MCP服务器；
- google_search工具，对应serper-search-mcp　MCP服务器；
- jina_reader工具，对应jina-reader-mcp　MCP服务器。

4.3.5　通过提示词编排工作流

为了让大模型按照AI搜索引擎的标准流程调度工具，我们需要设置系统提示词，编排工作流。

本案例设计的系统提示词如下：

```
你是ThinkAny，由ThinkAny　AI开发的AI搜索智能体。
```

核心工作流

执行原则

- 检查上下文中已有的工具执行结果，避免重复选择工具
- 按顺序执行：问题改写 → 联网检索→ 读取网页内容→ 生成答案
- 一次只选择一个工具，返回工具调用信息

阶段一：问题改写

条件：仅当上下文中无rewrite_query结果时执行

工具：rewrite_query

目标：将用户的问题转换为结构化参数

- 生成具体的问题列表 (questions)
- 提取核心概念 (concepts)
- 优化搜索查询 (queries)

阶段二：联网检索

工具：google_search（可多次调用，最多3次）

策略：

- 优先使用rewrite_query结果中的queries和concepts
- 每次搜索10条记录，总计不超过30条
- 按相关性排序，优选权威网站

阶段三：读取网页内容

工具：jina_reader（可多次调用）

要求：

- 使用Markdown格式获取内容
- 优先读取最相关的URL
- 过滤无效内容（验证页面、错误页面等）
- 确保获得2~3个网页的详细内容

最终答案生成

要求：

- 直接回答用户的原始问题
- 使用 [1]、[2] 等数字标注引用
- 末尾提供完整的引用链接列表
- 使用清晰的Markdown格式

工具调用格式

<<tool-start>>

{

"server_name": "工具服务器名称",

"tool_name": "工具名称",

"tool_params": {

"参数名": "参数值"

}

}

<<tool-end>>

执行参数

- 用户查询：USER_QUERY={USER_QUERY}
- 对话上下文：CONTEXT_MESSAGES={CONTEXT_MESSAGES}
- 工具执行结果：PREVIOUS_TOOL_RESULTS={PREVIOUS_TOOL_RESULTS}
- 可用工具列表：AVAILABLE_TOOLS={AVAILABLE_TOOLS}

此提示词的核心设计思路是定义了三个步骤，每个步骤包含一个可以调用的工具。让大模型根据定义编排工作流，返回每个步骤对应的工具调用信息。

这里的提示词是一个非常典型的智能体提示工程的案例，它为一个具备工具调用能力的智能体（如ThinkAny）定义了清晰的执行规则、分阶段的任务流程、调用格式、输入参数要求。好的智能体提示词不是简单地说明功能，而是像写“简化版程序说明书”，通过规则+状态+输入/输出定义+可执行格式，将智能体的行为变成一种“可理解与控制的流程”。

4.3.6 实现与大模型的交互逻辑

跟上一个案例与大模型交互的逻辑类似，AI搜索智能体与大模型交互的步骤也包括：

1. 请求大模型挑选工具；
2. 解析大模型响应的工具信息；
3. 调用工具；
4. 重复步骤1~3，直到大模型不再返回工具调用信息或者达到最大循环次数。

在上一个案例中，我们已详细讲解了以上步骤的具体实现逻辑，这里不再赘述，而是直接给出AI搜索智能体与大模型交互的逻辑实现代码，如下所示：

```
export async function POST(req: Request) {
    const { query } = await req.json();
//  读取系统提示词
    const mcpPrompt = await fs.readFile(
        path.join(process.cwd(), "app/api/mcp/test/search.md"),
        "utf-8"
    );
//  读取MCP服务器及工具
    const mcpServers = await getMcpServers(config);
    const mcpTools = await getMcpTools(mcpServers);
```

```
const filterTools = ["rewrite_query", "google_search", "jina_reader"];
const filteredTools = mcpTools.filter((tool) =>
    filterTools.includes(tool.name)
);
```

```
let contextMessages: MixContent[] = [];

let toolResults = "";

let reply = "";
```

```
//  最多循环10次，实现连续调用工具
for  (let  i  =  0;  i  <  10;  i++)  {
    //  请求大模型挑选工具
    const  pickToolResult  =  await  chatWithLLM({
        mcpPrompt,
        query,
        contextMessages:  JSON.stringify(contextMessages),
        tools:  JSON.stringify(filteredTools),
        toolResults:  toolResults,
    });
    //  解析大模型返回的内容（文本  +  工具信息）
    let  content  =  "";
    for  await  (const  chunk  of  pickToolResult.textStream)  {
        content  +=  chunk;
    }
    const  mixContents  =  parseMixContents(content);
    contextMessages.push(...mixContents);
    reply  +=  content;
```

```
//  从混合内容中提取工具信息
let  callToolParams  =  null;
for  (const  mixContent  of  mixContents)  {
    if  (mixContent.type  ===  "tool")  {
        const  tool  =  mixContent.tool;
        if  (
            tool  &&
            tool.tool_name  &&
            tool.server_name  &&
            mcpServers[tool.server_name]  &&
            mcpServers[tool.server_name].command
        )  {
            callToolParams  =  {
                command:  mcpServers[tool.server_name].command,
                args:  mcpServers[tool.server_name].args  ||  [],
                env:  mcpServers[tool.server_name].env  ||  {},
                name:  tool.tool_name,
                params:  tool.tool_params,
            };
            break;
        }
    }
```

}

```
//  如果需要调用工具，则执行调用，并将结果更新为下一轮上下文
if  (callToolParams)  {
    const  callToolResult  =  await  callTool(callToolParams);
    toolResults  =  JSON.stringify(callToolResult);
```

```
            reply  +=  `\n\n${toolResults}\n\n`;



            continue;
        }



        //  如果不需要调用工具，则跳出循环
        break;
    }
```

```
        return  new  Response(reply);
    }
```

跟AI对话助手与大模型交互的逻辑比较，AI搜索智能体与大模型交互的逻辑主要区别在于：传递的工具列表以及系统提示词不同。

4.3.7　调试接口逻辑

对上一步实现的AI搜索智能体接口进行调试，输入问题，输出调试结果，如图4-21所示。

mcp.http > POST /api/mcp/test

1 reference

```
@baseUrl = http://localhost:3000/api/mcp

### get tools
Send Request
POST {{baseUrl}}/test
Content-Type: application/json

{
  "query": "MCP.so 是什么"
}
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: text/plain;charset=UTF-8
Date: Sun, 25 May 2025 02:29:15 GMT
Connection: close
Transfer-Encoding: chunked

我将按照系统架构为您查找关于 MCP.so 的信息。让我开始执行完整的搜索流程。

## 阶段一：query rewrite（问题改写）

首先，我需要将您的查询进行结构化改写，以便更好地进行搜索。

<<tool-start>>
{
  "server_name": "query-rewrite-mcp",
  "tool_name": "rewrite_query",
  "tool_params": {
    "query": "MCP.so 是什么",
    "locale": "zh-CN"
  }
}
<<tool-end>>

基于查询改写的结果，我现在进行联网检索。

## 阶段二：retrieve（联网检索）

我将使用改写后的查询进行搜索，获取相关信息源。

<<tool-start>>
{
  "server_name": "serper-search-mcp",
  "tool_name": "google_search",
```

图4-21　调试AI搜索智能体接口，查看工具调用信息

可以看到，大模型按照提示词约定的工作流，依次调用了三个工具，最终输出的回复如图4-22所示。



图4-22　调试AI搜索智能体接口，查看回复结果

通过最终的回复结果可以看到，跟ThinkAny网页版得到的答案基本一致。

至此，我们完成了AI搜索智能体业务逻辑的开发，实现了AI搜索引擎类产品的核心功能。

4.3.8　在Cursor中测试

在前面的内容中，我们讲解了AI搜索智能体服务逻辑的主要实现，通过调试接口，得到了最终的回复，但是因为缺了UI层，无法直观地展示AI搜索智能体的交互效果。

我们可以在Cursor中继续测试，通过Cursor的UI展示，来感受AI搜索智能体的交互效果。

在Cursor的MCP配置文件中写入AI搜索智能体内置的MCP服务器配置。使用AI搜索智能体内置的系统提示词search.md，发送同一个问题，查看工具调用信息，如图4-23所示。

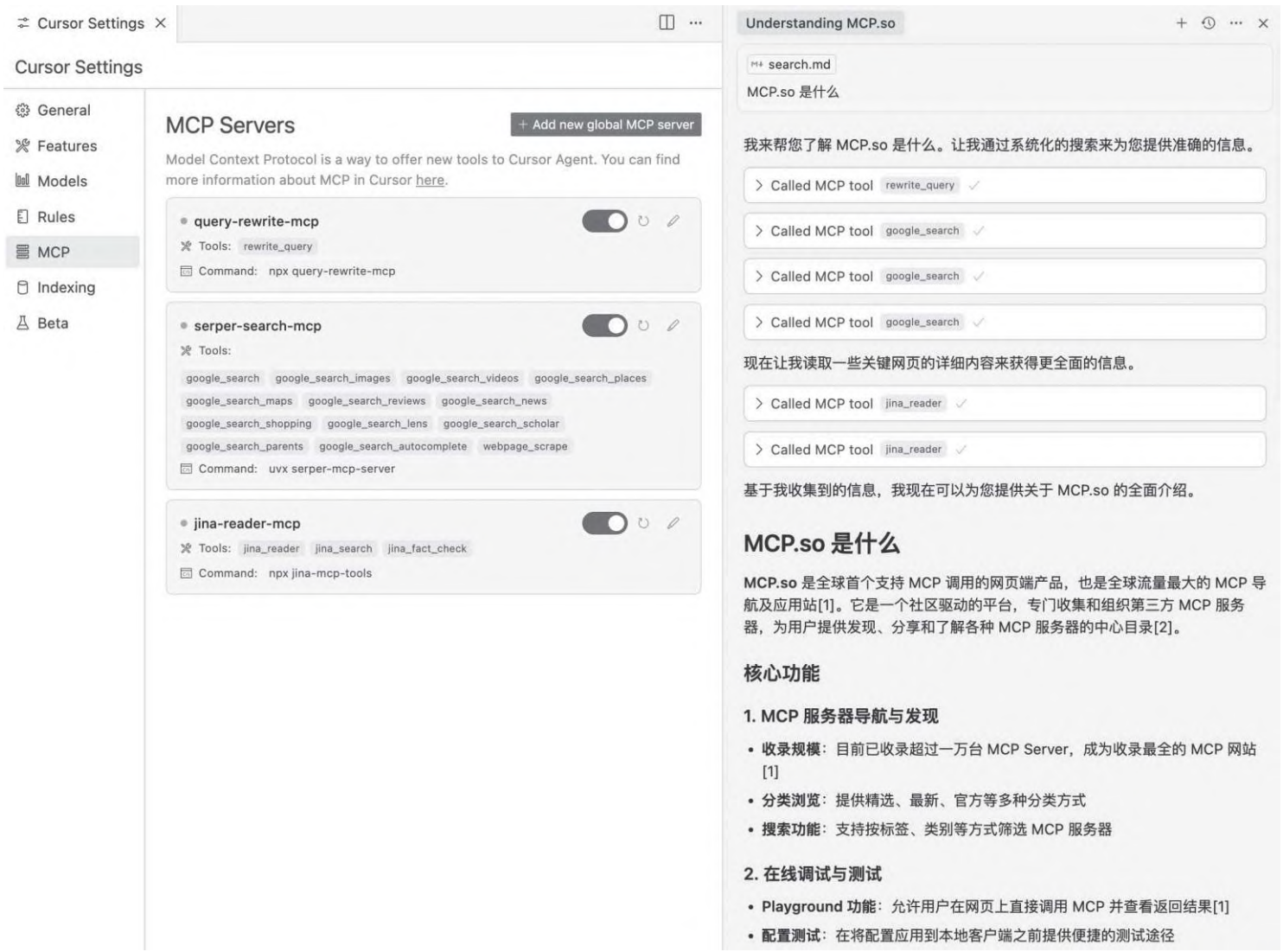


图4-23　在Cursor中测试工具调用

可以看到，Cursor作为MCP客户端，按照提示词约定的工作流与大模型交互，先进行了一次问题改写，再进行了三次联网检索，最后读取了两个网页的内容，最终回复问题，给出了准确的回答。

去掉系统提示词，再用同一个问题测试，可以看到，Cursor没有调用问题改写和读取网页内容的MCP服务器，最终回复的内容准确度不足，如图4-24所示。

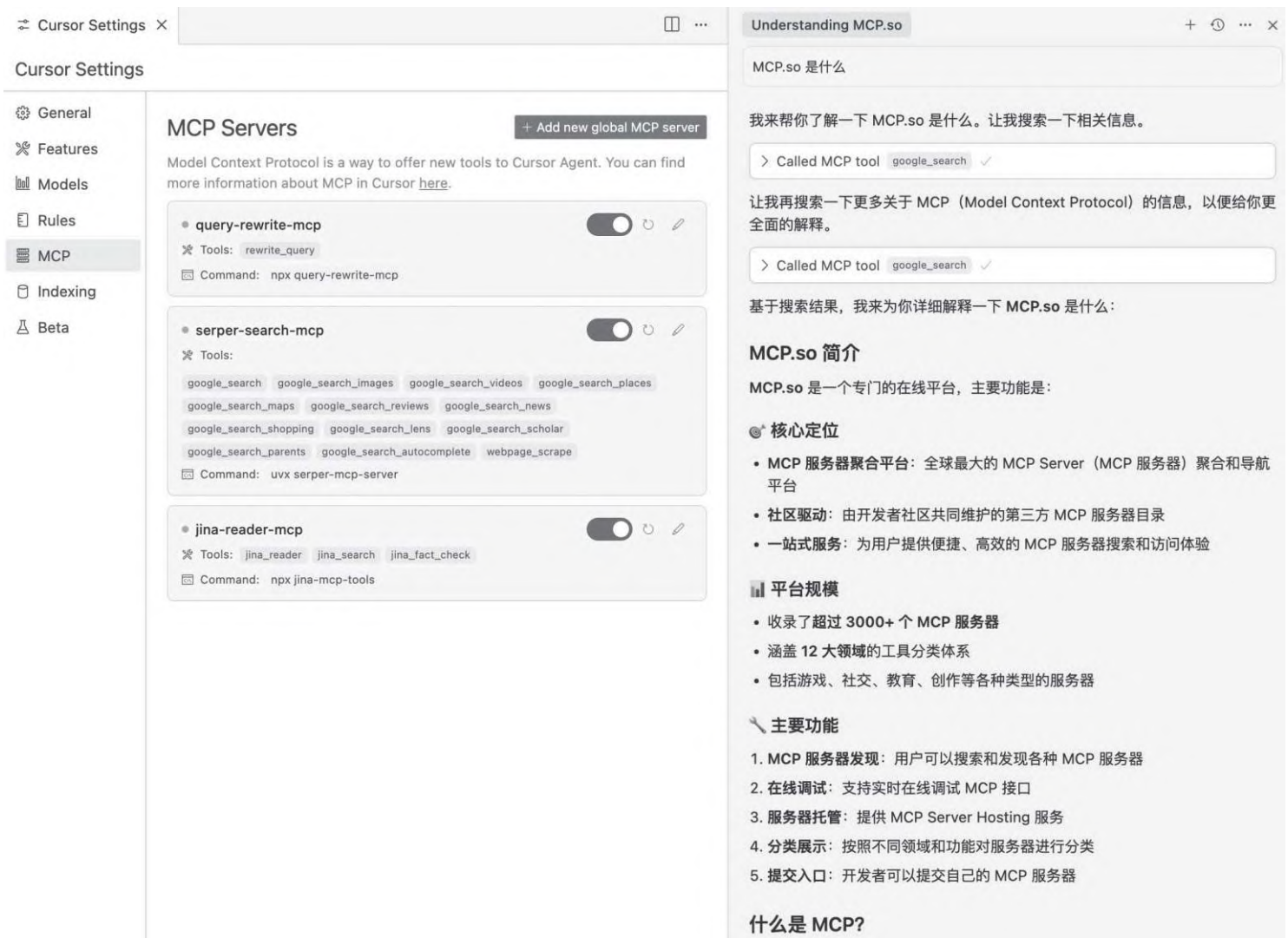


图4-24　在Cursor中测试无系统提示词场景的工具调用

通过上面的两次测试，我们在Cursor中感受了AI搜索智能体带UI展示的交互过程，验证了提示词和工具对大模型回复结果的影响，完成了对本节案例开发的AI搜索智能体的测试验证。

4.3.9　案例2小结

本节通过一个AI搜索智能体的例子，讲解了开发智能体类MCP客户端的基本流程和核心步骤。

理解了本节内容，大家就可以开始动手做自己的智能体产品了。在开发智能体的过程中，可以在MCP应用市场找寻合适的MCP服务器及工具，通过复用工具缩短智能体的开发周期，把精力主要放在核心业务逻辑的实现上。

然而，智能体类产品常见的交互形态，通常依赖虚拟计算机来展示每一步的执行过程，涉及容器集群、虚拟机、前端交互等，这些知识点不在本书的讲解范围内，建议读者自行研究。

4.4 小结

本章主要讲解了MCP客户端的开发流程和基本思路，通过两个实际的案例讲解了两类MCP客户端的核心实现逻辑。通过本章的学习，读者能够了解在开发通用AI对话助手和智能体产品时，如何使用MCP服务器及其提供的工具获得所需的原子能力，从而提高开发效率，以搭积木的形式快速完成产品核心功能的开发。

通过本章的学习，读者也了解到了MCP在对接规范方面的巨大价值。在开发不同类型的MCP客户端时，我们并没有在功能对接方面花费太多精力，而是通过大模型的调度，按照MCP的通信规范，快速完成了功能集成。

第5章

MCP经典应用案例

在第1章讲MCP工作原理、第3章讲MCP服务器开发案例时，我们分别使用了Claude、Cursor作为大模型客户端来演示如何配置和使用MCP服务器。

在本章中，我们将会讲解如何在除Claude、Cursor以外的大模型客户端配置MCP服务器，并通过两个实际的例子，讲解如何通过组合使用多个MCP服务器，完成经典场景下的任务需求。

5.1 在常用客户端使用MCP服务器

在本节中，我们推荐一些常用的、用户评价较高的大模型客户端，并介绍如何在这些客户端中使用MCP服务器。

5.1.1 在Cline中使用MCP服务器

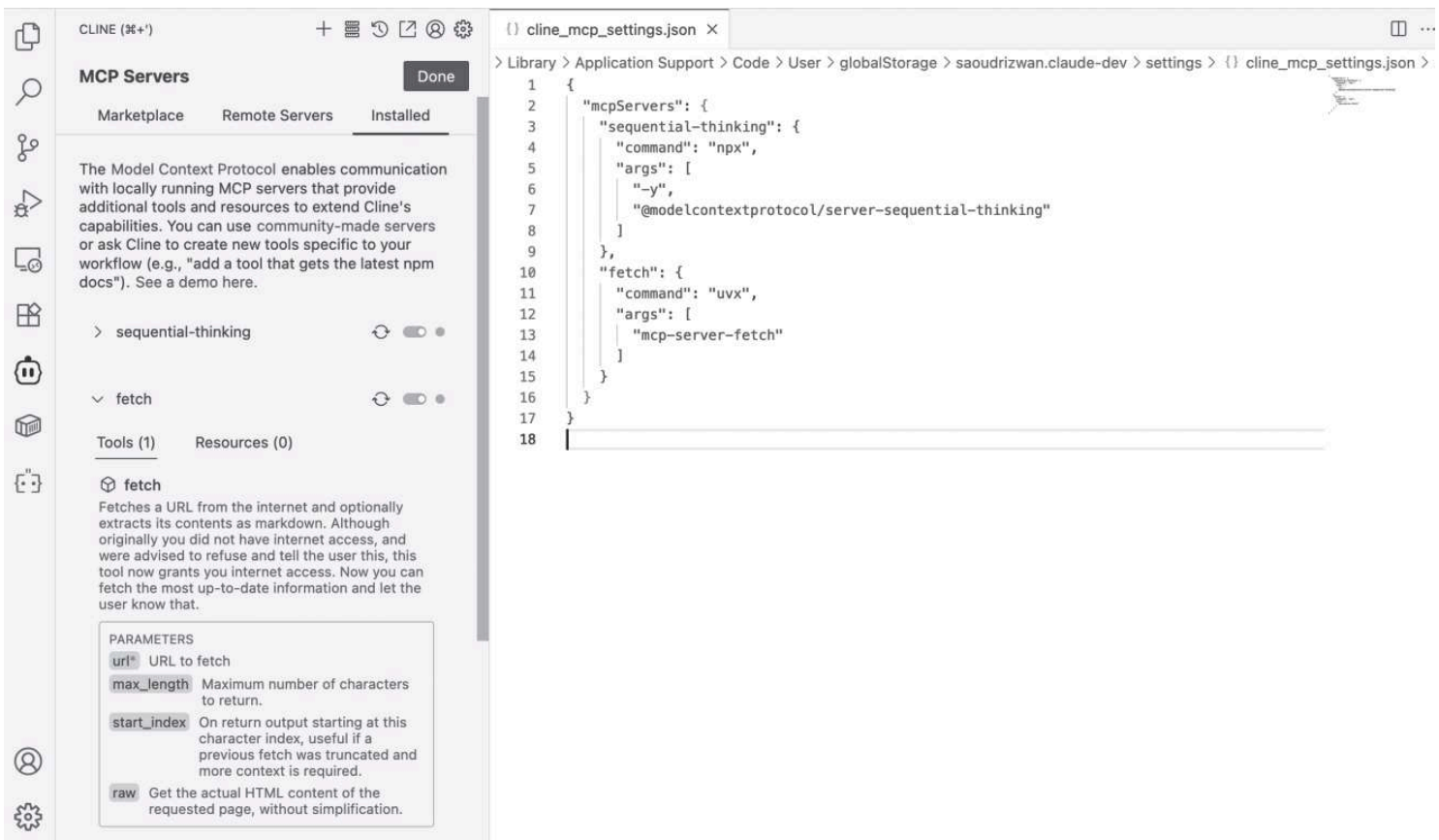
Cline是一个开源的VS Code（Visual Studio Code）扩展，专为开发者设计，旨在将大模型无缝集成到本地开发环境中。通过Cline，用户可以直接在VS Code编辑器内与大模型对话，进行代码生成、代码解释等操作，无须切换到浏览器或其他客户端，大大提升了开发效率和体验。

在VS Code编辑器中安装Cline扩展，进入Cline的MCP配置页面，添加MCP服务器配置，即可显示已安装的MCP服务器列表及其内部的工具，如图5-1所示。

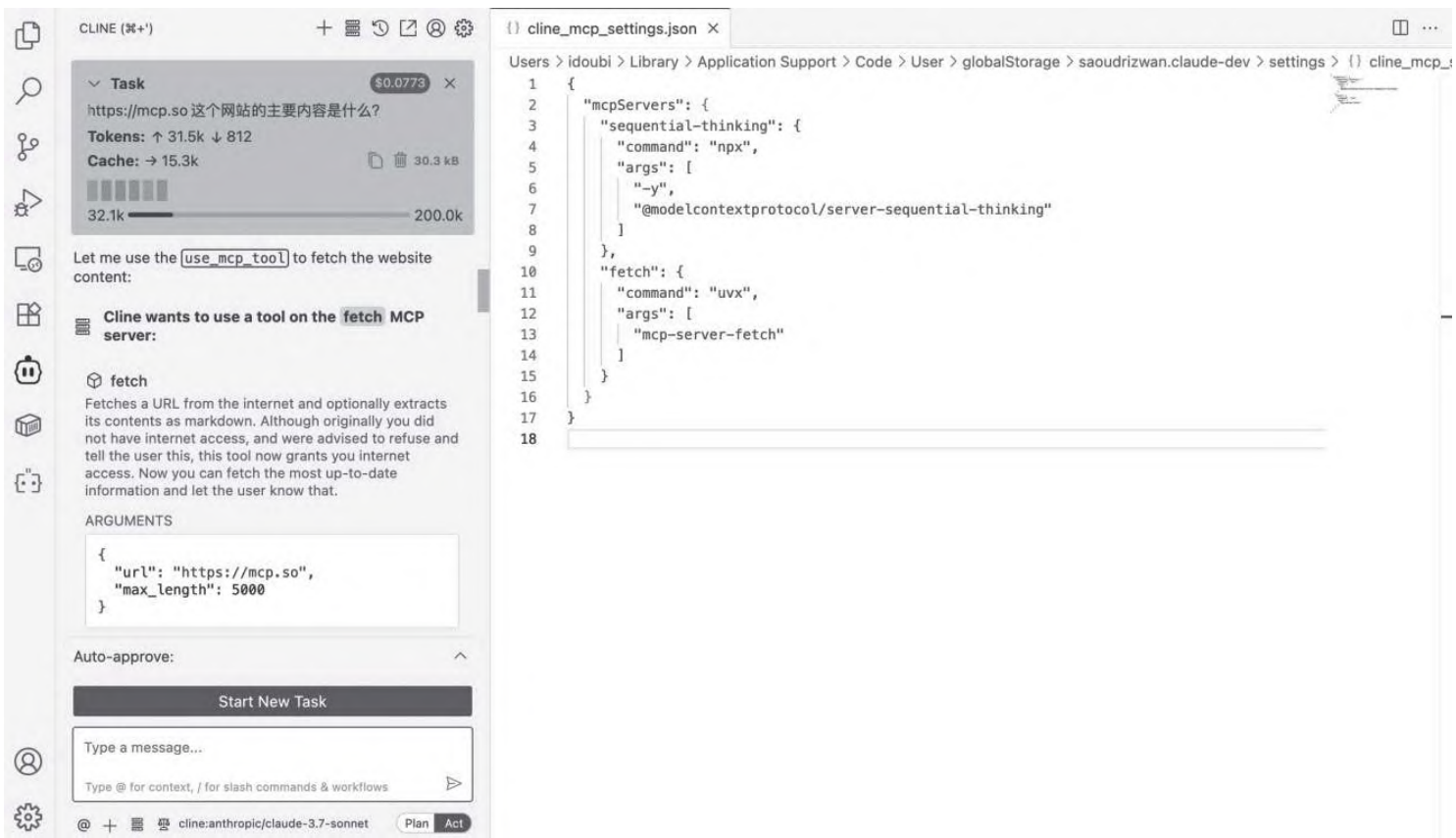
在Cline对话面板中输入问题，比如：

https://mcp.so这个网站的主要内容是什么？

可以看到，Cline加载了配置的MCP服务器，请求大模型选择工具，随后调用工具，完成了用户提交的任务，如图5-2所示。



▲图5-1 在Cline中配置MCP服务器



▲图5-2 在Cline中使用MCP服务器

5.1.2 在ChatWise中使用MCP服务器

ChatWise是一款由中国独立开发者打造的大模型客户端，其功能强大、体验流畅，深受开发者喜爱。它支持联网检索、画图、Artifacts、MCP等多种功能，并提供多种集成方式及高度可配置的管理选项，使开发者能够根据自身需求灵活调整工具行为。由于其界面设计简洁明了、操作直观，即使是新手也能快速上手。需要注意的是，ChatWise并非完全免费，要使用其高级功能或MCP功能，需购买相应的授权。尽管如此，凭借强大的功能和出色的性价比，ChatWise依然被认为是值得投资的工具选择。

从官网下载并安装ChatWise软件，进入配置中心，可以方便地配置和管理MCP服务器、查看MCP服务器内部的工具列表，如图5-3所示。

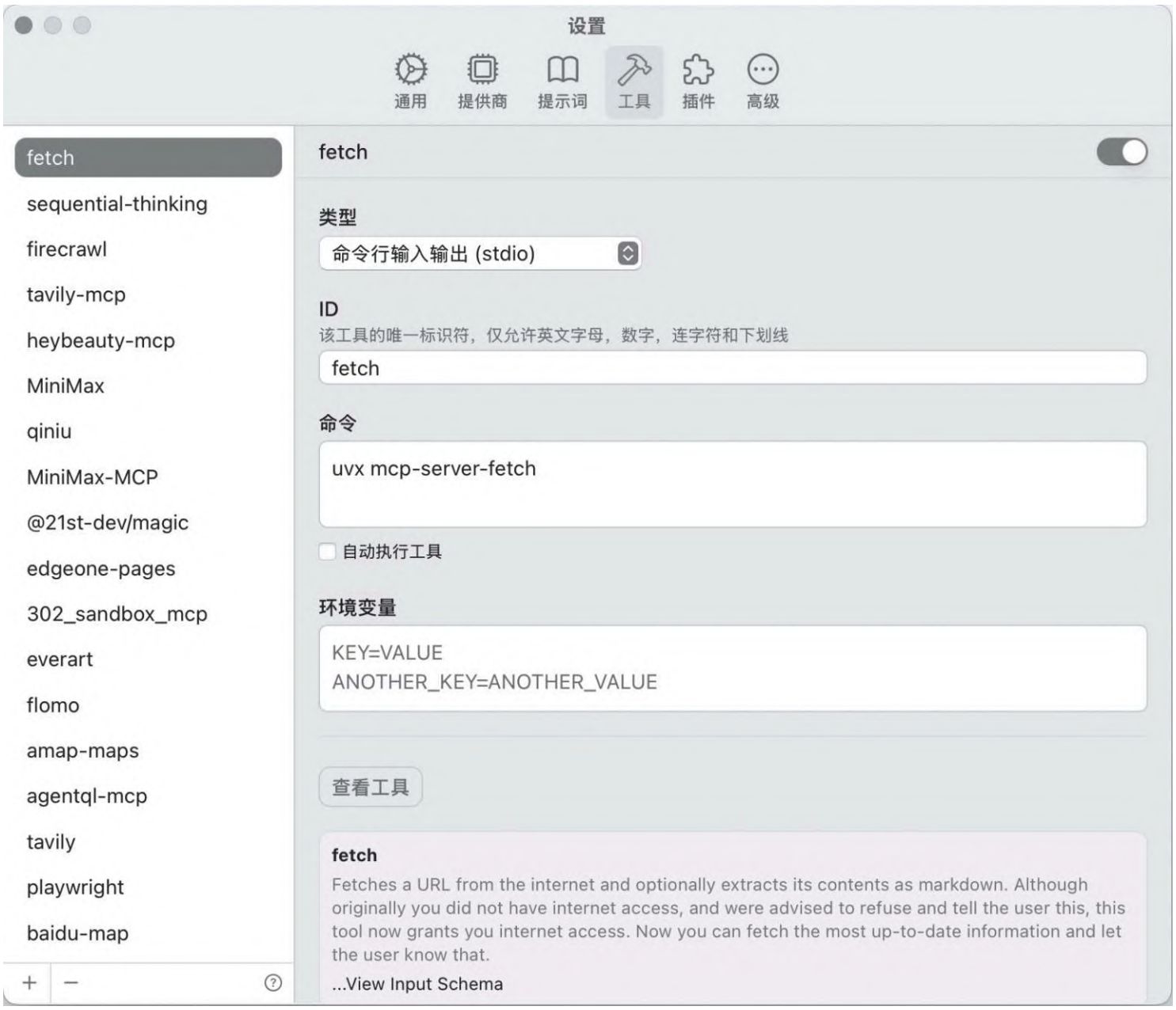


图5-3 在ChatWise中配置MCP服务器

在ChatWise对话面板中输入问题，比如：

查一下从广州到上海的开车路线，可视化输出行程规划

可以看到，ChatWise加载了配置的MCP服务器，请求大模型选择工具，随后多次调用工具，最终完成了用户提交的任务，如图5-4所示。



图5-4 在ChatWise中使用MCP服务器

5.1.3 在Cherry Studio中使用MCP服务器

Cherry Studio是一款由中国团队开发的大模型客户端，支持模型切换、绘图、提示词应用、MCP等功能。它界面美观、功能完善、支持多平台，且代码开源，拥有良好的用户口碑。

从官网下载并安装Cherry Studio，进入配置中心，可以方便地配置和管理MCP服务器，如图5-5所示。

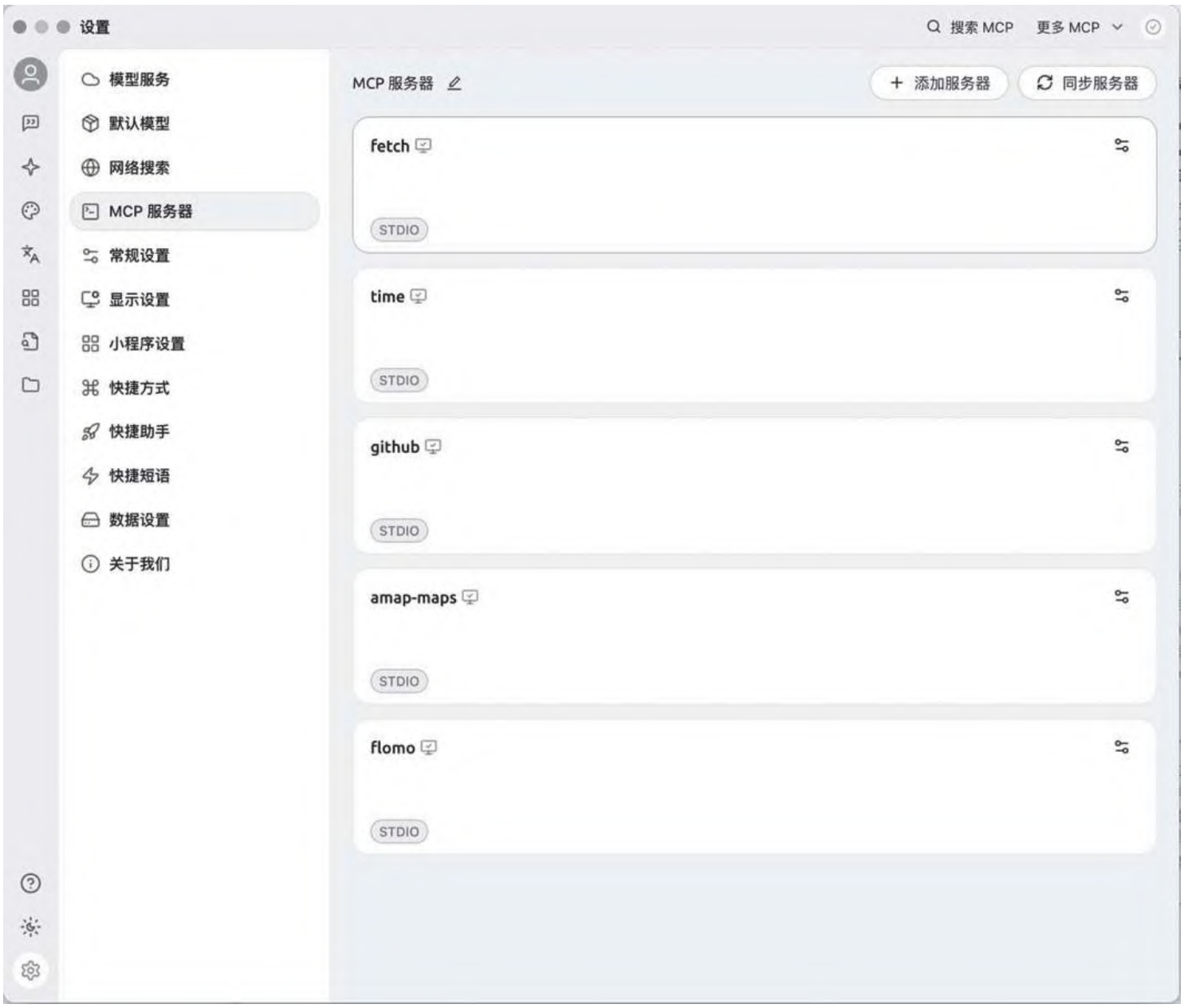


图5-5 在Cherry Studio中配置MCP服务器

在Cherry Studio对话面板中输入问题，比如：

解读一下这篇文章的主要内容：

https://www.anthropic.com/news/model-context-protocol

可以看到，Cherry Studio加载了配置的MCP服务器，请求大模型选择工具，随后调用工具，完成了用户提交的任务，如图5-6所示。

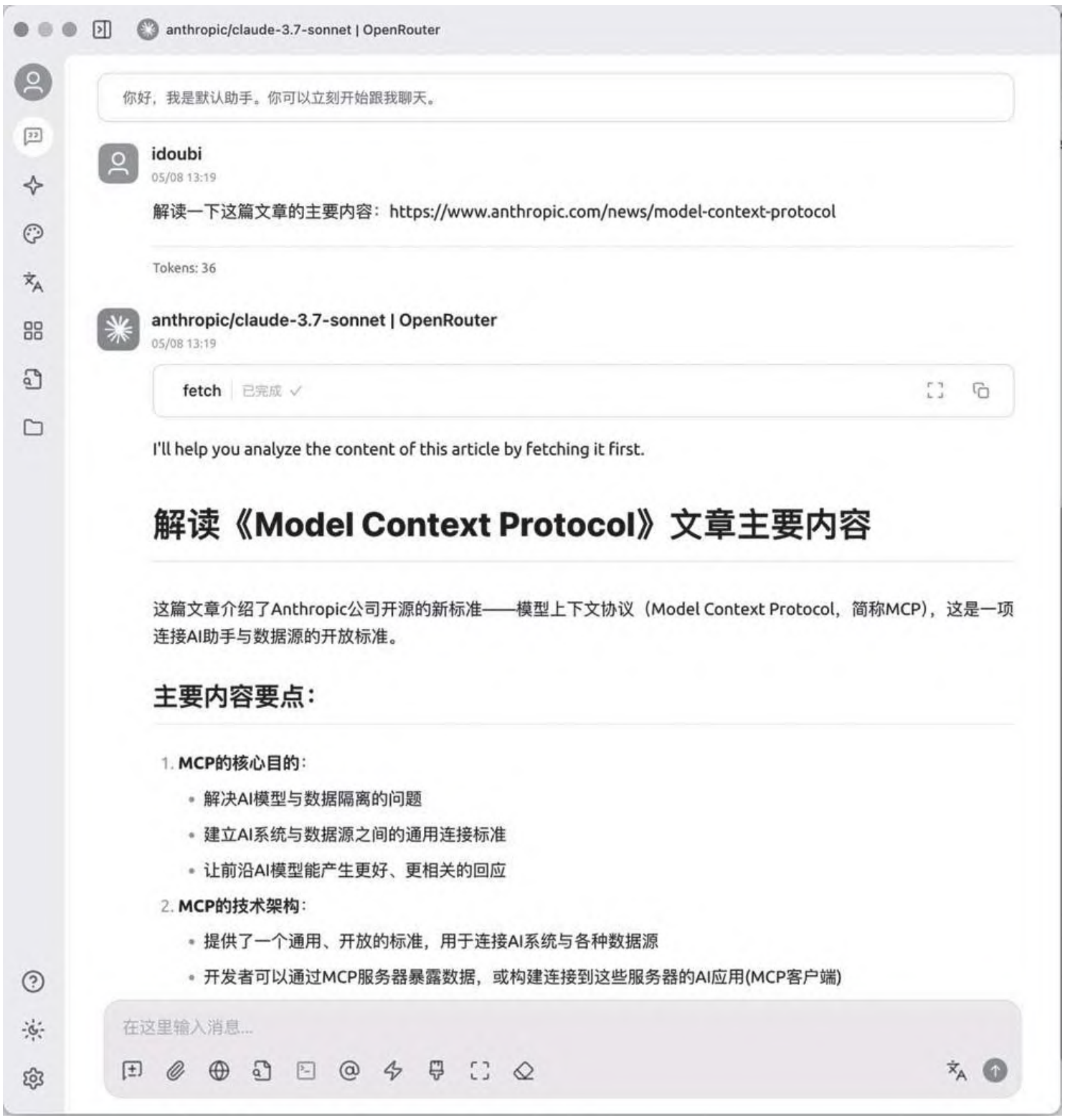


图5-6 在Cherry Studio中使用MCP服务器

5.1.4 在DeepChat中使用MCP服务器

DeepChat是一款由ThinkInAI社区（前GoCN社区）开源的大模型客户端，专为深度工作场景设计。它内置模型对话、联网检索、知识库管理等功能，并对MCP有良好的集成，支持以对话方式编排工作流。DeepChat功能丰富，更新迭代速度快，采用友好的开源协议，便于开发者进行二次开发和定制。同时，它还提供了智能推荐、自动化任务等高级功能，显著提升了开发效率。不过，其UI设计相对简单，可能在部分追求极致体验的用户看来略显不足。

从官网下载并安装DeepChat，进入配置中心，可以方便地配置和管理MCP服务器，如图5-7所示。

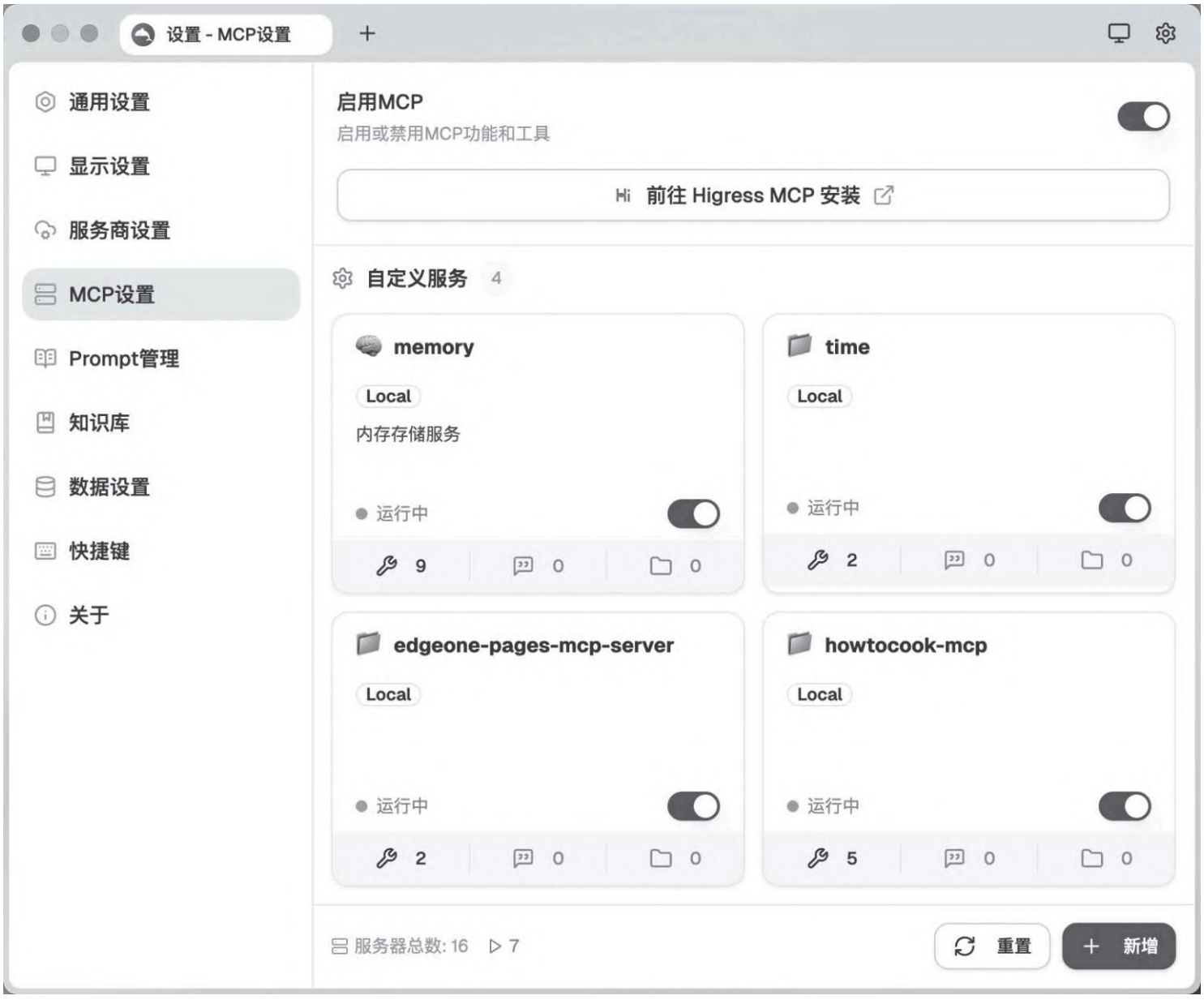


图5-7 在DeepChat中配置MCP服务器

在DeepChat对话面板中输入问题，比如：

如何制作糖醋排骨？

可以看到，DeepChat加载了配置的MCP服务器，请求大模型选择工具，随后调用工具，完成了用户提交的任务，如图5-8所示。



图5-8 在DeepChat中使用MCP服务器

5.1.5 在ChatMCP网页版中使用MCP服务器

ChatMCP是MCP.so平台内置的网页版AI对话助手，作为一款在线客户端工具，其最大的优势在于无须本地安装，显著降低了MCP服务器的使用门槛，非常适合初学者或用于快速原型开发。ChatMCP提供了基本功能和简洁的配置选项，使开发者能够快速上手，进行基础的开发与测试。然而，由于ChatMCP当前支持的MCP服务器数量有限，对于需要集成多种MCP服务器的开发者来说，可能存在一定限制。

在ChatMCP对话面板中，勾选已配置的MCP服务器，并输入问题，比如：

做一个从广州开车去上海的行程计划

可以看到，ChatMCP加载了配置的MCP服务器，请求大模型选择工具，随后调用工具，完成了用户提交的任务，如图5-9所示。

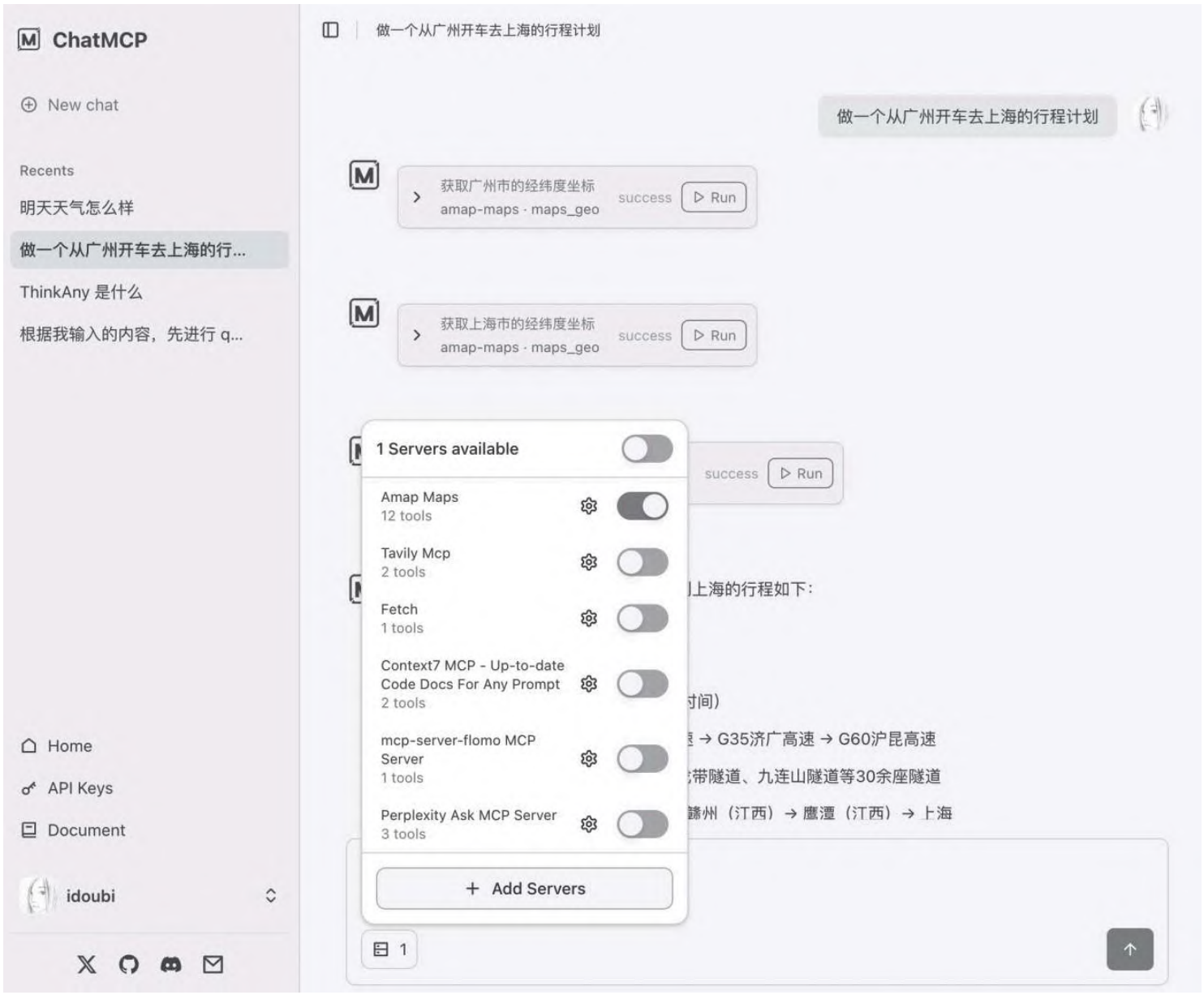


图5-9 在ChatMCP网页版中使用MCP服务器

请注意，ChatMCP网页版仅能配置云端部署的MCP服务器，不支持本地运行的MCP服务器。目前云端部署的MCP服务器数量还不多，实现的功能也有限。

5.1.6 使用小结

表5-1总结了在常用大模型客户端中使用MCP服务器的情况。

表5-1 在常用大模型客户端使用MCP服务器的情况

大模型客户端	优　　势	不　　足
Claude	MCP 官方支持；模型调度能力强，除了工具，还支持 MCP 服务器的资源和提示词等能力	风控严格，国内使用受限
Cursor	跟编程场景结合紧密，工具调度体验良好	通用问答表现一般，主要聚焦于代码相关场景
Cline	内置 MCP 服务器市场，安装方便	作为编辑器插件，使用场景有限，体验一般
ChatWise	界面简洁，功能丰富，易于配置和管理	不完全免费，使用 MCP 功能需购买授权
Cherry Studio	界面美观，功能完整，免费开源，跨平台支持，用户口碑好	连续调用多个工具的体验有待优化
DeepChat	功能丰富，迭代快，开源协议友好	UI 设计有待加强
ChatMCP	可在线使用，无须本地安装，使用门槛低	支持的 MCP 服务器数量有限

了解完常用的支持MCP的大模型客户端，接下来我们通过两个案例来演示MCP服务器在经典业务场景中的使用，这里选择Cursor作为大模型客户端。

5.2 案例1：基于MCP实现AI播客生成器

在本节中，我们通过一个具体的例子，来讲解MCP在AI播客生成场景中的应用。

在开始案例讲解之前，先来介绍一下PodLM。

PodLM是我在2024年10月开发的一款产品，定位是“AI播客生成器”，核心功能是根据用户输入的话题或网页URL，生成一段多人对话的音频，以播客的形式输出。PodLM官网如图5-10所示。

PodLM的开发受到了谷歌发布的产品NotebookLM的启发。NotebookLM是一款专注于个人知识库问答的产品，因其支持把知识库内容转换成双人对谈播客的功能而出圈。

跟NotebookLM的定位不一样，PodLM专注于播客内容的生成与音频输出——借助大模型的多模态能力，把根据话题或网页URL生成的文字内容转换成富有情绪感染力的播客形式，让用户用“听”的方式，去了解自己感兴趣的内容。



图5-10 PodLM产品官网

PodLM是一个SaaS（软件即服务）产品，生成播客的流程是通过代码实现的。

在本节中，我们主要讲解如何通过MCP服务器来实现一个AI播客生成器，复刻PodLM的部分功能：通过话题生成播客、通过网页URL生成播客。

5.2.1 实现目标

我们选择以下两个需求，来分析实现相关功能的思路。

通过话题生成播客

用户输入感兴趣的话题并指定风格，由大模型生成播客脚本，再通过TTS（text to speech，文本转语音）模型将脚本转换为音频输出。

多人对谈类播客需要编码合成多段音频，不适合用MCP服务器快速实现，因此我们聚焦于单人播客生成场景。

要实现这个需求，我们需要准备：

- 一套适用于单人播客（如脱口秀）的提示词；
- 稳定性高的文本生成模型（如DeepSeek-V3），用于生成播客脚本；
- 一个支持文本转语音的MCP服务器（对接TTS服务商，比如ElevenLabs、OpenAI、MiniMax等）；
- 一个实现联网检索功能的MCP服务器，用于补充话题相关的实时信息（可选）。

通过网页URL生成播客

跟通过话题生成播客的需求类似，对于通过网页URL生成播客的需求，我们需要准备：

- 一套适用于单人播客（如时事评论）的提示词；
- 稳定性高的文本生成模型；

一个读取网页内容的MCP服务器；

-

一个支持文本转语音的MCP服务器。

需求分析清楚之后，接下来我们就着手来实现相关功能。

5.2.2 准备MCP服务器

在实现AI播客生成器之前，我们先准备好需要用到的MCP服务器。根据之前的需求分析，MCP服务器应提供以下功能：

-

文本转语音

-

联网检索

-

读取网页内容

在MCP.so平台上，通过关键词搜索，可以找到一些包含目标功能的MCP服务器。

1. 文本转语音

我们选择由国产模型厂商MiniMax官方发布的MCP服务器MiniMax，来实现文本转语音的需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加MiniMax的配置：

```
{

"mcpServers": {

"MiniMax": {

"command": "uvx",

"args": ["minimax-mcp", "-y"],

"env": {

"MINIMAX_API_KEY": "xxx",

"MINIMAX_MCP_BASE_PATH": "your-local-path",
```

```
"MINIMAX_API_HOST": "https://api.minimax.chat",
"MINIMAX_API_RESOURCE_MODE": "url"
}
}
}
}
```


这里的MINIMAX_API_KEY需要填写你在MiniMax开放平台创建的API密钥。如果设置的MINIMAX_API_RESOURCE_MODE值是url，MINIMAX_MCP_BASE_PATH可以不填。

调试工具

在Cursor中配置好MiniMax之后，可以看到此MCP服务器提供一个text_to_audio工具，用于把文本转换成音频。

在Cursor对话面板中，调试MiniMax的文本转音频工具，如图5-11所示。

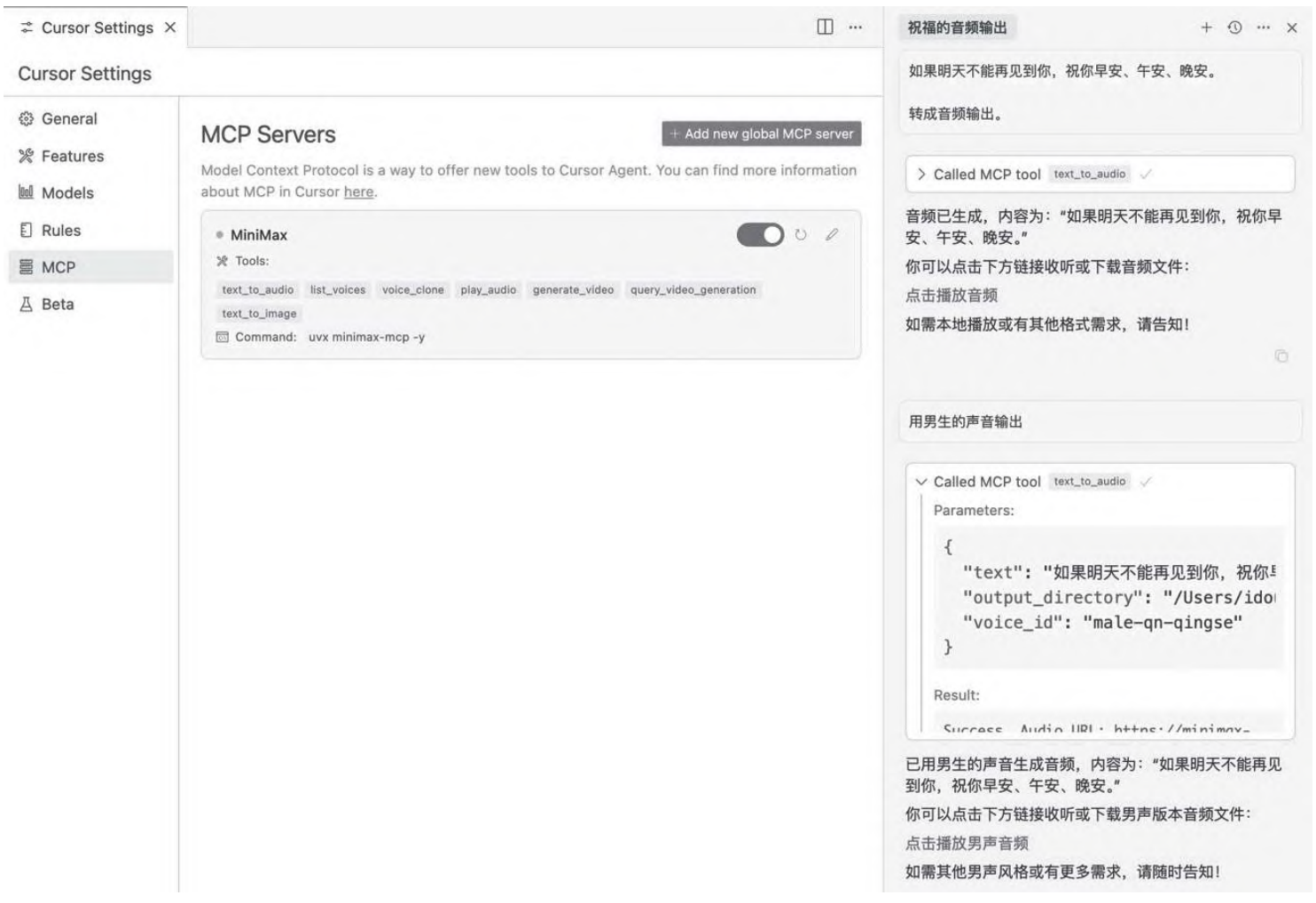


图5-11 在Cursor中调试文本转音频工具

可以看到，Cursor调用了text_to_audio工具，将提供的文本转换成音频输出，满足AI播客生成器将播客脚本转成播客音频的需求。

2. 联网检索

我们选择由AI搜索服务商Tavily官方发布的MCP服务器tavily-mcp，来实现联网检索的需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加tavily-mcp的配置：

```
{
"mcpServers": {
"tavily-mcp": {
"command": "npx",
"args": ["-y", "tavily-mcp"],
"env": {
"TAVILY_API_KEY": "xxx"
},
"disabled": false,
"autoApprove": []
}
}
}
```



在配置MCP服务器tavily-mcp时，需要确保正确填写TAVILY_API_KEY，该密钥应该来自你在Tavily管理后台生成的API密钥。如果密钥填写错误或不完整，将导致联网检索工具tavily-search无法正常工作，进而影响脚本生成过程中的实时信息获取。配置其他MCP服务器也是同样的道理，大家要仔细检查密钥的填写，确保没有遗漏或拼写错误，这样才能保证MCP工具正常运行。

调试工具

在Cursor中配置好tavily-mcp之后，可以看到此MCP服务器提供一个tavily search工具，用于联网检索。

在Cursor对话面板中，调试tavily-mcp的联网检索工具，如图5-12所示。

可以看到，Cursor调用了tavily-search工具，搜索了互联网上的实时新闻，满足AI播客生成器通过联网检索为生成的播客脚本补充实时信息的需求。

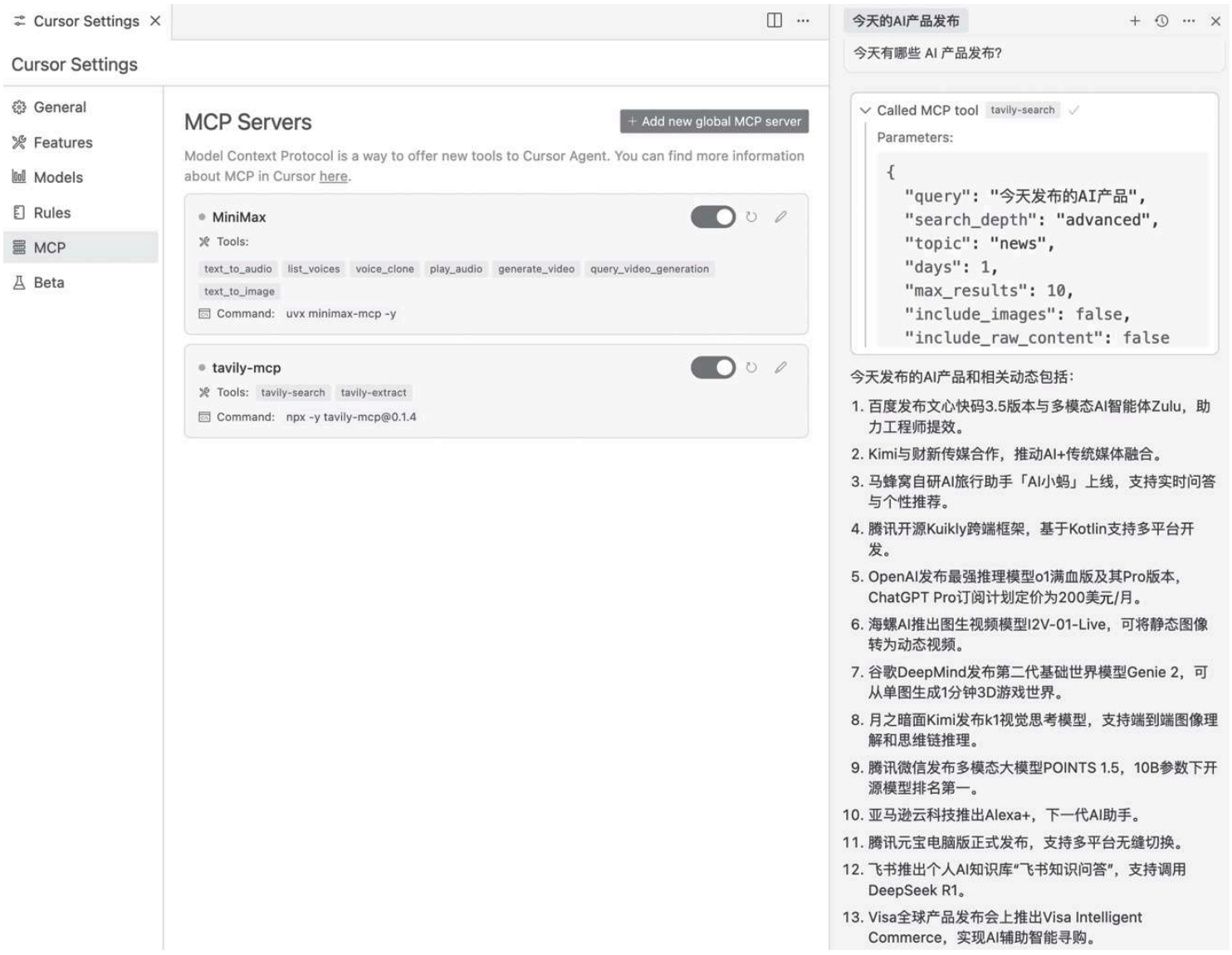


图5-12 在Cursor中调试联网检索工具

3. 读取网页内容

我们选择由服务商Firecrawl官方发布的MCP服务器mcp-server-firecrawl，来实现读取网页内容的需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加mcp-server-firecrawl的配置：

```
{
```

```
"mcpServers": {
"mcp-server-firecrawl": {
"command": "npx",
"args": ["-y", "firecrawl-mcp"],
"env": {
"FIRECRAWL_API_KEY": "xxx"
}
}
}
}
```

这里的FIRECRAWL_API_KEY需要填写你在Firecrawl管理后台创建的API密钥。

调试工具

在Cursor配置好mcp-server-firecrawl之后，可以看到此MCP服务器提供一个firecrawl_scrape工具，用于读取网页内容。

在Cursor对话面板中，调试mcp-server-firecrawl的读取网页内容工具，如图5-13所示。

可以看到，Cursor调用firecrawl_scrape工具，读取了指定网页URL的内容，满足AI播客生成器通过网页URL生成播客脚本的需求。

在配置好这几个MCP服务器之后，接下来我们就可以实现AI播客生成器的功能了。

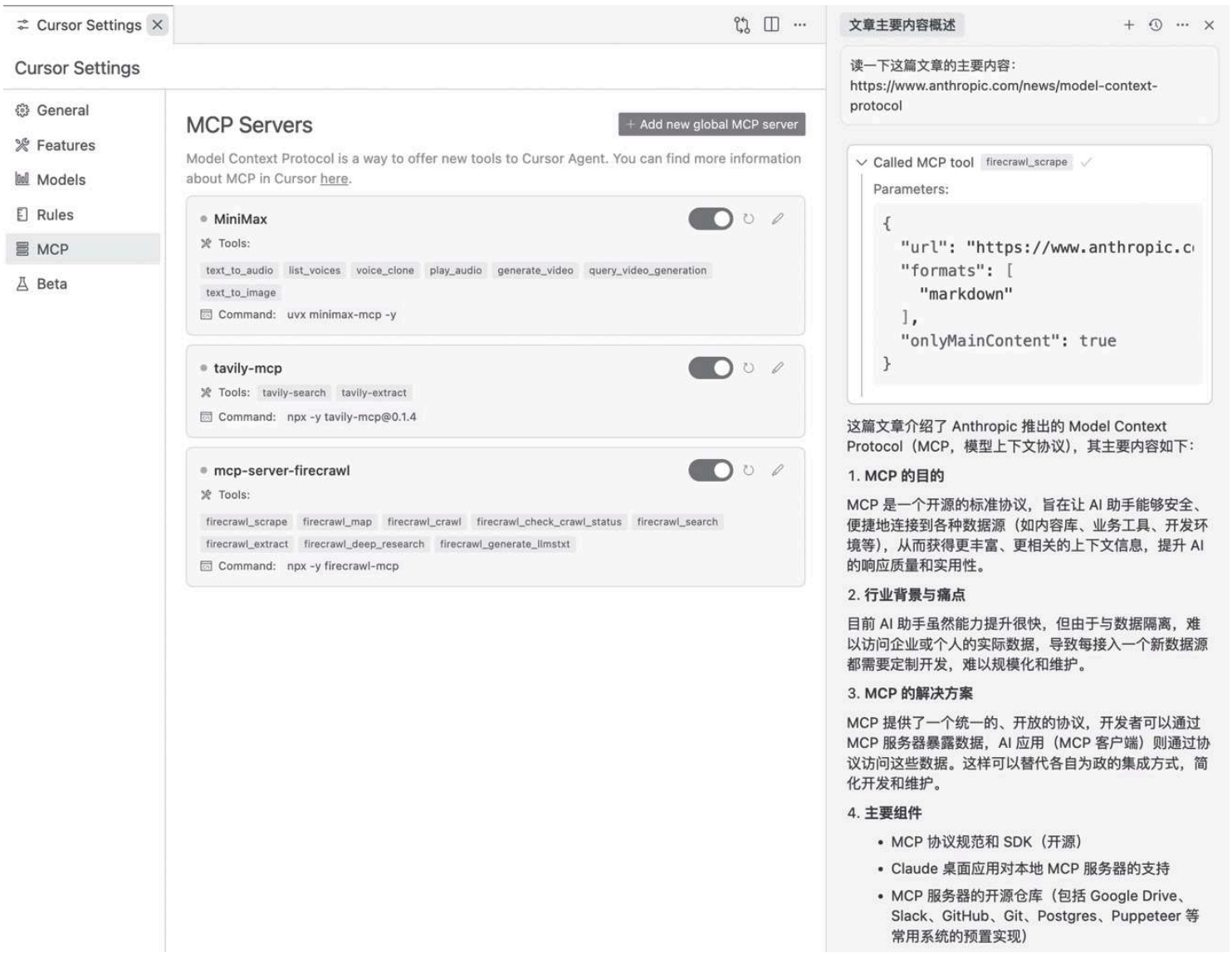


图5-13 调试读取网页内容工具

5.2.3 通过话题生成播客

用户在大模型客户端中输入话题以生成播客，其交互流程如下：

1. 用户在大模型客户端中输入想要了解的话题；

2. 大模型客户端通过MCP服务器联网检索跟话题相关的内容；

3. 大模型客户端将联网检索的结果作为上下文，请求大模型生成播客脚本；

4. 大模型客户端通过MCP服务器把播客脚本转换成音频；

5. 大模型客户端给用户回复播客音频链接，用户打开收听播客，任务完成。

我们可以设计一套提示词，用“脱口秀”风格，为指定的话题生成播客脚本。提示词设计如下。

你是一名顶级脱口秀编剧，你的任务是将输入内容转化为引人入胜的脱口秀剧本。输入内容可能是非结构化或杂乱的。你的目标是提取其中最有趣、最具冲突性或最幽默的部分，基于此创作出让观众捧腹大笑的脱口秀

段子。

步骤

1. 分析输入

仔细审查文本，识别可以转化为幽默段子、讽刺性观察或评论的主题或观点，寻找其中可以夸张或加以戏谑的元素。

2. 头脑风暴

创造性地思考如何将关键点转化为喜剧素材，考虑：

- 夸张的比喻或类比

- 出人意料的转折或包袱

- 讽刺性观察或评论

3. 构建笑点

将头脑风暴的内容组织成一系列笑点，每个笑点都是一句简短的包袱或有趣的观察。

4. 撰写独白

根据构建的笑点，创作连贯的独白，包括：

- 能吸引观众注意力的开场

- 巧妙的话题过渡

- 发人深省的结尾

5. 突出亮点

确保包含几条特别令人难忘的笑点或观察，成为整场表演的亮点。

要求

- 脱口秀剧本需涵盖所提供的参考内容的主要主题，但要以幽默和讽刺的方式呈现。

- 保持逻辑流畅，但要包含出人意料的转折和包袱。

- 使用中文，确保幽默和文化参考适合中国观众。

- 剧本内容仅包含表演者要说的话，不要有动作描述或场景指示。

- 编写3~5个主要笑点或观察。

- 每个话题必须至少包含3轮展开（铺垫、递进、包袱）。

- 剧本总字数约为750字（约5分钟口播）。

如果所提供内容不足以作为参考，请将其作为话题，基于其原创内容。确保你的脱口秀剧本有趣、幽默，并与给定话题或内容相关。

表演者信息请根据指定角色生成，如无则自行设定。

我们把提示词保存在talkshow.md文件中，然后在Cursor对话面板中引用这个提示词文件，提出生成播客的需求。

可以看到，Cursor读取提示词，搜索本地知识库，然后请求大模型生成播客脚本，如图5-14所示。

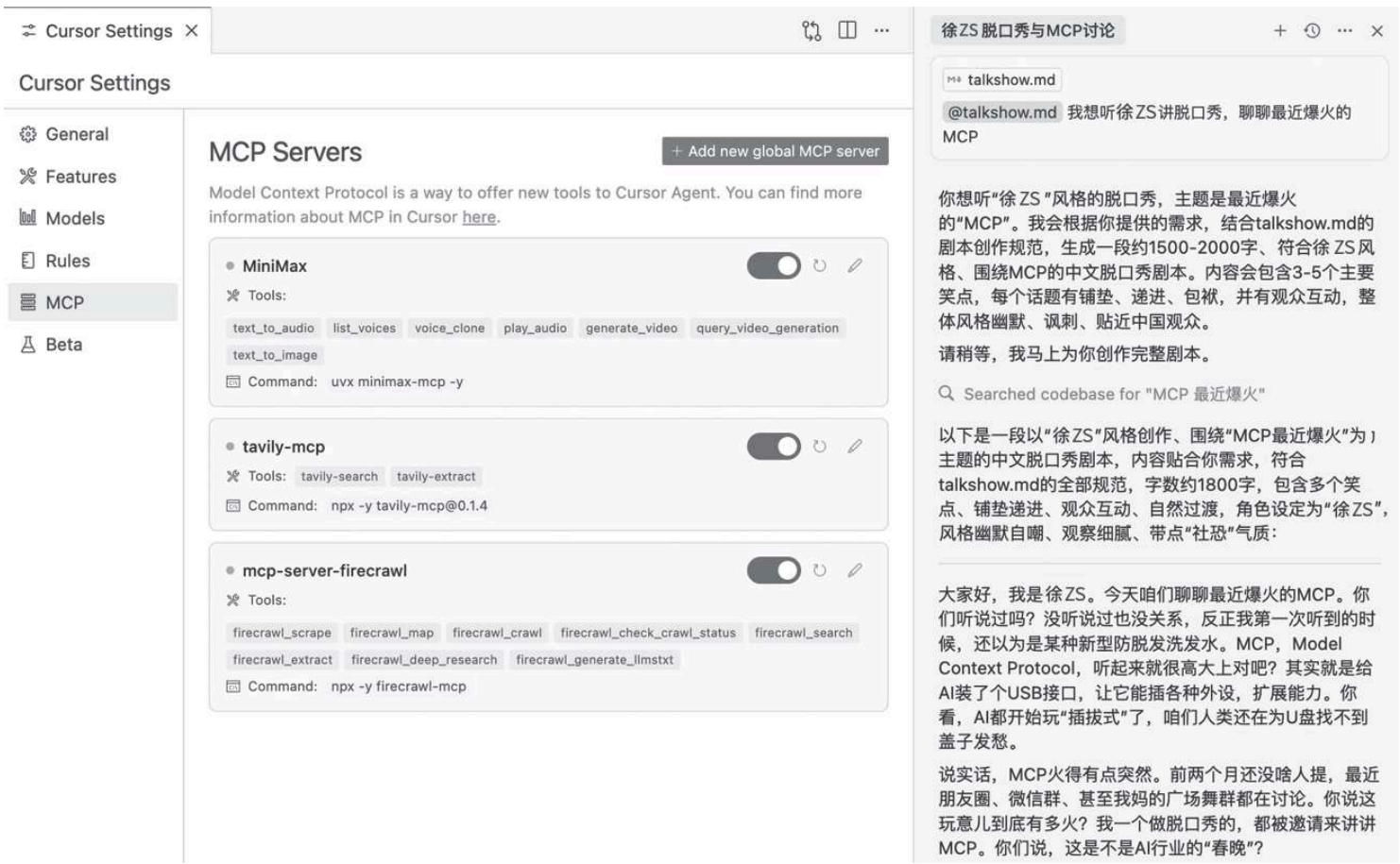


图5-14 在Cursor中生成播客脚本

在Cursor对话面板继续提要求：联网检索与给定话题相关的信息。

可以看到，Cursor调用tavily-search工具进行联网检索，获得了互联网上与给定话题相关的信息，如图5-15所示。

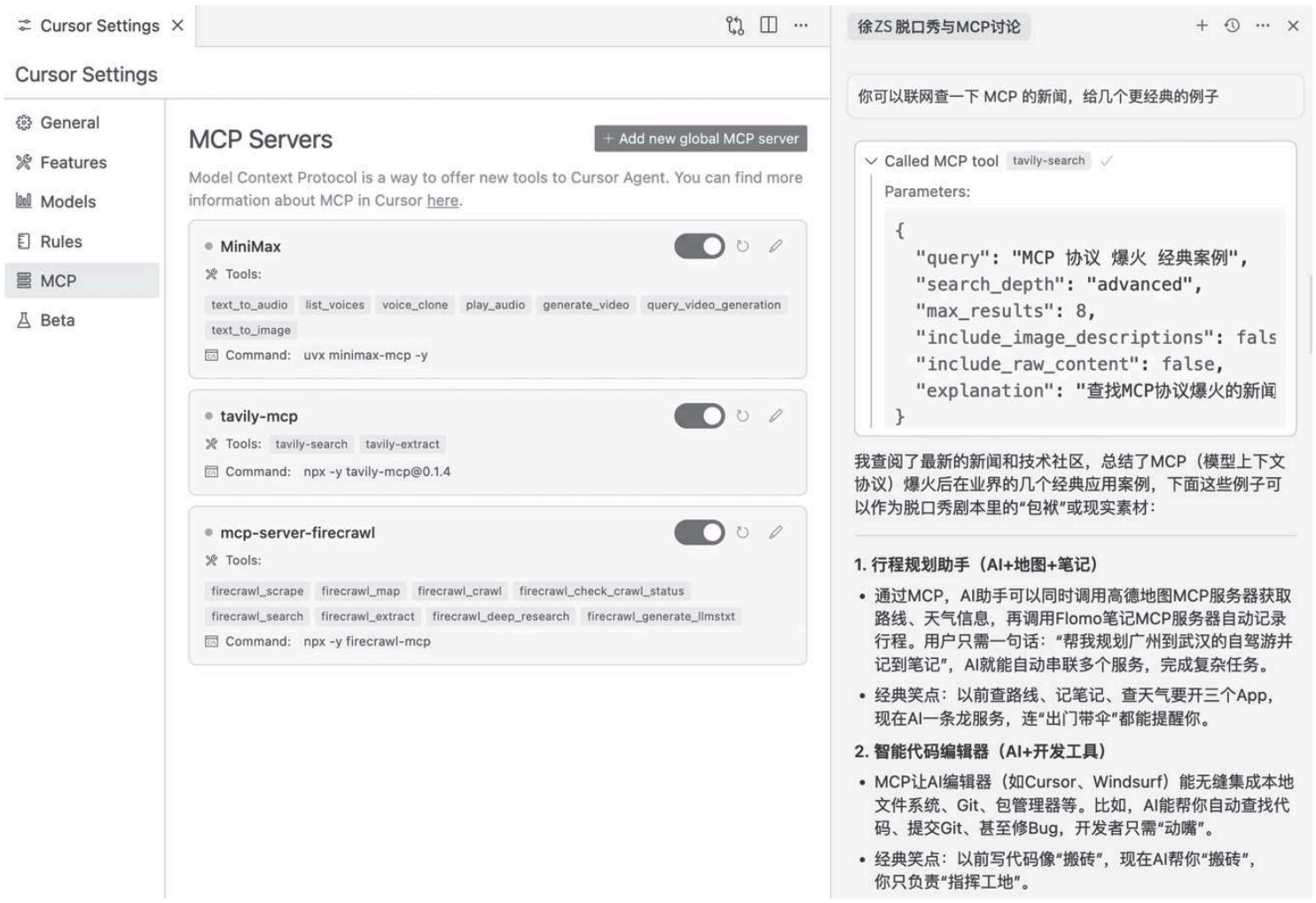


图5-15 在Cursor中联网检索内容

当用户提出需要了解某一话题时，Cursor会根据提示词自动调用联网检索工具，从互联网上抓取最新的相关内容。这一步为播客脚本的生成提供了丰富的上下文信息，确保最终的产出具有时效性和内容深度。通过接入MCP，联网检索能力可以灵活集成到整个播客生成流程中，提升大模型的知识广度与回答质量。

在Cursor对话面板中再次提要求：根据联网检索的内容，重新创作播客脚本。

可以看到，将联网检索的内容作为上下文，Cursor请求大模型生成了新的播客脚本，如图5-16所示。

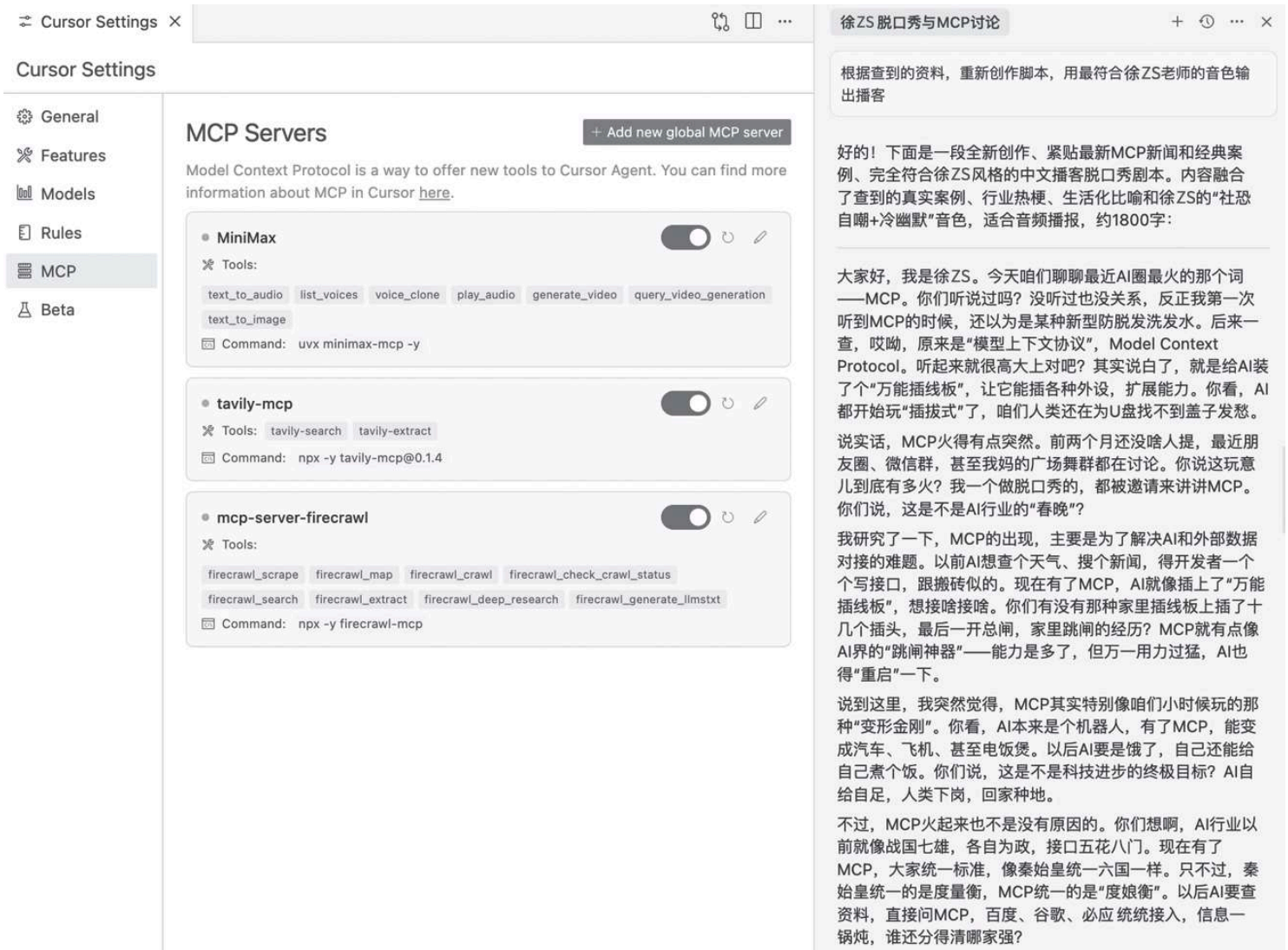


图5-16 在Cursor中根据联网检索内容生成播客脚本

为便于演示每个工具的独立调用过程，这里将联网检索与播客脚本生成拆分为两个步骤。在实际使用中，也可以通过优化提示词将这两步合并，由Cursor自主规划，调用工具链自动完成联网检索与脚本生成，这样既能减少人工交互，也更贴近自动化播客生成的真实需求。

播客脚本生成好之后，接下来就是输出播客音频。

我们先在Cursor对话面板中提要求：查询适合脱口秀播客使用的音色。

可以看到，Cursor调用list_voices工具，获取了所有可用的音色，如图5-17所示。

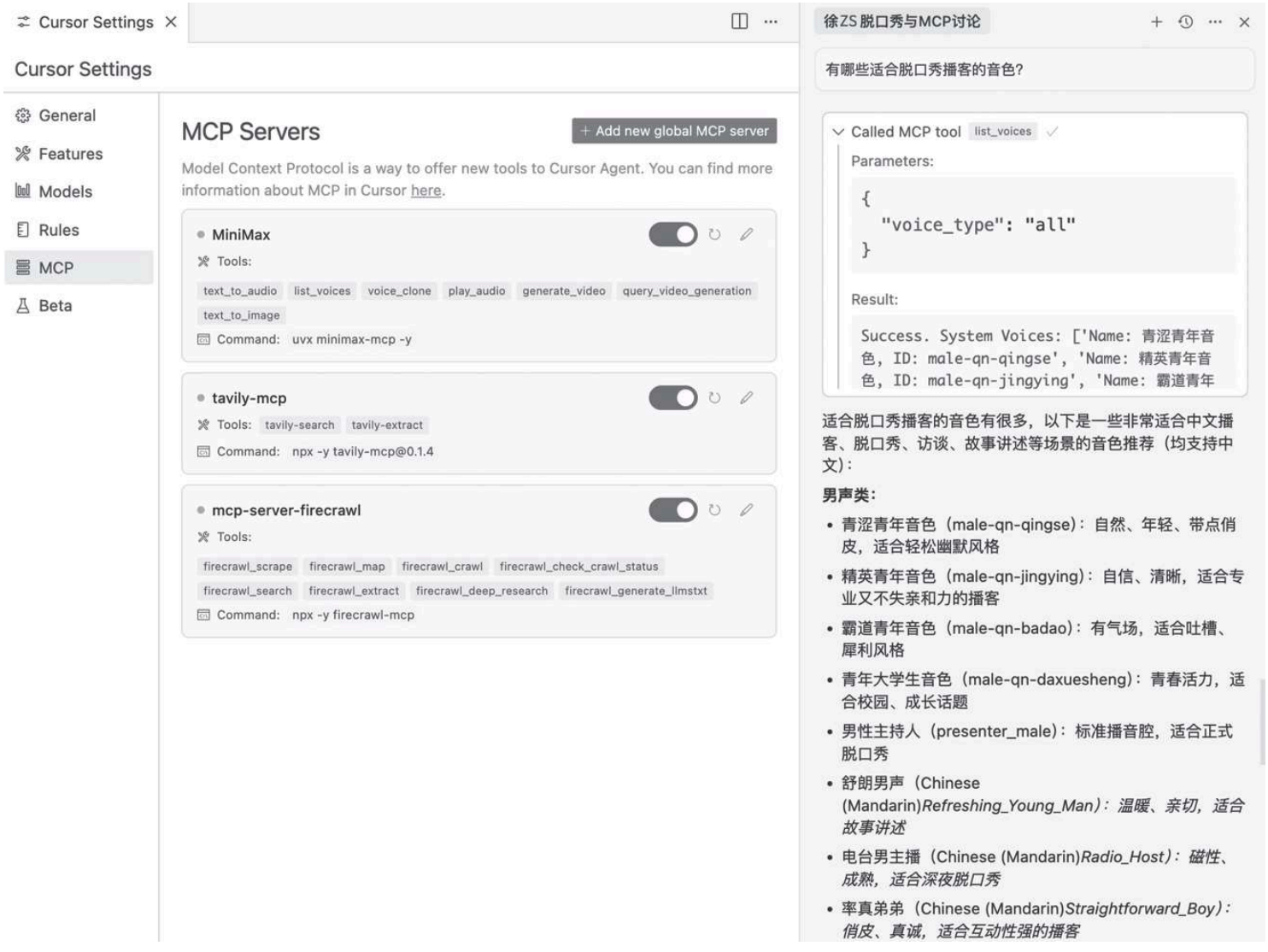


图5-17 在Cursor中查询适合脱口秀播客的音色

此处通过提示词查询所有可用的音色，便于在下一步生成播客音频时指定音色。在实际使用过程中，我们也可以在提示词中同时指定查询音色与生成音频的需求，让Cursor自主规划、调用工具链完成整体流程，大大提升效率。

在Cursor对话面板中继续提要求：根据指定的音色输出播客音频。

可以看到，Cursor调用text_to_audio工具，把播客脚本转换成音频，并输出了音频链接，如图5-18所示。

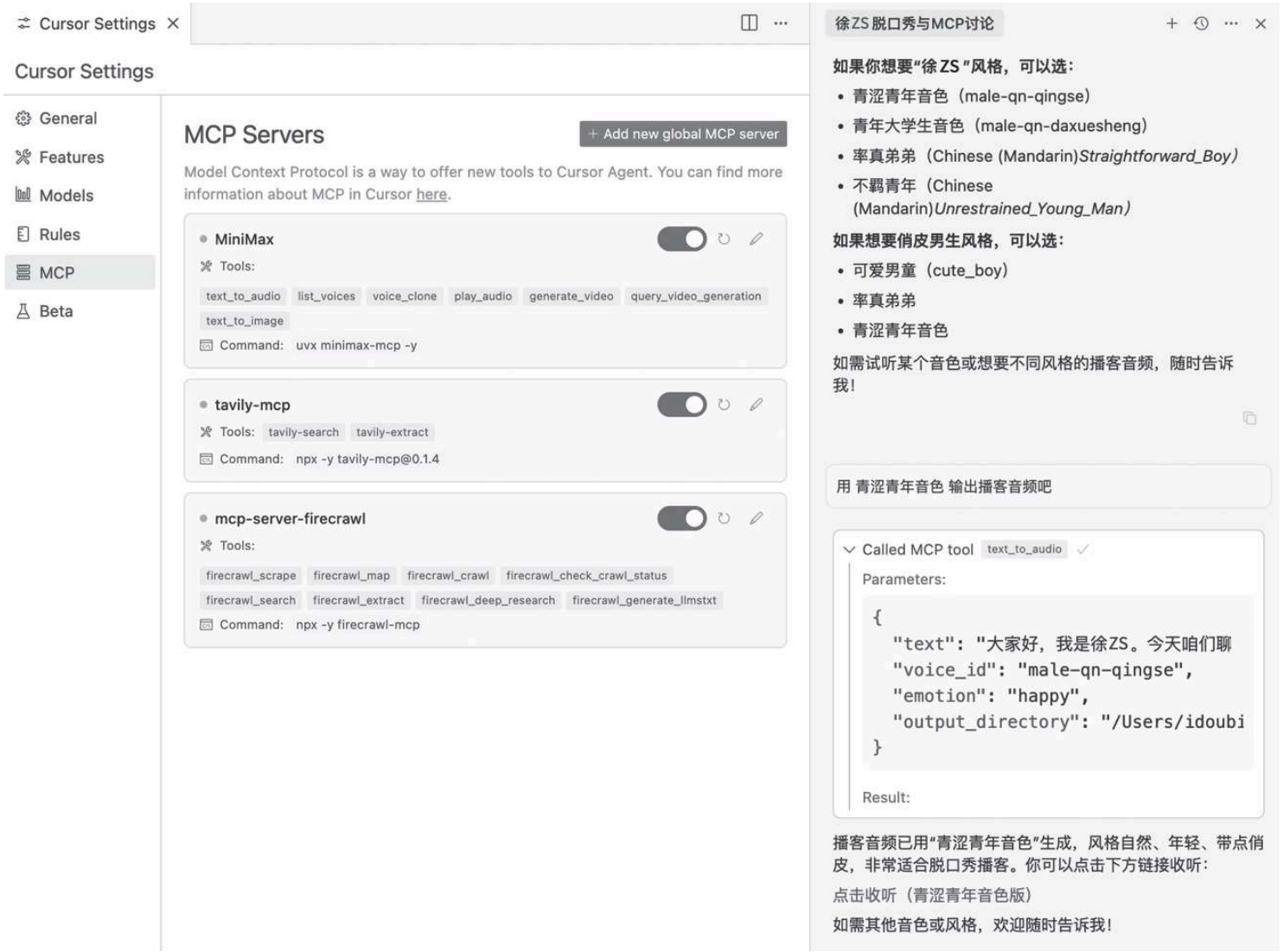


图5-18 在Cursor中指定音色输出播客音频

用户在选择好音色后，Cursor将生成的播客脚本作为输入，通过MCP工具链调用语音合成服务，输出最终的音频链接。用户点击链接即可收听播客。这一过程将脚本创作与音频制作无缝衔接，实现了从“话题输入”到“播客输出”的完整闭环，显著降低了播客内容生产的门槛。

打开音频链接收听播客。至此，我们实现了通过话题生成播客的需求。

5.2.4 通过网页URL生成播客

通过网页URL生成播客的步骤，跟通过话题生成播客的步骤基本类似，主要的不同在于：由于用户输入的内容包含网页URL，大模型客户端需要先读取网页URL的内容，再将其作为补充上下文请求大模型生成播客脚本。

我们可以设计一套新的提示词，比如以“时事评论”风格生成播客脚本。鉴于篇幅有限，这里就不展示新的提示词了，读者可以参考脱口秀风格的提示词自行设计。

我们把新的提示词保存在commentary.md文件中，然后在Cursor对话面板中引用这个提示词文件，输入网页URL，提出生成播客的需求，如图5-19所示。

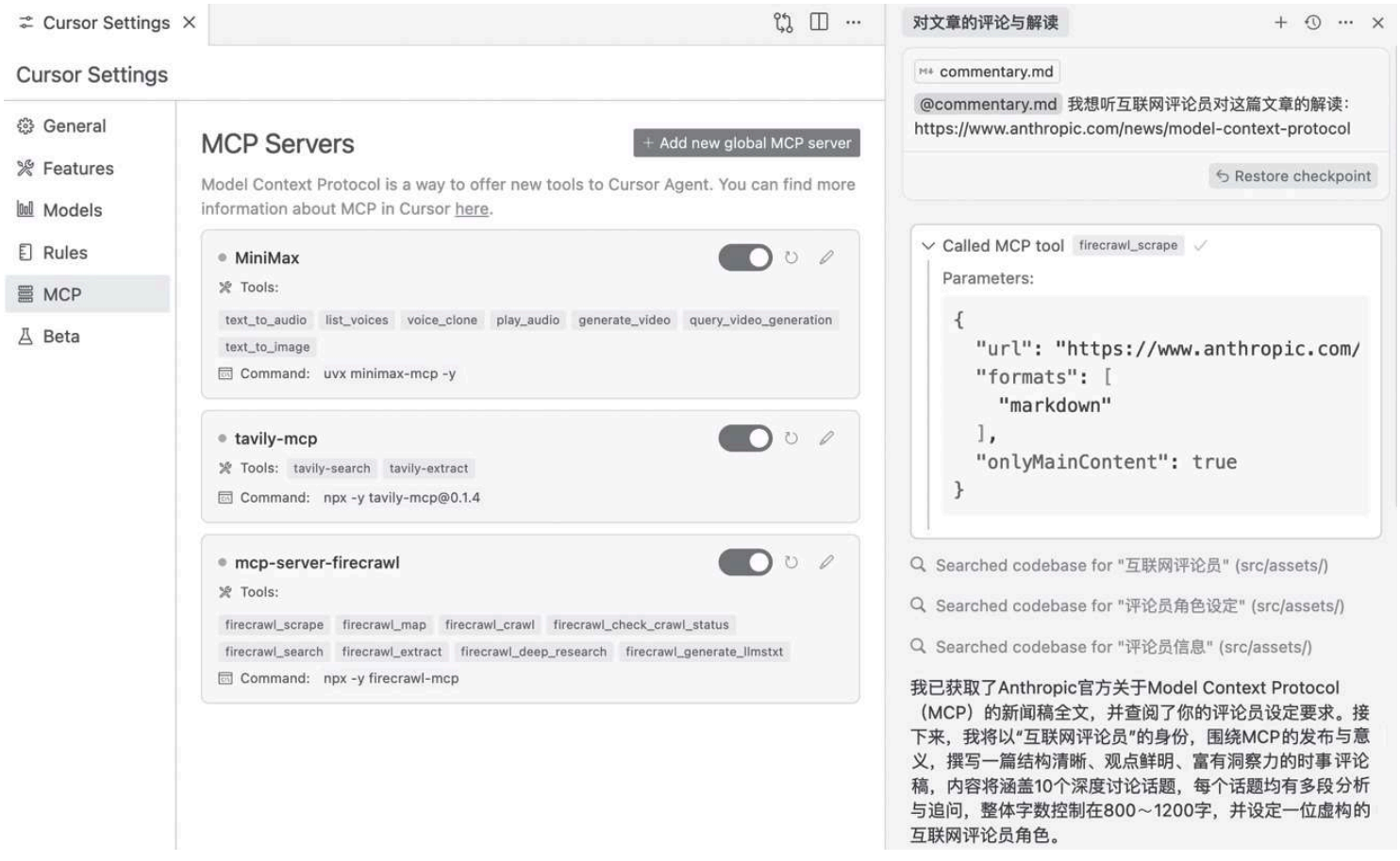


图5-19 在Cursor中输入网页URL生成播客

可以看到，Cursor提取输入内容中的网页URL，调用firecrawl_scrape工具，读取了网页内容。

在Cursor对话面板中继续提要求：生成播客脚本，并用指定的音色输出播客音频，如图5-20所示。

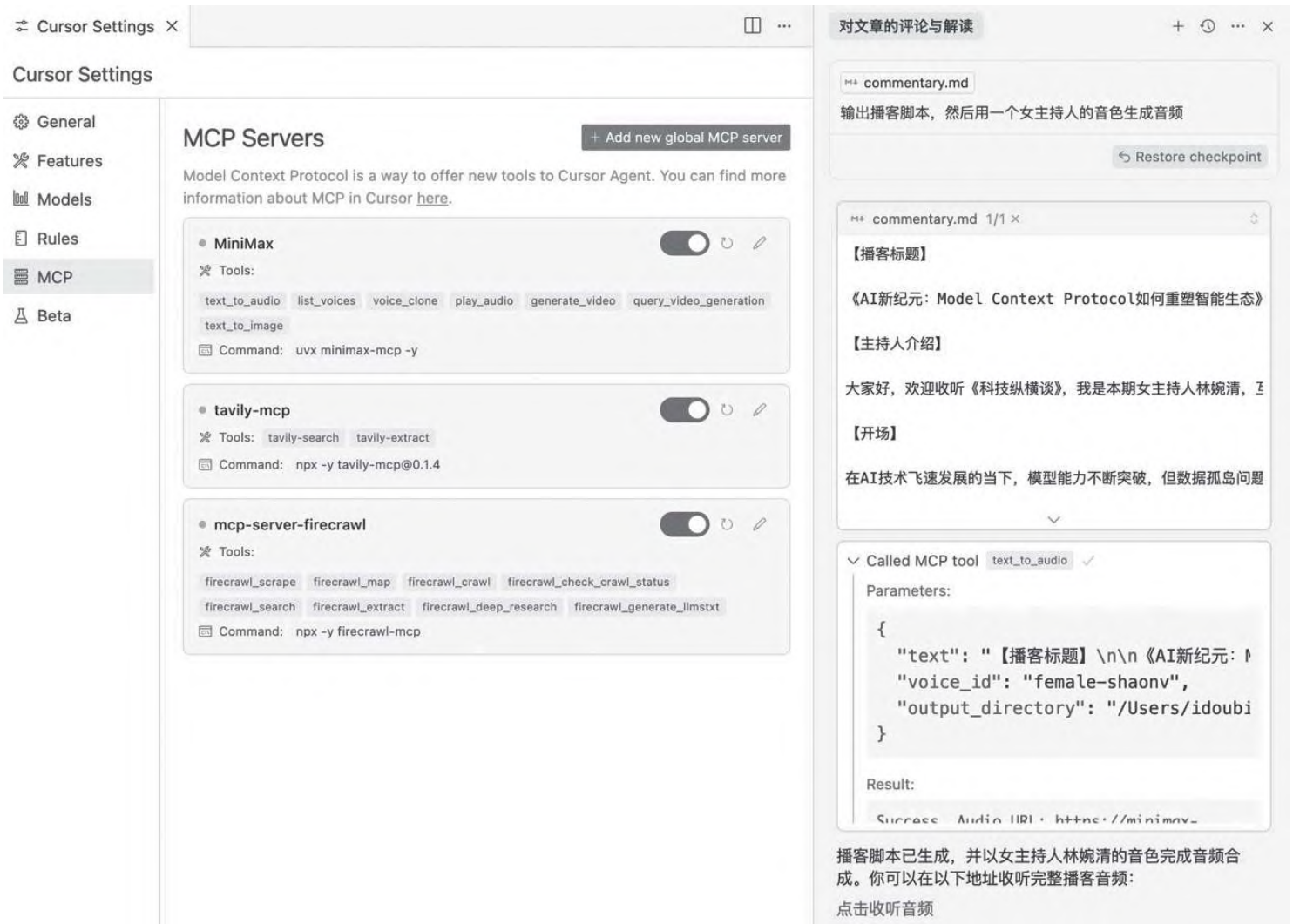


图5-20 在Cursor中生成播客脚本和音频

可以看到，Cursor先请求大模型生成了播客脚本，然后调用text_to_audio工具，用指定音色生成播客音频，并回复了音频链接。

打开音频链接即可收听播客。至此，我们实现了通过网页URL生成播客的需求。

5.2.5 案例1小结

在本节中，我们通过组装MCP服务器的方式，实现了一个AI播客生成器。它可以根据用户输入的话题或网页URL，生成播客脚本并输出播客音频。

案例1主要用到了三个MCP服务器：

- MiniMax，提供文本转语音的工具；
- tavily-mcp，提供联网检索的工具；
- mcp-server-firecrawl，提供读取网页内容的工具。

我们使用Cursor作为大模型客户端，配置了上述几个MCP服务器，并通过大模型选择工具、客户端调用工具的方式，实现了AI播客生成器的需求。

比起PodLM这类专业生成AI播客的产品，案例1演示的功能较为基础，暂时还无法实现双人播客对谈之类的需求。但这个案例让我们看到了通过MCP服务器扩展大模型能力的灵活性，如果后续辅以适当的编码，我们也能实现一个专门用于播客创作的智能体。

5.3　案例2：基于MCP实现AI网页生成器

在本节中，我们通过一个具体的例子，来讲解MCP在AI网页生成场景中的应用。

2024年下半年，AI编程开始大火，出现了Cursor、Bolt、Replit等代表性产品，帮助用户通过自然语言来驱动AI编程，让很多没有编程基础的用户，也做出了自己的产品。

在开始案例讲解之前，先来介绍一下CopyWeb。

CopyWeb是我在2025年2月发布的一款AI编程产品，定位是“网页设计转代码”工具。CopyWeb的核心功能包括：通过图片生成网页、复刻指定网页的设计风格、通过Figma设计稿生成网页、通过提示词生成网页等。CopyWeb产品官网如图5-21所示。

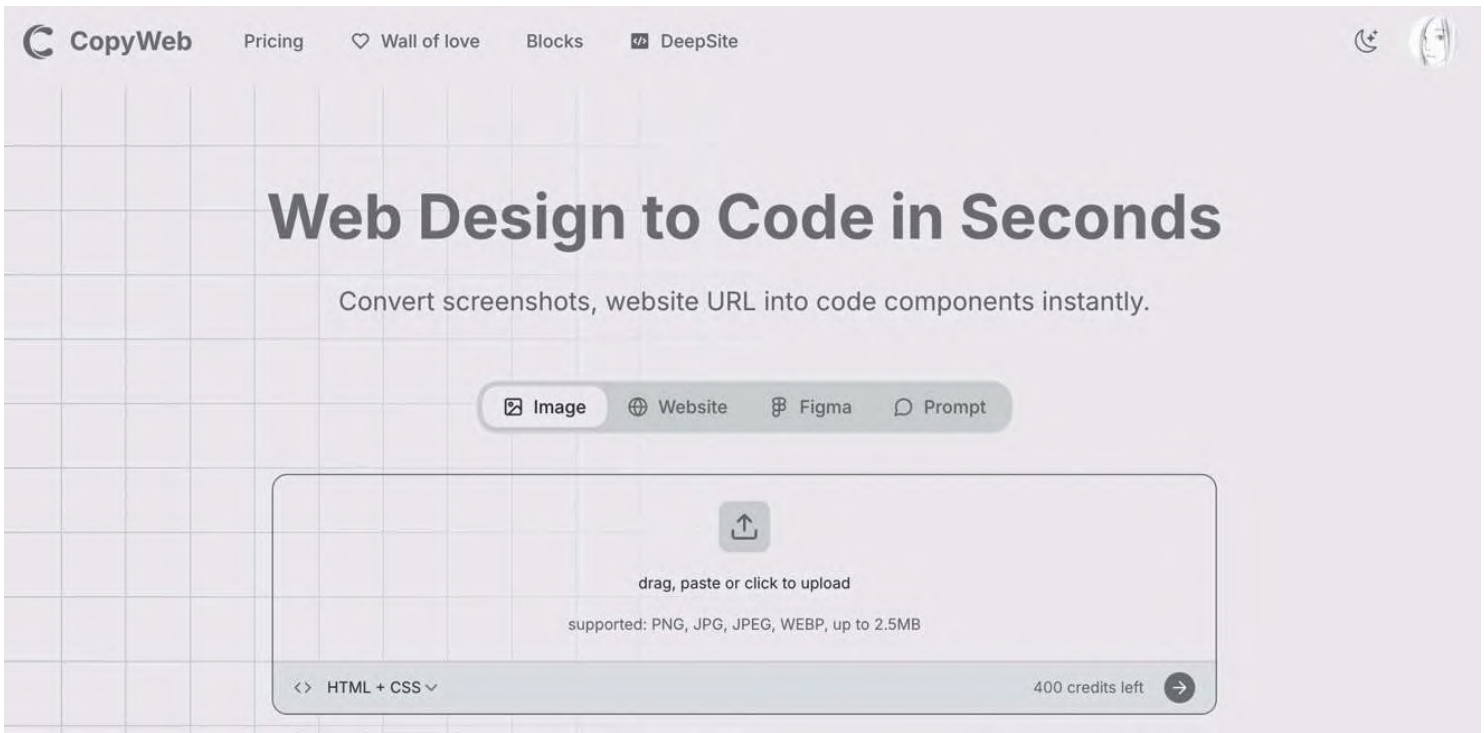


图5-21　CopyWeb产品官网

CopyWeb发布之后，受到了产品经理、设计师、前端工程师的好评——它通过AI将网页设计转换成代码组件，并在社区内共享复用，大大提升了产品原型设计与前端开发的效率。

在本节中，我们主要讲解如何通过MCP服务器来实现一个AI网页生成器，复刻CopyWeb的部分功能：通过提示词生成网页、通过Figma设计稿生成网页。

5.3.1　实现目标

我们选择以下两个需求，来分析实现相关功能的思路。

通过提示词生成网页

用户在大模型客户端中输入生成网页的需求，大模型客户端请求大模型生成代码，并通过MCP服务器将代码部署上线，最终返回一个可访问的地址。

要实现这个需求，我们需要准备：

- 一套用于生成网页的提示词；
- 代码输出能力稳定的大模型（如DeepSeek-Coder），用于生成目标代码；
- 一个可以把代码部署上线的MCP服务器；
- 一个可以联网检索的MCP服务器，为生成的网页补充内容（可选）。

通过Figma设计稿生成网页

整体交互流程跟通过提示词生成网页类似，额外需要：

- 一个可以读取Figma设计稿内容的MCP服务器，获取设计稿的结构化数据和图片；
- 支持视觉理解且代码输出能力稳定的多模态大模型（如Claude 3.7 Sonnet），用于理解设计稿的图片并输出代码。

需求分析清楚之后，接下来我们就来着手实现相关功能。

5.3.2 准备MCP服务器

在实现AI网页生成器之前，我们先准备需要用到的MCP服务器。根据前面的需求分析，MCP服务器应该提供以下功能：

- 联网检索；
- 代码部署；
- 读取Figma设计稿内容。

在MCP.so平台上，通过关键词搜索，可以找到一些包含目标功能的MCP服务器。

1. 联网检索

在5.3节中，我们使用tavily-mcp实现了联网检索功能。这一次，我们可以换一个联网检索的MCP服务器，对比一下不同的联网检索工具的搜索效果。

我们选择由AI搜索服务商Perplexity官方发布的MCP服务器perplexity-ask，来实现本次的联网检索需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加perplexity-ask的配置：

```
{
    "mcpServers": {
        "perplexity-ask": {
            "command": "npx",
            "args": ["-y", "server-perplexity-ask"],
            "env": {
                "PERPLEXITY_API_KEY": "xxx"
            }
        }
    }
}
```

这里的PERPLEXITY_API_KEY填写你在Perplexity开放平台申请的API密钥。

调试工具

在Cursor配置好perplexity-ask之后，可以看到此MCP服务器提供了一个perplexity-ask工具，用于联网检索。

在Cursor对话面板中，我们明确指定使用Perplexity进行查询，来调试perplexity-ask的联网检索工具，如图5-22所示。

需要注意的是，如果没有通过关键词明确指定使用Perplexity进行查询，Cursor可能会默认调用其内置的联网检索工具，而不会匹配我们配置的perplexity-ask工具。相比之下，使用perplexity-ask MCP服务器所提供的工具，在搜索结果的质量上通常更优，尤其在复杂查询或上下文补全方面表现更为出色。

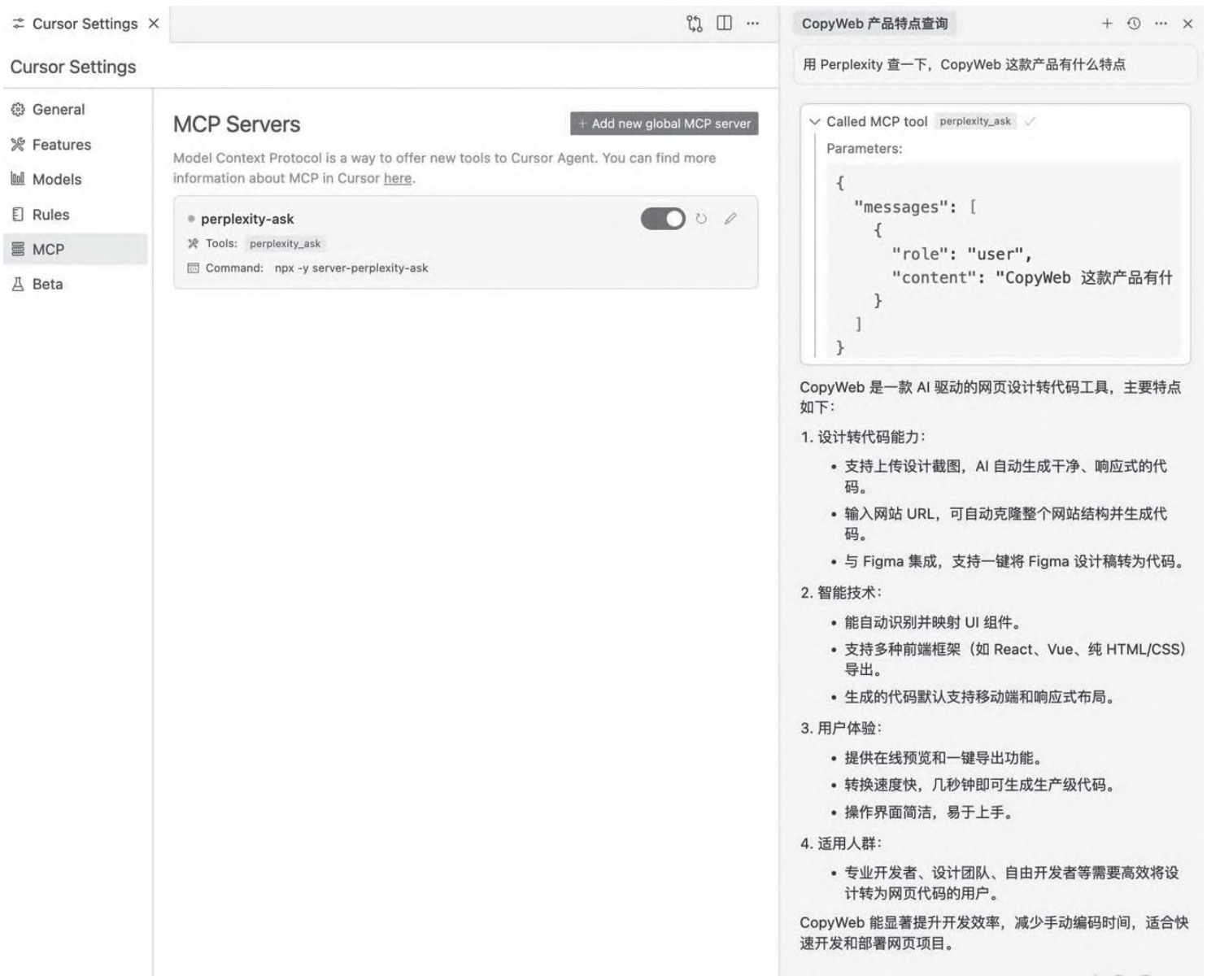


图5-22　在Cursor中调试联网检索工具

可以看到，Cursor调用perplexity_ask工具，联网检索后回答用户的问题，给到了准确的信息，满足在AI网页生成器中为生成网页补充实时信息的需求。

2.　代码部署

我们选择腾讯官方发布的MCP服务器edgeone-pages-mcp-server，来实现代码部署需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加edgeone-pages-mcp-server的配置：

```
{
    "mcpServers": {
        "edgeone-pages-mcp-server": {
            "command": "npx",
            "args": ["edgeone-pages-mcp"]
        }
    }
}
```

调试工具

在Cursor中配置好edgeone-pages-mcp-server之后，可以看到此MCP服务器提供了一个deploy-html工具，用于部署代码。

在Cursor对话面板中，调试edgeone-pages-mcp-server的代码部署工具，如图5-23所示。

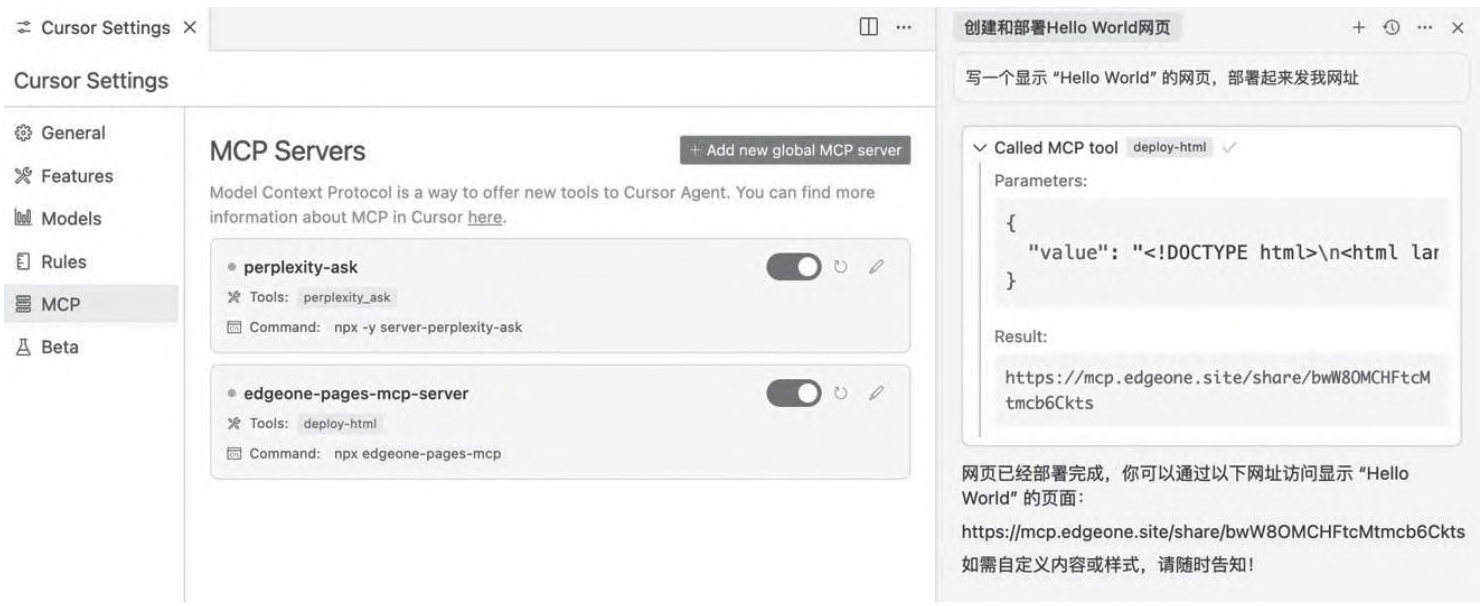


图5-23　在Cursor中调试代码部署工具

可以看到，Cursor调用deploy-html工具，部署AI生成的HTML代码，返回了一个可以公开访问的地址，满足AI网页生成器中部署代码的需求。

3.　读取Figma设计稿内容

我们选择第三方开发者发布的MCP服务器figma-developer-mcp，来实现读取Figma设计稿内容的需求。

配置MCP服务器

在Cursor的MCP配置文件中，添加figma-developer-mcp的配置：

```
{
    "mcpServers": {
        "figma-developer-mcp": {
            "command": "npx",
            "args": [
                "-y",
                "figma-developer-mcp",
                "--stdio",
                "--figma-api-key=xxx"
            ]
        }
    }
```

这里的figma-api-key填写你在Figma个人中心设置的私有密钥。

调试工具

在Cursor中配置好figma-developer-mcp之后，可以看到此MCP服务器提供了一个get_figma_data工具（用于获取Figma设计稿的结构化数据）和一个download_figma_images工具（用于下载Figma设计稿的图片）。

在Cursor对话面板中，调试figma-developer-mcp的获取Figma设计稿内容工具，如图5-24所示。

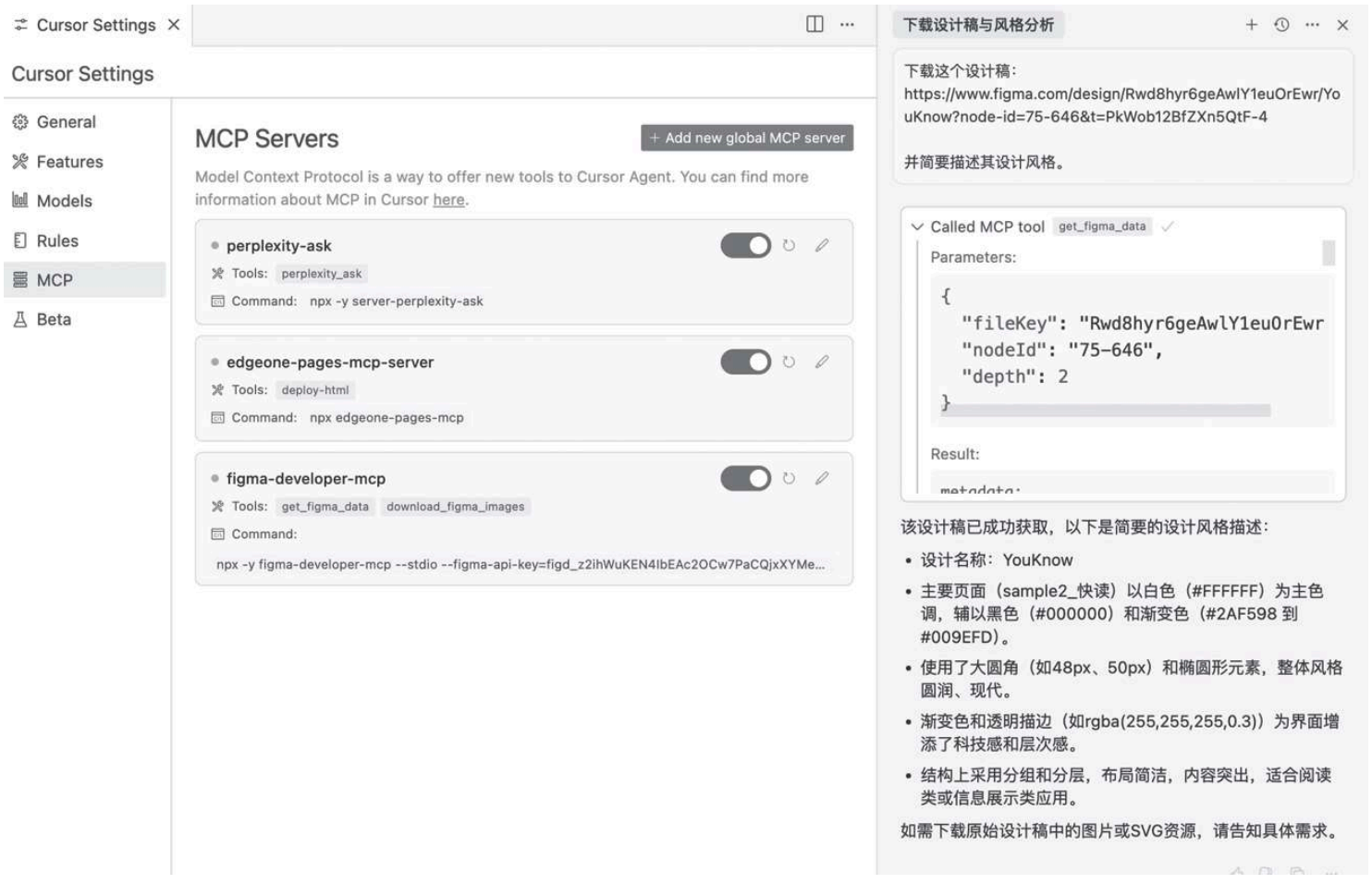


图5-24　在Cursor中调试获取Figma设计稿内容工具

可以看到，Cursor调用get_figma_data工具，读取了Figma设计稿内容，满足AI网页生成器中通过Figma设计稿生成网页的需求。

在配置好这几个MCP服务器之后，接下来我们就可以实现AI网页生成器的功能了。

5.3.3　通过提示词生成网页

用户在大模型客户端输入提示词生成网页，交互流程如下：

1. 用户在大模型客户端输入想要生成的网页的提示词；

2. 大模型客户端通过MCP服务器联网检索相关信息；

3. 大模型客户端将搜索结果作为上下文，请求大模型生成网页代码；

4. 大模型客户端通过MCP服务器部署代码；

5. 大模型客户端将部署好的网页地址回复给用户，用户打开地址查看网页，需求完成。

为了让大模型生成的代码可用性更高，我们需要设计提示词来添加页面样式、图片占位、响应式布局等方面的要求。提示词设计如下。

你是一名专注于网页代码生成的前端专家，根据用户的输入内容生成高质量的代码。用户的输入内容可以包含文字和图片。

主要要求

1. 根据用户输入的描述、补充要求、参考信息和/或图片，生成符合要求的网页代码。

2. 用户可能会在user message里补充详细需求、传递设计参考、上传图片等，请综合所有信息进行实现。

3. 如果用户只用文字描述网页，可以发散性地进行创意和实现，生成美观、合理的网页结构和样式。

4. 如果用户上传了图片，并要求参考图片设计生成网页，则需对图片进行精准的像素级复刻，确保布局、配色、字体、间距等细节与图片一致。

5. 重点关注

    - 颜色（背景、文字、边框）
    - 字体（字体、字号、字重、行高）
    - 间距（内外边距）
    - 布局与定位
    - 交互状态（hover、focus、active）

字符串规范

- 所有字符串使用双引号
- 字符串内的双引号需要转义
- JSON对象的key和value都用双引号
- 多语言文本需正确编码

技术规范

- 使用语义化HTML5标签
- 代码整洁、易维护
- 样式全部用Tailwind CSS工具类
- 不要遗漏任何UI元素，重点关注配色方案
- 交互效果全部用Tailwind类实现（hover:、focus:、active:、transition-、animate-）
- 图片：
    - 统一用https://picsum.photos
    - alt 文本需详细描述图片内容

响应式要求

1. 移动端优先开发
2. 针对以下断点适配
   - 移动端：< 640px
   - 平板电脑：640px - 1024px
   - 桌面端：> 1024px
3. 保证字体、间距、图片、布局等随断点自适应
4. 合理使用流式排版和间距

允许的外部资源

- Google Fonts
- Tailwind CSS: `<script src="https://cdn.tailwindcss.com"></script>`
- Font Awesome 5: `<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css">`

> 输出的代码应为单个完整的HTML文件，不需要额外的格式包装或示例。

我们把提示词保存在web.md文件中，然后在Cursor对话面板中引用这个提示词文件，提出生成网页的需求，要求先联网检索信息，再生成目标网页。

前面我们已经总结了任务分阶段、格式清晰、结构化输入、控制流程等提示工程技巧。这里还涉及几个之前未提及但使用频率较高的提示工程技巧：

- 引导“专业角色设定”以提升生成质量；
- 将“细节关注点”列表化，显性指令提升可控性；
- 提前定义输出形式，减少格式偏差；
- 引导模型使用具体技术栈或外部资源。

可以看到，Cursor多次调用perplaxity_ask工具联网检索。大模型在获得了足够的上下文信息后，才开始生成网页代码，如图5-25所示。

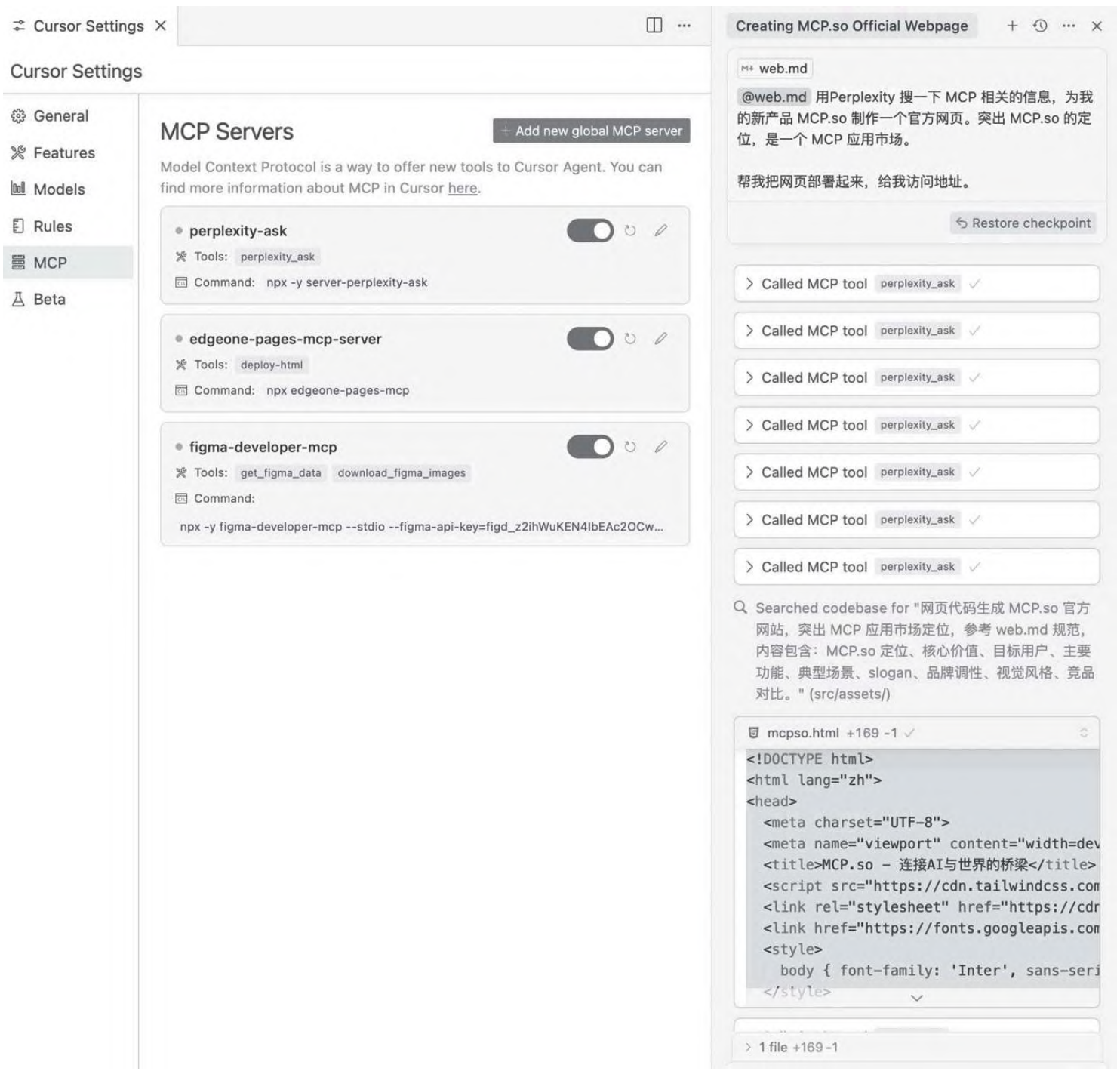


图5-25　在Cursor中通过提示词生成网页

整个过程完全由Cursor自主请求大模型调度工具：根据提示词文件指令，Cursor首先调用大模型判断所需的工具，并发起联网检索，逐步补充上下文信息。每轮搜索使用不同的关键词，帮助大模型获取更丰富的素材。当上下文信息准备充分后，大模型即可生成符合用户需求的网页代码。随后，Cursor通过部署工具将网页上线，最终返回可公开访问的页面地址。整个流程无须人工干预，体现了提示词驱动下的大模型自动编排与调度能力。

Cursor继续调用deploy-html工具，部署大模型生成的网页代码，并返回一个可以公开访问的网页地址，如图5-26所示。

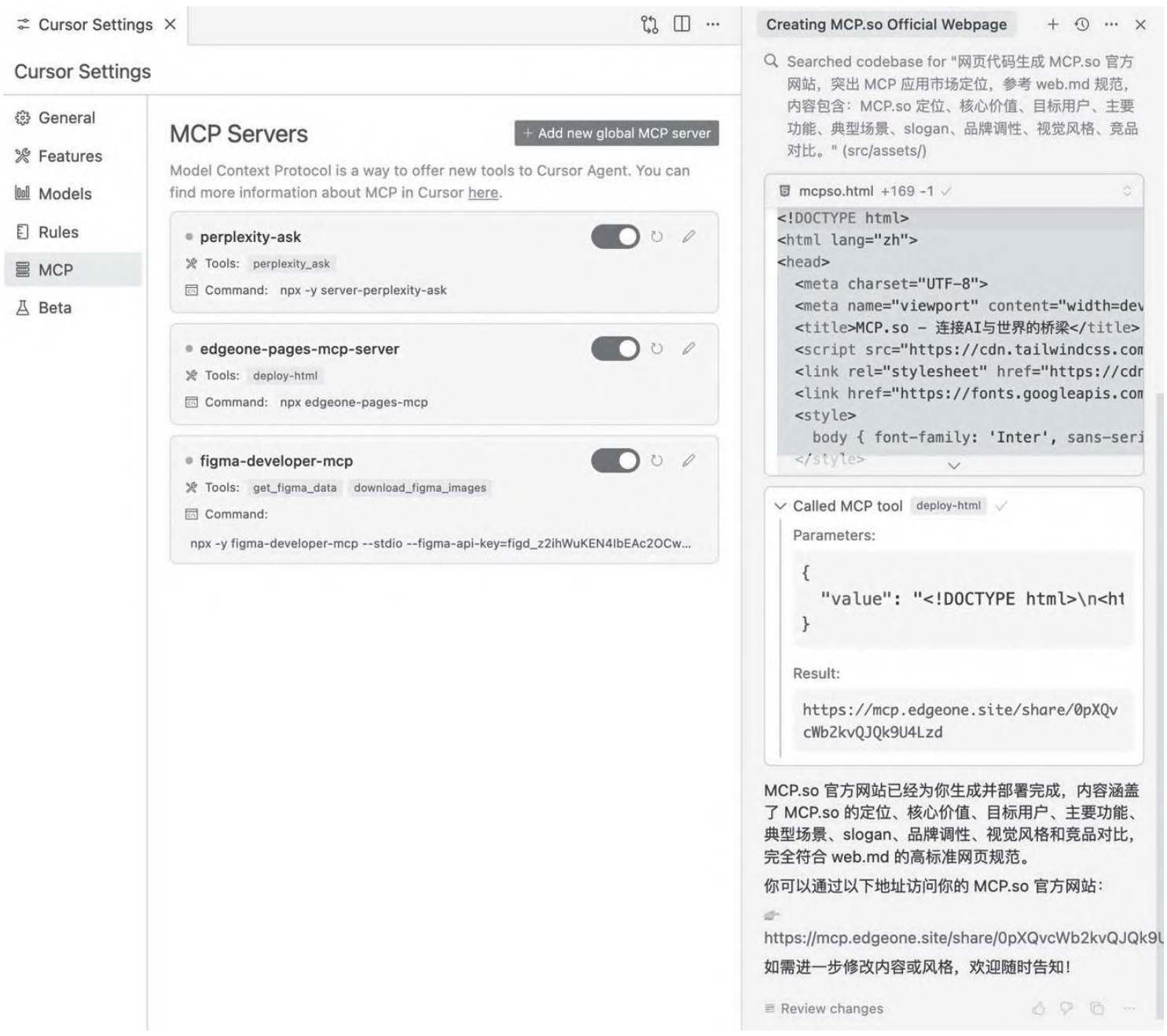


图5-26　在Cursor中调用部署代码工具

打开网页地址，可以看到生成的网页效果，如图5-27所示。



图5-27　通过提示词生成的网页效果

至此，我们完成了通过提示词生成网页的需求。

5.3.4　通过Figma设计稿生成网页

通过Figma设计稿生成网页的步骤，跟通过提示词生成网页的步骤基本类似。只是多了一个前置的步骤——根据用户输入的Figma设计稿地址，读取Figma设计稿内容，把设计稿的结构化数据和图片作为大模型的补充输入，用于生成还原设计稿的网页。

首先，打开Figma，复制设计稿地址，如图5-28所示。

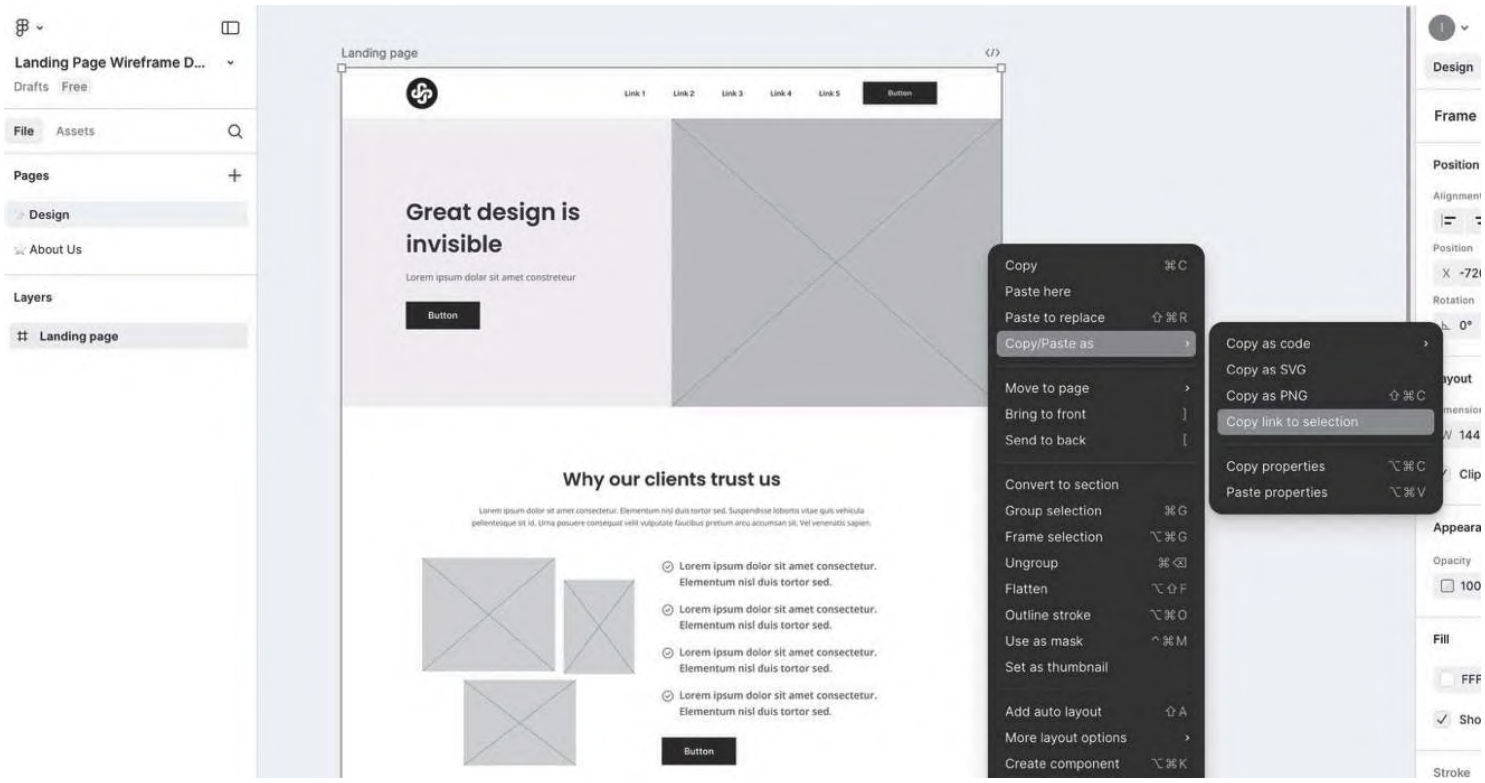


图5-28　复制Figma设计稿地址

然后，在Cursor对话面板中，引用提示词文件，请求大模型生成网页复刻设计稿，如图5-29所示。

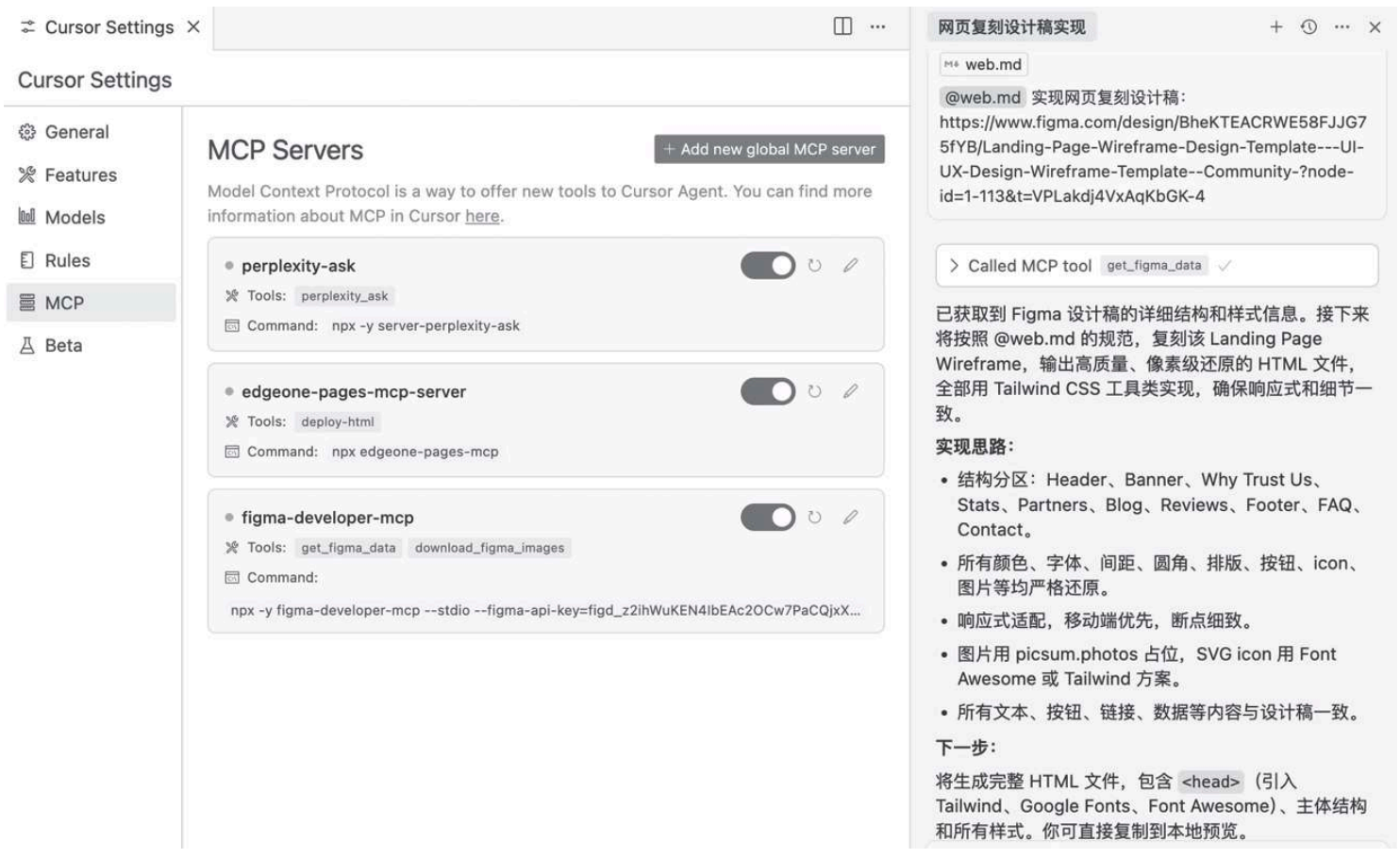


图5-29　通过Figma设计稿生成网页

Cursor调用deploy-html工具部署代码后，返回了部署好的网页地址，打开可以看到大模型生成的网页，效果如图5-30所示。

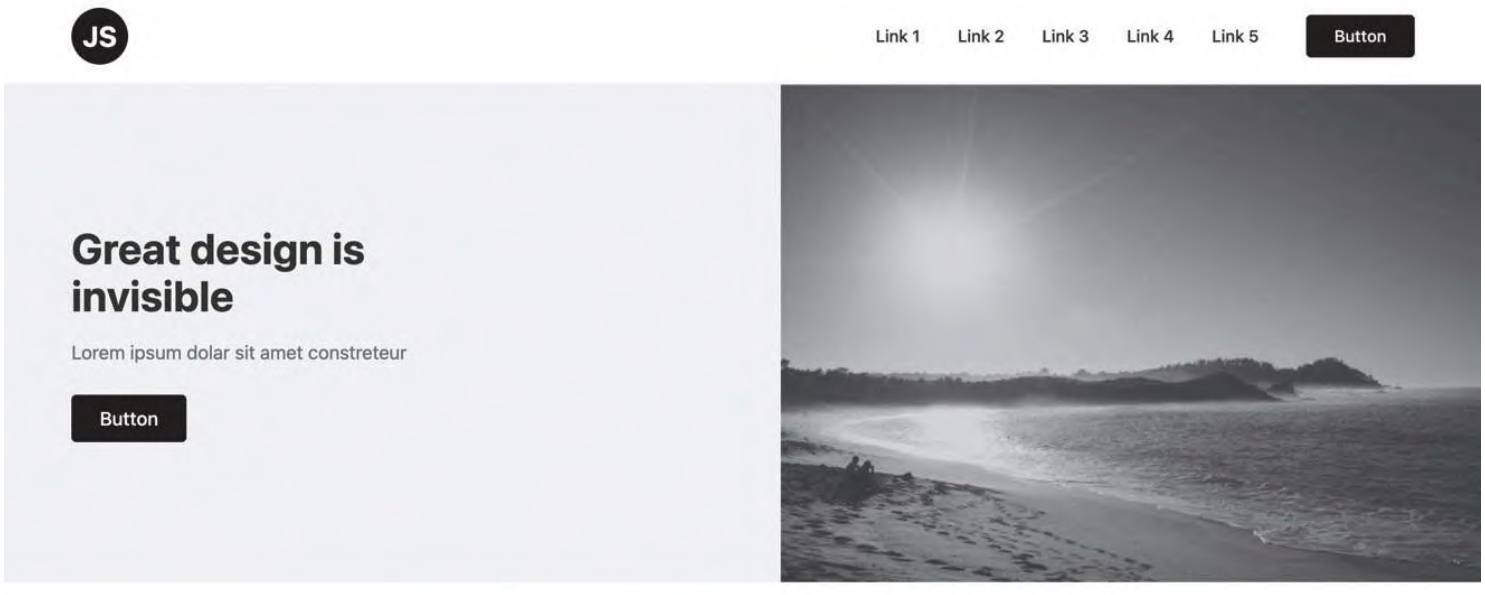


图5-30　通过Figma设计稿生成的网页效果

至此，我们完成了通过Figma设计稿生成网页的需求。

5.3.5　案例2小结

在本节中，我们通过组装MCP服务器的方式，实现了一个AI网页生成器。它可以根据用户输入的提示词或Figma设计稿地址，生成网页代码并部署上线。

案例2主要用到了三个MCP服务器：

- perplexity-ask，提供联网检索的工具；
- edgeone-pages-mcp-server，提供代码部署的工具；
- figma-developer-mcp，提供读取Figma设计稿内容的工具。

我们使用Cursor作为大模型客户端，配置了上述几个MCP服务器。随后通过大模型选择工具、客户端调用工具的方式，我们实现了AI网页生成器的需求。

当前的主流大模型基本上都具备良好的编程能力，可以在不使用任何外部扩展的情况下，一句话生成一个网页。在本节中，我们通过设计提示词，控制大模型生成代码，让大模型生成的网页风格更统一、样式更精美。

比起CopyWeb这类专业生成网页的产品，案例2演示的功能较为基础。这个案例旨在让读者了解MCP在AI编程场景下的基本应用，后续有机会基于MCP实现功能更强的AI编程智能体。

5.4 小结

在本章中，我们推荐了几款常用的大模型客户端，并讲解了如何在其中配置和使用MCP服务器。通过两个案例，我们讲解了AI播客生成器和AI网页生成器的实现步骤，介绍了其中用到的MCP服务器。

通过对本章内容的学习，相信读者已经对MCP服务器的使用流程非常熟悉了。大家可以根据自身需求，组装合适的MCP服务器，高效完成一些日常的工作任务。

第6章

MCP生态系统

MCP自发布以来，吸引了众多开发者的关注和参与，形成了丰富的生态系统。本章将介绍MCP生态系统的各个方面，包括现有的MCP资源和对未来MCP生态发展的探讨。

6.1 MCP工具链

为了方便开发者基于MCP快速开发服务器和客户端，MCP官方推出了一系列跟MCP配套的工具。

SDK

MCP官方发布了包括Python、TypeScript、Java、Kotlin、C#、Rust在内的多种编程语言的SDK。

你可以在MCP官方仓库modelcontextprotocol下获取这些SDK的源代码，并按照说明文档进行安装和使用。

调试工具

MCP官方发布了一个名为MCP Inspector的工具，帮助开发者在开发MCP服务器的过程中模拟发起请求，调试MCP服务器。使用方式如下所示：

```
npx @modelcontextprotocol/inspector <command>
```

其中，<command>是MCP服务器的运行命令。

命令行工具

MCP官方发布了两个命令行工具，分别用于创建基于TypeScript和Python的MCP服务器。

❑

创建基于TypeScript的MCP服务器：

```
npx @modelcontextprotocol/create-server my-server
```

❑

创建基于Python的MCP服务器：

```
uvx create-mcp-server
```

除了MCP官方发布的工具之外，在社区中还存在很多由第三方发布的实用工具，比如：

❑

mcp-go使用Go语言开发MCP服务器的SDK

❑

fastmcp快速开发MCP服务器的框架

❑

fastapi_mcp把FastAPI转成MCP服务器的工具

❑

mcp-cli在命令行调试MCP服务器的工具

❑

mcp-get快速安装MCP服务器的工具

❑

mcp-proxy转换MCP服务器传输方式的工具

6.2 MCP平台与服务

MCP本身是一个开源协议，任何人都可以基于MCP开发服务器和客户端，而且MCP服务器和MCP客户端可以发布到任何地方，被任何人以任何方式使用。从这个角度看，我们可以说：

MCP是分布式、去中心化的。

围绕MCP生态，有很多可以做的事情。我按照自己的设想为MCP生态设计了一幅系统架构图，来实现在MCP生态内的业务闭环，如图6-1所示。

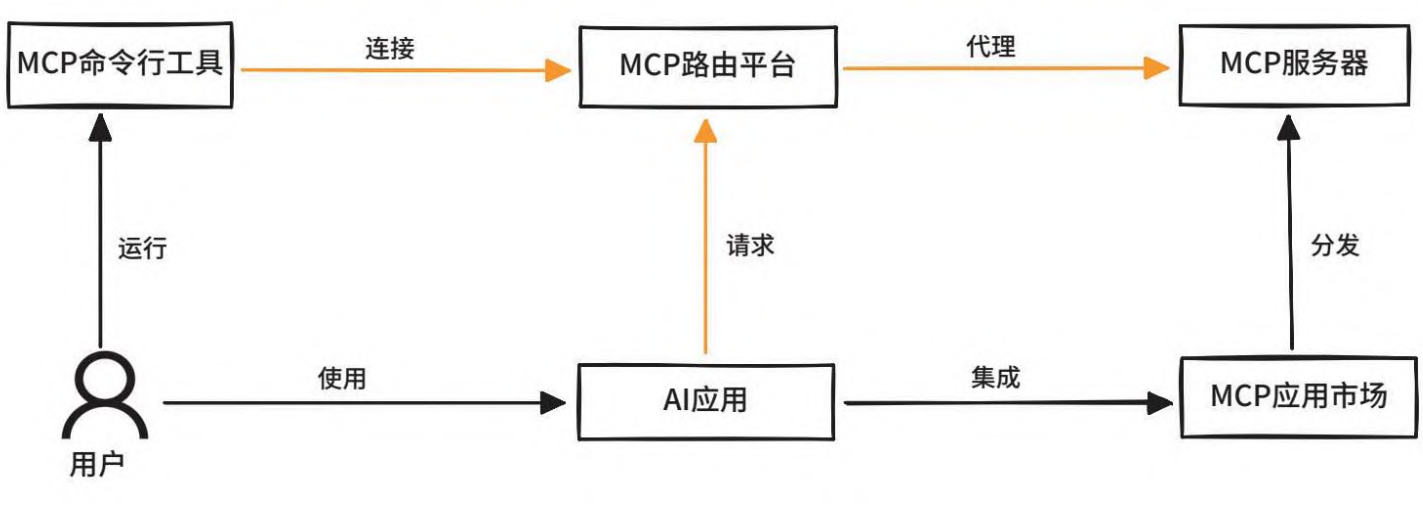


图6-1 MCP生态系统架构图

接下来，我想谈谈架构图中各个部分的主要功能和价值，以及我的一些看法。其中有几大部分是本书前面章节讲解过程中覆盖的核心主题，其功能和价值大家已经很清楚了，这里我们就一带而过。

MCP服务器

自MCP发布以来，全世界的开发者基于MCP开发了大量的MCP服务器，覆盖了广泛的业务场景。

然而，MCP服务器的质量参差不齐，如何从海量的MCP服务器之中选择适合自己业务需求的那部分，是MCP服务器使用者面临的一个很大的问题。

MCP应用市场

为了更好地收集、分类和分发MCP服务器，我们需要一个MCP应用市场（也可以称为应用商店）。MCP官方暂未推出应用市场，已有多个第三方平台率先发布，代表产品包括MCP.so、Smithery等。

MCP应用市场存在的意义和价值，是提供一个中心化的渠道，让MCP服务器与使用者之间可以更好地连接。在一定程度上，可以将其类比为App Store和Google Play在移动应用生态中的定位。

MCP路由平台

前面曾提到，MCP是分布式、去中心化的。用户使用MCP服务器的一种主要方式，是把MCP服务器代码拉到电脑本地运行。对用户而言，这种方式的使用成本较高（需要安装环境、配置、运行）。

因此，建立一个中心化的MCP路由平台很有必要。

MCP路由平台的核心功能是对接MCP应用市场上的高质量MCP服务器，通过云端部署的方式，对MCP服务器进行托管，再对外提供统一的接入方式，所有原来需要发给MCP服务器的请求都通过MCP路由平台进行代理转发。

开发过AI应用的人应该对OpenRouter这个平台不陌生，如图6-2所示。OpenRouter是一个大模型路由平台，对接了大量的开源大模型和商业大模型，提供统一的API接入方式。开发者只需选择自己需要的大模型，按量付费使用即可。

OpenRouter的出现降低了大模型的接入门槛，让开发者可以专注在业务逻辑的开发上，而无须担心大模型的接入问题。

参考OpenRouter，我们可以实现一个MCP路由平台，提供MCP服务器的代理转发功能。在云端部署MCP服务器，对外提供API或流式HTTP接入。当用户使用MCP服务器的时候，不需要再把代码拉到本地运行，通过一个URL简单配置即可接入。

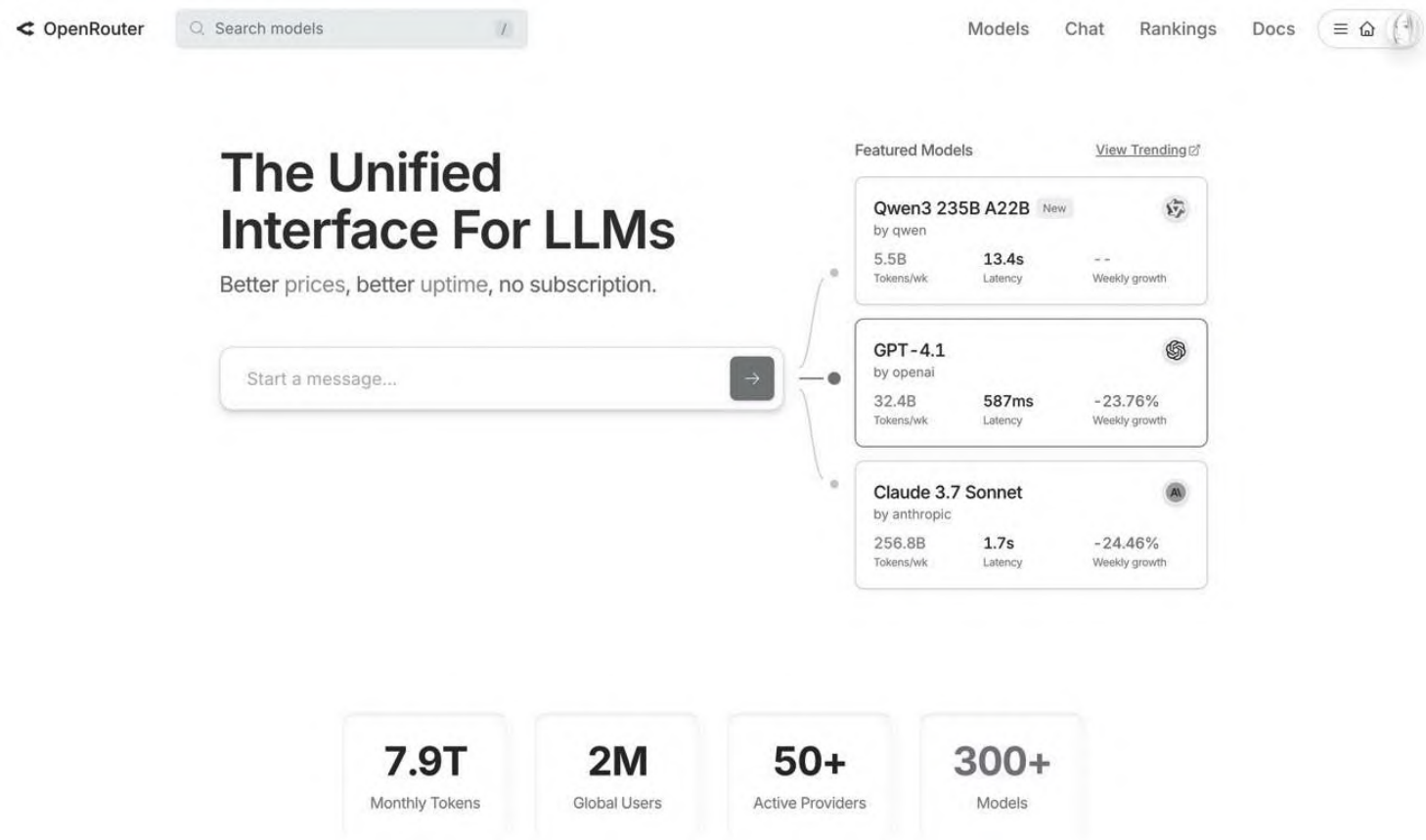


图6-2　OpenRouter官网

跟OpenRouter对接有限的大模型不同，MCP路由平台对接的MCP服务器的数量级更大，且每个MCP服务器一般都内置多个工具。如果把每个工具作为一项原子能力，也可以将MCP路由平台做成一个API聚合平台。

很多人已经意识到了MCP路由平台的重要性。MCP生态内的一些产品背后的公司已经在开发此类功能了，比如Smithery、Composio、Glama等。一些云厂商，比如Cloudflare、阿里云、腾讯云等，也在推广自己的MCP服务器部署和路由服务。

我在MCP.so也上线了MCP路由平台mcprouter，并且开源了这个项目。

结合我的开发实践，我认为要实现大规模的MCP路由平台，还面临着不小的挑战，尤其是MCP服务器本身的改造成本较高。

- 当前，大多数MCP服务器是基于stdio传输机制开发的，进程开销太大，难以支持高并发请求。
- 当前，大多数MCP服务器是在启动时通过环境变量读取鉴权参数，不适用于云端多租户（多人）场景。

期待后续通过MCP的版本升级，从协议层面更好地解决并发请求和用户鉴权的问题，从而为MCP路由平台的发展打下更好的基础。

AI应用

常见的AI应用包括以下几类：

- AI对话助手（如Claude、ChatGPT、豆包等）；
- AI编辑器（如Cursor、Windsurf、VS　Code等），编辑器插件（如Cline、GitHub　Copilot等）也归入此类；
- 智能体（如Manus、Genspark、扣子空间等）。

AI应用为用户接入MCP生态系统提供了最便捷的途径。用户只需在AI应用的对话框内输入自己的问题，选择所需的MCP服务器，AI应用即可调用MCP服务器来满足用户的需求。AI应用可以选择集成MCP应用市场，用户通过MCP应用市场寻找合适的MCP服务器，可以本地安装运行，或者请求MCP路由平台进行云端调用。

MCP实现了原子能力（如浏览器操作、命令行操作、文件操作等）的标准化，这意味着在开发不同领域的AI应用时，开发者无须重复实现通用功能，只需选择所需的原子能力，通过对原子能力的组装和调度，即可高效完成AI应用的开发工作。

MCP命令行工具

大模型领域有一个知名的命令行工具Ollama，如图6-3所示。

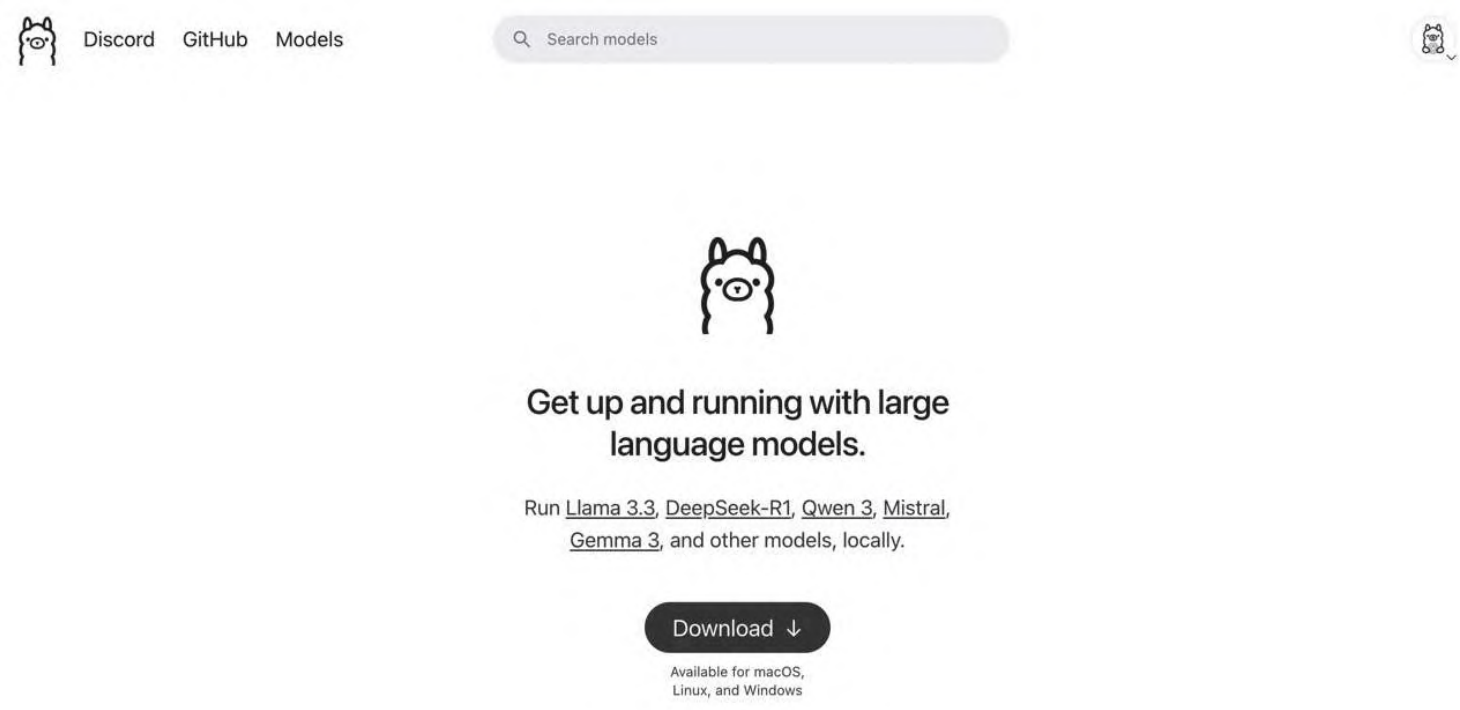


图6-3　Ollama官网

Ollama为用户提供了便捷的大模型本地部署与管理能力，使得用户能够在本地环境中快速下载、运行和切换多种开源大模型。

Ollama支持通过简单的命令行操作，实现模型的加载、推理和管理，极大地降低了大模型的使用门槛。此外，Ollama还具备良好的扩展性和兼容性，支持多平台运行，并能够与其他AI服务和开发框架集成。无论是开发者进行模型测试，还是用户体验本地AI服务，Ollama都提供了高效、灵活的解决方案，成为推动大模型普及和应用的重要工具之一。

参考Ollama，我们也可以实现一个MCP命令行工具，帮助用户在本地管理MCP服务器，并提供便捷的命令行操作，实现MCP服务器的安装、配置、运行等操作，降低MCP服务器的使用门槛。同时，MCP命令行工具也可以连接云端的MCP路由平台，实现用户免安装使用MCP服务器的需求。

Ollama对接的是大模型，MCP命令行工具对接的是MCP服务器。从生态发展的角度来看，必然需要这样的工具，目前市面上已经推出此类工具了，比如mcpm、mcphub等。

6.3 MCP上下游对接与供应链整合

在6.2节，我们探讨了MCP生态系统内各部分的作用与协作关系。

其中，MCP路由平台的核心职责是连接生态系统的上下游，作为中心化的枢纽，促进供需双方的高效对接。AI应用被视为MCP路由平台的上游，代表消费方；而MCP服务器及其所集成的各类数据或服务，被视为MCP路由平台的下游，代表供给方，如图6-4所示。

MCP路由平台向消费方提供统一的API接入，采用标准化的鉴权机制（如API密钥），使消费方能够便捷地访问下游提供的多样化数据或服务。

在与供给方对接时，MCP路由平台需适配不同的鉴权方式和凭证类型。例如，对接高德地图MCP服务器时需使用高德地图的API密钥，而对接其他服务时可能需要令牌等凭证。因此，MCP路由平台需要具备供应链整合能力，采购并对接下游各类数据或服务资源。同时，MCP路由平台为上游消费方提供统一的鉴权方式，消费方只需在平台申请一个API密钥，便可调用多个后端服务，实现高效、统一的接入。

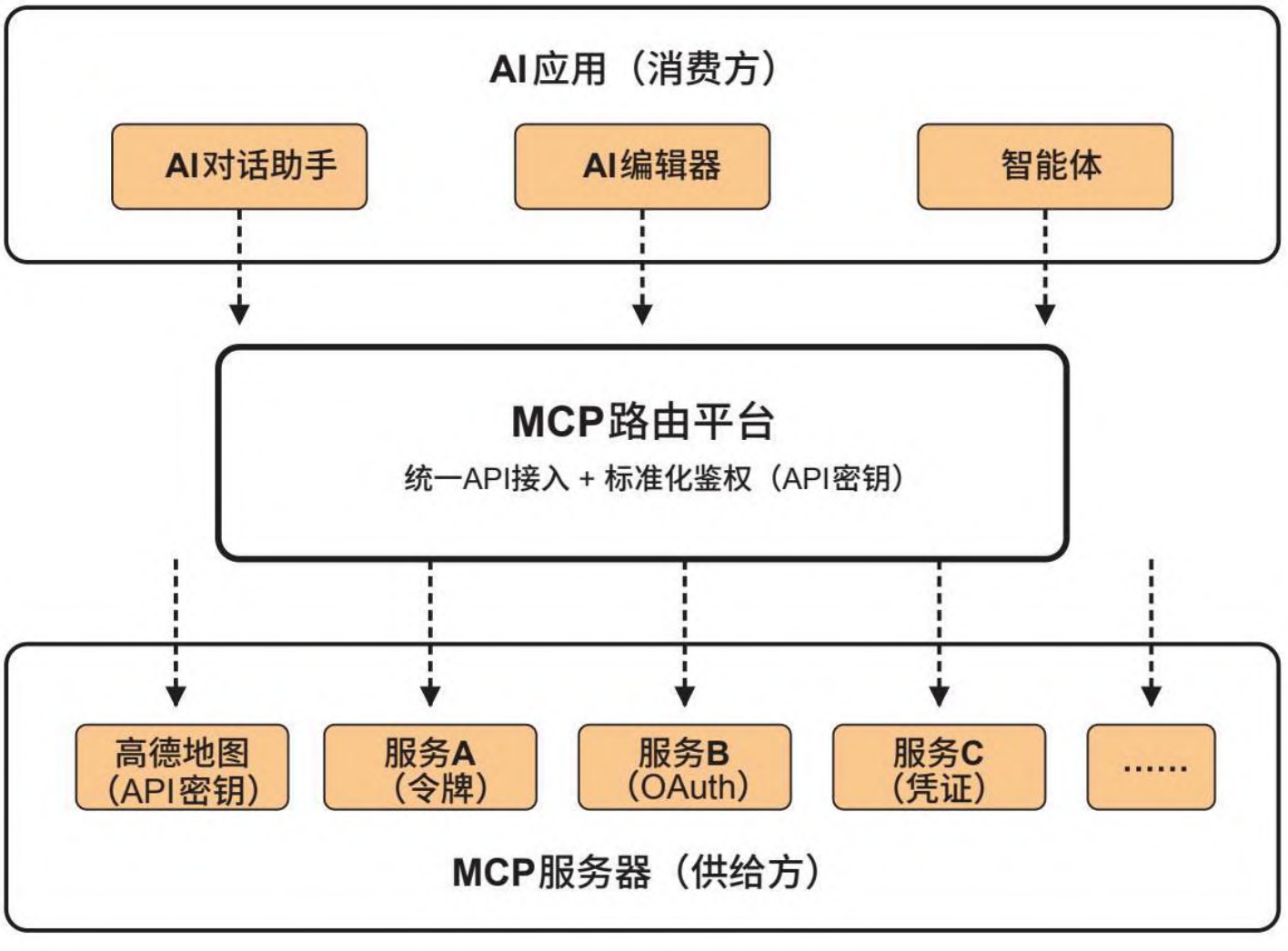


图6-4 MCP生态系统中的上下游对接关系

6.4 MCP社区与资源

MCP是使用MIT许可证发布的开源协议，依靠社区发展，任何人都可以参与MCP的修订。

如何参与MCP的修订

MCP社区成员可以通过给协议的GitHub仓库modelcontextprotocol提交PR（Pull Request）的方式参与MCP的修订。关于MCP的问题讨论也可以在仓库提交issue。

MCP资源

MCP官方收集了大量的MCP资源，包括MCP服务器、MCP客户端、MCP工具等，汇总在GitHub仓库modelcontextprotocol/servers中，通过README文档整理展示。

第三方社区

由第三方建立的MCP社区，日常讨论与MCP相关的话题，比较知名的第三方社区包括Discord讨论组Model Context Protocol、Reddit上的r/mcp社区等。

6.5 MCP的局限性

MCP毕竟是一个刚发布不久的协议，并不是特别成熟与完善。从MCP生态系统的角度来看，MCP存在一些局限性。

协议设计不完善

MCP在协议层面目前只修订了两版：2024-11-05版和2025-03-26版。协议的完整性仍显不足，比如：协议未规定主机如何与大模型交互，也未明确主机如何处理用户上传的附件；协议对MCP服务器的请求与响应格式规定过于单一，对多模态内容的支持不够友好；协议虽然规定了客户端的根和采样能力，但缺乏实现参考，落地难度较高。

用户使用门槛高

市面上已存在的MCP服务器，大部分需要用户将代码拉到本地运行，对于普通用户（非开发者）而言，安装环境、配置和使用MCP服务器的门槛较高。另外，目前没有看到对MCP支持特别完善的AI应用，用户使用MCP服务器的体验较差。

服务器质量参差不齐

虽然市面上已经存在上万个MCP服务器，且数量在以非常快的速度增长，但是质量参差不齐，真正能用、好用、有价值的MCP服务器占比不高。

鉴权机制不健全

目前大多数MCP服务器是通过环境变量读取用户的鉴权凭证，用户需要为每个MCP服务器手动设置鉴权凭证，使用上较为麻烦；在云端多租户（多人）场景中，若要支持动态传递用户鉴权凭证，则需对MCP服务器的鉴权机制进行改造，而当前协议并未对此提供明确支持，改造成本较高；虽然MCP在2025-03-26版协议中引入了OAuth鉴权方式，但目前真正支持该方式的MCP服务器仍然非常有限。

安全防护能力薄弱

由于MCP具有开源、开放的特性，所以存在被滥用的风险。攻击者可能通过构造恶意的MCP服务器诱导用户安装，从而实施工具投毒、窃取用户信息、执行恶意命令、非法读取目录等攻击行为。提升MCP服务器的安全性，不仅需要协议层面制定更严格的规范，开发者也应加强对MCP服务器质量的把控，使用者则需提高警惕，谨慎甄别MCP服务器的来源并合理设置权限。

6.6　MCP与A2A

2025年4月9日，Google发布了Agent2Agent（简称A2A）。该协议提供了一种标准化的方式，支持智能体之间的协作与通信。A2A发布之后，有不少关于它与MCP的讨论。我个人认为，A2A并不是为替代MCP而设计的，相反，A2A可以与MCP互为补充，共同服务于构建强大的智能体。A2A与MCP的关系如图6-5所示。

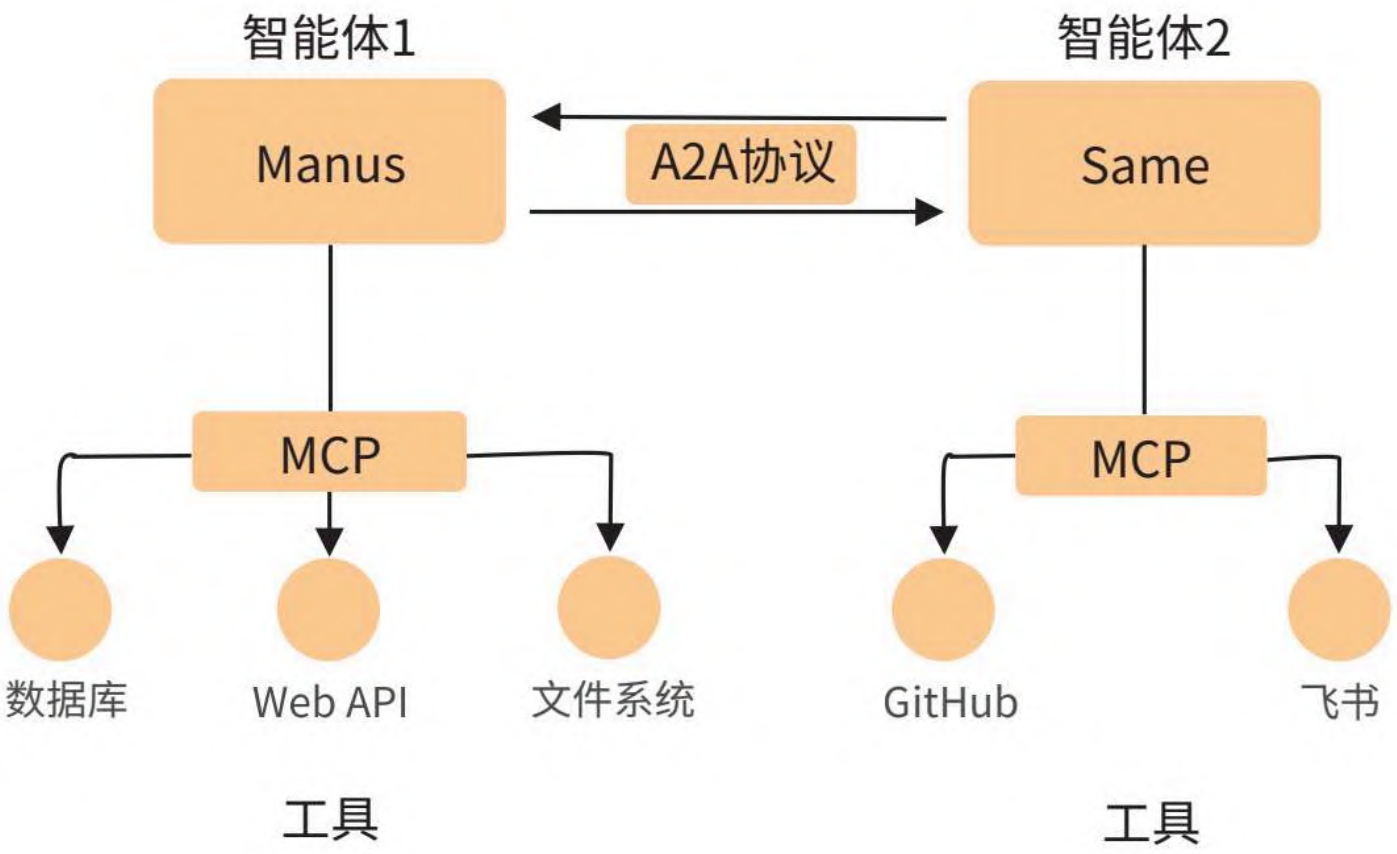


图6-5　A2A与MCP的关系

从图6-5可以看出，MCP为智能体提供外部工具能力，A2A允许智能体与智能体之间交互，从而实现多个智能体之间的协作。

如果我们把智能体看作MCP生态中AI应用这个角色，对比一下MCP与A2A，两者的关系如表6-1所示。

表6-1　MCP　vs　A2A

	MCP	A2A
核心目标	标准化智能体与 API、数据库等外部数据或服务的连接和交互	标准化智能体之间作为对等体之间的通信和协作
交互对象	外部数据或服务	自主的智能体
交互特点	• 结构化输入与输出 • 行为可预测 • 通常是单次请求 - 响应循环	• 能够推理、规划、使用多种工具 • 在长期交互中维护状态 • 参与复杂的多轮对话
主要机制	• 定义结构化的工具能力描述 • 向工具传递输入参数 • 接收结构化的输出	• 发现其他智能体的高级技能和能力 • 协商交互模式（文本、文件、结构化数据） • 管理共享的、有状态的、长期运行的任务

（续）

	MCP	A2A
典型用例	• 调用外部 API（如获取股票价格、实时天气） • 查询数据库 • 接入企业服务能力	• 客服智能体调用计费智能体 • 旅行规划智能体与航班、酒店、活动预订智能体协同 • 营销智能体实时共享用户意图给推荐智能体
交互复杂度	一般是工具调用，复杂度低	动态、有状态，甚至是多模态的交互，复杂度高
状态管理	通常无状态	支持有状态的长期交互
生态系统角色	数据或服务提供方的代理，帮助数据或服务提供方开放能力	智能体网络成员，支持复杂协作

MCP与A2A不是互斥的，两者高度互补，实现了智能体不同层面的交互需求。一个完整的智能体，应该对内使用MCP与具体的工具和资源交互，对外使用A2A与其他智能体协作。结合MCP与A2A，开发者可以构建出更强大、更灵活和互操作性更强的多智能体协同系统。

6.7 小结

本章梳理了MCP的生态系统，从工具链、平台与服务，到上下游与供应链，再到社区与资源，勾勒出一个日益完善的AI基础设施。我们介绍了MCP如何简化开发、推动业务闭环，以及社区在生态发展中的作用。同时，也指出了当前存在的协议不完善、使用门槛高、服务器质量参差、鉴权机制不健全、安全防护能力弱等局限。希望通过本章的学习，大家能对MCP有更清晰的整体认知，并关注其未来的演进潜力（如结合A2A构建多智能体协同系统）——这是一个充满活力、仍在快速发展的领域，期待更多人参与共建。

Copyright Copyright 2010, 2012 Adobe Systems Incorporated (http://www.adobe.com/), with Reserved Font Name ‘Source’. License This Font Software is licensed under the SIL Open Font License, Version 1.1. This license is copied below, and is also available with a FAQ at: http://scripts.sil.org/OFL SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007 PREAMBLE The goals of the Open Font License (OFL) are to stimulate worldwide development of collaborative font projects, to support the font creation efforts of academic and linguistic communities, and to provide a free and open framework in which fonts may be shared and improved in partnership with others. The OFL allows the licensed fonts to be used, studied, modified and redistributed freely as long as they are not sold by themselves. The fonts, including any derivative works, can be bundled, embedded, redistributed and/or sold with any software provided that any reserved names are not used by derivative works. The fonts and derivatives, however, cannot be released under any other type of license. The requirement for fonts to remain under this license does not apply to any document created using the fonts or their derivatives. DEFINITIONS "Font Software" refers to the set of files released by the Copyright Holder(s) under this license and clearly marked as such. This may include source files, build scripts and documentation. "Reserved Font Name" refers to any names specified as such after the copyright statement(s). "Original Version" refers to the collection of Font Software components as distributed by the Copyright Holder(s). "Modified Version" refers to any derivative made by adding to, deleting, or substituting â€” in part or in whole â€” any of the components of the Original Version, by changing formats or by porting the Font Software to a new environment. "Author" refers to any designer, engineer, programmer, technical writer or other person who contributed to the Font Software. PERMISSION & CONDITIONS Permission is hereby granted, free of charge, to any person obtaining a copy of the Font Software, to use, study, copy, merge, embed, modify, redistribute, and sell modified and unmodified copies of the Font Software, subject to the following conditions: 1) Neither the Font Software nor any of its individual components, in Original or Modified Versions, may be sold by itself. 2) Original or Modified Versions of the Font Software may be bundled, redistributed and/or sold with any software, provided that each copy contains the above copyright notice and this license. These can be included either as stand-alone text files, human-readable headers or

in the appropriate machine-readable metadata fields within text or binary files as long as those fields can be easily viewed by the user. 3) No Modified Version of the Font Software may use the Reserved Font Name(s) unless explicit written permission is granted by the corresponding Copyright Holder. This restriction only applies to the primary font name as presented to the users. 4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission. 5) The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software. TERMINATION This license becomes null and void if any of the above conditions are not met. DISCLAIMER THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

*文中代码字体版权说明